

Programación Web

Programación IV



Ing. Alejandro Di Battista

Programación IV — Introducción a JavaScript

Del lenguaje del navegador a una plataforma full stack

JavaScript está presente en casi todas las aplicaciones web que usamos. Responde cuando hacemos clic en un botón, valida un formulario, actualiza una pantalla sin recargarla y permite construir interfaces completas. Pero su alcance ya no termina en el navegador: también puede recibir solicitudes HTTP, consultar bases de datos, automatizar tareas y ejecutar servicios del lado del servidor.

¿Cómo llegó un lenguaje creado para agregar pequeñas interacciones a una página a ocupar tantos lugares distintos? Para responder esa pregunta conviene comenzar por el problema que le dio origen y separar tres conceptos que suelen confundirse: **el lenguaje**, **el motor que lo ejecuta** y **el entorno que le permite interactuar con el mundo**.

Al finalizar este apunte deberíamos poder:

- explicar por qué surgió JavaScript y reconocer los principales hitos de su evolución;
 - distinguir JavaScript, ECMAScript, un motor y un entorno de ejecución;
 - comprender por qué decir que JavaScript es solamente “interpretado” resulta incompleto;
 - describir, a nivel conceptual, qué hace la compilación justo a tiempo;
 - explicar qué agrega TypeScript y qué significa transpilar;
 - diferenciar tipado estático de dinámico y coerción fuerte de débil;
 - relacionar esas decisiones con el desarrollo web frontend y backend.
-

1. La Web antes de JavaScript

En los primeros años de la Web, una página se parecía más a un documento que a una aplicación. El navegador pedía un archivo HTML a un servidor, lo recibía y lo mostraba. Si el usuario quería realizar otra acción, normalmente debía solicitar una página nueva.

HTML podía expresar la **estructura** del documento y, con el tiempo, CSS permitió describir mejor su **presentación**. Faltaba una tercera pieza: una forma de expresar **comportamiento** dentro del navegador.

Imaginemos un formulario de registro en aquella Web. Sin un lenguaje ejecutándose en el navegador, incluso una verificación sencilla —por ejemplo, comprobar que un campo no estuviera vacío— podía requerir este recorrido:

1. el usuario completaba el formulario;
2. el navegador enviaba los datos al servidor;

3. el servidor detectaba el error;
4. el servidor generaba una nueva respuesta;
5. el navegador cargaba otra página para mostrar el mensaje.

La necesidad era concreta: poder reaccionar inmediatamente a lo que ocurría en la página. Hacía falta un lenguaje suficientemente accesible para pequeños programas, integrado al navegador y capaz de responder a eventos como un clic, el envío de un formulario o la carga de un documento.

2. El nacimiento y la evolución de JavaScript

JavaScript fue creado en 1995 por Brendan Eich mientras trabajaba en Netscape. Su primera aparición pública estuvo ligada a Netscape Navigator 2. Aunque sus primeros nombres fueron Mocha y luego LiveScript, terminó llamándose JavaScript. El nombre aprovechaba la popularidad de Java en ese momento, pero los dos lenguajes no son versiones uno del otro: tienen modelos, objetivos e historias diferentes.

El éxito de JavaScript produjo rápidamente un nuevo problema. Otros navegadores necesitaban implementar un lenguaje compatible, pero una tecnología controlada por una sola empresa no era una base saludable para toda la Web. Netscape llevó entonces la propuesta a Ecma International. El trabajo de estandarización comenzó en 1996 y la primera edición del estándar **ECMA-262** fue aprobada en junio de 1997.

Aquí aparecen dos nombres que conviene relacionar correctamente:

- **ECMAScript** es el lenguaje definido por el estándar ECMA-262.
- **JavaScript** es el nombre con el que usamos habitualmente una implementación de ese lenguaje y su ecosistema.

En la conversación cotidiana ambos nombres suelen referirse al mismo lenguaje. Sin embargo, "ECMAScript" resulta útil cuando hablamos con precisión de la especificación o de una edición, como ES5 o ES2015.

Una línea de tiempo mínima

Año	Hito	Por qué fue importante
1995	Nace JavaScript en Netscape	Las páginas pueden incorporar comportamiento ejecutado en el navegador.
1997	Primera edición de ECMA-262	El lenguaje obtiene una especificación independiente y compartida.
1999	ECMAScript 3	Consolida buena parte del JavaScript clásico utilizado durante años.
2005	Se populariza el enfoque AJAX	Las páginas comienzan a intercambiar datos con el servidor sin recargarse por completo.
2008	Google publica Chrome y el motor V8	La competencia entre motores impulsa grandes mejoras de rendimiento.
2009	ECMAScript 5 y primera versión de Node.js	El lenguaje madura y V8 comienza a utilizarse ampliamente fuera del navegador.
2012	Microsoft presenta TypeScript	Se agrega verificación estática de tipos sobre JavaScript.
2015	ECMAScript 2015, también llamado ES6	Incorpora módulos, clases, promesas, <code>let</code> , <code>const</code> , funciones flecha y muchas otras mejoras.
Desde 2016	Ediciones anuales de ECMAScript	La evolución pasa a ser gradual y predecible, en lugar de depender de grandes saltos.

Esta historia muestra una transformación. JavaScript nació para programas pequeños dentro de páginas, pero la Web empezó a exigir aplicaciones cada vez más complejas. Los motores se hicieron más rápidos, el estándar incorporó mejores abstracciones y el ecosistema llevó el lenguaje a nuevos entornos.

3. ¿Qué significa "ejecutar JavaScript"?

Cuando escribimos este código:

```
const mensaje = "Hola, Programación IV";  
console.log(mensaje.toUpperCase());
```

nosotros vemos texto. El procesador, en cambio, solo puede ejecutar instrucciones de máquina. Entre una cosa y la otra debe existir un traductor.

Tradicionalmente se presentan dos modelos:

- Un **compilador** traduce el programa antes de ejecutarlo y suele producir otro programa o código de máquina.

- Un **intérprete** analiza las instrucciones y las va ejecutando mediante otro programa.

Esta distinción sirve como primera aproximación, pero los motores modernos combinan estrategias. Por eso, afirmar simplemente que “JavaScript es interpretado” deja afuera una parte importante de lo que sucede.

Más precisamente:

ECMAScript define cómo debe comportarse el lenguaje, pero no obliga a los motores a utilizar una única estrategia de ejecución.

Un motor moderno puede interpretar una representación intermedia y, al mismo tiempo, compilar ciertas partes a código de máquina. V8 —utilizado por Chrome y Node.js— combina un intérprete con compiladores de distintos niveles. Otros motores, como SpiderMonkey de Firefox o JavaScriptCore de Safari, poseen arquitecturas propias con la misma clase de objetivo: comenzar a ejecutar pronto y optimizar lo que realmente lo necesita.

4. Compilación justo a tiempo: JIT

JIT significa *Just-In-Time*, o compilación justo a tiempo. La idea consiste en compilar durante la ejecución, usando información que solamente puede conocerse mientras el programa está funcionando.

De manera conceptual, el proceso puede verse así:

```
código fuente
  ↓
análisis sintáctico
  ↓
representación interna / bytecode
  ↓
ejecución y recolección de información
  ↓
optimización de las partes más utilizadas
  ↓
código de máquina especializado
```

Veámoslo desde primeros principios. En JavaScript una misma función puede recibir valores de distintos tipos:

```
function sumar(a, b) {
  return a + b;
}

sumar(10, 20);           // 30
sumar("Hola, ", "Ana"); // "Hola, Ana"
```


Antes de ejecutar el programa, el motor no siempre puede asumir qué valores recibirá `sumar`. Durante la ejecución, en cambio, puede observar su uso. Si una función se invoca muchas veces con números, el motor puede producir una versión optimizada para ese patrón.

El razonamiento simplificado es el siguiente:

1. **Leer y analizar.** El motor verifica la sintaxis y construye una representación interna del programa.
2. **Comenzar rápidamente.** Genera una forma ejecutable inicial, como bytecode, sin gastar demasiado tiempo en optimizaciones.
3. **Observar.** Mientras el programa corre, registra qué funciones se usan mucho y qué clases de valores reciben.
4. **Optimizar.** Compila las zonas "calientes" a código de máquina más especializado.
5. **Desoptimizar si es necesario.** Si dejan de cumplirse las suposiciones, vuelve a una versión más general sin cambiar el resultado que exige el lenguaje.

Supongamos que esta función procesa miles de precios:

```
function calcularTotal(precios) {  
  let total = 0;  
  
  for (const precio of precios) {  
    total += precio;  
  }  
  
  return total;  
}
```

Si siempre recibe arreglos de números, un motor podría optimizar ese camino. Si más tarde aparece un texto, quizá tenga que abandonar esa optimización. Este comportamiento es una posibilidad interna del motor, no un contrato que el código deba intentar controlar.

La conclusión importante no es memorizar los nombres de cada componente de V8. Es comprender que los motores actuales buscan equilibrar dos necesidades opuestas:

- **iniciar rápido**, sin compilar exhaustivamente todo el programa;
- **ejecutar rápido**, invirtiendo más trabajo en las partes que se repiten.

Por eso resulta más correcto decir:

JavaScript es un lenguaje cuya implementación moderna suele combinar interpretación y compilación JIT.

5. TypeScript y la transpilación

A medida que las aplicaciones JavaScript crecieron, también crecieron las relaciones que una persona debía mantener en su cabeza: qué datos recibe una función, qué propiedades tiene un objeto, qué puede devolver una operación y qué partes dependen de ellas.

Consideremos esta función JavaScript:

```
function aplicarDescuento(precio, porcentaje) {  
  return precio - precio * porcentaje / 100;  
}
```

El programa no expresa si `precio` debe ser un número, si `porcentaje` puede faltar o qué devuelve la función. Quizá esos contratos estén en la documentación; quizá solo sean un acuerdo implícito del equipo.

TypeScript permite escribirlos en el código:

```
function aplicarDescuento(  
  precio: number,  
  porcentaje: number  
): number {  
  return precio - precio * porcentaje / 100;  
}  
  
aplicarDescuento(10_000, 15); // correcto  
aplicarDescuento("10000", 15); // error antes de ejecutar
```

TypeScript fue presentado por Microsoft en 2012 para actuar como un **verificador estático de tipos para JavaScript**. Incluye la sintaxis de JavaScript y agrega, entre otras herramientas, anotaciones de tipos, interfaces y tipos genéricos.

Sin embargo, el navegador y Node.js ejecutan JavaScript. Por eso el código TypeScript debe convertirse en JavaScript antes de llegar al entorno de ejecución:

```
TypeScript (.ts)  
├── verificación de tipos  
└── emisión / transpilación  
    ↓  
    JavaScript (.js)  
    ↓  
    motor de JavaScript
```

Por ejemplo, este código TypeScript:

```
const duplicar = (valor: number): number => valor * 2;
```

puede producir un JavaScript equivalente a este:

```
const duplicar = (valor) => valor * 2;
```

Las anotaciones `: number` desaparecen. Ayudaron durante el desarrollo, pero no existen durante la ejecución.

¿Compilar o transpilar?

Compilar es traducir un programa a otra representación. **Transpilar** suele utilizarse para un caso más específico: traducir de un lenguaje fuente a otro lenguaje fuente de un nivel semejante. Como TypeScript produce JavaScript, es habitual decir que se transpila.

Las dos palabras no son contradictorias. `tsc` se llama *TypeScript Compiler*, realiza verificación estática y puede emitir JavaScript. En proyectos modernos también es común separar responsabilidades: TypeScript verifica los tipos sin emitir archivos y otra herramienta transforma el código.

La transpilación también permite adaptar JavaScript moderno a entornos más antiguos. Por ejemplo, una herramienta puede transformar cierta sintaxis reciente en otra equivalente que comprenda un navegador anterior. Esto cambia la forma del código, no necesariamente incorpora las APIs que falten; para estas últimas puede ser necesario un *polyfill*.

Una limitación fundamental

Como los tipos de TypeScript desaparecen, no validan automáticamente los datos que llegan desde fuera del programa:

```
type Usuario = {  
  id: number;  
  nombre: string;  
};  
  
const respuesta = await fetch("/api/usuario/1");  
const usuario: Usuario = await respuesta.json();
```

La anotación comunica lo que esperamos, pero no obliga al servidor a enviar esa estructura. Los datos provenientes de una API, un formulario, un archivo o una base de datos cruzan una frontera y deben validarse durante la ejecución.

TypeScript puede detectar inconsistencias en nuestro programa antes de ejecutarlo; no puede convertir una entrada externa no confiable en una entrada correcta.

6. Lenguaje, motor y entorno de ejecución

Para entender cómo JavaScript puede utilizarse tanto en frontend como en backend, debemos separar tres capas.

6.1 El lenguaje

ECMAScript define elementos como:

- valores: números, textos, booleanos, objetos, funciones;
- sintaxis: `if`, `for`, `class`, `import`, funciones flecha;
- reglas: alcance de variables, comparaciones, conversiones;
- objetos estándar: `Array`, `Object`, `Promise`, `Map`, `Math`.

Este código pertenece al lenguaje y puede ejecutarse en distintos entornos:

```
const notas = [7, 9, 6];
const promedio = notas.reduce((suma, nota) => suma + nota, 0) / notas.length;

console.log(promedio);
```

6.2 El motor

El motor implementa la especificación y ejecuta el código. También administra memoria, recolecta objetos que ya no se utilizan y aplica estrategias como JIT.

Algunos motores conocidos son:

- **V8**, usado por Chrome y Node.js;
- **SpiderMonkey**, usado por Firefox;
- **JavaScriptCore**, usado por Safari.

6.3 El entorno o anfitrión

El lenguaje por sí solo no sabe qué es una página web, un archivo del disco o una solicitud HTTP. El entorno incorpora esas capacidades mediante APIs.

En un navegador podemos escribir:

```
const boton = document.querySelector("#guardar");

boton.addEventListener("click", () => {
  document.body.classList.add("guardado");
});
```

`document`, el DOM y los eventos son APIs del navegador; no forman parte del núcleo de ECMAScript.

En Node.js podemos escribir:

```
import { readFile } from "node:fs/promises";

const contenido = await readFile("alumnos.json", "utf8");
console.log(contenido);
```

El acceso al sistema de archivos es provisto por Node.js. Un navegador común no expone `node:fs` porque su entorno y sus reglas de seguridad son diferentes.

Podemos resumir la relación así:

```
Aplicación
  ↓ usa
APIs del entorno (DOM, archivos, red, procesos...)
  ↓ se apoya en
Motor de JavaScript (V8, SpiderMonkey, JavaScriptCore...)
  ↓ implementa
ECMAScript
```

Esta separación explica por qué un código puede ser JavaScript válido y, aun así, no funcionar en todos los lugares: quizá dependa de una API que el entorno actual no ofrece.

7. JavaScript en frontend y backend

Frontend: código cerca del usuario

En el navegador, JavaScript suele encargarse de:

- responder a eventos del usuario;
- leer y modificar el DOM;
- administrar el estado de la interfaz;
- validar datos para ofrecer una respuesta inmediata;
- pedir o enviar información a un servidor;
- actualizar partes de la pantalla.

```
const formulario = document.querySelector("#inscripcion");

formulario.addEventListener("submit", async (evento) => {
  evento.preventDefault();

  const datos = new FormData(formulario);

  await fetch("/api/inscripciones", {
    method: "POST",
    body: datos,
  });
});
```

El frontend no debe considerarse una frontera de seguridad. Un usuario puede modificar el código que corre en su navegador o construir manualmente una solicitud. La validación del frontend mejora la experiencia; el backend debe volver a validar todo lo importante.

Backend: código cerca de los datos y las reglas del sistema

Con la aparición de Node.js, JavaScript obtuvo un entorno ampliamente adoptado fuera del navegador. Node.js utiliza V8 e incorpora APIs para red, archivos, procesos y otras tareas del sistema.

En el backend, JavaScript puede:

- recibir solicitudes HTTP;
- autenticar y autorizar usuarios;
- aplicar reglas de negocio;
- validar entradas no confiables;
- consultar o modificar una base de datos;
- integrarse con otros servicios;
- construir y devolver una respuesta.

```
import { createServer } from "node:http";

const servidor = createServer((solicitud, respuesta) => {
  respuesta.writeHead(200, { "content-type": "application/json" });
  respuesta.end(JSON.stringify({ mensaje: "Hola desde el backend" }));
});

servidor.listen(3000);
```

Usar el mismo lenguaje en ambos lados facilita compartir conocimientos y, en algunos casos, reglas y tipos. No significa que frontend y backend sean lo mismo: se ejecutan en máquinas diferentes, poseen capacidades diferentes y protegen responsabilidades diferentes.

8. Tipado estático y tipado dinámico

Un **tipo** representa una categoría de valores y las operaciones que tienen sentido sobre ellos. Sumar números, convertir un texto a mayúsculas o recorrer una colección son operaciones válidas para ciertos tipos y no para otros.

Al comparar sistemas de tipos conviene separar dos preguntas:

1. **¿Cuándo se comprueba la compatibilidad de tipos?** Estático frente a dinámico.
2. **¿Cuánto convierte automáticamente el lenguaje entre tipos?** Más fuerte frente a más débil o coercitivo.

Estas dimensiones están relacionadas, pero no son equivalentes.

Tipado estático

En un sistema estático, una herramienta analiza los tipos antes de ejecutar el programa. Las variables y expresiones tienen tipos que el analizador puede verificar, ya sea porque los escribimos o porque los infiere.

```
let cantidad: number = 3;
cantidad = "tres"; // error de tipos
```

Ventajas principales:

- detecta muchas inconsistencias antes de llegar al usuario;
- mejora el autocompletado y la navegación del editor;
- vuelve explícitos los contratos entre partes del sistema;
- brinda más seguridad al cambiar nombres, mover código o refactorizar;
- reduce la cantidad de estados que una persona debe considerar.

Costos y límites:

- agrega una etapa de análisis y, a veces, anotaciones;
- requiere aprender el sistema de tipos y resolver sus errores;
- puede introducir complejidad si se modela más de lo necesario;
- no demuestra que el programa sea correcto ni reemplaza las pruebas;
- no valida por sí mismo datos externos durante la ejecución.

Tipado dinámico

En un sistema dinámico, los valores poseen tipos durante la ejecución y una variable puede referirse a valores de tipos diferentes a lo largo del tiempo.

```
let resultado = 42;
resultado = "cuarenta y dos";
resultado = { valor: 42 };
```

JavaScript permite las tres asignaciones. Esto no significa que “no tenga tipos”: `42`, `"cuarenta y dos"` y `{ valor: 42 }` pertenecen a categorías diferentes. Significa que la variable `resultado` no queda restringida estáticamente a una de ellas.

Podemos consultar una parte de esa información durante la ejecución:

```
console.log(typeof 42);           // "number"
console.log(typeof "hola");       // "string"
console.log(typeof true);         // "boolean"
console.log(typeof { nombre: "Ana" }); // "object"
```

Ventajas principales:

- permite comenzar con poco código ceremonial;
- resulta flexible para prototipos, scripts y datos cambiantes;
- algunas abstracciones se expresan sin describir previamente todos sus tipos;
- el ciclo inicial de experimentación puede ser muy directo.

Costos y límites:

- ciertas incompatibilidades aparecen solamente al ejecutar el camino afectado;
- una función comunica menos sobre lo que espera si no tiene documentación;
- los cambios en sistemas grandes pueden tener consecuencias más difíciles de rastrear;
- el editor y las herramientas deben inferir más información incompleta.

No existe un ganador universal. El valor de cada estrategia depende del tamaño del programa, su vida útil, el equipo y el costo de un error. TypeScript surgió, precisamente, para conservar el modelo de ejecución de JavaScript y sumar comprobaciones estáticas cuando el sistema crece.

9. Tipado fuerte, tipado débil y coerción

“Fuertemente tipado” y “débilmente tipado” no poseen una definición universal tan precisa como estático y dinámico. Distintos autores clasifican un mismo lenguaje de maneras diferentes. En una introducción, la idea más útil es observar cuánto convierte el lenguaje de forma automática valores de un tipo a otro.

JavaScript realiza numerosas **coerciones implícitas**, por lo que suele describirse como débilmente tipado o, de manera más precisa, como un lenguaje dinámico y coercitivo.

```
console.log("5" + 1); // "51"
console.log("5" - 1); // 4
console.log(false == 0); // true
console.log(false === 0); // false
```

¿Por qué los dos primeros resultados son diferentes?

- El operador `+` también sirve para concatenar textos. Ante un texto y un número, convierte el número a texto.
- El operador `-` solo representa resta numérica. Por eso intenta convertir `"5"` a número.
- `==` permite conversiones antes de comparar.
- `===` compara sin esa coerción y, como los tipos son distintos, devuelve `false`.

La coerción puede ser cómoda, pero también puede esconder un error. Pensemos en un dato obtenido de un formulario:

```
const precio = 1200;
const cantidad = "2"; // los campos de texto entregan texto

console.log(precio * cantidad); // 2400
console.log(precio + cantidad); // "12002"
```

El primer resultado puede hacernos creer que los datos eran correctos. El segundo revela que `cantidad` nunca fue convertida de manera intencional.

Una solución más clara explicita la conversión y valida el resultado:

```
const precio = 1200;
const entrada = "2";
const cantidad = Number(entrada);

if (!Number.isFinite(cantidad) || cantidad <= 0) {
  throw new Error("La cantidad debe ser un número positivo");
}

const total = precio * cantidad;
```

La enseñanza no debería reducirse a "JavaScript convierte tipos y eso está mal". La coerción forma parte de las reglas del lenguaje. El objetivo es reconocer cuándo ocurre y evitar que una conversión implícita tome una decisión de negocio por nosotros.

Como regla inicial:

- preferir `===` y `!==` para comparar;
- convertir las entradas de manera explícita;
- validar los datos en las fronteras del sistema;

- no confiar en que un resultado aparentemente correcto prueba que el tipo de origen era correcto.

Las dos dimensiones juntas

Lenguaje	Momento de comprobación	Tendencia frente a conversiones
C# / Java	Estático	Generalmente fuerte
Python	Dinámico	Generalmente fuerte
JavaScript	Dinámico	Coercitivo; suele clasificarse como débil
TypeScript	Estático durante el desarrollo	Conserva el comportamiento dinámico de JavaScript al ejecutarse

La tabla es orientativa, en especial en la segunda columna conceptual. Lo esencial es notar que **dinámico no significa necesariamente débil**: Python es el contraejemplo habitual. Tampoco TypeScript crea un nuevo entorno fuertemente tipado durante la ejecución; después de borrar los tipos, se ejecutan las reglas normales de JavaScript.

10. De JavaScript a TypeScript: una evolución gradual

TypeScript no reemplaza JavaScript. Se construye sobre él. Un programa JavaScript válido puede ser el punto de partida de una migración en la que los contratos se agregan gradualmente.

Supongamos que una aplicación maneja productos:

```
function calcularTotal(productos) {  
  return productos.reduce(  
    (total, producto) => total + producto.precio,  
    0  
  );  
}
```

La función depende de varias suposiciones:

- `productos` debe ser recorrible;
- cada elemento debe tener una propiedad `precio`;
- cada `precio` debería ser numérico;
- la función devolverá un número si todas esas condiciones se cumplen.

TypeScript convierte esas suposiciones en un contrato verificable:

```

type Producto = {
  id: number;
  nombre: string;
  precio: number;
};

function calcularTotal(productos: Producto[]): number {
  return productos.reduce(
    (total, producto) => total + producto.precio,
    0
  );
}

```

Ahora el editor puede anticipar propiedades, detectar un nombre mal escrito y avisar si intentamos usar un precio textual. Además, TypeScript infiere muchos tipos: no es necesario anotar cada variable.

```

const productos: Producto[] = [
  { id: 1, nombre: "Teclado", precio: 35_000 },
  { id: 2, nombre: "Mouse", precio: 18_000 },
];

const total = calcularTotal(productos);
// TypeScript infiere que total es number.

```

La evolución puede ser gradual:

1. conservar archivos JavaScript existentes;
2. activar comprobaciones sobre ellos y documentar tipos con JSDoc;
3. convertir primero los módulos más importantes a `.ts`;
4. endurecer las reglas a medida que el equipo resuelve los casos anteriores;
5. validar en tiempo de ejecución todas las entradas externas.

Este enfoque explica el éxito de TypeScript en proyectos web grandes: permite incorporar seguridad estática sin abandonar JavaScript, su ecosistema ni sus entornos de ejecución.

11. Un caso full stack: los tipos atraviesan fronteras

Imaginemos una pantalla que envía un nuevo producto al servidor.

En el navegador, el valor de un campo llega como texto:

```

const precioIngresado = document.querySelector("#precio").value;
// Por ejemplo: "35000"

```

La interfaz debe convertirlo y ofrecer un mensaje temprano si es inválido. Sin embargo, el servidor no puede confiar en que la solicitud provino de esa interfaz ni en que su validación fue ejecutada.

El recorrido correcto es:

```
campo de formulario
  ↓ texto
conversión y validación en frontend
  ↓ solicitud HTTP
validación nuevamente en backend
  ↓ dato confiable para esa operación
regla de negocio y persistencia
```

TypeScript puede compartir una descripción como `Producto` entre frontend y backend, y así detectar desacuerdos durante el desarrollo. Aun así, la solicitud HTTP contiene datos de ejecución. El backend debe comprobar que `precio` existe, que es un número finito y que cumple las reglas del negocio.

De aquí surge una regla que acompañará toda la materia:

Los tipos estáticos describen contratos dentro del programa; la validación protege las fronteras por las que ingresan datos reales.

12. Ideas que conviene conservar

1. JavaScript surgió para incorporar comportamiento a la Web, pero su uso se expandió mucho más allá de las primeras páginas interactivas.
2. ECMAScript es la especificación estandarizada del lenguaje; JavaScript es el nombre habitual de sus implementaciones y su ecosistema.
3. El estándar define resultados, no una única técnica interna de ejecución.
4. Los motores modernos combinan interpretación, bytecode y compilación JIT; por eso “solo interpretado” es una simplificación insuficiente.
5. El motor ejecuta el lenguaje; el entorno aporta capacidades como DOM, archivos, procesos o red.
6. El navegador y Node.js pueden ejecutar JavaScript, pero ofrecen APIs y responsabilidades diferentes.
7. JavaScript posee tipado dinámico: los valores tienen tipos y las variables pueden referirse a valores de tipos distintos.
8. JavaScript realiza coerciones implícitas; conocerlas y convertir de forma intencional evita resultados sorpresivos.
9. TypeScript verifica tipos antes de ejecutar y luego produce JavaScript. Sus tipos no permanecen en tiempo de ejecución.

10. La verificación estática, la validación en ejecución y las pruebas resuelven problemas diferentes; se complementan.
-

13. Preguntas de repaso

1. ¿Qué necesidad de la Web original ayudó a resolver JavaScript?
2. ¿Qué diferencia existe entre JavaScript y ECMAScript?
3. ¿Por qué no es completamente preciso definir JavaScript como un lenguaje interpretado?
4. ¿Qué información puede aprovechar un compilador JIT que un compilador previo a la ejecución todavía no posee?
5. ¿Qué diferencia hay entre un motor y un entorno de ejecución?
6. ¿Por qué `document.querySelector` funciona en el navegador y no forma parte del núcleo de ECMAScript?
7. ¿Qué significa que una variable de JavaScript sea dinámicamente tipada?
8. ¿Por qué tipado dinámico y tipado débil no son sinónimos?
9. ¿Qué resultados producen `"5" + 1` y `"5" - 1`? ¿Qué regla explica la diferencia?
10. ¿Qué agrega TypeScript y qué ocurre con sus tipos antes de ejecutar el programa?
11. ¿Por qué una interfaz TypeScript no alcanza para validar el cuerpo de una solicitud HTTP?
12. ¿Qué responsabilidades pueden compartirse entre frontend y backend y cuáles deben permanecer separadas?

Ejercicio breve

Analizar el siguiente código sin ejecutarlo:

```
const cantidad = "3";
const precio = 500;

const subtotal = cantidad * precio;
const descripcion = "Total: $" + cantidad * precio;
const resultado = cantidad + precio;

console.log(subtotal);
console.log(descripcion);
console.log(resultado);
```

Para cada variable:

1. anticipar su valor y su tipo;
2. identificar las coerciones realizadas;
3. reescribir el programa para convertir y validar `cantidad` explícitamente;
4. proponer una versión TypeScript de la operación.

Fuentes y lecturas recomendadas

- [ECMA-262, primera edición \(1997\)](#)
- [Archivo de ediciones de ECMAScript — Ecma International](#)
- [JavaScript: historia y definición — MDN](#)
- [Diez años de V8 — V8](#)
- [El intérprete Ignition y la compilación JIT — V8](#)
- [Introducción a Node.js — documentación oficial](#)
- [TypeScript para quien comienza a programar — documentación oficial](#)
- [Manual de TypeScript — documentación oficial](#)

Programación IV — Variables y tipos básicos de JavaScript

Nombres, valores, operadores y conversiones

Un programa transforma información. Para poder hacerlo necesita representar valores —un precio, un nombre, una cantidad, una respuesta afirmativa— y necesita conservar algunos de ellos para utilizarlos más adelante.

En JavaScript, cada valor pertenece a un tipo. El número `25`, el texto `"25"` y el booleano `true` son tres valores distintos, con operaciones y significados diferentes. Una variable permite asociar uno de esos valores con un nombre dentro del programa.

En esta clase construiremos esas ideas desde la base. Primero veremos qué es una variable, cómo se declara y durante qué parte del programa existe. Después estudiaremos los tipos primitivos, sus literales, operadores y funciones habituales. Finalmente explicaremos las conversiones explícitas y automáticas que conectan unos tipos con otros.

Al finalizar este apunte deberíamos poder:

- distinguir declaración, inicialización y asignación;
- elegir correctamente entre `const`, `let` y `var`;
- explicar el alcance y la vida de una variable;
- escribir identificadores válidos y nombres expresivos;
- reconocer los siete tipos primitivos de JavaScript;
- representar booleanos, números y cadenas mediante literales;
- utilizar sus operadores y funciones estándar más importantes;
- interpretar expresiones considerando precedencia y asociatividad;
- explicar los valores *truthy* y *falsy* y el cortocircuito lógico;
- realizar conversiones explícitas y anticipar las coerciones automáticas.

1. De un valor a una variable

El valor `1200` puede representar el precio de un producto. Podríamos escribirlo directamente cada vez que lo necesitamos:

```
console.log(1200 * 2);  
console.log(1200 * 3);
```

Pero el programa no expresa qué significa ese número. Si le damos un nombre, la intención se vuelve visible:

```
const precioUnitario = 1200;

console.log(precioUnitario * 2);
console.log(precioUnitario * 3);
```

Podemos imaginar una variable como una etiqueta que permite encontrar un valor. En términos más precisos, una declaración crea una **vinculación** entre un identificador y un valor dentro de cierto entorno del programa.

Esta precisión será importante más adelante. Una variable de JavaScript no queda asociada para siempre a un tipo. Si fue declarada con `let`, puede referirse primero a un número y después a un texto:

```
let respuesta = 42;
respuesta = "cuarenta y dos";
```

Los valores tienen tipos. La variable es el nombre mediante el cual accedemos al valor actual.

Declarar, inicializar y asignar

Son tres acciones relacionadas, pero no idénticas:

```
let cantidad;      // declaración
cantidad = 3;      // asignación

let precio = 1500; // declaración e inicialización
precio = 1800;     // nueva asignación
```

- **Declarar** introduce el nombre en un alcance.
- **Inicializar** le entrega su primer valor.
- **Asignar** cambia el valor al que se encuentra vinculada una variable ya existente.

Una declaración con `let` que no posee inicializador recibe el valor `undefined` cuando su declaración es ejecutada:

```
let observacion;

console.log(observacion); // undefined
```

Una declaración con `const`, en cambio, debe inicializarse en el mismo momento:


```
const comision = "4K1";  
// const turno; // SyntaxError: falta el inicializador
```

2. Tres formas de declarar: `const`, `let` y `var`

JavaScript ofrece tres palabras para declarar variables. No son tres maneras intercambiables de escribir lo mismo: difieren en reasignación, alcance, redeclaración e inicialización.

Característica	<code>const</code>	<code>let</code>	<code>var</code>
Permite reasignar	No	Sí	Sí
Exige valor inicial	Sí	No	No
Alcance de bloque	Sí	Sí	No
Permite redeclarar en el mismo alcance	No	No	Sí
Acceso antes de la declaración	Error	Error	<code>undefined</code>
Uso recomendado en código moderno	Por defecto	Cuando habrá reasignación	Evitar salvo código legado

2.1 `const` : una vinculación que no cambia

`const` impide que el nombre sea asignado a otro valor:

```
const nombreMateria = "Programación IV";  
  
// nombreMateria = "Laboratorio IV";  
// TypeError: no se puede reasignar una constante
```

Esto permite comunicar una decisión: después de inicializar `nombreMateria`, el programa no debería reemplazar su valor.

Por esa razón, la regla práctica es:

Usar `const` por defecto. Cambiar a `let` solamente cuando la reasignación forme parte del problema.

`const` no vuelve inmutable al valor. Protege la vinculación, no el interior de un objeto:

```
const alumno = { nombre: "Ana", regular: true };

alumno.regular = false; // permitido: cambia el objeto

// alumno = { nombre: "Luis", regular: true };
// no permitido: intenta reemplazar la vinculación
```

Esta diferencia será desarrollada al estudiar objetos. Por ahora alcanza con recordar que, para valores primitivos —que son inmutables—, una constante tampoco puede “modificar el valor por dentro”.

2.2 `let` : una vinculación reasignable

`let` se utiliza cuando el valor asociado al nombre debe cambiar:

```
let intentosRestantes = 3;

intentosRestantes = intentosRestantes - 1;
intentosRestantes -= 1;
```

Los casos típicos son acumuladores, contadores o estados que evolucionan:

```
const notas = [8, 6, 9];
let suma = 0;

for (const nota of notas) {
  suma += nota;
}

const promedio = suma / notas.length;
```

`notas` y `promedio` no necesitan ser reasignados, por lo que se declaran con `const`. `suma` sí cambia durante el recorrido y se declara con `let`.

2.3 `var` : la declaración histórica

Antes de ECMAScript 2015, `var` era la forma habitual de declarar variables. Sigue siendo parte del lenguaje por compatibilidad, pero sus reglas resultan menos locales y favorecen errores que `let` y `const` evitan.

```
if (true) {
  var mensaje = "visible fuera del bloque";
}

console.log(mensaje); // funciona
```

Un bloque delimitado por llaves no crea un alcance nuevo para `var`. Si se encuentra dentro de una función, su alcance es toda la función; si se declara en el nivel superior de un script tradicional, posee alcance global.

Además, `var` permite redeclarar el mismo nombre:

```
var estado = "pendiente";  
var estado = "enviado"; // permitido
```

Con `let` o `const`, la redeclaración en el mismo alcance es un error y ayuda a detectar una posible confusión:

```
let estado = "pendiente";  
// let estado = "enviado"; // SyntaxError
```

Regla de elección

Para código moderno:

1. elegir `const` si el nombre no será reasignado;
2. elegir `let` si la reasignación es necesaria;
3. reconocer `var` para poder leer código existente, pero evitarlo en código nuevo.

La elección no depende de si el valor "es importante". Depende de si la **vinculación** necesitará cambiar.

3. Alcance: ¿desde dónde puede utilizarse un nombre?

El **alcance**, o *scope*, es la región del código desde la cual una vinculación resulta accesible.

Alcance global

Una declaración situada fuera de funciones y bloques pertenece al nivel superior del script o módulo:

```
const nombreAplicacion = "Gestión Académica";  
  
function mostrarNombre() {  
  console.log(nombreAplicacion);  
}
```

El uso excesivo de variables globales dificulta saber qué parte del programa puede modificarlas. Los módulos permiten limitar mejor ese alcance.

Alcance de función

Los parámetros y las variables declaradas dentro de una función no son accesibles desde afuera:

```
function calcularDoble(numero) {  
  const resultado = numero * 2;  
  return resultado;  
}  
  
console.log(calcularDoble(5)); // 10  
// console.log(resultado);    // ReferenceError
```

`numero` y `resultado` pertenecen a la ejecución de `calcularDoble`.

Alcance de bloque

Un bloque es una región delimitada por llaves. `let` y `const` respetan ese límite:

```
const usuarioAutenticado = true;  
  
if (usuarioAutenticado) {  
  const mensaje = "Bienvenido";  
  let segundos = 5;  
  
  console.log(mensaje, segundos); // accesibles  
}  
  
// console.log(mensaje); // ReferenceError  
// console.log(segundos); // ReferenceError
```

Los bloques de `if`, `for`, `while` y los bloques independientes pueden crear alcances léxicos.

Alcances anidados y sombreado

Un alcance interno puede utilizar nombres del externo. También puede declarar un nuevo nombre igual, que temporalmente **sombrea** al anterior:

```
const mensaje = "mensaje exterior";  
  
{  
  const mensaje = "mensaje interior";  
  console.log(mensaje); // "mensaje interior"  
}  
  
console.log(mensaje); // "mensaje exterior"
```

El sombreado es válido, pero puede reducir la claridad. Conviene utilizarlo solamente cuando los dos significados resultan evidentes.

4. Vida de una variable: ¿durante cuánto tiempo existe?

El alcance responde **dónde** puede usarse un nombre. Su vida responde **durante cuánto tiempo** se conserva la vinculación y, eventualmente, el valor al que hace referencia.

Una vinculación local suele crearse al entrar en su entorno y deja de ser necesaria cuando ese entorno ya no puede alcanzarse. El motor puede recuperar automáticamente la memoria mediante el recolector de basura.

Sin embargo, una función interna puede conservar acceso a variables del entorno donde fue creada. Ese mecanismo se denomina **clausura** o *closure*:

```
function crearContador() {  
  let cuenta = 0;  
  
  return function incrementar() {  
    cuenta += 1;  
    return cuenta;  
  };  
}  
  
const siguiente = crearContador();  
  
console.log(siguiente()); // 1  
console.log(siguiente()); // 2
```

Aunque `crearContador` terminó, la función `incrementar` todavía puede alcanzar `cuenta`. Por eso esa vinculación continúa viva.

No necesitamos dominar todavía las clausuras. El ejemplo muestra una idea fundamental: la vida de una variable no depende solamente de la línea donde fue declarada, sino de si algún código todavía puede alcanzarla.

5. Declaraciones anticipadas y zona muerta temporal

Antes de ejecutar las instrucciones de un alcance, el motor registra sus declaraciones. A este comportamiento se lo suele llamar **hoisting**. Sin embargo, `var`, `let` y `const` no quedan utilizables de la misma manera.

Con `var`, el nombre existe desde el inicio del alcance y se inicializa con `undefined`:

```
console.log(puntaje); // undefined  
var puntaje = 10;
```

Una forma conceptual de imaginarlo es:

```
var puntaje;  
console.log(puntaje);  
puntaje = 10;
```

Con `let` y `const`, la vinculación también pertenece al alcance desde su comienzo, pero no puede utilizarse antes de la declaración:

```
// console.log(puntaje); // ReferenceError  
let puntaje = 10;
```

La región entre el comienzo del alcance y la inicialización se denomina **zona muerta temporal**, o *Temporal Dead Zone*.

La enseñanza práctica es sencilla:

Declarar cada nombre cerca de su primer uso y no depender de accesos anteriores a la declaración.

6. Identificadores y convenciones de nombres

El nombre de una variable es un **identificador**. JavaScript establece reglas sintácticas para decidir qué identificadores son válidos; el equipo establece convenciones para decidir cuáles son claros.

Reglas del lenguaje

Un identificador habitual puede contener letras, dígitos, `_` y `$`, pero no puede comenzar con un dígito:

```
const cantidad2 = 10;    // válido  
const _resultado = 20;   // válido  
const $elemento = null;  // válido  
  
// const 2cantidad = 10; // inválido
```

JavaScript distingue mayúsculas y minúsculas:

```
const nombre = "Ana";
const Nombre = "Luis";

console.log(nombre === Nombre); // false
```

Los identificadores pueden usar muchos caracteres Unicode:

```
const año = 2026;
```

Aunque es válido, algunos equipos prefieren limitar los identificadores a caracteres sin tildes para facilitar búsquedas, herramientas e intercambio entre distribuciones de teclado. Lo importante es acordar una convención y aplicarla de manera consistente.

Las palabras reservadas del lenguaje no pueden utilizarse como nombres de variables:

```
// const const = 3;
// const import = "archivo";
// const if = true;
```

Convenciones habituales

En JavaScript se utiliza normalmente **camelCase** para variables y funciones:

```
const cantidadDeAlumnos = 35;
const fechaDeInscripcion = "2026-08-10";

function calcularPromedio() {
  // ...
}
```

El primer término comienza en minúscula y cada término siguiente comienza en mayúscula.

Otras convenciones frecuentes son:

- **PascalCase** para clases: `CuentaBancaria`, `ClientePremium`;
- **MAYUSCULAS_CON_GUIONES_BAJOS** para constantes universales o de configuración: `MAX_INTENTOS`, `IVA_GENERAL`;
- sustantivos para valores: `precio`, `alumno`, `fechaCreacion`;
- verbos para funciones: `calcularTotal`, `buscarCliente`, `enviarMensaje`;
- preguntas para booleanos: `esValido`, `tienePermiso`, `puedeEditar`;
- plurales para colecciones: `alumnos`, `productos`, `mensajes`.

No toda declaración con `const` necesita escribirse en mayúsculas. Una variable local como `const usuario = ...` no representa necesariamente una constante universal. Las mayúsculas se reservan para valores cuyo significado es fijo en todo el dominio o módulo.

Un nombre debe expresar intención

Compárese:

```
const p = 1200;  
const c = 3;  
const t = p * c;
```

con:

```
const precioUnitario = 1200;  
const cantidad = 3;  
const total = precioUnitario * cantidad;
```

La segunda versión no necesita un comentario para explicar la operación.

También conviene incluir la unidad cuando puede haber ambigüedad:

```
const duracionMs = 1500;  
const distanciaKm = 12.5;  
const precioCentavos = 199_900;
```

Un buen nombre no describe la sintaxis del valor; describe qué representa dentro del problema.

7. Tipos y valores primitivos

Un tipo agrupa valores que comparten una representación conceptual y un conjunto de operaciones válidas. JavaScript posee siete tipos primitivos y un gran grupo adicional: los objetos.

Tipo	Utilidad principal	Ejemplo	Resultado de <code>typeof</code>
<code>boolean</code>	representar verdad o falsedad	<code>true</code>	<code>"boolean"</code>
<code>number</code>	representar cantidades enteras o decimales	<code>42</code> , <code>3.14</code>	<code>"number"</code>
<code>bigint</code>	representar enteros de precisión arbitraria	<code>9007199254740993n</code>	<code>"bigint"</code>
<code>string</code>	representar texto	<code>"Hola"</code>	<code>"string"</code>
<code>undefined</code>	señalar que no se asignó o no existe un valor	<code>undefined</code>	<code>"undefined"</code>
<code>null</code>	representar intencionalmente ausencia de valor	<code>null</code>	<code>"object"</code> por una particularidad histórica
<code>symbol</code>	crear identificadores únicos	<code>Symbol("id")</code>	<code>"symbol"</code>

Todo lo que no es un valor primitivo es un **objeto**. Los arreglos, las funciones, las fechas y los objetos literales pertenecen a esta categoría general.

Propiedades de los primitivos

Los valores primitivos son inmutables. Una operación no modifica el valor original: produce otro valor.

```
const nombre = "ana";
const nombreEnMayusculas = nombre.toUpperCase();

console.log(nombre);           // "ana"
console.log(nombreEnMayusculas); // "ANA"
```

Aunque una cadena es primitiva, podemos escribir `nombre.toUpperCase()`. JavaScript permite acceder a métodos mediante objetos envoltorio como `String`, `Number` y `Boolean`. Esa ayuda no significa que debamos construir esos envoltorios manualmente:

```
const correcto = false;
const evitar = new Boolean(false);

console.log(Boolean(correcto)); // false
console.log(Boolean(evitar));   // true: todo objeto es truthy
```

Como regla general, usamos `Boolean()`, `Number()` y `String()` como funciones de conversión, sin `new`.

8. Expresiones, operadores y precedencia

Una **expresión** es una porción de código que produce un valor:

```
2 + 3           // produce 5
precio * cantidad // produce un número
edad >= 18      // produce un booleano
nombre.toUpperCase() // produce una cadena
```

Un **operador** combina, transforma o consulta operandos. En `2 + 3`, `+` es el operador y `2` y `3` son sus operandos.

Precedencia

La precedencia determina cómo se agrupan operadores diferentes cuando no hay paréntesis:

```
const resultado = 2 + 3 * 4;
console.log(resultado); // 14
```

La multiplicación posee mayor precedencia que la suma, por lo que la expresión se interpreta así:

```
2 + (3 * 4)
```

Los paréntesis cambian el agrupamiento:

```
const resultado = (2 + 3) * 4;
console.log(resultado); // 20
```

Asociatividad

Cuando los operadores poseen la misma precedencia, la asociatividad indica cómo se agrupan:

```
20 - 5 - 2
// (20 - 5) - 2 = 13: asociatividad de izquierda a derecha

2 ** 3 ** 2
// 2 ** (3 ** 2) = 512: exponenciación de derecha a izquierda
```

La asignación también se agrupa desde la derecha:

```
let a;
let b;

a = b = 5;
// equivale a: a = (b = 5)
```

Orden práctico de precedencia

No hace falta memorizar toda la tabla del lenguaje. Conviene reconocer estos grupos, ordenados aproximadamente de mayor a menor precedencia:

Grupo	Operadores representativos
Agrupamiento y acceso	<code>()</code> , <code>obj.prop</code> , <code>funcion()</code>
Operadores unarios	<code>!</code> , <code>typeof</code> , <code>++</code> , <code>--</code> , <code>~</code>
Exponenciación	<code>**</code>
Multiplicativos	<code>*</code> , <code>/</code> , <code>%</code>
Aditivos	<code>+</code> , <code>-</code>
Desplazamiento de bits	<code><<</code> , <code>>></code> , <code>>>></code>
Relacionales	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>
Igualdad	<code>===</code> , <code>!==</code> , <code>==</code> , <code>!=</code>
Bit a bit	<code>&</code> , luego <code>^</code> , luego <code>^</code>
AND lógico	<code>&&</code>
OR lógico y valor nulo	<code>,</code> <code>??</code>
Condicional	<code>condicion ? valor1 : valor2</code>
Asignación	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , entre otros

Si la lectura de una expresión exige recordar varios niveles, los paréntesis pueden hacer explícita la intención:

```
const puedeIngresar = estaActivo && (esDocente || esAlumno);
```

Los paréntesis no son solamente una herramienta matemática. También son documentación.

9. Booleanos: representar decisiones

El tipo `boolean` posee solamente dos valores:

```
const activo = true;
const bloqueado = false;
```

Sus literales son `true` y `false`, siempre en minúsculas. Se utilizan para representar condiciones, resultados de comparaciones y estados con dos posibilidades.

Un nombre booleano debería poder leerse como una pregunta:

```
const esMayorDeEdad = edad >= 18;
const tieneSaldo = saldo > 0;
const puedeComprar = esMayorDeEdad && tieneSaldo;
```

Operadores de comparación

Las comparaciones producen booleanos:

Operador	Significado
<code><</code>	menor que
<code><=</code>	menor o igual que
<code>></code>	mayor que
<code>>=</code>	mayor o igual que
<code>===</code>	igualdad estricta
<code>!==</code>	desigualdad estricta
<code>==</code>	igualdad con conversión automática
<code>!=</code>	desigualdad con conversión automática

Para código de aplicación se prefieren `===` y `!==`:

```
console.log(5 === 5);    // true
console.log(5 === "5");  // false

console.log(5 == "5");   // true: convierte antes de comparar
```

La igualdad estricta no intenta convertir tipos. Esto vuelve la expresión más local y predecible: si comparamos un número, esperamos realmente un número.

Operadores lógicos

Operador	Nombre	Idea
<code>!a</code>	NOT	niega la condición
<code>a && b</code>	AND	ambas condiciones deben ser verdaderas
<code>a b</code>	OR	al menos una condición debe ser verdadera

Cuando los operandos son booleanos, su tabla de verdad es:

a	b	!a	a && b	a b
false	false	true	false	false
false	true	true	false	true
true	false	false	false	true
true	true	false	true	true

```
const tieneUsuario = true;
const tieneClave = true;

const puedeIniciarSesion = tieneUsuario && tieneClave;
const necesitaAyuda = !puedeIniciarSesion;
```

`!` tiene mayor precedencia que `&&`, y `&&` tiene mayor precedencia que `||`:

```
const permitido = esAdministrador || esDocente && estaActivo;

// Se interpreta como:
const permitidoExplicito = esAdministrador || (esDocente && estaActivo);
```

Aunque los resultados coincidan, los paréntesis pueden comunicar mejor la regla de negocio.

Contextos booleanos

JavaScript permite utilizar cualquier valor donde espera una condición:

```
if (nombre) {
  console.log("El nombre no está vacío");
}
```

Antes de decidir, convierte conceptualmente el valor a booleano. Los valores que se convierten en `false` se denominan **falsy**. Los demás son **truthy**.

Los valores falsy habituales son:

```
false
0
-0
0n
""           // cadena vacía
null
undefined
NaN
```

Todo lo demás es truthy, incluyendo valores que a primera vista pueden confundir:

```
Boolean("false") // true: es una cadena no vacía
Boolean("0")     // true: es una cadena no vacía
Boolean([])      // true: es un objeto
Boolean({})      // true: es un objeto
```

Los navegadores conservan además una excepción histórica y muy particular, `document.all`, que se comporta como falsy. No es un valor que deba utilizarse para diseñar aplicaciones.

Podemos convertir explícitamente mediante `Boolean()` o una doble negación:

```
const tieneNombre = Boolean(nombre);
const tieneCorreo = !!correo;
```

`Boolean()` suele expresar mejor la intención para quien está aprendiendo. `!!` es una forma idiomática basada en negar dos veces.

Los booleanos no necesitan una gran colección de métodos propios. La mayor parte de su utilidad proviene de las comparaciones, los operadores lógicos y la función de conversión `Boolean()`.

Evaluación de cortocircuito

`&&` y `||` evalúan de izquierda a derecha y pueden evitar el operando derecho:

```
const usuario = null;

usuario && console.log(usuario.nombre);
// console.log no se ejecuta porque usuario es falsy
```

Esto se denomina **cortocircuito**. El operando izquierdo ya puede determinar si es necesario continuar.

Además, `&&` y `||` no siempre devuelven `true` o `false`: devuelven uno de sus operandos originales.

```
console.log("Ana" && "Docente"); // "Docente"
console.log("" && "Docente");    // ""

console.log("" || "Sin nombre"); // "Sin nombre"
console.log("Ana" || "Sin nombre"); // "Ana"
```

Las reglas son:

- `a && b` devuelve el primer operando falsy; si no encuentra ninguno, devuelve el último.
- `a || b` devuelve el primer operando truthy; si no encuentra ninguno, devuelve el último.

El operador de coalescencia nula, `??`, se parece a `||`, pero solo usa el valor alternativo cuando el primero es `null` o `undefined`:

```
const cantidad = 0;

console.log(cantidad || 10); // 10: 0 es falsy
console.log(cantidad ?? 10); // 0: no es null ni undefined
```

`??` resulta preferible cuando `0`, `false` o `""` son valores válidos.

No se puede mezclar `??` directamente con `&&` o `||` sin paréntesis:

```
const valor = (entrada || respaldo) ?? valorFinal;
```

10. Números: cantidades y operaciones

JavaScript posee dos tipos numéricos primitivos:

- `number`, para la mayoría de los cálculos;
- `bigint`, para enteros cuya magnitud o precisión supera el rango entero seguro de `number`.

10.1 El tipo `number`

`number` utiliza el formato binario IEEE 754 de doble precisión. El mismo tipo representa enteros, decimales, infinitos y el valor especial `NaN`.

```
const alumnos = 35;
const temperatura = 21.5;
const saldo = -1500;
```

JavaScript no posee tipos primitivos separados llamados `int`, `float` o `double`. Todos esos literales habituales son valores `number`.

Formas de escribir literales numéricos

```
const decimal = 255;
const conDecimales = 3.1416;
const exponencial = 1.5e3;      // 1500
const binario = 0b11111111;    // 255
const octal = 0o377;           // 255
const hexadecimal = 0xff;      // 255
const legible = 1_000_000;     // 1000000
```

Los prefijos indican la base de representación del literal. Una vez creado, el valor no “recuerda” si fue escrito en decimal, binario o hexadecimal: sigue siendo el mismo número.

Los separadores `_` mejoran la lectura y no forman parte del valor.

Operadores aritméticos

Operador	Operación	Ejemplo	Resultado
<code>+</code>	suma	<code>7 + 3</code>	<code>10</code>
<code>-</code>	resta	<code>7 - 3</code>	<code>4</code>
<code>*</code>	multiplicación	<code>7 * 3</code>	<code>21</code>
<code>/</code>	división	<code>7 / 2</code>	<code>3.5</code>
<code>%</code>	resto	<code>7 % 3</code>	<code>1</code>
<code>**</code>	potencia	<code>2 ** 3</code>	<code>8</code>
<code>++</code>	incremento	<code>contador++</code>	suma uno
<code>--</code>	decremento	<code>contador--</code>	resta uno

El resto `%` permite, por ejemplo, reconocer números pares:

```
const numero = 14;
const esPar = numero % 2 === 0;
```

Los operadores de asignación combinada expresan actualización:

```
let total = 100;

total += 20; // total = total + 20
total *= 2;  // total = total * 2
```

Conviene utilizar `++` y `--` cuando su propósito de contador sea evidente. En expresiones más complejas, sus variantes prefija y sufija pueden producir diferencias difíciles de leer:


```
let a = 5;
const antes = a++; // antes vale 5; luego a vale 6

let b = 5;
const despues = ++b; // primero b vale 6; despues vale 6
```

Comparaciones numéricas

```
const edad = 20;

console.log(edad >= 18); // true
console.log(edad < 65); // true
console.log(edad === 20); // true
```

Si los operandos poseen tipos distintos, algunos operadores comparativos realizan conversiones. La práctica recomendada es convertir las entradas en la frontera y comparar valores del mismo tipo.

Valores numéricos especiales

```
console.log(1 / 0); // Infinity
console.log(-1 / 0); // -Infinity
console.log(Number("x")); // NaN
```

`NaN` significa *Not a Number*, pero su tipo sigue siendo `number`:

```
console.log(typeof NaN); // "number"
```

Además, `NaN` no es igual a sí mismo:

```
console.log(NaN === NaN); // false
console.log(Number.isNaN(NaN)); // true
```

Para comprobarlo debe utilizarse `Number.isNaN()`.

Precisión y enteros seguros

Los decimales se representan en binario y algunos no pueden almacenarse exactamente:

```
console.log(0.1 + 0.2); // 0.30000000000000004
console.log(0.1 + 0.2 === 0.3); // false
```

No es un error exclusivo de JavaScript, sino una consecuencia de la representación binaria de punto flotante.

Para cantidades monetarias suele ser conveniente trabajar con la unidad mínima como entero:

```
const precioCentavos = 199_900;
```

`number` representa exactamente los enteros desde `Number.MIN_SAFE_INTEGER` hasta `Number.MAX_SAFE_INTEGER`:

```
console.log(Number.MAX_SAFE_INTEGER); // 9007199254740991
```

Fuera de ese rango, dos enteros matemáticamente distintos pueden compartir la misma representación.

Funciones y métodos numéricos habituales

El objeto estándar `Number` ofrece comprobaciones y conversiones:

```
Number.isFinite(25);           // true
Number.isInteger(25.0);        // true
Number.isSafeInteger(25);      // true
Number.isNaN(NaN);            // true

Number.parseInt("42", 10);      // 42
Number.parseFloat("3.14");      // 3.14
```

Siempre conviene indicar la base al utilizar `parseInt`:

```
Number.parseInt("101", 2);      // 5
Number.parseInt("101", 10);     // 101
```

Los valores numéricos exponen métodos de formato:

```
const precio = 1234.567;

precio.toFixed(2);              // "1234.57"
precio.toPrecision(4);         // "1235"
precio.toString(16);           // "4d2.9126e978d5"
```

Estos métodos devuelven cadenas. Formatear no modifica el valor numérico original.

El objeto estándar `Math` reúne operaciones matemáticas:

```

Math.abs(-8);           // 8
Math.round(3.6);        // 4
Math.floor(3.9);        // 3
Math.ceil(3.1);         // 4
Math.trunc(3.9);        // 3
Math.min(4, 2, 9);      // 2
Math.max(4, 2, 9);      // 9
Math.sqrt(81);          // 9
Math.pow(2, 3);         // 8, equivalente a 2 ** 3
Math.random();          // valor desde 0 inclusive hasta 1 exclusivo

```

`Math.random()` resulta útil para simulaciones o juegos simples, pero no genera aleatoriedad apta para contraseñas, tokens o seguridad.

10.2 Operadores orientados a bits

Los operadores bit a bit trabajan con la representación binaria de enteros.

Operador	Operación
<code>a & b</code>	AND bit a bit
<code>a b</code>	OR bit a bit
<code>a ^ b</code>	XOR bit a bit
<code>~a</code>	NOT bit a bit
<code>a << n</code>	desplazamiento a la izquierda
<code>a >> n</code>	desplazamiento a la derecha con signo
<code>a >>> n</code>	desplazamiento a la derecha sin signo

Ejemplo con `5` y `3`:

```

5 = 0101
3 = 0011

5 & 3 = 0001 = 1
5 | 3 = 0111 = 7
5 ^ 3 = 0110 = 6

```

```

console.log(5 & 3); // 1
console.log(5 | 3); // 7
console.log(5 ^ 3); // 6
console.log(5 << 1); // 10
console.log(5 >> 1); // 2

```

Un uso real consiste en representar banderas o permisos:

```
const LEER = 1 << 0;    // 001
const EDITAR = 1 << 1;  // 010
const ELIMINAR = 1 << 2; // 100

const permisos = LEER | EDITAR; // 011
const puedeEditar = (permisos & ELIMINAR) !== 0;
```

Cuando operan sobre valores `number`, los operadores de bits los convierten a enteros de 32 bits. Por eso no deben utilizarse como atajo general para truncar decimales ni para procesar enteros grandes.

10.3 El tipo `bigint`

`bigint` representa enteros de precisión arbitraria. Su literal termina con `n`:

```
const poblacionEstelar = 9_007_199_254_740_993n;
const otro = BigInt("9007199254740993");
```

No representa decimales:

```
const cociente = 5n / 2n;
console.log(cociente); // 2n: descarta la parte fraccionaria
```

No se mezclan directamente `number` y `bigint` en operaciones aritméticas:

```
// 10n + 2; // TypeError
10n + 2n;    // 12n
```

La conversión debe ser explícita y solo debe realizarse después de considerar si puede perder precisión:

```
BigInt(10); // 10n
Number(10n); // 10
```

`bigint` soporta la mayoría de los operadores aritméticos y de bits, pero no el desplazamiento sin signo `>>>`.

11. Cadenas: representar texto

El tipo `string` representa secuencias de texto. JavaScript ofrece tres formas habituales de escribir literales de cadena.

Comillas simples y dobles

```
const simple = 'Hola';  
const doble = "Hola";
```

No existe una diferencia de comportamiento entre ambas. Se elige una convención para el proyecto y se utiliza la otra cuando evita escapes innecesarios:

```
const frase1 = "La materia se llama 'Programación IV'";  
const frase2 = 'El alumno respondió "presente"';
```

También puede escaparse un carácter con `\`:

```
const mensaje = "Ella dijo: \"Hola\"";  
const ruta = "C:\\documentos\\clase";  
const dosLineas = "Primera línea\nSegunda línea";  
const tabulado = "Nombre\tNota";
```

Algunas secuencias de escape frecuentes son:

Secuencia	Significado
<code>\n</code>	nueva línea
<code>\t</code>	tabulación
<code>\"</code>	barra invertida
<code>\"</code>	comilla doble
<code>'</code>	comilla simple

Comillas invertidas y plantillas literales

Las comillas invertidas —*backticks*— crean plantillas literales:

```
const nombre = "Ana";  
const nota = 9;  
  
const resumen = `${nombre} obtuvo ${nota} puntos`;
```

La construcción `${expresion}` evalúa la expresión, convierte su resultado a texto y lo inserta:

```
const precio = 1200;
const cantidad = 3;

const detalle = `Total: ${precio * cantidad}`;
// "Total: $3600"
```

Esto se denomina **interpolación**. Suele ser más legible que concatenar muchas partes:

```
const concatenado = nombre + " obtuvo " + nota + " puntos";
const interpolado = `${nombre} obtuvo ${nota} puntos`;
```

Las plantillas también pueden ocupar varias líneas:

```
const correo = `Hola, ${nombre}:

Tu inscripción fue confirmada.
Saludos.`;
```

Los saltos y espacios escritos entre los backticks forman parte de la cadena resultante.

Las cadenas son inmutables

No podemos cambiar un carácter dentro de una cadena existente:

```
const lenguaje = "javascript";

lenguaje[0] = "J"; // no modifica la cadena

const corregido = "J" + lenguaje.slice(1);
console.log(corregido); // "Javascript"
```

Los métodos crean nuevas cadenas.

Operadores de cadenas

+ concatena cuando la operación involucra una cadena:

```
const nombreCompleto = "Ana" + " " + "Pérez";
```

+= concatena y reasigna:

```
let mensaje = "Hola";
mensaje += ", Ana";
```

Las comparaciones entre cadenas son lexicográficas y sensibles a mayúsculas, minúsculas y Unicode:

```
console.log("Ana" === "ana"); // false
console.log("20" < "3");      // true: compara como texto
```

La segunda expresión no compara veinte con tres. Compara los caracteres iniciales `"2"` y `"3"`.

Propiedades y métodos habituales

```
const materia = "  Programación IV  ";

materia.length;           // 19, incluye espacios
materia.trim();           // "Programación IV"
materia.toUpperCase();     // "  PROGRAMACIÓN IV  "
materia.toLowerCase();     // "  programación iv  "
materia.includes("IV");    // true
materia.startsWith("Programa"); // false: comienza con espacios
materia.endsWith("IV");   // false: termina con espacios
```

Para extraer y buscar partes:

```
const codigo = "TUP-P4-2026";

codigo[0];                // "T"
codigo.at(-1);            // "6"
codigo.indexOf("P4");     // 4
codigo.slice(0, 3);       // "TUP"
codigo.substring(4, 6);   // "P4"
```

Para reemplazar y dividir:

```
"hola mundo".replace("mundo", "curso");
// "hola curso"

"uno, dos, tres".split(", ");
// ["uno", "dos", "tres"]
```

`replace()` reemplaza la primera coincidencia textual. `replaceAll()` reemplaza todas:

```
"1-2-3".replaceAll("-", "/"); // "1/2/3"
```

Los métodos pueden encadenarse porque cada uno devuelve un nuevo valor:

```
const entrada = " ANA@EJEMPLO.COM ";  
const correoNormalizado = entrada.trim().toLowerCase();
```

Una nota sobre longitud y Unicode

`length` cuenta unidades de código UTF-16, no necesariamente los caracteres que una persona percibe en pantalla:

```
console.log("😊".length); // 2
```

Para la mayoría de los textos iniciales esto no genera dificultad. En sistemas internacionales, cortar cadenas por índices requiere considerar Unicode con mayor cuidado.

12. `undefined` y `null`: dos formas de ausencia

Ambos valores expresan que no hay un dato utilizable, pero comunican intenciones diferentes.

`undefined`

`undefined` suele indicar que un valor todavía no fue asignado, que una propiedad no existe o que una función no devolvió un resultado explícito:

```
let telefono;  
console.log(telefono); // undefined  
  
const alumno = { nombre: "Ana" };  
console.log(alumno.telefono); // undefined  
  
function registrar() {  
  console.log("Registrado");  
}  
  
console.log(registrar()); // undefined
```

Técnicamente, `undefined` es un valor global y no un literal especial de la gramática como `null`, aunque en el uso cotidiano se escribe directamente.

`null`

`null` representa normalmente una ausencia elegida de manera intencional:

```
const fechaDeBaja = null;
```


Aquí no olvidamos asignar el dato: afirmamos que el alumno no posee fecha de baja.

Una particularidad histórica es:

```
console.log(typeof null); // "object"
```

Eso no convierte a `null` en un objeto. Para comprobarlo se utiliza igualdad estricta:

```
if (fechaDeBaja === null) {  
  console.log("El alumno continúa activo");  
}
```

`null` y `undefined` se comparan así:

```
null === undefined; // false  
null == undefined;  // true, por la regla especial de igualdad flexible
```

El operador `??` permite tratar ambos como ausencia sin descartar otros valores falsy válidos.

13. `symbol`: identificadores únicos

Un `symbol` es un valor primitivo único e inmutable. No posee sintaxis literal; se crea mediante `Symbol()`:

```
const idInterno = Symbol("id");  
const otroId = Symbol("id");  
  
console.log(idInterno === otroId); // false
```

La descripción `"id"` ayuda a depurar, pero no determina la identidad. Cada llamada crea un valor diferente.

Su uso principal es actuar como clave de propiedades sin colisionar con otras claves:

```
const identificador = Symbol("identificador");  
  
const alumno = {  
  nombre: "Ana",  
  [identificador]: 123,  
};
```

Es un tipo básico del lenguaje, aunque aparece con menos frecuencia en los primeros programas. También existen símbolos estándar que permiten personalizar ciertos comportamientos internos de los objetos.

14. Inspeccionar el tipo con `typeof`

El operador `typeof` devuelve una cadena que describe el tipo general de un valor:

```
typeof true;           // "boolean"
typeof 42;             // "number"
typeof 42n;            // "bigint"
typeof "hola";         // "string"
typeof undefined;      // "undefined"
typeof Symbol("id");   // "symbol"
typeof {};             // "object"
typeof [];             // "object"
typeof function () {}; // "function"
typeof null;           // "object", particularidad histórica
```

`typeof` distingue bien la mayoría de los primitivos, pero no alcanza para clasificar todos los objetos. Para un arreglo se utiliza:

```
Array.isArray([]); // true
```

Y para comprobar `null`:

```
valor === null;
```

15. Conversiones explícitas

Convertir un tipo significa producir un nuevo valor de otro tipo. Cuando la conversión aparece escrita por el programador es **explícita**.

Convertir a cadena

```
String(42);           // "42"
String(true);         // "true"
String(null);         // "null"
String(undefined);    // "undefined"
```

Los números también pueden expresarse en otras bases:

```
const valor = 255;

valor.toString(2); // "11111111"
valor.toString(16); // "ff"
```

Convertir a número

```
Number("42"); // 42
Number("3.14"); // 3.14
Number(""); // 0
Number(" "); // 0
Number(true); // 1
Number(false); // 0
Number(null); // 0
Number(undefined); // NaN
Number("42px"); // NaN
```

`Number()` exige que la cadena completa represente un número, aparte de espacios laterales.

`Number.parseInt()` y `Number.parseFloat()` pueden leer un prefijo numérico:

```
Number.parseInt("42px", 10); // 42
Number.parseFloat("3.14kg"); // 3.14
```

Esa tolerancia puede ser útil al analizar formatos controlados, pero también puede aceptar una entrada que debería rechazarse. La función se elige según la regla del problema.

Una conversión robusta de un campo numérico puede escribirse así:

```
function convertirCantidad(entrada) {
  const texto = entrada.trim();

  if (texto === "") {
    throw new Error("La cantidad es obligatoria");
  }

  const cantidad = Number(texto);

  if (!Number.isInteger(cantidad) || cantidad < 0) {
    throw new Error("La cantidad debe ser un entero no negativo");
  }

  return cantidad;
}
```

Convertir y validar son pasos distintos: `Number("-2")` convierte correctamente, pero la regla de negocio puede rechazar el resultado.

Convertir a booleano

```
Boolean(1);      // true
Boolean(0);      // false
Boolean("hola"); // true
Boolean("");     // false
Boolean("false"); // true
Boolean(null);   // false
```

`Boolean()` no interpreta el significado humano de la palabra `"false"`. Solo aplica la regla de truthy y falsy: toda cadena no vacía es truthy.

Convertir a `bigint`

```
BigInt("9007199254740993"); // 9007199254740993n
BigInt(42);                  // 42n
```

Una cadena decimal produce un error porque `bigint` solamente representa enteros:

```
// BigInt("3.14"); // SyntaxError
```

16. Coerción: la conversión automática de tipos

JavaScript también convierte valores de manera automática. Esa conversión implícita se denomina **coerción**.

No se trata de decisiones aleatorias del motor. Cada operador y cada contexto poseen reglas definidas por el lenguaje. La dificultad aparece cuando esas reglas no coinciden con la intención del programador.

16.1 Contextos booleanos

Las condiciones de `if`, `while`, el operador ternario y los operadores lógicos aplican la conversión booleana:

```
if ("hola") {
  console.log("Se ejecuta porque la cadena es truthy");
}
```

No cambian el valor original; lo interpretan como condición.

16.2 Interpolación y conversión a texto

Una expresión dentro de una plantilla se convierte a cadena:

```
const cantidad = 3;
const disponible = true;

`${cantidad}`; // "3"
`${disponible}`; // "true"
```

16.3 La regla especial del operador +

`+` puede representar suma numérica o concatenación. Después de obtener valores primitivos, si uno de los operandos es una cadena, concatena como texto:

```
5 + 2; // 7
"5" + 2; // "52"
5 + "2"; // "52"
"5" + true; // "5true"
```

La evaluación ocurre de izquierda a derecha:

```
1 + 2 + "3"; // "33": primero 1 + 2, después 3 + "3"
"1" + 2 + 3; // "123": la concatenación comienza desde el primer paso
```

Los paréntesis pueden cambiar el resultado:

```
"Total: " + 2 + 3; // "Total: 23"
"Total: " + (2 + 3); // "Total: 5"
```

La interpolación suele expresar mejor esta intención:

```
`Total: ${2 + 3}`; // "Total: 5"
```

16.4 Los demás operadores aritméticos

`-`, `*`, `/`, `%` y `**` no concatenan. Intentan convertir sus operandos a valores numéricos compatibles:

```
"10" - 2; // 8
"10" * 2; // 20
"10" / 2; // 5
"10" ** 2; // 100
```

Si la conversión no produce un número válido, el resultado suele ser `NaN` :

```
"diez" * 2; // NaN
```

Que una operación “funcione” no demuestra que la entrada posea el tipo correcto:

```
const precio = 1200;
const cantidad = "2";

precio * cantidad; // 2400
precio + cantidad; // "12002"
```

La solución es convertir una vez, validar y luego operar con un tipo conocido.

16.5 Comparaciones relacionales

Si ambos operandos son cadenas, `<`, `>`, `<=` y `>=` comparan lexicográficamente. En otros casos habituales intentan una comparación numérica:

```
"20" < "3"; // true: comparación textual
20 < "3";   // false: "3" se convierte a 3
```

Evitar operandos de tipos diferentes vuelve la comparación más fácil de revisar.

16.6 Igualdad flexible y estricta

`==` y `!=` aplican un algoritmo de conversión cuando los tipos no coinciden:

```
0 == false;      // true
"" == 0;         // true
"5" == 5;        // true
null == undefined; // true
```

`===` y `!==` no convierten:

```
0 === false;     // false
"" === 0;        // false
"5" === 5;       // false
null === undefined; // false
```

Por eso la igualdad estricta debe ser la opción habitual. La igualdad flexible posee reglas válidas y deterministas, pero obliga a conocer más información para interpretar una línea.

16.7 Coerciones limitadas para `bigint` y `symbol`

JavaScript evita algunas conversiones automáticas que podrían perder información o resultar ambiguas:

```
// 1n + 1;           // TypeError: mezcla bigint y number
// +1n;             // TypeError: el + unario no admite bigint
// "id: " + Symbol("id"); // TypeError
```

Las conversiones deben hacerse de manera explícita cuando correspondan:

```
1n + BigInt(1);           // 2n
`id: ${String(Symbol("id"))}`; // "id: Symbol(id)"
```

17. Una estrategia segura para datos de entrada

Los formularios, parámetros de URL, archivos y cuerpos HTTP llegan frecuentemente como texto o como datos cuya forma no controlamos. Una secuencia saludable es:

```
entrada externa
  ↓
comprobar presencia y formato
  ↓
convertir explícitamente
  ↓
validar reglas del dominio
  ↓
operar con un valor confiable
```

Ejemplo:

```
const precioTexto = "1999.50";
const cantidadTexto = "3";

const precio = Number(precioTexto);
const cantidad = Number(cantidadTexto);

if (!Number.isFinite(precio) || precio < 0) {
  throw new Error("Precio inválido");
}

if (!Number.isInteger(cantidad) || cantidad <= 0) {
  throw new Error("Cantidad inválida");
}

const total = precio * cantidad;
```

Esta versión parece más extensa que confiar en la coerción, pero hace visibles las decisiones:

- qué tipos espera el cálculo;
- qué entradas son inválidas;
- en qué punto se realiza la conversión;
- qué condiciones pertenecen al negocio.

TypeScript puede agregar verificación estática alrededor de este código, pero la entrada sigue necesitando validación durante la ejecución.

18. Errores frecuentes y su explicación

Confundir `const` con inmutabilidad profunda

```
const configuracion = { tema: "claro" };
configuracion.tema = "oscuro"; // permitido
```

`const` impide reasignar `configuracion`; no congela el objeto.

Usar `var` esperando alcance de bloque

```
if (true) {
  var visible = "también afuera";
}

console.log(visible);
```


`var` posee alcance de función o global, no de bloque.

Interpretar una cadena no vacía como su significado humano

```
Boolean("false"); // true
```

La conversión observa si la cadena está vacía, no si contiene la palabra "false".

Utilizar `||` cuando cero es válido

```
const intentos = 0;

const conOr = intentos || 3; // 3
const conNullish = intentos ?? 3; // 0
```

`||` descarta cualquier falsy. `??` reemplaza únicamente `null` o `undefined`.

Formatear un número y creer que continúa siendo numérico

```
const precio = 12.5;
const formateado = precio.toFixed(2);

typeof formateado; // "string"
```

`toFixed()` produce texto listo para mostrar.

Usar `parseInt` sin decidir si se aceptará texto adicional

```
Number.parseInt("12px", 10); // 12
Number("12px");              // NaN
```

La primera función acepta un prefijo numérico. La segunda exige una representación numérica completa.

Comparar tipos diferentes con `==`

```
"0" == false; // true
"0" === false; // false
```

Convertir primero y comparar después reduce sorpresas.

19. Ideas que conviene conservar

1. Una variable vincula un nombre con un valor dentro de un alcance.
 2. Declarar, inicializar y asignar son operaciones diferentes.
 3. `const` evita reasignar la vinculación; no vuelve inmutable a un objeto.
 4. `let` se utiliza cuando la reasignación forma parte del problema.
 5. `var` posee alcance de función y reglas históricas; se evita en código moderno.
 6. El alcance indica dónde puede usarse un nombre; su vida indica cuánto tiempo permanece alcanzable.
 7. Los buenos nombres expresan significado y unidad, no solamente el tipo del dato.
 8. JavaScript posee siete tipos primitivos: boolean, number, bigint, string, undefined, null y symbol.
 9. Los valores primitivos son inmutables.
 10. La precedencia determina el agrupamiento de operadores; los paréntesis hacen explícita la intención.
 11. Todos los valores son truthy salvo el conjunto específico de valores falsy.
 12. `&&`, `||` y `??` evalúan con cortocircuito y devuelven operandos, no necesariamente booleanos.
 13. `number` representa enteros y decimales con punto flotante; no todos los decimales son exactos.
 14. Los operadores de bits convierten valores `number` a enteros de 32 bits.
 15. Las plantillas literales permiten interpolación y cadenas multilínea.
 16. `undefined` suele expresar falta de asignación; `null`, ausencia intencional.
 17. Una conversión explícita aparece escrita; una coerción es aplicada automáticamente por el contexto.
 18. El operador `+` es especial porque puede sumar o concatenar.
 19. La igualdad estricta evita conversiones implícitas.
 20. Los datos externos deben convertirse y validarse antes de utilizarlos en reglas del sistema.
-

20. Preguntas de repaso

1. ¿Qué diferencia existe entre declarar, inicializar y asignar?
2. ¿Por qué se recomienda `const` como primera elección?
3. ¿En qué se diferencian el alcance de `var` y el de `let`?
4. ¿Qué es la zona muerta temporal?
5. ¿Por qué una variable capturada por una clausura puede seguir viva después de terminar una función?
6. ¿Qué reglas debe cumplir un identificador?
7. ¿Qué convención se utiliza habitualmente para variables y funciones?
8. ¿Cuáles son los siete tipos primitivos?
9. ¿Por qué `typeof null` requiere una aclaración?
10. ¿Qué diferencia existe entre precedencia y asociatividad?

11. ¿Cuáles son los valores falsy?
12. ¿Por qué `"false"` es truthy?
13. ¿Qué valor devuelve `"Ana" && "Docente"`?
14. ¿Cuándo conviene `??` en lugar de `||`?
15. ¿Por qué `0.1 + 0.2` no es exactamente `0.3`?
16. ¿Qué sucede con los operandos `number` de una operación bit a bit?
17. ¿Qué diferencias hay entre `number` y `bigint`?
18. ¿Qué permiten las comillas invertidas que no ofrecen directamente las simples o dobles?
19. ¿Qué diferencia conceptual existe entre `undefined` y `null`?
20. ¿Por qué `Number("12px")` y `Number.parseInt("12px", 10)` producen resultados distintos?
21. ¿Cómo decide `+` entre suma y concatenación?
22. ¿Por qué se recomienda `===` en lugar de `==`?

Ejercicio 1: anticipar valores y tipos

Sin ejecutar el código, completar el valor y el tipo de cada resultado:

```
const a = 2 + 3 * 4;  
const b = (2 + 3) * 4;  
const c = "2" + 3;  
const d = "10" - 4;  
const e = "" || "sin dato";  
const f = 0 ?? 10;  
const g = Boolean("false");  
const h = 5n / 2n;
```

Ejercicio 2: corregir una entrada

El siguiente programa calcula mal algunos totales:

```
const precio = "1500.50";  
const cantidad = "2";  
const descuento = "10";  
  
const subtotal = precio * cantidad;  
const total = subtotal - subtotal * descuento / 100;  
  
console.log("Total: $" + total);
```

Reescribirlo para:

1. convertir explícitamente las entradas;
2. rechazar valores no numéricos;
3. exigir una cantidad entera positiva;
4. exigir un descuento entre `0` y `100`;

5. mostrar el resultado con dos decimales mediante una plantilla literal.

Ejercicio 3: diseñar nombres

Reemplazar los nombres poco expresivos:

```
const x = 3000;  
const y = 5;  
const z = 0.21;  
const r = x * y * (1 + z);
```

Indicar qué representa cada valor y qué unidad utiliza.

Fuentes y lecturas recomendadas

- Gramática, declaraciones, tipos y literales — MDN
- Tipos y estructuras de datos de JavaScript — MDN
- Gramática léxica e identificadores — MDN
- Precedencia de operadores — MDN
- Expresiones y operadores — MDN
- Referencia de `Number` — MDN
- Referencia de `BigInt` — MDN
- Plantillas literales — MDN
- Coerción de tipos — MDN
- ECMAScript Language Specification — TC39

Programación IV — El tipo booleano en JavaScript

Los booleanos son uno de los tipos de datos primitivos de JavaScript. Representan una idea extremadamente simple: una condición puede ser verdadera o falsa.

```
true;
false;
```

Aunque sólo existen esos dos valores booleanos, JavaScript permite utilizar prácticamente cualquier valor como si fuera una condición. Para comprender correctamente este comportamiento hay que distinguir entre:

- valores booleanos;
- valores *truthy* y *falsy*;
- conversión explícita e implícita a booleano;
- operadores lógicos `!`, `&&` y `||`;
- evaluación por cortocircuito.

1. El tipo `boolean`

JavaScript tiene dos valores booleanos: `true` y `false`. Podemos almacenarlos directamente en variables:

```
let activo = true;
let terminado = false;

console.log(typeof activo);
// "boolean"
```

También suelen aparecer como resultado de comparaciones:

```
let edad = 20;
let mayor = edad >= 18;

console.log(mayor);
// true
```

Las expresiones relacionales y de igualdad producen valores booleanos:

```
10 > 5;      // true
10 < 5;      // false
10 === 10;   // true
10 !== 10;   // false
```

Por eso los booleanos son fundamentales para controlar el flujo de ejecución:

```
if (edad >= 18) {
  console.log("Es mayor de edad");
}
```

2. Una condición no necesita ser un booleano

En muchos lenguajes, la condición de un `if` debe ser estrictamente un valor booleano. JavaScript es más flexible.

```
let nombre = "Ana";

if (nombre) {
  console.log("Hay un nombre");
}
```

`nombre` no contiene un booleano, sino un `string`. Sin embargo, JavaScript permite utilizarlo como condición porque, cuando necesita decidir entre verdadero y falso, convierte implícitamente el valor a booleano.

Conceptualmente podemos imaginar que ocurre algo parecido a esto:

```
if (Boolean(nombre)) {
  console.log("Hay un nombre");
}
```

La conversión de una cadena no vacía produce `true`:

```
Boolean("Ana");
// true
```

Por lo tanto, el cuerpo del `if` se ejecuta. Esta conversión automática es una manifestación del sistema de tipos dinámico y de las coerciones implícitas de JavaScript.

3. Valores *truthy* y *falsy*

Cuando JavaScript convierte un valor a booleano, el resultado sólo puede ser `true` o `false`.

- Los valores que se convierten en `false` se denominan ***falsy***.
- Los valores que se convierten en `true` se denominan ***truthy***.

Es importante notar que un valor *truthy* no necesariamente es el booleano `true`:

```
"Hola" === true;  
// false  
  
Boolean("Hola");  
// true
```

Por eso decimos que `"Hola"` es *truthy*, no que sea el booleano `true`.

4. Valores *falsy*

JavaScript tiene un conjunto pequeño de valores que se consideran falsos al convertirse a booleano:

```
false;  
0;  
-0;  
0n;  
"";  
null;  
undefined;  
NaN;
```

Podemos comprobarlo con la función `Boolean`:

```
Boolean(false);    // false  
Boolean(0);        // false  
Boolean("");       // false  
Boolean(null);     // false  
Boolean(undefined); // false  
Boolean(NaN);      // false
```

Esto permite escribir:

```
let nombre = "";

if (nombre) {
  console.log("Hay un nombre");
}
```

Como la cadena vacía `""` es *falsy*, el cuerpo del `if` no se ejecutará.

5. Valores *truthy*

Prácticamente todo lo que no es *falsy* es *truthy*:

```
Boolean(1);      // true
Boolean(-1);     // true
Boolean(3.14);   // true
Boolean("Hola"); // true
Boolean("false"); // true
Boolean([]);     // true
Boolean({});     // true
```

Hay algunos casos que inicialmente pueden resultar sorprendentes. Una cadena que contiene `"false"` es *truthy*:

```
Boolean("false");
// true
```

No importa lo que diga el texto: lo importante es que se trata de una cadena no vacía. Por el mismo motivo, `"0"` también es *truthy*:

```
Boolean("0");
// true
```

Los arrays y los objetos también son *truthy*, incluso cuando están vacíos:

```
Boolean([]); // true
Boolean({}); // true
```

6. Conversión explícita a booleano

Podemos pedir explícitamente la conversión mediante la función `Boolean`:


```
Boolean(valor);
```

Por ejemplo:

```
Boolean(10);      // true
Boolean(0);       // false
Boolean("Hola");  // true
Boolean("");      // false
Boolean(null);    // false
Boolean(undefined); // false
```

Esto resulta especialmente útil para experimentar y comprender las reglas de conversión de JavaScript.

7. Conversión mediante **!!**

Existe otra forma muy habitual de convertir un valor a booleano:

```
!!valor;
```

Para entenderla conviene recordar que **!** es el operador de negación lógica:

```
!true;    // false
!false;   // true
```

El operador **!** convierte primero su operando a booleano y luego niega el resultado. Por ejemplo:

```
!"Hola";
```

Conceptualmente ocurre lo siguiente:

```
!Boolean("Hola");
```

Como **"Hola"** es *truthy*, se convierte en **true** y luego se niega:

```
!true;
// false
```

Si volvemos a negar el resultado, obtenemos el valor booleano original:

```
!!"Hola";  
// true
```

Por eso, `!!valor` es una forma abreviada de escribir `Boolean(valor)`:

```
!!"Hola";    // true  
!!"";        // false  
!!42;        // true  
!!0;         // false  
!!null;      // false
```

8. Conversión implícita

Muchas conversiones ocurren automáticamente en JavaScript:

```
let cantidad = 10;  
  
if (cantidad) {  
  console.log("Hay elementos");  
}
```

`cantidad` contiene un número, no un booleano. Sin embargo, el `if` necesita tomar una decisión lógica y JavaScript realiza implícitamente esta conversión:

```
Boolean(cantidad);
```

Como `Boolean(10)` es `true`, la condición se considera verdadera.

Lo mismo ocurre en otras estructuras de control:

```
while (valor) {  
  // Repetir mientras valor sea truthy.  
}
```

También sucede con los operadores lógicos `!`, `&&` y `||`.

Esta capacidad se denomina **coerción de tipos**: JavaScript convierte automáticamente un valor de un tipo en otro cuando el contexto lo requiere.

9. El operador AND: `&&`

El operador lógico AND se escribe `&&`. Su tabla de verdad es la siguiente:

A	B	A <code>&&</code> B
false	false	false
false	true	false
true	false	false
true	true	true

Por ejemplo:

```
let edad = 25;
let tieneEntrada = true;

if (edad >= 18 && tieneEntrada) {
  console.log("Puede ingresar");
}
```

Las dos condiciones deben ser verdaderas. Sin embargo, en JavaScript `&&` tiene una característica adicional muy importante: **no necesariamente devuelve `true` o `false`; puede devolver directamente uno de sus operandos.**

10. Cortocircuito en `&&`

Consideremos la expresión:

```
A && B;
```

Para que la expresión completa sea verdadera, `A` debe ser *truthy*. Si `A` es *falsy*, ya conocemos el resultado lógico y no hace falta evaluar `B`.

```
false && algo();
```

En este caso, `algo()` ni siquiera se ejecuta. A este comportamiento se lo denomina **evaluación por cortocircuito**.

Podemos comprobarlo con una función:

```
function prueba() {  
  console.log("Se ejecutó");  
  return true;  
}  
  
false && prueba();
```

No se imprime nada. JavaScript encontró un valor *false* y ya sabe que un AND no puede resultar verdadero.

11. && devuelve el primer valor *false*

En JavaScript, && devuelve el primer valor *false* que encuentra:

```
10 && 20;  
// 20
```

10 es *truthy*, por lo que todavía no se puede determinar el resultado. Entonces se evalúa 20. Como no quedan más operandos, 20 es el resultado.

Veamos una expresión más larga:

```
10 && 0 && 30;  
// 0
```

La evaluación ocurre de izquierda a derecha:

1. 10 es *truthy*: continuar.
2. 0 es *false*: detenerse.
3. 30 nunca se evalúa.

Otro ejemplo:

```
"Juan" && 25 && "Hola";  
// "Hola"
```

Todos los operandos son *truthy*, por lo que la evaluación llega hasta el último.

A && B && C && D busca el primer valor *false*. Si no encuentra ninguno, devuelve el último operando.

12. El operador OR: `||`

El operador lógico OR se escribe `||`. Su tabla de verdad es la siguiente:

A	B	A <code> </code> B
false	false	false
false	true	true
true	false	true
true	true	true

Nuevamente, JavaScript no necesariamente devuelve un booleano: puede devolver uno de los operandos.

13. Cortocircuito en `||`

Consideremos esta expresión:

```
A || B;
```

Para que un OR sea verdadero alcanza con que `A` sea *truthy*. Por lo tanto, si `A` es *truthy*, no existe ninguna necesidad de evaluar `B`.

```
true || algo();
```

En este caso, `algo()` no se ejecuta. Esto también es evaluación por cortocircuito.

14. `||` devuelve el primer valor *truthy*

Podemos pensar el comportamiento de `||` de esta manera:

`||` devuelve el primer valor *truthy* que encuentra.

Por ejemplo:

```
0 || 20;  
// 20
```

`0` es *falsy*, por lo que JavaScript continúa. `20` es *truthy* y se convierte en el resultado.

Otro ejemplo:

```
"" || null || "Juan" || "Pedro";  
// "Juan"
```

La evaluación ocurre de izquierda a derecha:

1. `""` es *falsy*: continuar.
2. `null` es *falsy*: continuar.
3. `"Juan"` es *truthy*: detenerse.
4. `"Pedro"` nunca se evalúa.

Si ninguno de los primeros operandos es *truthy*, se devuelve el último:

```
0 || "" || null;  
// null
```

15. La simetría entre `&&` y `||`

Una forma sencilla de recordar ambos operadores es:

- `&&` busca algo que haga **falsa** la expresión;
- `||` busca algo que haga **verdadera** la expresión.

Por eso:

- `&&` devuelve el primer valor *falsy*;
- `||` devuelve el primer valor *truthy*.

Si no encuentran un valor que determine anticipadamente el resultado, devuelven el último operando.

```
10 && 20 && 30;  
// 30, porque ninguno fue falsy  
  
0 || "" || 30;  
// 30, porque los anteriores fueron falsy
```

16. ¿Por qué devuelven el valor y no `true` o `false`?

Esta particularidad resulta muy útil en JavaScript. Supongamos que queremos acceder a una propiedad sólo cuando existe un usuario:

```
let nombre = usuario && usuario.nombre;
```

Si `usuario` es *false*, JavaScript se detiene y devuelve ese valor. Si es *truthy*, continúa y devuelve `usuario.nombre`.

Otro patrón tradicional permite elegir un valor alternativo:

```
let nombre = nombreIngresado || "Anónimo";
```

Si el usuario ingresó un nombre:

```
let nombreIngresado = "Ana";  
  
"Ana" || "Anónimo";  
// "Ana"
```

Si ingresó una cadena vacía:

```
let nombreIngresado = "";  
  
"" || "Anónimo";  
// "Anónimo"
```

Los operadores lógicos pueden utilizarse no sólo para construir expresiones booleanas, sino también para seleccionar valores.

17. Un modelo mental sencillo

Cuando JavaScript encuentra `A && B`, podemos imaginar el siguiente comportamiento:

```
if (!A) {  
  return A;  
} else {  
  return B;  
}
```

Cuando encuentra `A || B`, podemos imaginar esto:

```
if (A) {  
  return A;  
} else {  
  return B;  
}
```

No es literalmente la implementación interna del lenguaje, pero es un excelente modelo mental:

```
0 && 100;  
// 0  
  
0 || 100;  
// 100
```

18. Lo fundamental

Un booleano sólo puede contener `true` o `false`. Sin embargo, JavaScript permite utilizar otros tipos de valores en contextos lógicos gracias a la conversión implícita a booleano.

Los valores se clasifican en:

- **falsy**: al convertirse producen `false`;
- **truthy**: al convertirse producen `true`.

La conversión explícita puede realizarse mediante cualquiera de estas formas:

```
Boolean(valor);  
!!valor;
```

Los operadores `&&` y `||` utilizan cortocircuito:

- `A && B` busca el primer valor *falsy*;
- `A || B` busca el primer valor *truthy*.

La idea más importante es que estos operadores no convierten necesariamente el resultado a `true` o `false`. Devuelven directamente el valor que permite determinar el resultado de la expresión.

```
"Hola" && 42;    // 42  
0 && 42;          // 0  
"Hola" || 42;    // "Hola"  
0 || 42;         // 42
```


Esta combinación de valores *truthy* y *falsy*, coerción implícita, cortocircuito y retorno del operando explica buena parte de la expresividad —y también algunas de las rarezas— de las expresiones lógicas en JavaScript.

El tipo `number` en JavaScript

En JavaScript, `number` es el tipo numérico principal. Una particularidad importante es que **no hay tipos separados como `int`, `float` o `double`** al nivel del lenguaje: tanto los enteros como los números con decimales son valores de tipo `number`.

```
let edad = 25;
let temperatura = 36.5;
let negativo = -10;

console.log(typeof edad);      // "number"
console.log(typeof temperatura); // "number"
```

Internamente, los valores `number` se representan normalmente mediante el formato de punto flotante **IEEE 754 de 64 bits**.

Esto explica algunas particularidades famosas:

```
0.1 + 0.2
// 0.30000000000000004
```

No es un problema específico de JavaScript: es consecuencia de representar en binario números reales con una cantidad finita de bits.

1. Literales numéricos

Un **literal** es una forma de escribir directamente un valor dentro del código.

Decimal

```
let a = 123;
let b = 3.1416;
let c = -45;
```

También existe notación científica:

```
let a = 1e3;    // 1000
let b = 2.5e2;  // 250
let c = 3e-3;   // 0.003
```

Binario

Se utiliza el prefijo `0b`:

```
let n = 0b1010;  
console.log(n); // 10
```

Octal

Se utiliza `0o`:

```
let n = 0o17;  
console.log(n); // 15
```

Hexadecimal

Se utiliza `0x`:

```
let n = 0xFF;  
console.log(n); // 255
```

Por ejemplo:

```
let a = 42;  
let b = 0b101010;  
let c = 0o52;  
let d = 0x2A;  
  
console.log(a, b, c, d);  
// 42 42 42 42
```

Los cuatro contienen **el mismo valor**. Lo único que cambia es cómo se escribió ese valor en el programa.

2. Conversión a `number`

JavaScript tiene **tipado dinámico**: una variable no queda asociada permanentemente con un tipo.

```
let x = 10;  
  
x = "hola";  
x = true;
```

Además, JavaScript realiza muchas **conversiones implícitas**, también llamadas coerciones.

Conversión implícita

```
"10" * 2  
// 20
```

JavaScript necesita realizar una multiplicación y convierte `"10"` a número.

```
"10" - 2    // 8  
"10" / 2    // 5  
"10" * 2    // 20
```

Pero `+` tiene una particularidad: también sirve para concatenar strings.

```
"10" + 2  
// "102"
```

Por eso:

```
10 + 20      // 30  
"10" + 20    // "1020"  
  
10 * 2       // 20  
"10" * 2     // 20
```

Esta coerción automática es una de las características que hacen que JavaScript sea considerado un lenguaje de **tipado débil**.

3. Conversión explícita

La forma general para convertir algo a número es:

```
Number(valor)
```

Por ejemplo:

```
Number("123")    // 123  
Number("3.14")   // 3.14  
Number(true)     // 1  
Number(false)    // 0  
Number("")       // 0
```

Si la conversión no tiene sentido:

```
Number("hola")  
// NaN
```

NaN significa **Not a Number**, aunque:

```
typeof NaN  
// "number"
```

Para verificarlo conviene:

```
Number.isNaN(x)
```

parseInt() y parseFloat()

```
parseInt("123")      // 123  
parseFloat("12.34")  // 12.34
```

parseInt() puede recibir una base:

```
parseInt("1010", 2)  // 10  
parseInt("FF", 16)   // 255  
parseInt("100", 8)   // 64
```

Hay una diferencia conceptual importante:

```
Number("123px")      // NaN  
parseInt("123px")     // 123
```

Number() intenta convertir **todo el valor**.

parseInt() intenta leer una parte inicial que pueda interpretar como entero.

4. Operadores aritméticos

Operador Operación Ejemplo ----- + suma 10 + 3
→ 13 - resta 10 - 3 → 7 * multiplicación 10 * 3 → 30 / división 10 / 3 → 3.333...
% resto 10 % 3 → 1 ** potencia 10 ** 3 → 1000

El operador % es especialmente útil para divisibilidad:

```
n % 2 === 0
```

indica que `n` es par.

También tenemos incremento y decremento:

```
x++;  
x--;  
  
++x;  
--x;
```

La diferencia entre prefijo y sufijo aparece cuando utilizamos el resultado:

```
let x = 10;  
let a = x++; // a = 10, después x = 11
```

frente a:

```
let x = 10;  
let a = ++x; // primero x = 11, después a = 11
```

5. Precedencia

Los operadores no se evalúan simplemente de izquierda a derecha.

```
2 + 3 * 4  
// 14
```

porque `*` tiene mayor precedencia que `+`.

Una simplificación útil del orden es:

```
()  
**  
* / %  
+ -  
comparaciones  
igualdades  
operadores lógicos
```

Ejemplos:

```
2 + 3 * 4      // 14
(2 + 3) * 4    // 20
10 % 4 + 2     // 4
2 ** 3 * 4     // 32
```

Cuando una expresión comienza a ser difícil de leer, es preferible usar paréntesis aunque no sean estrictamente necesarios.

6. Operadores de comparación

Las comparaciones producen valores booleanos:

```
10 > 5
// true
```

Operador Significado ----- < menor > mayor <= menor o igual
>= mayor o igual === igualdad estricta !== desigualdad estricta == igualdad con coerción !=
desigualdad con coerción

JavaScript distingue entre igualdad estricta y no estricta:

```
10 == "10"    // true
10 === "10"   // false
```

Con `==`, JavaScript intenta convertir tipos antes de comparar.

Con `===`, compara valor **y tipo**.

Por eso, en código moderno generalmente se prefiere `===` y `!==`.

7. Operaciones bit a bit --- bitwise

Los operadores **bitwise** permiten trabajar con los bits individuales de un entero.

Operador Operación ----- & AND bit a bit | OR
bit a bit ^ XOR bit a bit ~ NOT bit a bit << desplazamiento izquierda >> desplazamiento
derecha conservando signo >>> desplazamiento derecha sin signo

Por ejemplo:

```
let a = 0b1100;
let b = 0b1010;

a & b
```

Bit por bit:

```
  1100
& 1010
-----
  1000
```

Resultado: 8.

OR bit a bit

```
  1100
| 1010
-----
  1110
```

```
0b1100 | 0b1010
// 14
```

XOR bit a bit

```
  1100
^ 1010
-----
  0110
```

```
0b1100 ^ 0b1010
// 6
```

8. ¿Qué pasa con los `number` al usar bitwise?

Aunque normalmente `number` trabaja con números IEEE 754 de 64 bits, **la mayoría de las operaciones bitwise convierten sus operandos a enteros de 32 bits** antes de realizar la operación.


```
5.8 | 0
// 5
```

Conceptualmente:

```
5.8
↓
entero de 32 bits
↓
00000000 00000000 00000000 00000101
```

De ahí viene el viejo truco:

```
x | 0
```

para forzar un valor al dominio de enteros de 32 bits:

```
12.9 | 0    // 12
-12.9 | 0    // -12
```

Pero **no debe confundirse con** `Math.floor()` :

```
Math.floor(-12.9)  // -13
-12.9 | 0          // -12
```

Además, la conversión bitwise está limitada al rango de 32 bits.

Para truncar explícitamente es más claro:

```
Math.trunc(12.9)  // 12
```

9. Desplazamientos

`<<` mueve los bits hacia la izquierda.

```
let x = 0b00000101;
x << 1
```

produce:

```
00000101    5
  ↓
00001010    10
```

Para valores donde no hay overflow:

```
5 << 1    // 10
5 << 2    // 20
```

Desplazar `n` posiciones hacia la izquierda equivale conceptualmente a multiplicar por `2^n`, dentro de las limitaciones de 32 bits.

Desplazamiento hacia la derecha

```
20 >> 1    // 10
20 >> 2    // 5
```

Conceptualmente:

```
10100    20
01010    10
00101     5
```

`>>` conserva el signo.

`>>>` realiza desplazamiento hacia la derecha rellenando con ceros.

10. Bits como conjunto de opciones

Una aplicación clásica consiste en representar **varias opciones simultáneas dentro de un único número**.

Supongamos permisos de usuario:

```
const LEER      = 1 << 0;  // 0001
const ESCRIBIR  = 1 << 1;  // 0010
const BORRAR    = 1 << 2;  // 0100
const ADMIN     = 1 << 3;  // 1000
```

Cada bit representa una opción independiente.

LEER	0001
ESCRIBIR	0010
BORRAR	0100
ADMIN	1000

Activar opciones: OR

```
let permisos = LEER | ESCRIBIR;
```

0001	LEER
0010	ESCRIBIR

0011	

Podemos agregar otra opción:

```
permisos |= BORRAR;
```

Ahora `permisos` vale `0111`: tiene tres opciones simultáneamente.

Consultar una opción: AND

```
if ((permisos & ESCRIBIR) !== 0) {
  console.log("Puede escribir");
}
```

AND funciona como una máscara que deja pasar únicamente el bit que queremos consultar.

Apagar una opción

Para apagar un bit usamos AND con la máscara negada:

```
permisos &= ~BORRAR;
```

Por ejemplo:

0111
& 1011

0011

Quedan activos `LEER` y `ESCRIBIR`.

Alternar una opción con XOR

```
permisos ^= ADMIN;
```

Si `ADMIN` estaba apagado, se enciende. Si estaba encendido, se apaga.

11. El patrón completo

```
const LEER      = 1 << 0;
const ESCRIBIR  = 1 << 1;
const BORRAR    = 1 << 2;
const ADMIN     = 1 << 3;

let permisos = 0;

// prender
permisos |= LEER;
permisos |= ESCRIBIR;

// preguntar
if ((permisos & LEER) !== 0) {
  console.log("Puede leer");
}

// apagar
permisos &= ~ESCRIBIR;

// alternar
permisos ^= ADMIN;
```

Un solo entero puede representar muchas opciones simultáneas:

0000	ninguna
0001	LEER
0011	LEER + ESCRIBIR
0101	LEER + BORRAR
1011	LEER + ESCRIBIR + ADMIN
1111	todas

Una forma práctica de recordar los operadores es:

	prender bits
&	consultar/filtrar bits
& ~	apagar bits
^	alternar bits
<<	colocar un bit en una posición
>>	desplazar bits

La idea fundamental es que dejamos de pensar el número solamente como una cantidad y empezamos a utilizar **cada uno de sus bits como información independiente**.

27. Recorrer correctamente un string

1. Para trabajar correctamente con Unicode moderno conviene usar `codePointAt()`, iteradores (`for...of`, `[...s]`) y normalización cuando corresponda.
1. Para **orden alfabético humano**, no alcanza con `<`, `>` o `sort()`; conviene `localeCompare()` o `Intl.Collator`.

Ahora podemos profundizar en las consecuencias prácticas.

La forma aparentemente obvia es:

```
const texto = "A😄B";

for (let i = 0; i < texto.length; i++) {
  console.log(texto[i]);
}
```

Pero `length` e índices trabajan con unidades UTF-16. Por eso el emoji queda dividido.

Conceptualmente obtenemos:

```
A
<primera mitad de 😄>
<segunda mitad de 😄>
B
```

En cambio:

```
for (const caracter of texto) {
  console.log(caracter);
}
```

produce:

```
A
😄
B
```

Lo mismo ocurre con:

```
[... "A😄B"]
// ["A", "😄", "B"]
```

Regla práctica

Para procesamiento Unicode:

```
for (const c of texto) {  
    // ...  
}
```

suele ser preferible a:

```
for (let i = 0; i < texto.length; i++) {  
    const c = texto[i];  
}
```

28. Code units, code points y grafemas

Hay que distinguir **tres conceptos**.

Supongamos:

```
const s = "😄";
```

Tenemos:

Grafema visible:	😄
Code point:	U+1F600
Code units UTF-16:	D83D DE00

Entonces:

```
s.length  
// 2  
  
[...s].length  
// 1
```

Pero todavía queda una dificultad.

Consideremos:

```
const s = "a\u0301";
```

Visualmente:

á

Sin embargo:

```
[...s]  
// ["a", ""]
```

Hay **dos code points**, aunque una persona ve **una letra**.

Así que tenemos:

code unit ≠ code point ≠ carácter visual

El término técnico para este último es **grapheme cluster** o *grupo de grafemas*.

29. Los emoji hacen visible el problema

Consideremos:



Visualmente es un solo emoji: una familia.

Pero está construido combinando varios code points.

Por eso:

```
const familia = "👨👩👧👦";  
  
familia.length  
// mucho mayor que 1  
  
[...familia].length  
// también mayor que 1
```

Aunque el usuario ve:



como una sola unidad visual.

Por eso cortar strings arbitrariamente puede generar resultados bastante desagradables.

30. Intl.Segmenter

JavaScript moderno puede segmentar texto según reglas lingüísticas.

```
const segmenter =  
  new Intl.Segmenter("es", {  
    granularity: "grapheme"  
  });  
  
const texto = "Hola 👨‍👩‍👧";  
  
const partes =  
  [...segmenter.segment(texto)];  
  
console.log(partes);
```

Esto permite trabajar mucho mejor con lo que el usuario percibe como caracteres.

Es una distinción conceptual importante:

```
"😄".length  
  ↓  
unidades UTF-16  
  
[..."😄"].length  
  ↓  
code points  
  
Intl.Segmenter  
  ↓  
grafemas percibidos
```

31. Concatenación

La operación fundamental entre strings es unirlos:

```
const nombre = "Ana";

const mensaje =
  "Hola " + nombre;
```

El operador `+` está sobrecargado.

Puede significar suma:

```
2 + 3
// 5
```

o concatenación:

```
"2" + "3"
// "23"
```

Esto nos lleva directamente a una particularidad importante de JavaScript.

32. Conversión implícita a `string`

JavaScript tiene coerción automática de tipos.

```
"Edad: " + 25
// "Edad: 25"
```

El número `25` fue convertido implícitamente en:

```
"25"
```

Esto puede provocar resultados sorprendentes:

```
1 + 2
// 3
```

pero:

```
"1" + 2
// "12"
```

y:

```
1 + "2"  
// "12"
```

Una vez que `+` actúa como concatenación, los operandos correspondientes se convierten a texto.

Por eso:

```
1 + 2 + "3"
```

se evalúa de izquierda a derecha:

```
1 + 2  
↓  
3  
  
3 + "3"  
↓  
"33"
```

Resultado:

```
"33"
```

Mientras:

```
"1" + 2 + 3
```

produce:

```
"1" + 2  
↓  
"12"  
  
"12" + 3  
↓  
"123"
```

33. Conversión explícita

Podemos convertir explícitamente a string:

```
String(123)
// "123"

String(true)
// "true"

String(null)
// "null"

String(undefined)
// "undefined"
```

También:

```
const n = 123;

n.toString();
// "123"
```

Hay una diferencia práctica:

```
String(null)
// "null"
```

pero:

```
null.toString()
// TypeError
```

Por eso `String(valor)` es una conversión general más segura.

34. Conversión de números con distintas bases

`toString()` tiene una capacidad particularmente interesante:

```
const n = 42;

n.toString(10)
// "42"

n.toString(2)
// "101010"

n.toString(16)
// "2a"
```

Es decir:

```
número
  ↓
representación textual
```

Por ejemplo:

```
255..toString(16)
// "ff"
```

Esto es importante conceptualmente: `"101010"` no es un número binario almacenado misteriosamente de otra manera.

Es simplemente un **string que representa un número en base 2**.

35. Comparación lexicográfica

Volvamos a:

```
"casa" < "cosa"
// true
```

La idea es comparar posición por posición:

```
casa
cosa
|
c = c
```

Seguimos:

```
casa
cosa
|
a < 0
```

Y ya tenemos el resultado.

Esto es parecido a buscar palabras en un diccionario.

Pero la comparación básica de JavaScript utiliza valores UTF-16, no reglas lingüísticas de un idioma.

36. Una consecuencia muy importante

```
"10" < "2"  
// true
```

Esto suele sorprender al principio.

Pero estamos comparando **strings**, no números.

La primera posición es:

```
"10"  
"2"  
  
'1' < '2'
```

Por lo tanto:

```
"10" < "2"  
// true
```

En cambio:

```
10 < 2  
// false
```

Es una diferencia fundamental entre:

```
valor numérico
```

y:

representación textual del valor

37. El problema clásico al ordenar números almacenados como strings

Consideremos:

```
const valores = [  
  "1",  
  "2",  
  "10",  
  "20",  
  "3"  
];  
  
valores.sort();
```

Obtendremos algo parecido a:

```
[  
  "1",  
  "10",  
  "2",  
  "20",  
  "3"  
]
```

Perfectamente correcto **como texto**.

Incorrecto si pretendíamos interpretar esos strings como números.

Entonces:

```
valores.sort(  
  (a, b) => Number(a) - Number(b)  
);
```

produce el orden numérico esperado.

38. Comparación lingüística

Para personas queremos algo diferente.

Supongamos:

```
const palabras = [  
  "nube",  
  "Ñandú",  
  "árbol",  
  "zapato",  
  "avión"  
];
```

No queremos razonar en términos de posiciones Unicode.

Queremos decir:

"ordená esto como lo ordenaría alguien que habla español".

Ahí usamos:

```
palabras.sort(  
  (a, b) => a.localeCompare(b, "es")  
);
```

O mejor, si vamos a comparar muchas veces:

```
const español =  
  new Intl.Collator("es");  
  
palabras.sort(español.compare);
```

`Intl.Collator` encapsula las reglas de **collation**: ordenamiento lingüístico.

39. Comparaciones ignorando mayúsculas y acentos

Para determinadas búsquedas podemos querer considerar equivalentes:

ÁRBOL
árbol
Arbol
arbol

Podemos construir:

```
const comparador =  
  new Intl.Collator("es", {  
    sensitivity: "base"  
  });
```

Y comparar:

```
comparador.compare("árbol", "ARBOL")  
// 0
```

El `0` significa:

para los criterios de este comparador son equivalentes.

Esto suele ser mucho más correcto que hacer manualmente:

```
a.toLowerCase() === b.toLowerCase()
```

porque el tratamiento lingüístico de mayúsculas, acentos y alfabetos es más complejo de lo que parece.

40. Template literals: hay más que interpolación

Habíamos visto:

```
const nombre = "Ana";  
  
const mensaje =  
  `Hola ${nombre}`;
```

Pero `${}` acepta expresiones completas:

```
const precio = 100;
const cantidad = 3;

const mensaje =
  `Total: ${precio * cantidad}`;
```

Resultado:

```
Total: $300
```

También:

```
`${usuario.activo ? "Activo" : "Inactivo"}`
```

o:

```
`${productos.length} productos`
```

La idea fundamental es:

```
texto estático + expresión JS + texto estático
```

41. Tagged templates

Los template literals tienen una capacidad bastante más profunda:

```
html`<h1>${titulo}</h1>`
```

o:

```
sql`
  SELECT *
  FROM usuarios
  WHERE id = ${id}
`
```

Aquí `html` y `sql` pueden ser **funciones**.

Por ejemplo:

```
function mostrar(partes, valor) {  
    console.log(partes);  
    console.log(valor);  
}  
  
const nombre = "Ana";  
  
mostrar`Hola ${nombre}!`;
```

La función recibe separadamente:

```
["Hola ", "!"]  
  
y  
  
"Ana"
```

Esto permite procesar la interpolación antes de construir el resultado.

Es la base conceptual de APIs como:

```
html`...`  
sql`...`  
css`...`
```

y permite, por ejemplo, escapar valores, construir consultas parametrizadas o generar estructuras.

42. String.raw

Normalmente:

```
console.log("A\nB");
```

interpreta `\n` como salto de línea.

Con:

```
String.raw`A\nB`
```

obtenemos literalmente:

A\nB

Puede resultar útil trabajando con expresiones regulares, rutas y otros textos donde las barras invertidas tienen significado.

43. Strings y expresiones regulares

Los strings suelen procesarse mediante expresiones regulares.

Por ejemplo:

```
const texto = "Juan tiene 57 años";

texto.match(/\d+/);
// ["57"]
```

Reemplazar:

```
"hola    mundo".replace(/\s+/g, " ");
// "hola mundo"
```

Separar:

```
"uno, dos; tres".split(/[;,]/);
// ["uno", " dos", " tres"]
```

Y Unicode vuelve a aparecer aquí.

Las expresiones regulares modernas tienen el flag:

/u

para trabajar con semántica Unicode:

/🤪/u

y propiedades Unicode:

/\p{Letter}/u

Por ejemplo:

```
"ñ".match(/\p{Letter}/u);  
// ["ñ"]
```

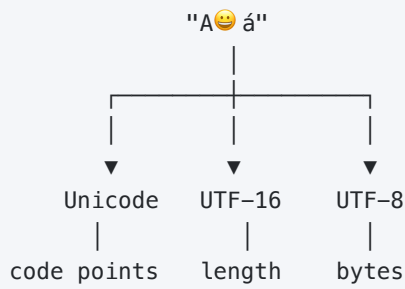
Esto evita pensar que "letra" significa simplemente `[A-Za-z]`, que en un mundo Unicode es una definición bastante pobre de letra.

44. Un experimento que resume casi todo

Este pequeño programa merece ser ejecutado:

```
const s = "A👉á";  
  
console.log("Texto:", s);  
  
console.log(  
  "length:",  
  s.length  
);  
  
console.log(  
  "Code points:",  
  [...s]  
);  
  
console.log(  
  "Unicode:",  
  [...s].map(  
    c =>  
      "U+" +  
      c.codePointAt(0)  
        .toString(16)  
        .toUpperCase()  
  )  
);  
  
console.log(  
  "UTF-8:",  
  [...new TextEncoder().encode(s)]  
);
```

Estamos observando **el mismo texto desde distintas capas**:



Y esa es probablemente la idea más importante de todo el tema:

Un string no es "una lista de bytes".

El texto tiene una representación abstracta mediante Unicode; JavaScript expone sus strings históricamente como unidades UTF-16, mientras que UTF-8 aparece normalmente cuando necesitamos convertir ese texto en bytes para almacenarlo o transmitirlo.

Entendiendo esa separación, dejan de parecer misteriosos `"😊".length === 2`, `TextEncoder`, `codePointAt()`, los problemas con acentos y buena parte de las rarezas del procesamiento de texto en JavaScript.

Expresiones regulares en JavaScript, desde primeros principios

Una **expresión regular** —*regular expression*, o **regex**— es una forma de describir **patrones de texto**.

La idea fundamental es esta:

En lugar de preguntar «¿este texto es exactamente "gato" ?», podemos preguntar «¿este texto tiene la forma que estoy buscando?».

Por ejemplo, queremos reconocer cualquiera de estos textos:

```
gato
gata
gatos
gatas
```

Podríamos escribir muchas comparaciones:

```
texto === "gato" ||
texto === "gata" ||
texto === "gatos" ||
texto === "gatas"
```

Pero también podemos describir el patrón:

```
/gat[oa]s?/
```

Eso ya es una expresión regular.

1. Antes de las regex: buscar texto literalmente

Supongamos:

```
const texto = "El gato duerme";
```

JavaScript ya puede buscar cadenas:

```
texto.includes("gato");
```

Resultado:

```
true
```

También podemos encontrar la posición:

```
texto.indexOf("gato");
```

Resultado:

```
3
```

Esto funciona perfectamente cuando conocemos **exactamente** lo que queremos buscar.

El problema aparece cuando queremos buscar una **forma**, no una cadena concreta.

Por ejemplo:

```
381-555-1234  
381-421-8877  
351-555-9999
```

Queremos reconocer:

```
tres dígitos  
un guion  
tres dígitos  
un guion  
cuatro dígitos
```

Ahí aparece la utilidad de las expresiones regulares.

2. Una regex es un objeto que describe un patrón

En JavaScript podemos escribir una expresión regular entre barras `/`.

```
const patron = /gato/;
```


Podemos probarla sobre un texto:

```
const patron = /gato/;  
  
console.log(patron.test("Tengo un gato"));
```

Resultado:

```
true
```

Pero:

```
patron.test("Tengo un perro");
```

da:

```
false
```

`test()` responde simplemente:

¿Existe alguna parte del texto que coincida con este patrón?

3. El caso más sencillo: caracteres literales

La regex:

```
/gato/
```

significa:

```
g  
seguido de a  
seguido de t  
seguido de o
```

Por eso encuentra:

```
/gato/.test("un gato negro");
```

pero no:

```
/gato/.test("una gata negra");
```

Hasta aquí no hemos ganado demasiado respecto de:

```
texto.includes("gato");
```

La potencia comienza cuando algunos caracteres dejan de significar literalmente lo que representan.

4. Los metacaracteres

En una expresión regular existen caracteres especiales llamados **metacaracteres**.

Algunos de los más importantes son:

```
.    cualquier carácter  
[]   conjunto de caracteres  
^    comienzo  
$    final  
*    cero o más  
+    uno o más  
?    cero o uno  
{ }  cantidad  
( )  grupo  
|    alternativa  
\\   escape
```

Vamos a construirlos uno por uno.

5. El punto `.`: cualquier carácter

La expresión:

```
/g.to/
```

significa:

```
g
cualquier carácter
t
o
```

Entonces coincide con:

```
gato
geto
gito
goto
guto
g1to
g-to
```

Por ejemplo:

```
/g.to/.test("gato");
```

da:

```
true
```

y:

```
/g.to/.test("g1to");
```

también:

```
true
```

Pero:

```
/g.to/.test("gto");
```

da:

```
false
```

porque el punto representa **exactamente un carácter**.

6. Conjuntos con `[]`

Muchas veces no queremos «cualquier carácter», sino uno dentro de ciertas posibilidades.

Por ejemplo:

```
/gat[oa]/
```

significa:

```
gat  
seguido de o o a
```

Entonces reconoce:

```
gato  
gata
```

Ejemplo:

```
/gat[oa]/.test("gata");
```

```
true
```

7. Rangos

Dentro de `[]` podemos especificar rangos.

Por ejemplo:

```
/[0-9]/
```

significa:

```
cualquier dígito entre 0 y 9.
```

Entonces:

```
/[0-9]/.test("abc7xyz");
```

da:

```
true
```

Podemos hacer lo mismo con letras:

```
/[a-z]/
```

una letra minúscula.

```
/[A-Z]/
```

una letra mayúscula.

Y podemos combinar:

```
/[a-zA-Z0-9]/
```

que significa:

```
una letra minúscula  
o una letra mayúscula  
o un número
```

8. Negar un conjunto

Dentro de `[]`, el símbolo `^` tiene un significado especial si aparece primero.

```
/[^0-9]/
```

significa:

```
cualquier carácter que no sea un dígito.
```

Por ejemplo:

```
/[^0-9]/.test("123A456");
```

da:

```
true
```

porque aparece `A`.

9. Clases abreviadas

Como ciertos patrones son extremadamente frecuentes, regex tiene abreviaturas.

Dígitos

```
\d
```

aproximadamente:

```
[0-9]
```

Ejemplo:

```
/\d/.test("Tengo 57 años");
```

```
true
```

No dígito

```
\D
```

significa:

```
cualquier carácter que no sea un dígito.
```

Caracteres de palabra

```
\w
```

tradicionalmente equivale aproximadamente a:

```
letras inglesas
dígitos
—
```

es decir:

```
A-Z
a-z
0-9
—
```

Ojo: para trabajar seriamente con Unicode, acentos, ñ, alfabetos no latinos, etc., conviene no asumir que `\w` significa «cualquier letra humana».

No carácter de palabra

```
\W
```

Espacios

```
\s
```

reconoce caracteres de espacio en blanco:

```
espacio
tabulación
salto de línea
...
```

Por ejemplo:

```
/\s/.test("Hola mundo");
```

da:

```
true
```

No espacio

\S

10. El problema de la cantidad

Hasta ahora:

/\d/

significa:

un dígito.

Pero un número como:

12345

tiene muchos dígitos.

Necesitamos poder decir:

repita este patrón cierta cantidad de veces.

Para eso existen los **cuantificadores**.

11. + : una o más veces

/\d+/

significa:

uno o más dígitos consecutivos.

Entonces encuentra:


```
1
42
123
987654321
```

Por ejemplo:

```
/\d+/.test("Tengo 57 años");
```

da:

```
true
```

Podemos extraer la coincidencia:

```
"Tengo 57 años".match(/\d+/);
```

obtendremos una coincidencia cuyo contenido principal es:

```
57
```

12. `*`: cero o más veces

```
/a*/
```

significa:

```
cero o más letras a.
```

Reconoce:

```
""
"a"
"aa"
"aaa"
"aaaaaaa"
```

La diferencia es importante:

a+ una o más
a* cero o más

Por eso * puede coincidir incluso sin consumir ningún carácter.

13. ?: cero o una vez

/a?/

significa:

la a puede aparecer o no.

Esto permite expresar elementos opcionales.

Por ejemplo:

/gatos?/

Hay que leerlo cuidadosamente.

El ? afecta solamente a la s.

Por tanto significa:

gato
seguido opcionalmente por s

Reconoce:

gato
gatos

No reconoce gata.

Para eso podríamos escribir:

/gat[oa]s?/

que reconoce:

gato
gata
gatos
gatas

14. `{n}` : exactamente cierta cantidad

Podemos indicar una cantidad precisa.

```
/\d{3}/
```

significa:

exactamente tres dígitos consecutivos.

Reconoce:

123
381
999

Entonces podemos empezar a describir nuestro teléfono:

```
/\d{3}-\d{3}-\d{4}/
```

Esto significa:

3 dígitos
-
3 dígitos
-
4 dígitos

Ejemplo:

```
const telefono = /\d{3}-\d{3}-\d{4}/;  
  
telefono.test("381-555-1234");
```

Resultado:

```
true
```

15. {mínimo,máximo}

También podemos escribir:

```
/\d{2,4}/
```

significa:

```
entre 2 y 4 dígitos.
```

Por ejemplo:

```
12  
123  
1234
```

También:

```
/\d{2,}/
```

significa:

```
al menos dos dígitos.
```

16. Hasta ahora buscamos «dentro» del texto

Consideremos:

```
/\d{3}/.test("abc123xyz");
```

Da:

```
true
```

Pero observá esto:

```
/\d{3}/.test("123456");
```

También da:

```
true
```

aunque quizás queríamos comprobar que **todo el texto** tuviera exactamente tres dígitos.

Para eso necesitamos los **anclajes**.

17. [^]: comienzo del texto

```
/^Hola/
```

significa:

```
Hola
```

 debe aparecer al comienzo.

Entonces:

```
/^Hola/.test("Hola mundo");
```

da:

```
true
```

pero:

```
/^Hola/.test("Te digo Hola");
```

da:

```
false
```

18. \$: final del texto

```
/mundo$/
```

significa:

```
mundo debe estar al final.
```

```
/mundo$/.test("Hola mundo");
```

```
true
```

pero:

```
/mundo$/.test("mundo cruel");
```

```
false
```

19. Validar un texto completo

Combinando ambos:

```
/^\d{3}$/
```

significa:

```
comienzo  
exactamente tres dígitos  
final
```

Por tanto:

```
/^\d{3}$/ .test("123");
```

da:

```
true
```

pero:

```
/^\d{3}$/.test("1234");
```

da:

```
false
```

Y:

```
/^\d{3}$/.test("abc123");
```

también:

```
false
```

Esta diferencia es fundamental:

```
/\d{3}/
```

busca tres dígitos **en alguna parte**.

Mientras que:

```
/^\d{3}$/
```

exige que **todo el texto** consista exactamente en tres dígitos.

20. Agrupar con ()

Supongamos que queremos reconocer:

```
ja  
jaja  
jajaja  
jajajaja
```

Podemos agrupar:

```
/(ja)+/
```

El cuantificador **+** ahora afecta al grupo completo:

```
(ja)
```

no solamente a la **a**.

Así:

```
/^ (ja)+ $/.test("jajaja");
```

da:

```
true
```

21. Alternativas con **|**

El símbolo:

```
|
```

significa **o**.

Por ejemplo:

```
/gato|perro/
```

significa:

```
gato  
o  
perro
```

Entonces:

```
/gato|perro/.test("Tengo un perro");
```


da:

```
true
```

Con grupos se vuelve mucho más útil:

```
/(gato|perro)s?/
```

reconoce:

```
gato  
gatos  
perro  
perros
```

22. Escapar caracteres especiales

El punto:

```
.
```

significa «cualquier carácter».

Si realmente queremos buscar un punto debemos **escaparlo**:

```
/\./
```

Por ejemplo:

```
/\./\.test("Hola.");
```

da:

```
true
```

El `\` dice:

```
interpretá el siguiente carácter literalmente.
```

Para buscar:

```
3.14
```

podemos usar:

```
/3\.14/
```

Sin el escape:

```
/3.14/
```

también coincidiría con:

```
3X14  
3-14  
3a14
```

porque `.` significa cualquier carácter.

23. Las expresiones regulares en JavaScript son objetos

Cuando escribimos:

```
const patron = /\d+/;
```

estamos creando un objeto de tipo:

```
RegExp
```

Podemos verificarlo:

```
console.log(typeof patron);
```

Resultado:

```
object
```

y:

```
patron instanceof RegExp;
```

da:

```
true
```

También podemos crearlo explícitamente:

```
const patron = new RegExp("\\d+");
```

Esto equivale aproximadamente a:

```
const patron = /\d+/;
```

Pero aparece una molestia: dentro de un string de JavaScript también debemos escapar `\`.

Por eso:

```
/\d+/
```

se convierte en:

```
new RegExp("\\d+")
```

Cuando el patrón es conocido de antemano, normalmente es más cómodo usar:

```
/.../
```

`new RegExp()` es especialmente útil cuando construimos el patrón dinámicamente.

Por ejemplo:

```
const palabra = "gato";  
  
const regex = new RegExp(palabra);
```

24. Flags: modificar cómo trabaja la regex

Después de la `/` final podemos colocar opciones llamadas **flags**.

Por ejemplo:

```
/gato/i
```

La `i` significa:

```
ignorar diferencias entre mayúsculas y minúsculas.
```

Entonces:

```
/gato/i.test("GAT0");
```

da:

```
true
```

También reconoce:

```
Gato  
gato  
GaTo
```

25. El flag `g`: búsqueda global

Normalmente una regex encuentra la primera coincidencia.

Por ejemplo:

```
"1 gato, 2 perros y 3 peces".match(/\d/);
```

encuentra inicialmente:

```
1
```

Pero con:

```
/\d/g
```

pedimos todas las coincidencias:

```
"1 gato, 2 perros y 3 peces".match(/\d/g);
```

Resultado:

```
["1", "2", "3"]
```

`g` significa **global**.

26. Métodos principales

En JavaScript, las regex aparecen principalmente mediante métodos de `RegExp` y de `String`.

Los más importantes son:

```
test()  
match()  
matchAll()  
search()  
replace()  
split()
```

27. `test()` : ¿existe una coincidencia?

```
const texto = "Tengo 42 años";  
  
console.log(/\d+/.test(texto));
```

Resultado:

```
true
```

Es ideal para validaciones sencillas.

28. `match()` : encontrar coincidencias

```
const texto = "Tengo 42 años";  
  
const resultado = texto.match(/\d+/);  
  
console.log(resultado);
```

La parte principal encontrada será:

```
42
```

Con `g`:

```
const texto = "Compré 3 panes, 2 leches y 10 huevos";  
  
console.log(texto.match(/\d+/g));
```

Resultado:

```
["3", "2", "10"]
```

29. `replace()` : buscar y reemplazar

Las regex son extremadamente útiles para transformar texto.

Por ejemplo:

```
const texto = "Juan    Pérez";  
  
const limpio = texto.replace(/\s+/g, " ");  
  
console.log(limpio);
```

Resultado:

```
Juan Pérez
```

La expresión:

```
/\s+/g
```

significa:

```
uno o más espacios  
en todas las posiciones
```

Y los sustituimos por un único espacio.

30. `split()` : separar utilizando un patrón

Normalmente hacemos:

```
"a,b,c".split(",");
```

Pero podemos separar utilizando una regex.

Por ejemplo, queremos aceptar espacios variables:

```
Juan,Pedro,Ana  
Juan, Pedro, Ana  
Juan , Pedro ,Ana
```

Podemos hacer:

```
const texto = "Juan , Pedro,  Ana";  
  
const nombres = texto.split(/\s*,\s*/);  
  
console.log(nombres);
```

Resultado:

```
["Juan", "Pedro", "Ana"]
```

La expresión:

```
\s*,\s*
```

significa:

cero o más espacios
,
cero o más espacios

31. Capturas

Los paréntesis no solamente agrupan. También **capturan** la parte del texto que coincidió.

Supongamos una fecha:

24/08/2026

Podemos describirla así:

`/(\d{2})\(/(\d{2})\(/(\d{4})/`

Tenemos tres grupos:

<code>(\d{2})</code>	día
<code>(\d{2})</code>	mes
<code>(\d{4})</code>	año

Ejemplo:

```
const fecha = "24/08/2026";

const resultado = fecha.match(/(\d{2})\(/(\d{2})\(/(\d{4})/);

console.log(resultado[0]);
console.log(resultado[1]);
console.log(resultado[2]);
console.log(resultado[3]);
```

Resultado:

24/08/2026
24
08
2026

Es decir:


```
resultado[0] = coincidencia completa
resultado[1] = primer grupo
resultado[2] = segundo grupo
resultado[3] = tercer grupo
```

32. Grupos con nombre

JavaScript moderno permite dar nombres a las capturas.

```
const regex =
  /(?!<dia>\d{2})\/(?!<mes>\d{2})\/(?!<anio>\d{4})/;
```

Entonces:

```
const resultado = "24/08/2026".match(regex);

console.log(resultado.groups.dia);
console.log(resultado.groups.mes);
console.log(resultado.groups.anio);
```

Resultado:

```
24
08
2026
```

Esto suele ser mucho más legible que trabajar con posiciones numéricas.

33. Un ejemplo completo: extraer información

Tenemos:

```
const texto = "Alumno: Juan Pérez – Legajo: 12345";
```

Queremos obtener el legajo.

Podemos buscar:

```
/Legajo:\s*(\d+)/
```

Descompongámoslo:

Legajo:	texto literal
\s*	cero o más espacios
(\d+)	uno o más dígitos, y además los capturamos

Entonces:

```
const texto = "Alumno: Juan Pérez - Legajo: 12345";  
  
const resultado = texto.match(/Legajo:\s*(\d+)/);  
  
console.log(resultado[1]);
```

Resultado:

12345

Eso muestra el uso conceptual más importante de regex:

convertir texto aparentemente desestructurado en datos estructurados.

34. Límites de palabra `\b`

Consideremos:

`/gato/`

También encuentra `gato` dentro de:

gatopardo

porque ahí aparecen esos cuatro caracteres.

Si queremos la palabra `gato`, podemos usar:

`/\bgato\b/`

`\b` significa aproximadamente:

límite de palabra.

Entonces:

```
/\bgato\b/.test("El gato duerme");
```

da:

true

mientras:

```
/\bgato\b/.test("El gatopardo duerme");
```

da:

false

35. Greedy: los cuantificadores son «codiciosos»

Supongamos:

```
const texto = "<b>hola</b><b>mundo</b>";
```

Queremos obtener el contenido entre `<b>` y `</b>`.

Podríamos intentar:

```
/<b>.*</b>/
```

Pero `.*` es **greedy**: intenta consumir todo lo posible.

Por eso puede capturar:

```
<b>hola</b><b>mundo</b>
```

en lugar de solamente:

```
<b>hola</b>
```

36. Cuantificadores no codiciosos

Podemos agregar `?` después del cuantificador:

```
.*?
```

Entonces:

```
/<b>.*?<\b>/
```

busca la coincidencia más corta posible.

Conceptualmente:

```
.*    todo lo posible  
.*?   lo mínimo posible
```

También existen:

```
+?  
*?  
{n,m}?
```

37. Unicode

JavaScript trabaja con Unicode, y las expresiones regulares modernas tienen soporte específico mediante el flag:

```
u
```

Por ejemplo:

```
/.u
```

maneja correctamente los puntos de código Unicode en situaciones donde una regex tradicional puede comportarse de forma inesperada con caracteres representados mediante pares sustitutos UTF-16.

Además existen las **Unicode Property Escapes**.

Por ejemplo:

```
/\p{Letter}/u
```

significa:

```
cualquier carácter Unicode cuya propiedad sea «letra».
```

Esto es mucho más apropiado para texto internacional que:

```
/[A-Za-z]/
```

Por ejemplo:

```
/\p{Letter}+/u.test("José");
```

```
true
```

Y podemos combinarlo con `g`:

```
const palabras = "José, María y Ñandú".match(/\p{Letter}+/gu);  
console.log(palabras);
```

38. Una regex se puede pensar como un pequeño lenguaje

Llegados a este punto, conviene cambiar la forma de pensar.

No hay que memorizar regex completas.

Hay que aprender a leerlas como un lenguaje.

Por ejemplo:

```
/^[A-Z][a-z]+$
```

podemos traducirla paso por paso:

```
^  
comienzo  
  
[A-Z]  
una mayúscula  
  
[a-z]+  
una o más minúsculas  
  
$  
final
```

Por tanto describe textos como:

```
Juan  
Pedro  
Maria
```

pero no:

```
juan  
JUAN  
Juan123
```

39. Construir una regex en lugar de adivinarla

Supongamos que necesitamos validar un código con esta forma:

```
ABC-1234
```

No intentemos escribir la regex completa mentalmente.

Partimos de la especificación.

Tres letras mayúsculas:

```
[A-Z]{3}
```

Después un guión:

```
[A-Z]{3}-
```

Después cuatro números:

```
[A-Z]{3}-\d{4}
```

Y queremos validar el texto completo:

```
 /^[A-Z]{3}-\d{4}$/
```

Probamos:

```
const codigo = /^[A-Z]{3}-\d{4}$/;  
  
console.log(codigo.test("ABC-1234")); // true  
console.log(codigo.test("AB-1234"));  // false  
console.log(codigo.test("abc-1234"));  // false
```

Esta es la manera correcta de trabajar:

```
problema → componentes → regex.
```

No al revés.

40. Un ejemplo más real: nombre y número

Supongamos que recibimos líneas:

```
Juan Perez,123  
Ana Gomez,456  
Pedro López,789
```

Queremos separar nombre y número.

Podemos usar:

```
 /^(.+),(\d+)$/
```

Traducimos:

```
^  
  
(.+)  
uno o más caracteres  
capturados  
  
,  
coma literal  
  
(\d+)  
uno o más números  
capturados  
  
$
```

Entonces:

```
const linea = "Juan Perez,123";  
  
const match = linea.match(/^(.+),(\d+)$/);  
  
console.log(match[1]);  
console.log(match[2]);
```

Resultado:

```
Juan Perez  
123
```

41. Pero regex no reemplaza a un parser

Esto es importante.

Las expresiones regulares son excelentes para estructuras relativamente simples:

```
números  
fechas  
códigos  
líneas  
identificadores  
fragmentos de texto  
separadores  
normalización
```


No son necesariamente la herramienta adecuada para analizar estructuras complejas como:

```
HTML completo
JavaScript
JSON
SQL
lenguajes de programación
```

Por ejemplo, para JSON:

```
JSON.parse(texto);
```

es muchísimo mejor que intentar construir una regex monstruosa.

Cuando una expresión regular comienza a parecer un conjuro para invocar a Cthulhu, probablemente sea momento de usar un parser.

42. La idea teórica detrás de una expresión regular

En términos conceptuales, una regex describe un **lenguaje**.

Por ejemplo:

```
/\d{3}/
```

describe el conjunto:

```
000
001
002
...
998
999
```

No enumera esas 1000 posibilidades.

Describe una **regla generadora** para todas ellas.

Podemos decir:

```
L = { todos los strings formados por exactamente tres dígitos }
```

La regex:

```
\d{3}
```

es una representación compacta de ese conjunto.

43. Regex y autómatas

Esta idea conecta con teoría de la computación.

Una expresión regular puede convertirse conceptualmente en una máquina que lee caracteres uno por uno.

Para:

```
/ab+c/
```

la máquina tendría estados similares a:

```
inicio
|
a
v
estado 1
|
b
v
estado 2
|\
| b
|/
|
c
v
aceptado
```

Reconoce:

```
abc
abbc
abbbc
abbbbbc
```

pero no:

```
ac
ab
bac
```

Esta máquina es un **autómata finito**.

De ahí viene el nombre «expresiones regulares»: originalmente describen precisamente los **lenguajes regulares**.

Los motores modernos de regex han agregado características que van más allá de las expresiones regulares matemáticas puras, pero la intuición sigue siendo excelente.

44. Resumen mental

No conviene aprender regex como una colección de jeroglíficos. Conviene recordar unas pocas ideas.

```
texto literal
gato
```

```
cualquier carácter
.
```

```
uno de estos caracteres
[abc]
```

```
un dígito
\d
```

```
un espacio
\s
```

```
repetición
*  cero o más
+  uno o más
?  cero o uno
```

```
cantidad
{3}
{2,5}
{2,}
```

```
posición
^   comienzo
$   final
\b  límite de palabra
```

```
estructura
() grupo
|  alternativa
```

Y en JavaScript:

```
regex.test(texto)
texto.match(regex)
texto.matchAll(regex)
texto.replace(regex, reemplazo)
texto.split(regex)
```

La habilidad importante no es memorizar una expresión monstruosa.

La habilidad importante es poder mirar una necesidad como:

```
«tres letras, después un guion, después cuatro números»
```

y construir tranquilamente:

```
/^[A-Z]{3}-\d{4}$/
```

Una regex no es más que eso: **una descripción compacta de qué secuencia de caracteres consideramos válida o interesante.**

Programación IV — Tipos compuestos en JavaScript

Arrays, objetos, `Map`, `Set`, referencias y recorridos

Los tipos primitivos permiten representar valores individuales: un número, una cadena, un booleano o una ausencia. Sin embargo, una aplicación rara vez trabaja con datos aislados. Un curso tiene alumnos, un carrito contiene productos y cada producto reúne un nombre, un precio y una categoría.

Para modelar esas relaciones necesitamos **tipos compuestos**: valores capaces de organizar otros valores. En JavaScript, los arrays y los objetos son las estructuras fundamentales. `Map` y `Set` agregan colecciones especializadas para asociaciones clave–valor y conjuntos sin repetidos.

Estas estructuras comparten una característica decisiva: son objetos y se manejan mediante **referencias**. Comprender esa idea permite explicar por qué dos variables pueden modificar la misma colección, por qué una copia con `...` solamente copia un nivel y por qué las actualizaciones inmutables construyen valores nuevos.

Al finalizar este apunte deberíamos poder:

- declarar arrays, acceder a sus elementos y extraer rangos;
- representar tablas mediante arrays de arrays y matrices;
- elegir entre `for`, `for...of`, `for...in` y los métodos de arrays;
- buscar, transformar, filtrar, ordenar y reducir colecciones;
- distinguir métodos mutables de operaciones inmutables;
- explicar identidad, asignación por referencia, copia superficial y copia profunda;
- desestructurar arrays y objetos;
- crear objetos mediante sintaxis literal y acceder a sus propiedades;
- utilizar propiedades abreviadas, métodos abreviados y claves calculadas;
- recorrer las propiedades propias de un objeto;
- utilizar `Map` para asociaciones dinámicas y `Set` para valores únicos.

1. De un dato individual a una estructura

Supongamos que queremos conservar las notas de un alumno. Con variables individuales podríamos escribir:

```
const nota1 = 8;
const nota2 = 6;
const nota3 = 9;
```

Esta representación funciona para tres notas conocidas de antemano, pero no expresa que forman una colección. Tampoco permite recorrerlas de manera general.

Un array reúne los valores bajo una misma estructura:

```
const notas = [8, 6, 9];
```

Ahora supongamos que queremos representar a un alumno. Un nombre, un legajo y una regularidad no son elementos equivalentes ni se identifican naturalmente por una posición. Un objeto permite nombrar cada parte:

```
const alumno = {
  nombre: "Ana Pérez",
  legajo: 12345,
  regular: true,
  notas: [8, 6, 9],
};
```

Las dos estructuras resuelven problemas diferentes:

- un **array** representa una secuencia ordenada de elementos;
- un **objeto** representa una entidad mediante propiedades con nombre.

Aunque en JavaScript los arrays son técnicamente objetos especializados, conviene elegir la estructura según el significado del dato y no según lo que la sintaxis permite hacer.

2. Arrays: secuencias ordenadas

Un array es una colección ordenada cuyos elementos se asocian con índices enteros que comienzan en cero.

```
const lenguajes = ["JavaScript", "TypeScript", "C#"];
```

La relación entre posiciones e índices es:

posición humana:	primera	segunda	tercera
índice:	0	1	2
valor:	"JavaScript"	"TypeScript"	"C#"

Que el primer índice sea cero no es una decisión estética. El índice expresa el desplazamiento desde el comienzo de la colección: el primer elemento se encuentra a cero posiciones del inicio.

Declaración mediante un literal

La forma habitual de crear un array es el literal delimitado por corchetes:

```
const vacio = [];  
const edades = [19, 22, 21];  
const materias = ["Programación IV", "Base de Datos II"];
```

JavaScript permite mezclar tipos:

```
const mezcla = [42, "hola", true, null];
```

Sin embargo, una colección suele ser más fácil de procesar cuando sus elementos cumplen el mismo contrato conceptual. Un array de notas debería contener notas; un array de alumnos debería contener objetos alumno.

Otras formas de creación

`Array.of()` construye un array con los argumentos recibidos:

```
Array.of(3);           // [3]  
Array.of(1, 2, 3);    // [1, 2, 3]
```

`Array.from()` crea un array a partir de un iterable o de un valor similar a un array:

```
Array.from("TUP");    // ["T", "U", "P"]  
  
Array.from({ length: 5 }, (_, indice) => indice);  
// [0, 1, 2, 3, 4]
```

El constructor `new Array()` existe, pero posee un caso que suele confundir:

```
new Array(3);         // array con longitud 3 y tres posiciones vacías  
Array.of(3);          // [3]
```

Por claridad, se prefieren `[]`, `Array.of()` y `Array.from()` según la intención.

length : cantidad de posiciones

La propiedad `length` indica la longitud:

```
const notas = [8, 6, 9];

console.log(notas.length); // 3
```

En un array normal y denso, los índices válidos van desde `0` hasta `length - 1`.

3. Acceso y modificación por índice

Los corchetes permiten leer una posición:

```
const materias = ["Programación IV", "Base de Datos II", "Inglés II"];

console.log(materias[0]); // "Programación IV"
console.log(materias[1]); // "Base de Datos II"
```

También permiten modificarla:

```
const estados = ["pendiente", "pendiente", "pendiente"];

estados[1] = "aprobado";

console.log(estados);
// ["pendiente", "aprobado", "pendiente"]
```

Si el índice no existe, la lectura devuelve `undefined`:

```
const notas = [8, 6, 9];

console.log(notas[10]); // undefined
```

Asignar muy lejos del final crea posiciones vacías, llamadas *holes*:

```
const valores = [10, 20];
valores[5] = 60;

console.log(valores.length); // 6
// [10, 20, <3 empty items>, 60]
```

Los arrays dispersos tienen comportamientos particulares al recorrerlos. En código de aplicación conviene evitarlos y agregar elementos mediante operaciones explícitas.

Índices calculados

El índice puede ser cualquier expresión que produzca la posición deseada:

```
const notas = [8, 6, 9];
const indice = 2;

console.log(notas[indice]);    // 9
console.log(notas[indice - 1]); // 6
```

Acceso desde el final con `at()`

La sintaxis de corchetes no interpreta índices negativos como posiciones desde el final:

```
const notas = [8, 6, 9];

notas[-1]; // undefined
```

El método `at()` sí admite índices negativos:

```
notas.at(0); // 8
notas.at(-1); // 9
notas.at(-2); // 6
```

La forma tradicional de obtener el último elemento es:

```
notas[notas.length - 1]; // 9
```

4. Rangos de elementos

JavaScript no posee una sintaxis especial como `array[inicio:fin]`. El método habitual para extraer un rango es `slice()`.

```
const letras = ["a", "b", "c", "d", "e"];

const centro = letras.slice(1, 4);

console.log(centro); // ["b", "c", "d"]
console.log(letras); // el original no cambia
```

El índice inicial se incluye y el final se excluye. Esta convención permite calcular la cantidad mediante `fin - inicio`.

```
slice(1, 4)
    ↑  ↑
  incluye  excluye
```

Variantes frecuentes:

```
const datos = [10, 20, 30, 40, 50];

datos.slice(2);      // [30, 40, 50]
datos.slice(0, 3);   // [10, 20, 30]
datos.slice(-2);     // [40, 50]
datos.slice(1, -1);  // [20, 30, 40]
datos.slice();       // copia superficial completa
```

No debe confundirse `slice()` con `splice()`:

- `slice(inicio, fin)` extrae una copia y no modifica el original;
- `splice(inicio, cantidad, ...elementos)` elimina o inserta modificando el array original;
- `toSpliced()` ofrece una versión moderna que devuelve un array nuevo.

```
const original = ["a", "b", "c", "d"];

const nuevo = original.toSpliced(1, 2, "x", "y");

console.log(nuevo);    // ["a", "x", "y", "d"]
console.log(original); // ["a", "b", "c", "d"]
```

5. Arrays de arrays y matrices

Un elemento de un array puede ser otro array:

```
const cursos = [
  ["Ana", "Luis", "Marta"],
  ["Pedro", "Julia"],
];
```

El primer índice selecciona el array exterior y el segundo selecciona un elemento interior:

```
console.log(cursos[0]);    // ["Ana", "Luis", "Marta"]
console.log(cursos[0][1]); // "Luis"
console.log(cursos[1][0]); // "Pedro"
```

Matrices

Una matriz puede representarse como un array de filas:

```
const matriz = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9],  
];
```

Se accede mediante `matriz[filas][columna]`:

```
matriz[0][0]; // 1  
matriz[1][2]; // 6  
matriz[2][1]; // 8
```

JavaScript no exige que todas las filas tengan la misma longitud. Una matriz rectangular es una convención que el programa debe mantener:

```
const irregular = [  
  [1, 2],  
  [3, 4, 5],  
];
```

Crear una matriz sin compartir filas

Esta forma parece crear tres filas independientes, pero no lo hace:

```
const incorrecta = Array(3).fill(Array(3).fill(0));  
  
incorrecta[0][0] = 9;  
  
console.log(incorrecta);  
// [[9, 0, 0], [9, 0, 0], [9, 0, 0]]
```

`fill()` colocó en cada posición una referencia al mismo array interior.

La forma correcta crea una fila nueva para cada posición:

```
const matriz = Array.from(
  { length: 3 },
  () => Array(3).fill(0)
);

matriz[0][0] = 9;

console.log(matriz);
// [[9, 0, 0], [0, 0, 0], [0, 0, 0]]
```

La función se ejecuta tres veces y construye tres arrays diferentes.

Recorrer una matriz

Cuando necesitamos índices de fila y columna, un `for` anidado resulta claro:

```
const matriz = [
  [1, 2, 3],
  [4, 5, 6],
];

for (let fila = 0; fila < matriz.length; fila += 1) {
  for (let columna = 0; columna < matriz[fila].length; columna += 1) {
    console.log(`matriz[${fila}][${columna}] = ${matriz[fila][columna]}`);
  }
}
```

Si solo importan los valores, puede utilizarse `for...of`:

```
for (const fila of matriz) {
  for (const valor of fila) {
    console.log(valor);
  }
}
```

Una transformación inmutable también puede anidar `map()`:

```
const duplicada = matriz.map((fila) =>
  fila.map((valor) => valor * 2)
);

// [[2, 4, 6], [8, 10, 12]]
```

6. Tres formas fundamentales de recorrer

6.1 `for` tradicional

El `for` tradicional reúne inicialización, condición y actualización:

```
const notas = [8, 6, 9];

for (let indice = 0; indice < notas.length; indice += 1) {
  console.log(indice, notas[indice]);
}
```

Es apropiado cuando necesitamos:

- el índice numérico;
- avanzar con un paso diferente de uno;
- recorrer hacia atrás;
- comparar con elementos vecinos;
- controlar con precisión el comienzo y el final.

```
for (let indice = notas.length - 1; indice >= 0; indice -= 1) {
  console.log(notas[indice]);
}
```

`break` termina el bucle y `continue` salta a la siguiente iteración:

```
for (let indice = 0; indice < notas.length; indice += 1) {
  if (notas[indice] < 6) {
    continue;
  }

  console.log(`Nota aprobada: ${notas[indice]}`);

  if (notas[indice] === 10) {
    break;
  }
}
```

6.2 `for...of`: recorrer valores

`for...of` obtiene los valores producidos por un objeto iterable. Arrays, cadenas, `Map` y `Set` son iterables.

```
const notas = [8, 6, 9];

for (const nota of notas) {
  console.log(nota);
}
```

Cuando no necesitamos controlar el índice, suele ser la opción imperativa más legible.

Si necesitamos índice y valor, podemos recorrer `entries()` y desestructurar cada par:

```
for (const [indice, nota] of notas.entries()) {
  console.log(`La nota ${indice + 1} es ${nota}`);
}
```

6.3 `for...in`: recorrer nombres de propiedades

`for...in` recorre claves de propiedades enumerables expresadas como cadenas. No recorre directamente los valores.

```
const alumno = {
  nombre: "Ana",
  legajo: 12345,
  regular: true,
};

for (const propiedad in alumno) {
  console.log(propiedad, alumno[propiedad]);
}
```

Además de propiedades propias, puede recorrer propiedades enumerables heredadas. Para limitarlo a las propiedades del objeto:

```
for (const propiedad in alumno) {
  if (Object.hasOwn(alumno, propiedad)) {
    console.log(propiedad, alumno[propiedad]);
  }
}
```

No se recomienda `for...in` para arrays:

```
const notas = [8, 6, 9];

for (const clave in notas) {
  console.log(typeof clave); // "string"
}
```

Puede incluir propiedades adicionales y entrega índices como cadenas. Para arrays se utilizan `for`, `for...of` o métodos como `forEach()`.

Comparación rápida

Construcción	Recorre	Uso típico
<code>for</code>	posiciones controladas por el programa	índices, pasos, recorrido inverso
<code>for...of</code>	valores de un iterable	arrays, <code>Map</code> , <code>Set</code> , cadenas
<code>for...in</code>	nombres de propiedades enumerables	inspección de objetos, con cuidado por la herencia

7. Operaciones frecuentes sobre arrays

Los métodos de arrays permiten expresar la intención de una operación: agregar, buscar, transformar, seleccionar o resumir.

Agregar y quitar elementos

Estos métodos modifican el array original:

```
const nombres = ["Ana", "Luis"];

nombres.push("Marta");    // agrega al final
nombres.pop();            // quita y devuelve el último
nombres.unshift("Pedro"); // agrega al comienzo
nombres.shift();          // quita y devuelve el primero
```

`push()` y `unshift()` devuelven la nueva longitud. `pop()` y `shift()` devuelven el elemento quitado o `undefined` si el array estaba vacío.

`splice()` permite quitar, insertar o reemplazar en cualquier posición, pero también muta:

```
const letras = ["a", "b", "d"];

letras.splice(2, 0, "c");

console.log(letras); // ["a", "b", "c", "d"]
```

Buscar y comprobar

```
const notas = [8, 6, 9, 6];

notas.includes(9);           // true
notas.indexOf(6);           // 1
notas.lastIndexOf(6);       // 3
notas.find((nota) => nota < 7); // 6
notas.findIndex((nota) => nota < 7); // 1
notas.some((nota) => nota === 10); // false
notas.every((nota) => nota >= 6); // true
```

Las intenciones son diferentes:

- `includes(valor)` pregunta si aparece ese valor;
- `indexOf(valor)` devuelve su primera posición o `-1`;
- `find(predicado)` devuelve el primer elemento que cumple una condición;
- `findIndex(predicado)` devuelve su índice;
- `some(predicado)` pregunta si al menos uno cumple;
- `every(predicado)` pregunta si todos cumplen.

`find()`, `some()` y `every()` pueden terminar antes de recorrer toda la colección cuando ya conocen el resultado.

Transformar con `map()`

`map()` aplica una función a cada elemento y produce un array nuevo con la misma cantidad de posiciones:

```
const precios = [1000, 2500, 800];

const preciosConIva = precios.map((precio) => precio * 1.21);

console.log(preciosConIva); // [1210, 3025, 968]
console.log(precios);      // [1000, 2500, 800]
```

La función recibe valor, índice y array:

```
const etiquetas = precios.map(
  (precio, indice) => `Producto ${indice + 1}: ${precio}`
);
```

Se utiliza `map()` cuando cada elemento de entrada produce un elemento de salida.

Seleccionar con `filter()`

`filter()` conserva únicamente los elementos para los cuales el predicado devuelve un valor `truthy`:

```
const notas = [8, 4, 6, 9, 3];

const aprobadas = notas.filter((nota) => nota >= 6);

console.log(aprobadas); // [8, 6, 9]
console.log(notas);     // no cambia
```

El resultado puede tener desde cero hasta la misma cantidad de elementos que el original.

Acumular con `reduce()`

`reduce()` combina todos los elementos para obtener un único resultado:

```
const precios = [1000, 2500, 800];

const total = precios.reduce(
  (acumulado, precio) => acumulado + precio,
  0
);

console.log(total); // 4300
```

Los argumentos importantes son:

```
reduce(función reductora, valor inicial)
```

En cada paso, la función recibe el acumulador y el elemento actual. Es recomendable proporcionar siempre un valor inicial: hace explícito el tipo del resultado y define qué ocurre con un array vacío.

`reduce()` no es solamente para sumar. Puede construir objetos, mapas o agrupaciones, aunque debe utilizarse cuando esa reducción siga siendo legible.

Ejecutar efectos con `forEach()`

`forEach()` ejecuta una función por cada elemento y devuelve `undefined`:

```
const alumnos = ["Ana", "Luis", "Marta"];

alumnos.forEach((alumno, indice) => {
  console.log(`${indice + 1}. ${alumno}`);
});
```

Es apropiado para efectos como mostrar, registrar o enviar información. Si queremos construir un array, `map()` o `filter()` comunican mejor la intención.

Un `return` dentro del callback no termina el `forEach()` completo. Cuando se necesita `break`, suele convenir un `for...of`.

Aplanar y combinar

```
const grupos = ["Ana", "Luis", "Marta", "Pedro"];

grupos.flat(); // ["Ana", "Luis", "Marta", "Pedro"]
```

`flat(profundidad)` reduce niveles de anidamiento:

```
[1, [2, [3]]].flat(1); // [1, 2, [3]]
[1, [2, [3]]].flat(2); // [1, 2, 3]
```

`flatMap()` equivale conceptualmente a transformar y luego aplanar un nivel:

```
const frases = ["hola mundo", "programación web"];

const palabras = frases.flatMap(frase => frase.split(" "));
// ["hola", "mundo", "programación", "web"]
```

`concat()` une arrays sin modificar los originales:

```
const frontend = ["HTML", "CSS"];
const programacion = frontend.concat(["JavaScript"]);

// ["HTML", "CSS", "JavaScript"]
```

La sintaxis de expansión ofrece otra forma:

```
const programacion = [...frontend, "JavaScript"];
```

Ordenar e invertir

`sort()` modifica el array y, sin comparador, ordena como cadenas:

```
const numeros = [10, 2, 1];

numeros.sort();
console.log(numeros); // [1, 10, 2]
```

Para números se proporciona un comparador:

```
numeros.sort((a, b) => a - b); // ascendente
numeros.sort((a, b) => b - a); // descendente
```

`toSorted()` devuelve una copia ordenada y conserva el original:

```
const originales = [10, 2, 1];
const ordenados = originales.toSorted((a, b) => a - b);

console.log(ordenados); // [1, 2, 10]
console.log(originales); // [10, 2, 1]
```

De manera similar:

- `reverse()` muta y `toReversed()` copia;
- `splice()` muta y `toSpliced()` copia;
- asignar `array[indice]` muta y `with(indice, valor)` copia.

```
const notas = [8, 6, 9];
const corregidas = notas.with(1, 7);

console.log(corregidas); // [8, 7, 9]
console.log(notas);      // [8, 6, 9]
```

8. Mutación e inmutabilidad

Una operación **mutable** cambia la estructura existente. Una operación **inmutable** conserva la anterior y produce una nueva.

```
const original = [1, 2, 3];

original.push(4); // mutación
```

```
const original = [1, 2, 3];
const nuevo = [...original, 4]; // actualización inmutable
```

Ningún enfoque es universalmente correcto. La mutación local puede ser simple y eficiente. La inmutabilidad resulta valiosa cuando:

- necesitamos conservar estados anteriores;
- varias partes del programa comparten una estructura;

- una interfaz detecta cambios comparando referencias;
- queremos que una función no produzca efectos sobre sus argumentos;
- buscamos facilitar razonamiento, pruebas y depuración.

Tabla práctica

Intención	Muta	Devuelve una estructura nueva
agregar al final	<code>push()</code>	<code>[...array, elemento]</code> , <code>concat()</code>
agregar al inicio	<code>unshift()</code>	<code>[elemento, ...array]</code>
quitar del final	<code>pop()</code>	<code>slice(0, -1)</code>
quitar del inicio	<code>shift()</code>	<code>slice(1)</code>
insertar o eliminar	<code>splice()</code>	<code>toSpliced()</code>
reemplazar por índice	<code>array[i] = valor</code>	<code>with(i, valor)</code> , <code>map()</code>
ordenar	<code>sort()</code>	<code>toSorted()</code>
invertir	<code>reverse()</code>	<code>toReversed()</code>
rellenar	<code>fill()</code>	crear con <code>map()</code> o <code>Array.from()</code>

Actualizar un elemento de manera inmutable

```
const productos = [
  { id: 1, nombre: "Teclado", stock: 5 },
  { id: 2, nombre: "Mouse", stock: 8 },
];

const actualizados = productos.map((producto) =>
  producto.id === 2
    ? { ...producto, stock: producto.stock - 1 }
    : producto
);
```

El array es nuevo. El producto actualizado también es un objeto nuevo. El producto que no cambió conserva su referencia, lo cual evita copiar trabajo innecesario.

9. Identidad y asignación por referencia

Los primitivos se comparan por su valor:

```
console.log(5 === 5);           // true
console.log("hola" === "hola"); // true
```

Los objetos y arrays poseen identidad. Dos estructuras separadas no son iguales por contener los mismos datos:

```
console.log([1, 2] === [1, 2]); // false
console.log({ id: 1 } === { id: 1 }); // false
```

Cada literal crea un objeto diferente.

Dos variables, una misma estructura

La asignación no copia el array. Copia la referencia:

```
const originales = [8, 6, 9];
const alias = originales;

alias.push(10);

console.log(originales); // [8, 6, 9, 10]
console.log(alias);      // [8, 6, 9, 10]
console.log(alias === originales); // true
```

No existen dos arrays sincronizados. Existe un solo array alcanzado mediante dos variables.

Copia superficial de un array

Podemos crear otro array mediante expansión, `slice()` o `Array.from()`:

```
const original = [1, 2, 3];

const copia1 = [...original];
const copia2 = original.slice();
const copia3 = Array.from(original);

console.log(copia1 === original); // false
```

Agregar a la copia no modifica el array exterior original:

```
copia1.push(4);

console.log(original); // [1, 2, 3]
console.log(copia1);   // [1, 2, 3, 4]
```

Sin embargo, esas operaciones realizan una **copia superficial**: copian los elementos del primer nivel, no duplican los objetos interiores.

```
const original = [
  { nombre: "Ana", nota: 8 },
  { nombre: "Luis", nota: 6 },
];

const copia = [...original];

copia[0].nota = 10;

console.log(original[0].nota); // 10
```

Los arrays exteriores son diferentes, pero `original[0]` y `copia[0]` apuntan al mismo objeto.

```
original → [ referencia A, referencia B ]
           |
copia    → [ referencia A, referencia B ]
           ↓
        { nombre: "Ana", nota: 10 }
```

Copia profunda

Una copia profunda crea estructuras nuevas también en los niveles interiores. Cuando los valores son compatibles con el algoritmo de clonación estructurada puede utilizarse `structuredClone()`:

```
const original = [
  { nombre: "Ana", notas: [8, 9] },
];

const copiaProfunda = structuredClone(original);

copiaProfunda[0].notas.push(10);

console.log(original[0].notas); // [8, 9]
console.log(copiaProfunda[0].notas); // [8, 9, 10]
```

`structuredClone()` no pertenece al núcleo de ECMAScript: es una API ofrecida por navegadores y otros entornos modernos. Acepta muchos tipos y referencias circulares, pero no puede clonar funciones, elementos del DOM ni cualquier valor arbitrario.

A veces se observa este atajo:

```
const copia = JSON.parse(JSON.stringify(original));
```

No es una solución general de clonación. JSON pierde o transforma valores como `undefined`, `Date`, `Map`, `Set` y `BigInt`, no admite funciones y falla con referencias circulares.

La mejor pregunta no siempre es "¿cómo copio todo?". Con frecuencia alcanza con copiar solamente las ramas que la actualización modifica.

10. Desestructuración de arrays

La desestructuración permite extraer posiciones y vincularlas con nombres:

```
const coordenadas = [-26.83, -65.20];
const [latitud, longitud] = coordenadas;

console.log(latitud); // -26.83
console.log(longitud); // -65.20
```

La correspondencia es posicional. El primer nombre recibe el índice `0`, el segundo el índice `1` y así sucesivamente.

Omitir posiciones

```
const colores = ["rojo", "verde", "azul"];
const [, segundo] = colores;

console.log(segundo); // "verde"
```

Valores predeterminados

El valor predeterminado se utiliza cuando la posición contiene `undefined` o no existe:

```
const [nombre = "Sin nombre", nota = 0] = ["Ana"];

console.log(nombre); // "Ana"
console.log(nota); // 0
```

No reemplaza `null`, `false`, `0` ni una cadena vacía.

El resto de los elementos

```
const [primero, ...restantes] = [10, 20, 30, 40];

console.log(primero);    // 10
console.log(restantes); // [20, 30, 40]
```

El elemento rest debe ser el último y siempre crea un array nuevo.

Intercambiar variables

```
let izquierda = "A";
let derecha = "B";

[izquierda, derecha] = [derecha, izquierda];

console.log(izquierda); // "B"
console.log(derecha);   // "A"
```

Desestructuración anidada

```
const matriz = [
  [1, 2],
  [3, 4],
];

const [[primero], [, ultimo]] = matriz;

console.log(primero); // 1
console.log(ultimo);  // 4
```

La desestructuración no vuelve profunda a una copia. Si extraemos un objeto o array interior, obtenemos una referencia a ese mismo valor.

11. Objetos: entidades con propiedades

Un objeto reúne asociaciones entre **claves** y **valores**. Cada asociación se denomina propiedad.

```
const producto = {
  id: 1,
  nombre: "Teclado",
  precio: 35_000,
  disponible: true,
};
```


Las claves de un objeto son cadenas o símbolos. Cuando escribimos `id` o `precio` sin comillas dentro del literal, JavaScript los interpreta como nombres de propiedades.

Los valores pueden pertenecer a cualquier tipo, incluidos otros objetos, arrays o funciones:

```
const alumno = {
  nombre: "Ana Pérez",
  contacto: {
    correo: "ana@example.com",
    telefono: null,
  },
  notas: [8, 6, 9],
  saludar() {
    return `Hola, soy ${this.nombre}`;
  },
};
```

Cuando el valor de una propiedad es una función, suele llamarse **método**.

En el ejemplo, una llamada como `alumno.saludar()` hace que `this` se refiera a `alumno`. El valor de `this` depende de la forma en que se invoca la función; ese mecanismo merece un desarrollo específico al estudiar funciones y objetos con mayor profundidad.

Crear, actualizar y eliminar propiedades

```
const alumno = {
  nombre: "Ana",
};

alumno.legajo = 12345; // agrega
alumno.nombre = "Ana Pérez"; // actualiza
delete alumno.legajo; // elimina
```

Que el objeto haya sido declarado con `const` no impide estas mutaciones. Impide asignar otro objeto a la variable `alumno`.

12. Acceso con punto y con corchetes

JavaScript ofrece dos sintaxis para acceder a propiedades.

Notación de punto

```
const producto = {  
  nombre: "Teclado",  
  precio: 35_000,  
};  
  
console.log(producto.nombre);  
console.log(producto.precio);
```

Después del punto se escribe directamente el nombre de la propiedad. Debe tener una forma válida como identificador.

La notación de punto suele preferirse cuando la clave es conocida y fija porque resulta breve y legible.

Notación de corchetes

```
console.log(producto["nombre"]);  
console.log(producto["precio"]);
```

Dentro de los corchetes se evalúa una expresión. Esto permite claves dinámicas:

```
const propiedadElegida = "precio";  
  
console.log(producto[propiedadElegida]); // 35000  
console.log(producto.propiedadElegida); // undefined
```

`producto.propiedadElegida` busca literalmente una propiedad llamada `"propiedadElegida"`.

`producto[propiedadElegida]` primero evalúa la variable y busca `"precio"`.

Los corchetes también son necesarios para claves que no son identificadores válidos:

```
const datos = {  
  "nombre-completo": "Ana Pérez",  
  "código postal": "4000",  
};  
  
datos["nombre-completo"];  
datos["código postal"];
```

Propiedades anidadas y encadenamiento opcional

```
const alumno = {  
  nombre: "Ana",  
  contacto: {  
    correo: "ana@example.com",  
  },  
};  
  
console.log(alumno.contacto.correo);
```

Si una parte puede faltar, el encadenamiento opcional evita intentar leer sobre `null` o `undefined`:

```
const telefono = alumno.contacto?.telefono ?? "Sin teléfono";
```

`?.` no reemplaza la validación. Expresa que la ausencia es una posibilidad admitida en ese acceso.

Comprobar existencia

```
"nombre" in alumno;           // true, incluye propiedades heredadas  
Object.hasOwn(alumno, "nombre"); // true, solo propiedad propia
```

Comparar con `undefined` no siempre distingue ausencia de una propiedad presente cuyo valor es precisamente `undefined`:

```
const objeto = { dato: undefined };  
  
objeto.dato === undefined;    // true  
Object.hasOwn(objeto, "dato"); // true
```

13. Formas abreviadas de escribir objetos

Propiedades abreviadas

Cuando una variable y una propiedad deben tener el mismo nombre, puede evitarse la repetición:

```
const nombre = "Ana";
const legajo = 12345;

const formaCompleta = {
  nombre: nombre,
  legajo: legajo,
};

const formaAbreviada = {
  nombre,
  legajo,
};
```

`{ nombre }` significa "crear una propiedad llamada `nombre` cuyo valor proviene de la variable `nombre`".

Es muy útil al construir respuestas o parámetros:

```
function crearAlumno(nombre, legajo) {
  return { nombre, legajo, regular: true };
}
```

Métodos abreviados

```
const contador = {
  valor: 0,

  incrementar() {
    this.valor += 1;
  },
};
```

La forma extensa sería `incrementar: function () { ... }`. La forma abreviada comunica con claridad que se trata de un método.

Claves calculadas

Dentro de un literal, los corchetes permiten calcular el nombre de una propiedad:

```
const campo = "correo";
const valor = "ana@example.com";

const contacto = {
  [campo]: valor,
};

console.log(contacto.correo); // "ana@example.com"
```

Las claves calculadas son útiles para formularios, agrupaciones y transformaciones dinámicas.

```
const indice = 2;

const etiquetas = {
  [`campo${indice}`]: "valor",
};

// { campo2: "valor" }
```

Expansión de propiedades

La sintaxis `...` copia propiedades enumerables propias dentro de otro objeto:

```
const producto = {
  id: 1,
  nombre: "Teclado",
  precio: 35_000,
};

const actualizado = {
  ...producto,
  precio: 32_000,
};
```

Si una propiedad aparece más de una vez, gana el último valor:

```
const a = { valor: 1, ...producto };
const b = { ...producto, precio: 32_000 };
```

El orden de las expansiones forma parte del significado.

14. Funciones frecuentes para objetos

`Object.keys()` devuelve las claves propias, enumerables y de tipo cadena:

```
const alumno = {
  nombre: "Ana",
  legajo: 12345,
  regular: true,
};

Object.keys(alumno);
// ["nombre", "legajo", "regular"]
```

`Object.values()` devuelve los valores:

```
Object.values(alumno);  
// ["Ana", 12345, true]
```

`Object.entries()` devuelve pares `[clave, valor]`:

```
Object.entries(alumno);  
// [["nombre", "Ana"], ["legajo", 12345], ["regular", true]]
```

Estos métodos permiten utilizar operaciones de arrays:

```
for (const [clave, valor] of Object.entries(alumno)) {  
  console.log(`${clave}: ${valor}`);  
}
```

`Object.fromEntries()` realiza la transformación inversa:

```
const pares = [  
  ["tema", "oscuro"],  
  ["idioma", "es"],  
];  
  
const configuracion = Object.fromEntries(pares);  
// { tema: "oscuro", idioma: "es" }
```

Combinadas, estas funciones permiten transformar propiedades de manera inmutable:

```
const precios = {  
  teclado: 35_000,  
  mouse: 18_000,  
};  
  
const preciosConIva = Object.fromEntries(  
  Object.entries(precios).map(([nombre, precio]) => [  
    nombre,  
    precio * 1.21,  
  ])  
);
```

Otras funciones importantes son:

```
Object.hasOwn(alumno, "nombre"); // comprueba propiedad propia

Object.assign({}, alumno, { regular: false });
// copia superficial y combina en el objeto destino

Object.freeze(alumno);
// impide cambios directos en las propiedades del primer nivel
```

`Object.freeze()` es superficial: no congela automáticamente objetos anidados.

Los símbolos no aparecen en `Object.keys()`, `Object.values()` ni `Object.entries()`. Pueden obtenerse mediante `Object.getOwnPropertySymbols()`.

15. Desestructuración de objetos

La desestructuración de objetos extrae propiedades por nombre, no por posición:

```
const producto = {
  id: 1,
  nombre: "Teclado",
  precio: 35_000,
};

const { nombre, precio } = producto;

console.log(nombre); // "Teclado"
console.log(precio); // 35000
```

El orden del patrón no importa:

```
const { precio, nombre } = producto;
```

Cambiar el nombre local

```
const { nombre: nombreProducto } = producto;

console.log(nombreProducto); // "Teclado"
```

La propiedad se llama `nombre`; la nueva variable se llama `nombreProducto`.

Valores predeterminados

```
const { stock = 0 } = producto;  
  
console.log(stock); // 0
```

El predeterminado se utiliza solamente si la propiedad vale `undefined` o no existe, no si contiene `null`, `false`, `0` o `""`.

Propiedades restantes

```
const { id, ...datosEditables } = producto;  
  
console.log(id); // 1  
console.log(datosEditables);  
// { nombre: "Teclado", precio: 35000 }
```

`datosEditables` es un objeto nuevo con las propiedades enumerables propias restantes. Sus valores interiores continúan siendo copias superficiales.

Desestructuración anidada

```
const alumno = {  
  nombre: "Ana",  
  contacto: {  
    correo: "ana@example.com",  
  },  
};  
  
const {  
  nombre,  
  contacto: { correo },  
} = alumno;
```

Este patrón crea `nombre` y `correo`, pero no crea una variable llamada `contacto`.

Parámetros desestructurados

Una función puede expresar directamente qué propiedades utiliza:

```
function mostrarProducto({ nombre, precio, disponible = true }) {  
  const estado = disponible ? "disponible" : "sin stock";  
  return `${nombre}: ${precio} - ${estado}`;  
}
```


Esto resulta claro para objetos pequeños y contratos conocidos. En patrones demasiado profundos puede ser preferible recibir el objeto completo y desestructurarlo en líneas separadas.

16. Referencias y copias de objetos

La misma regla vista en arrays se aplica a todos los objetos:

```
const original = { nombre: "Ana", regular: true };
const alias = original;

alias.regular = false;

console.log(original.regular); // false
console.log(alias === original); // true
```

La expansión crea una copia superficial:

```
const original = {
  nombre: "Ana",
  contacto: {
    correo: "ana@example.com",
  },
};

const copia = { ...original };

console.log(copia === original); // false
console.log(copia.contacto === original.contacto); // true
```

Modificar una propiedad anidada compartida afecta a ambas estructuras:

```
copia.contacto.correo = "nuevo@example.com";

console.log(original.contacto.correo); // "nuevo@example.com"
```

Para una actualización inmutable de esa rama:

```
const actualizado = {
  ...original,
  contacto: {
    ...original.contacto,
    correo: "nuevo@example.com",
  },
};
```

Se copian el objeto exterior y el objeto `contacto`, porque ambos cambian en el nuevo estado. No es necesario clonar propiedades que no se modifican.

17. ¿Objeto o `Map` ?

Un objeto puede utilizarse como un diccionario simple:

```
const preciosPorCodigo = {  
  A10: 1000,  
  B20: 2500,  
};
```

Pero los objetos también modelan entidades con propiedades conocidas. `Map` está diseñado específicamente para una colección dinámica de pares clave–valor.

Necesidad	Objeto	<code>Map</code>
modelar una entidad con campos conocidos	excelente	poco habitual
claves de tipo cadena o símbolo	sí	sí
claves de cualquier tipo	no	sí
cantidad directa	<code>Object.keys(obj).length</code>	<code>map.size</code>
agregar y quitar frecuentemente	posible	diseñado para ello
iteración directa	no es iterable por defecto	sí
conversión directa a JSON	natural para datos compatibles	requiere transformación

Como regla orientativa:

- usar objetos para registros como `alumno`, `producto` o `configuracion`;
- usar `Map` para asociaciones dinámicas donde las claves son datos.

18. `Map` : pares clave–valor

Un `Map` conserva pares clave–valor en orden de inserción. Cualquier valor, incluso un objeto, puede actuar como clave.

Crear y cargar

```
const stockPorCodigo = new Map();

stockPorCodigo.set("A10", 5);
stockPorCodigo.set("B20", 8);
```

`set()` agrega o reemplaza y devuelve el mismo mapa, por lo que puede encadenarse:

```
const stock = new Map()
  .set("A10", 5)
  .set("B20", 8);
```

También puede inicializarse con un iterable de pares:

```
const stock = new Map([
  ["A10", 5],
  ["B20", 8],
]);
```

Operaciones frecuentes

```
stock.get("A10");    // 5
stock.has("A10");    // true
stock.size;          // 2
stock.delete("B20"); // true si existía
stock.clear();        // elimina todos los pares
```

`get()` devuelve `undefined` cuando la clave no existe. Si `undefined` puede ser un valor almacenado, se utiliza `has()` para distinguir ambos casos.

Las claves se comparan por identidad cuando son objetos:

```
const clave = { id: 1 };
const mapa = new Map();

mapa.set(clave, "encontrado");

mapa.get(clave);    // "encontrado"
mapa.get({ id: 1 }); // undefined: es otro objeto
```

Recorrer un `Map`

Cada elemento producido por el iterador es un par `[clave, valor]`. La desestructuración resulta natural:

```
const precios = new Map([
  ["teclado", 35_000],
  ["mouse", 18_000],
]);

for (const [producto, precio] of precios) {
  console.log(`${producto}: ${precio}`);
}
```

También pueden recorrerse partes específicas:

```
for (const clave of precios.keys()) {
  console.log(clave);
}

for (const valor of precios.values()) {
  console.log(valor);
}

for (const entrada of precios.entries()) {
  console.log(entrada); // [clave, valor]
}
```

`forEach()` está disponible, aunque el orden de parámetros es `valor, clave`:

```
precios.forEach((precio, producto) => {
  console.log(`${producto}: ${precio}`);
});
```

Convertir entre `Map`, arrays y objetos

```
const entradas = [...precios];
// [["teclado", 35000], ["mouse", 18000]]

const comoObjeto = Object.fromEntries(precios);
// { teclado: 35000, mouse: 18000 }

const nuevamenteMapa = new Map(Object.entries(comoObjeto));
```

La conversión a objeto solo conserva fielmente claves que puedan expresarse adecuadamente como cadenas o símbolos del objeto.

Actualización inmutable de un `Map`

`set()` muta el mapa. Para conservar el anterior, se crea primero una copia superficial:

```
const original = new Map([
  ["A10", 5],
]);

const actualizado = new Map(original);
actualizado.set("A10", 4);

console.log(original.get("A10")); // 5
console.log(actualizado.get("A10")); // 4
```

Los valores interiores siguen compartiendo referencias si son objetos.

19. Set : colecciones sin repetidos

Un `Set` almacena valores únicos y conserva el orden en que fueron agregados por primera vez.

Crear y operar

```
const categorias = new Set();

categorias.add("hardware");
categorias.add("software");
categorias.add("hardware");

console.log(categorias.size); // 2
```

El segundo `"hardware"` no crea otra entrada.

Operaciones frecuentes:

```
categorias.has("hardware"); // true
categorias.delete("software"); // true si existía
categorias.clear(); // elimina todos los valores
```

`add()` devuelve el mismo conjunto y permite encadenar:

```
const permisos = new Set()
  .add("leer")
  .add("editar");
```

Eliminar duplicados de un array

```
const etiquetas = ["web", "js", "web", "backend", "js"];
const unicas = [...new Set(etiquetas)];

console.log(unicas); // ["web", "js", "backend"]
```

El `Set` elimina repeticiones y la expansión vuelve a producir un array.

Identidad de objetos

Dos objetos con los mismos datos siguen siendo valores diferentes:

```
const primero = { id: 1 };
const segundo = { id: 1 };

const conjunto = new Set([primero, segundo]);

console.log(conjunto.size); // 2
```

Agregar dos veces la misma referencia sí evita la repetición:

```
const conjunto = new Set([primero, primero]);
console.log(conjunto.size); // 1
```

Si buscamos unicidad por `id`, debemos guardar los identificadores o construir una asociación apropiada.

Recorrer un `Set`

```
const tecnologias = new Set(["HTML", "CSS", "JavaScript"]);

for (const tecnologia of tecnologias) {
  console.log(tecnologia);
}
```

También dispone de `values()`, `keys()`, `entries()` y `forEach()`. En un conjunto, `keys()` y `values()` producen los mismos valores por compatibilidad con la interfaz de `Map`.

Operaciones de conjuntos

Los entornos modernos incluyen operaciones matemáticas que producen nuevos conjuntos:

```
const frontend = new Set(["Ana", "Luis", "Marta"]);
const backend = new Set(["Luis", "Pedro"]);

frontend.union(backend);
// Set { "Ana", "Luis", "Marta", "Pedro" }

frontend.intersection(backend);
// Set { "Luis" }

frontend.difference(backend);
// Set { "Ana", "Marta" }

frontend.isSubsetOf(backend); // false
frontend.isDisjointFrom(backend); // false
```

Estas operaciones se incorporaron recientemente y requieren comprobar el entorno objetivo si se trabaja con navegadores antiguos.

Una intersección compatible con entornos anteriores puede expresarse mediante arrays:

```
const interseccion = new Set(
  [...frontend].filter((persona) => backend.has(persona))
);
```

Actualización inmutable de un `Set`

`add()`, `delete()` y `clear()` mutan. Para conservar el conjunto anterior:

```
const original = new Set(["leer"]);
const actualizado = new Set(original);

actualizado.add("editar");

console.log(original); // Set { "leer" }
console.log(actualizado); // Set { "leer", "editar" }
```

20. Elegir una estructura

Estructura	Pregunta que responde	Acceso principal
Array	"¿Qué valores aparecen en esta secuencia y en qué orden?"	índice numérico
Objeto	"¿Cuáles son las características de esta entidad?"	nombre de propiedad
<code>Map</code>	"¿Qué valor está asociado dinámicamente con esta clave?"	clave de cualquier tipo
<code>Set</code>	"¿Este valor pertenece al conjunto?"	valor único

Ejemplos:

```
// Secuencia ordenada
const notas = [8, 6, 9];

// Entidad con campos conocidos
const alumno = {
  legajo: 12345,
  nombre: "Ana",
};

// Asociación dinámica por identificador
const alumnosPorLegajo = new Map([
  [12345, alumno],
]);

// Pertenencia sin duplicados
const permisos = new Set(["leer", "editar"]);
```

No existe una estructura mejor en abstracto. Existe una estructura que expresa mejor la relación que necesita el dominio.

21. Caso integrador: un carrito de compras

Un catálogo puede representarse como un array de objetos:

```
const productos = [
  { id: 1, nombre: "Teclado", precio: 35_000, categoria: "hardware" },
  { id: 2, nombre: "Mouse", precio: 18_000, categoria: "hardware" },
  { id: 3, nombre: "Curso JS", precio: 25_000, categoria: "software" },
];
```

Un `Map` permite encontrar rápidamente un producto por identificador:

```
const productosPorId = new Map(
  productos.map((producto) => [producto.id, producto])
);
```

Un `Set` permite obtener categorías únicas:


```
const categorias = new Set(
  productos.map((producto) => producto.categoria)
);

console.log([...categorias]);
// ["hardware", "software"]
```

El carrito puede almacenar líneas con producto y cantidad:

```
const carrito = [
  { productId: 1, cantidad: 2 },
  { productId: 3, cantidad: 1 },
];
```

Podemos construir el detalle mediante `map()`, desestructuración y acceso al mapa:

```
const detalle = carrito.map(({ productId, cantidad }) => {
  const producto = productosPorId.get(productId);

  if (!producto) {
    throw new Error(`No existe el producto ${productId}`);
  }

  const { nombre, precio } = producto;

  return {
    productId,
    nombre,
    precioUnitario: precio,
    cantidad,
    subtotal: precio * cantidad,
  };
});
```

Y calcular el total mediante `reduce()`:

```
const total = detalle.reduce(
  (acumulado, linea) => acumulado + linea.subtotal,
  0
);
```

La actualización inmutable de una cantidad puede hacerse con `map()`:

```
const carritoActualizado = carrito.map((linea) =>
  linea.productoId === 1
    ? { ...linea, cantidad: linea.cantidad + 1 }
    : linea
);
```

Este ejemplo combina responsabilidades claras:

- array para secuencias de productos y líneas;
- objeto para describir cada entidad;
- `Map` para localizar por clave;
- `Set` para valores únicos;
- desestructuración para extraer datos;
- `map()` para transformar;
- `reduce()` para agregar;
- expansión para actualizar sin mutar.

22. Errores frecuentes y su explicación

Esperar que la asignación copie

```
const copia = original;
```

La asignación crea otra referencia al mismo objeto. No crea una estructura independiente.

Esperar una copia profunda de `...`

```
const copia = { ...original };
```

La expansión copia un nivel. Los objetos interiores continúan compartidos.

Crear una matriz con filas compartidas

```
Array(3).fill(Array(3).fill(0));
```

Cada posición exterior recibe la misma referencia. Debe crearse cada fila con una función.

Confundir `slice()` con `splice()`

- `slice()` copia un rango;
- `splice()` modifica el array;
- `toSpliced()` devuelve una versión modificada sin alterar el original.

Ordenar números sin comparador

```
[10, 2, 1].sort(); // [1, 10, 2]
```

El orden predeterminado compara representaciones textuales. Para números se utiliza `(a, b) => a - b`.

Utilizar `map()` solamente para efectos

```
alumnos.map((alumno) => console.log(alumno));
```

Esto crea un array de `undefined` que no se utiliza. Para efectos corresponde `forEach()` o `for...of`.

Esperar que `forEach()` admita `break`

Un `return` termina solamente el callback actual. Si se necesita detener el recorrido se utiliza `for...of`, `find()`, `some()` o `every()`, según la intención.

Utilizar `for...in` para valores de un array

`for...in` entrega nombres de propiedades como cadenas y puede incluir propiedades no indexadas o heredadas. Para valores de un array se utiliza `for...of`.

Confundir una clave dinámica con su nombre literal

```
const clave = "precio";

producto.clave; // busca "clave"
producto[clave]; // busca "precio"
```

Crear que `Set` elimina objetos con datos repetidos

```
new Set([ { id: 1 }, { id: 1 } ]).size; // 2
```

Los objetos se distinguen por identidad. Para unicidad por `id`, se almacenan identificadores o se construye un `Map`.

23. Ideas que conviene conservar

1. Un array representa una secuencia ordenada y utiliza índices desde cero.
 2. `length` indica la cantidad de posiciones; el último índice habitual es `length - 1`.
 3. `slice()` copia un rango incluyendo el inicio y excluyendo el final.
 4. Una matriz es un array de arrays y se accede como `matriz[fila][columna]`.
 5. `Array.fill()` repite la misma referencia cuando recibe un objeto.
 6. `for` ofrece control explícito del índice; `for...of` recorre valores; `for...in` recorre nombres de propiedades.
 7. `map()` transforma, `filter()` selecciona y `reduce()` acumula.
 8. `forEach()` se utiliza para efectos y devuelve `undefined`.
 9. Algunos métodos mutan; otros producen estructuras nuevas. El nombre del método no siempre permite adivinarlo.
 10. Arrays y objetos se comparan por identidad, no por contenido.
 11. Una asignación copia una referencia; no duplica la estructura.
 12. La expansión, `slice()`, `Array.from()` y `Object.assign()` realizan copias superficiales.
 13. La desestructuración de arrays es posicional; la de objetos utiliza nombres de propiedades.
 14. La notación de punto usa una clave fija; los corchetes permiten una expresión dinámica.
 15. `{ nombre }` abrevia `{ nombre: nombre }`.
 16. `{ [expresion]: valor }` crea una clave calculada.
 17. `Object.keys()`, `Object.values()` y `Object.entries()` trabajan con propiedades propias, enumerables y de clave cadena.
 18. `Map` modela asociaciones dinámicas y admite claves de cualquier tipo.
 19. `Set` modela pertenencia y conserva valores únicos por identidad.
 20. Las actualizaciones inmutables copian la estructura exterior y las ramas interiores que cambian.
-

24. Preguntas de repaso

1. ¿Por qué el primer elemento de un array posee índice cero?
2. ¿Qué diferencia existe entre `array[-1]` y `array.at(-1)`?
3. ¿Qué elementos devuelve `slice(1, 4)`?
4. ¿En qué se diferencian `slice()`, `splice()` y `toSpliced()`?
5. ¿Cómo se representa y accede una matriz en JavaScript?
6. ¿Por qué `Array(3).fill([])` comparte el mismo array interior?

7. ¿Cuándo conviene un `for` tradicional en lugar de `for...of`?
8. ¿Qué recorre realmente `for...in`?
9. ¿Qué diferencias conceptuales existen entre `map()`, `filter()` y `reduce()`?
10. ¿Por qué `forEach()` no reemplaza siempre a un bucle?
11. ¿Qué métodos de ordenamiento y reemplazo mutan el array?
12. ¿Qué significa que dos variables compartan una referencia?
13. ¿Por qué `[1, 2] === [1, 2]` devuelve `false`?
14. ¿Qué diferencia hay entre copia superficial y profunda?
15. ¿Cómo se extrae el primer elemento y se agrupan los restantes mediante desestructuración?
16. ¿Cuándo debe utilizarse `objeto[clave]` en lugar de `objeto.clave`?
17. ¿Qué significa la abreviatura `{ nombre, edad }`?
18. ¿Cómo se cambia el nombre de una variable al desestructurar un objeto?
19. ¿Qué diferencia existe entre el operador `in` y `Object.hasOwnProperty()`?
20. ¿Cuándo conviene utilizar `Map` en lugar de un objeto?
21. ¿Cómo compara `Map` una clave que es un objeto?
22. ¿Cómo puede eliminarse la repetición de valores primitivos mediante `Set`?

Ejercicio 1: procesamiento de notas

Dado el array:

```
const notas = [8, 4, 10, 6, 3, 9];
```

Obtener sin modificarlo:

1. las notas aprobadas;
2. todas las notas incrementadas en un punto, con máximo `10`;
3. el promedio;
4. la primera nota desaprobada;
5. si existe al menos un `10`;
6. una copia ordenada de mayor a menor.

Ejercicio 2: matriz

Crear una matriz de `4 × 4` inicializada en cero sin compartir filas. Luego construir otra matriz, sin modificar la primera, que tenga `1` en la diagonal principal.

Ejercicio 3: actualización de objetos

Dado:

```
const alumno = {
  legajo: 12345,
  nombre: "Ana",
  contacto: {
    correo: "ana@example.com",
    telefono: null,
  },
  notas: [8, 6, 9],
};
```

Crear un nuevo objeto que:

1. conserve el original sin cambios;
2. actualice el correo;
3. agregue una nota `10`;
4. mantenga el mismo legajo y nombre;
5. permita verificar qué referencias fueron copiadas y cuáles se conservaron.

Ejercicio 4: índice y categorías

A partir de un array de productos:

1. crear un `Map` que los indexe por `id`;
2. crear un `Set` de categorías únicas;
3. recorrer el mapa mostrando `id`, nombre y precio;
4. convertir el mapa nuevamente en un array de productos;
5. explicar por qué dos objetos con el mismo `id` pueden ser elementos distintos de un `Set`.

Fuentes y lecturas recomendadas

- Colecciones indexadas y arrays — MDN
- Referencia de `Array` — MDN
- Desestructuración — MDN
- Bucles e iteración — MDN
- `for...of` — MDN
- `for...in` — MDN
- Trabajo con objetos — MDN
- Inicializadores de objetos — MDN
- Sintaxis de expansión — MDN
- Referencia de `Map` — MDN
- Referencia de `Set` — MDN
- Clonación estructurada — MDN

- ECMAScript Language Specification — TC39

Programación IV — Estructuras de control en JavaScript

Decisiones, bucles, `switch` y excepciones

Un programa sencillo ejecuta sus instrucciones en el orden en que fueron escritas. Una aplicación real necesita algo más: elegir caminos, repetir tareas, abandonar una operación o transferir el control cuando ocurre un error.

El **flujo de control** es el orden en el que se ejecutan las instrucciones. Las estructuras de control permiten modificar ese orden de manera explícita:

- `if`, `else` y `switch` seleccionan caminos;
- `while`, `do...while` y `for` repiten instrucciones;
- `break` y `continue` alteran una repetición;
- `return` termina una función;
- `throw` interrumpe el flujo normal y propaga una excepción.

En esta clase estudiaremos cada estructura a partir de la pregunta que responde. El objetivo no es solamente aprender la sintaxis, sino poder elegir la forma que haga visible la regla del problema.

Al finalizar este apunte deberíamos poder:

- escribir condiciones simples, encadenadas y anidadas;
- simplificar decisiones mediante expresiones booleanas y guardas;
- distinguir una sentencia `if` del operador ternario;
- construir bucles con `while`, `do...while` y `for`;
- anticipar el orden de evaluación de un bucle;
- utilizar `break` y `continue` sin ocultar la lógica principal;
- explicar el *fall-through* de un `switch`;
- elegir entre `for`, `for...of` y `for...in`;
- lanzar, propagar, capturar y enriquecer excepciones;
- reconocer cuándo una excepción es apropiada y cuándo conviene una decisión normal.

1. El flujo normal y sus desvíos

En ausencia de estructuras de control, JavaScript ejecuta una sentencia y continúa con la siguiente:


```
const precio = 1000;
const cantidad = 3;
const total = precio * cantidad;

console.log(total);
```

Podemos representar ese flujo como una línea:

```
instrucción 1 → instrucción 2 → instrucción 3 → fin
```

Una condición agrega una bifurcación:

```

inicio → decisión {
  | condición verdadera → camino A |
  | condición falsa → camino B |
} → continúa

```

Un bucle agrega un regreso controlado:

```

      | condición verdadera ←
      |
inicio → condición → cuerpo → actualización →
      |
      | condición falsa → continúa

```

Una excepción produce una salida abrupta y busca un manejador:

```

función C — throw
  ↑ descarta su ejecución pendiente
función B
  ↑ propaga
función A — catch → maneja o vuelve a lanzar

```

Todas estas construcciones responden a la misma pregunta: **¿cuál es la próxima instrucción que debe ejecutarse?**

2. `if` : ejecutar cuando se cumple una condición

La forma básica es:

```
if (condicion) {
  // se ejecuta si la condición es truthy
}
```

Ejemplo:

```
const edad = 20;

if (edad >= 18) {
  console.log("Puede registrarse");
}
```

La condición puede ser cualquier expresión. JavaScript la interpreta en un contexto booleano. Si es *truthy*, ejecuta el bloque; si es *falsy*, lo omite.

```
const nombre = "Ana";

if (nombre) {
  console.log(`Hola, ${nombre}`);
}
```

Aunque las llaves son opcionales para una sola sentencia, utilizarlas siempre evita ambigüedades al modificar el código:

```
if (edad >= 18) {
  console.log("Puede registrarse");
}
```

Condiciones expresivas

Una condición debería leerse como una pregunta del dominio:

```
const tieneEdadPermitida = edad >= 18;
const tieneDocumento = documento !== null;
const puedeRegistrarse = tieneEdadPermitida && tieneDocumento;

if (puedeRegistrarse) {
  registrarUsuario();
}
```

Nombrar una condición compleja puede comunicar mejor la regla y permite depurar cada parte.

No es necesario comparar un booleano con `true`:

```

if (estaActivo) {
  // ...
}

// Evitar si no existe una razón particular:
if (estaActivo === true) {
  // ...
}

```

La comparación explícita sí puede tener sentido cuando queremos distinguir el booleano `true` de otros valores truthy.

3. `else` : elegir entre dos caminos

`else` define qué hacer cuando la condición del `if` es falsy:

```

if (saldo >= importe) {
  console.log("Compra aprobada");
} else {
  console.log("Saldo insuficiente");
}

```

Solamente uno de los dos bloques se ejecuta.

```

saldo >= importe
├─ true  → compra aprobada
└─ false → saldo insuficiente

```

`else` no lleva una condición propia. Representa todos los casos que no fueron aceptados por el `if` anterior.

El operador condicional o ternario

Cuando la decisión produce uno de dos valores, el operador ternario puede ser más directo:

```
const estado = nota >= 6 ? "aprobado" : "desaprobado";
```

Su forma es:

```
condición ? valorSiVerdadera : valorSiFalsa
```

La versión con `if...else` necesita una variable reasignable:

```
let estado;

if (nota >= 6) {
  estado = "aprobado";
} else {
  estado = "desaprobado";
}
```

El ternario es una **expresión**: produce un valor. `if` es una **sentencia**: controla qué bloque se ejecuta.

Conviene utilizar el ternario para una elección breve. Los ternarios anidados suelen esconder la jerarquía de condiciones:

```
// Difícil de leer
const categoria = nota >= 9
  ? "destacado"
  : nota >= 6
    ? "aprobado"
    : "desaprobado";
```

En ese caso, un encadenamiento o una función con guardas resulta más claro.

4. Encadenamiento con `else if`

Cuando existen más de dos alternativas excluyentes, las condiciones pueden evaluarse en secuencia:

```
function clasificarNota(nota) {
  if (nota >= 9) {
    return "destacado";
  } else if (nota >= 6) {
    return "aprobado";
  } else if (nota >= 4) {
    return "recupera";
  } else {
    return "desaprobado";
  }
}
```

JavaScript prueba las condiciones de arriba hacia abajo y ejecuta solamente el primer bloque cuya condición sea `truthy`.

El orden forma parte del significado. Esta versión está mal ordenada:

```
if (nota >= 6) {  
    return "aprobado";  
} else if (nota >= 9) {  
    return "destacado"; // nunca se alcanza  
}
```

Toda nota mayor o igual que nueve también es mayor o igual que seis. El primer caso la captura antes.

Una cadena `else if` resulta apropiada cuando:

- las alternativas son mutuamente excluyentes;
- existe una prioridad clara;
- solamente debe ejecutarse una rama.

Si varias condiciones independientes pueden cumplirse, deben utilizarse varios `if`:

```
if (tieneErrores) {  
    mostrarErrores();  
}  
  
if (tieneAdvertencias) {  
    mostrarAdvertencias();  
}
```

5. Condiciones anidadas

Un `if` puede aparecer dentro de otro:

```
if (usuario !== null) {  
    if (usuario.estaActivo) {  
        if (usuario.esAdministrador) {  
            mostrarPanelDeAdministracion();  
        }  
    }  
}
```

El anidamiento expresa que una pregunta solamente tiene sentido después de responder otra. No es incorrecto por sí mismo, pero demasiados niveles desplazan la lógica principal hacia la derecha y obligan a recordar muchas condiciones.

Combinar condiciones

Si las condiciones describen una misma decisión, pueden combinarse:

```
if (
    usuario !== null &&
    usuario.estaActivo &&
    usuario.esAdministrador
) {
    mostrarPanelDeAdministracion();
}
```

El cortocircuito de `&&` evita acceder a `usuario.estaActivo` cuando `usuario` es `null`.

Con encadenamiento opcional:

```
if (usuario?.estaActivo && usuario.esAdministrador) {
    mostrarPanelDeAdministracion();
}
```

No todo anidamiento puede reemplazarse mecánicamente por `&&`. Si cada nivel tiene un `else` diferente, las ramas representan decisiones distintas y deben conservarse o reorganizarse con cuidado.

Leyes de De Morgan

Estas equivalencias ayudan a simplificar negaciones:

```
!(a && b) equivale a !a || !b
!(a || b) equivale a !a && !b
```

Ejemplo:

```
if (!(esDocente || esAlumno)) {
    rechazarAcceso();
}

// Equivale a:
if (!esDocente && !esAlumno) {
    rechazarAcceso();
}
```

La mejor versión es la que expresa con mayor claridad la regla del dominio.

6. Guardas y retorno temprano

Una **guarda** verifica al comienzo una condición que impide continuar. Si se cumple, la función termina temprano.

Versión anidada:

```
function procesarCompra(usuario, carrito) {
  if (usuario !== null) {
    if (usuario.estaActivo) {
      if (carrito.length > 0) {
        return confirmarCompra(usuario, carrito);
      } else {
        return { ok: false, motivo: "carrito vacío" };
      }
    } else {
      return { ok: false, motivo: "usuario inactivo" };
    }
  } else {
    return { ok: false, motivo: "falta usuario" };
  }
}
```

Versión con guardas:

```
function procesarCompra(usuario, carrito) {
  if (usuario === null) {
    return { ok: false, motivo: "falta usuario" };
  }

  if (!usuario.estaActivo) {
    return { ok: false, motivo: "usuario inactivo" };
  }

  if (carrito.length === 0) {
    return { ok: false, motivo: "carrito vacío" };
  }

  return confirmarCompra(usuario, carrito);
}
```

Las guardas dejan el camino principal al nivel izquierdo y separan cada impedimento.

Retornar o lanzar

Una guarda puede devolver un resultado esperado:

```
if (carrito.length === 0) {
  return { ok: false, motivo: "carrito vacío" };
}
```

También puede lanzar una excepción si continuar violaría el contrato de la función:

```
function calcularPromedio(notas) {
  if (!Array.isArray(notas)) {
    throw new TypeError("notas debe ser un array");
  }

  if (notas.length === 0) {
    throw new RangeError("no se puede promediar un array vacío");
  }

  return notas.reduce((suma, nota) => suma + nota, 0) / notas.length;
}
```

La elección depende del contrato: un resultado de negocio previsible suele devolverse; una imposibilidad para cumplir la operación puede representarse mediante una excepción.

7. **while** : repetir mientras una condición sea verdadera

La sintaxis es:

```
while (condicion) {
  // cuerpo
}
```

while evalúa la condición antes de cada repetición. Si inicialmente es falsy, el cuerpo no se ejecuta ninguna vez.

```
let numero = 1;

while (numero <= 5) {
  console.log(numero);
  numero += 1;
}
```

Orden de ejecución:

1. evaluar condición
2. si es falsa, terminar
3. ejecutar cuerpo
4. volver al paso 1

El progreso debe ser visible

Todo bucle necesita una razón para terminar. En el ejemplo, `numero += 1` acerca el estado a la condición de salida.


```
let numero = 1;

while (numero <= 5) {
  console.log(numero);
  // Falta actualizar numero: bucle infinito
}
```

No todos los bucles avanzan mediante un contador. También pueden consumir una colección o cambiar un estado:

```
const tareas = ["validar", "guardar", "notificar"];

while (tareas.length > 0) {
  const tarea = tareas.shift();
  console.log(`Procesando: ${tarea}`);
}
```

Esta versión muta la cola. Si la colección original debe conservarse, puede trabajarse sobre una copia.

Casos apropiados para `while`

`while` resulta natural cuando no sabemos de antemano cuántas repeticiones serán necesarias:

- reintentar hasta alcanzar un límite o tener éxito;
- consumir una cola mientras tenga elementos;
- leer hasta encontrar una marca de finalización;
- repetir un algoritmo hasta que su estado converja;
- mantener un servicio mientras continúe activo.

Ejemplo con cantidad máxima de intentos:

```
let intentos = 0;
let autenticado = false;

while (intentos < 3 && !autenticado) {
  autenticado = intentarAutenticacion();
  intentos += 1;
}
```

8. `do...while` : ejecutar al menos una vez

`do...while` evalúa la condición después del cuerpo:

```
do {  
  // cuerpo  
} while (condicion);
```

Por eso siempre realiza al menos una iteración:

```
let pagina = 1;  
const totalPaginas = 3;  
  
do {  
  console.log(`Procesando página ${pagina}`);  
  pagina += 1;  
} while (pagina <= totalPaginas);
```

Incluso si la condición inicial fuera falsa, el cuerpo se ejecutaría una vez.

```
let valor = 10;  
  
do {  
  console.log(valor); // 10  
} while (valor < 5);
```

`do...while` es útil cuando la primera acción debe ocurrir antes de poder decidir si se repite:

- mostrar un menú y luego evaluar la opción;
- solicitar un dato y luego validarlo;
- ejecutar una operación y decidir si debe reintentarse;
- procesar la primera página antes de saber si existe otra.

La diferencia esencial es:

Bucle	Momento de la condición	Mínimo de ejecuciones
<code>while</code>	antes del cuerpo	0
<code>do...while</code>	después del cuerpo	1

9. `break` : terminar una repetición

`break` abandona inmediatamente el bucle más cercano:

```
const numeros = [3, 8, -1, 10];

for (const numero of numeros) {
  if (numero < 0) {
    break;
  }

  console.log(numero);
}

// Muestra 3 y 8
```

`-1` actúa como una señal de finalización. El `10` nunca se procesa.

Un bucle deliberadamente infinito puede terminar mediante una guarda y `break`:

```
let indice = 0;

while (true) {
  if (indice >= datos.length) {
    break;
  }

  procesar(datos[indice]);
  indice += 1;
}
```

Esta forma puede ser válida cuando la condición de salida resulta más clara dentro del cuerpo. Aun así, si puede expresarse con precisión en la cabecera, suele preferirse:

```
while (indice < datos.length) {
  procesar(datos[indice]);
  indice += 1;
}
```

10. `continue`: pasar a la siguiente iteración

`continue` omite el resto del cuerpo actual y comienza la próxima iteración:

```
const valores = [10, null, 20, undefined, 30];
let total = 0;

for (const valor of valores) {
  if (valor === null || valor === undefined) {
    continue;
  }

  total += valor;
}

console.log(total); // 60
```

Puede verse como una guarda dentro del bucle: descarta temprano los elementos que no deben procesarse.

Sin `continue`, el mismo código necesita anidamiento:

```
for (const valor of valores) {
  if (valor !== null && valor !== undefined) {
    total += valor;
  }
}
```

Ambas versiones son correctas. `continue` resulta útil cuando deja un camino principal simple.

En un `for`, después de `continue` se ejecuta la expresión de actualización y luego se vuelve a evaluar la condición. En un `while`, salta directamente a la condición; por eso debemos asegurarnos de no omitir accidentalmente la actualización:

```
let indice = 0;

while (indice < valores.length) {
  const valor = valores[indice];
  indice += 1; // actualizar antes de cualquier continue

  if (valor === null) {
    continue;
  }

  procesar(valor);
}
```

Bucles anidados y etiquetas

Sin una etiqueta, `break` y `continue` afectan solamente al bucle más cercano.

JavaScript admite etiquetas para dirigirlos a un bucle exterior:

```

busqueda:
for (let fila = 0; fila < matriz.length; fila += 1) {
  for (let columna = 0; columna < matriz[fila].length; columna += 1) {
    if (matriz[fila][columna] === objetivo) {
      console.log(fila, columna);
      break busqueda;
    }
  }
}
}

```

Las etiquetas existen, pero su uso frecuente puede dificultar la lectura. Extraer la búsqueda a una función y utilizar `return` suele ofrecer una estructura más clara.

11. `switch` : seleccionar por un valor

`switch` compara una expresión con una serie de casos:

```

switch (expresion) {
  case valor1:
    // instrucciones
    break;

  case valor2:
    // instrucciones
    break;

  default:
    // si ningún caso coincide
}

```

Ejemplo:

```

function describirEstado(estado) {
  switch (estado) {
    case "pendiente":
      return "Todavía no fue procesado";

    case "aprobado":
      return "La operación fue aprobada";

    case "rechazado":
      return "La operación fue rechazada";

    default:
      return "Estado desconocido";
  }
}

```

`switch` resulta apropiado cuando una misma expresión se compara contra valores discretos. La comparación es estricta, equivalente en lo esencial a `===`:

```
const valor = "1";

switch (valor) {
  case 1:
    console.log("número");
    break;

  case "1":
    console.log("cadena"); // coincide
    break;
}
```

`default` es opcional y puede ubicarse en otra posición, aunque escribirlo al final suele ser más claro.

`break` o `return`

`break` termina el `switch` y continúa después de él:

```
let mensaje;

switch (estado) {
  case "ok":
    mensaje = "Correcto";
    break;

  default:
    mensaje = "Desconocido";
}

console.log(mensaje);
```

Si el `switch` está dentro de una función y cada caso devuelve un resultado, `return` ya termina la función y no necesita `break`.

12. *Fall-through*: continuar en el caso siguiente

Si un caso no termina con `break`, `return` o `throw`, la ejecución continúa por las instrucciones del caso siguiente. Este comportamiento se llama **fall-through**.

```
const nivel = 2;

switch (nivel) {
  case 1:
    console.log("Nivel básico");
    break;

  case 2:
    console.log("Nivel intermedio");
    // Falta break

  case 3:
    console.log("Nivel avanzado");
    break;
}
```

La salida es:

```
Nivel intermedio
Nivel avanzado
```

Después de encontrar `case 2`, JavaScript no vuelve a comparar `case 3`: simplemente continúa ejecutando sus instrucciones.

Fall-through accidental

La omisión de `break` es una fuente clásica de errores. Cuando no es intencional, cada caso debe finalizar explícitamente.

Fall-through intencional

Puede utilizarse para agrupar varios valores bajo el mismo comportamiento:

```
switch (dia) {
  case "sábado":
  case "domingo":
    console.log("Fin de semana");
    break;

  default:
    console.log("Día laborable");
}
```

Los primeros casos no tienen instrucciones; todos conducen al mismo bloque.

También puede acumular capacidades:

```
switch (rol) {
  case "administrador":
    permisos.add("eliminar");
    // continúa intencionalmente

  case "editor":
    permisos.add("editar");
    // continúa intencionalmente

  case "lector":
    permisos.add("leer");
    break;
}
```

Cuando el *fall-through* contiene instrucciones, debe comentarse para dejar claro que no falta un `break`.

Alcance dentro de `switch`

Todo el `switch` comparte un bloque léxico. Declarar el mismo nombre con `const` en varios casos produce un error, aunque solamente se ejecute uno:

```
switch (tipo) {
  case "a": {
    const mensaje = "Tipo A";
    console.log(mensaje);
    break;
  }

  case "b": {
    const mensaje = "Tipo B";
    console.log(mensaje);
    break;
  }
}
```

Los bloques adicionales `{}` crean un alcance independiente por caso.

13. `while` convertido en `for`

Muchos bucles con contador comparten tres partes:

1. inicialización;
2. condición;
3. actualización.

Con `while` aparecen separadas:


```
let indice = 0;

while (indice < alumnos.length) {
  console.log(alumnos[indice]);
  indice += 1;
}
```

El `for` tradicional las reúne en su cabecera:

```
for (let indice = 0; indice < alumnos.length; indice += 1) {
  console.log(alumnos[indice]);
}
```

Su forma general es:

```
for (inicializacion; condicion; actualizacion) {
  // cuerpo
}
```

El orden real es:

- | | |
|---------------------|---------------------------|
| 1. inicialización | una sola vez |
| 2. condición | antes de cada vuelta |
| 3. cuerpo | si la condición es truthy |
| 4. actualización | después del cuerpo |
| 5. volver al paso 2 | |

Cuándo elegir `for`

`for` es natural cuando la progresión está controlada por un contador:

```
for (let numero = 1; numero <= 10; numero += 1) {
  console.log(numero);
}
```

También permite otras progresiones:

```
for (let par = 0; par <= 20; par += 2) {
  console.log(par);
}

for (let indice = alumnos.length - 1; indice >= 0; indice -= 1) {
  console.log(alumnos[indice]);
}
```

Las tres expresiones son opcionales:

```
for (;;) {  
  if (debeTerminar()) {  
    break;  
  }  
}
```

Esto representa un bucle infinito equivalente a `while (true)`. Debe existir una salida clara dentro del cuerpo.

14. `for...of`: recorrer valores de un iterable

`for...of` recorre los valores producidos por un iterable:

```
const notas = [8, 6, 9];  
  
for (const nota of notas) {  
  console.log(nota);  
}
```

Es apropiado para arrays, cadenas, `Map`, `Set` y otras estructuras iterables.

```
for (const caracter of "TUP") {  
  console.log(caracter);  
}
```

Con un `Map`, cada valor iterado es un par `[clave, valor]`:

```
const stock = new Map([  
  ["A10", 5],  
  ["B20", 8],  
]);  
  
for (const [codigo, cantidad] of stock) {  
  console.log(`${codigo}: ${cantidad}`);  
}
```

Con un `Set`, recorre cada valor único:

```
const permisos = new Set(["leer", "editar"]);

for (const permiso of permisos) {
  console.log(permiso);
}
```

Si necesitamos índice y valor de un array:

```
for (const [indice, nota] of notas.entries()) {
  console.log(indice, nota);
}
```

`break` y `continue` funcionan normalmente en `for...of`.

Un objeto común no es iterable de manera directa:

```
const alumno = { nombre: "Ana", legajo: 12345 };

// for (const valor of alumno) {} // TypeError
```

Podemos convertir sus propiedades en un iterable mediante `Object.entries()`.

15. `for...in` : recorrer claves enumerables

`for...in` recorre nombres de propiedades enumerables de un objeto. Las claves obtenidas son cadenas:

```
const alumno = {
  nombre: "Ana",
  legajo: 12345,
  regular: true,
};

for (const propiedad in alumno) {
  console.log(propiedad, alumno[propiedad]);
}
```

Puede incluir propiedades enumerables heredadas. Para trabajar únicamente con propiedades propias:

```
for (const propiedad in alumno) {
  if (!Object.hasOwn(alumno, propiedad)) {
    continue;
  }

  console.log(propiedad, alumno[propiedad]);
}
```

En muchos casos resulta más directo combinar `Object.entries()` con `for...of`:

```
for (const [propiedad, valor] of Object.entries(alumno)) {
  console.log(propiedad, valor);
}
```

No se recomienda `for...in` para arrays porque recorre nombres de propiedades, no solamente valores ni necesariamente únicamente índices:

```
const notas = [8, 6, 9];
notas.descripcion = "Primer parcial";

for (const clave in notas) {
  console.log(clave);
}

// "0", "1", "2", "descripcion"
```

Elección rápida

Estructura	Recorrido habitual
array sin necesidad de índice	<code>for...of</code>
array con control de índice	<code>for</code>
<code>Map</code> o <code>Set</code>	<code>for...of</code>
propiedades propias de objeto	<code>for...of Object.entries(objeto)</code>
inspección de propiedades enumerables	<code>for...in</code> , considerando herencia

16. Elegir el bucle apropiado

Pregunta	Construcción sugerida
¿Se repite mientras cambie un estado y no conocemos la cantidad?	<code>while</code>
¿Debe ejecutarse al menos una vez?	<code>do...while</code>
¿Existe un contador con inicio, condición y paso?	<code>for</code>
¿Queremos cada valor de un iterable?	<code>for...of</code>
¿Queremos nombres de propiedades enumerables?	<code>for...in</code>
¿Queremos transformar un array completo?	<code>map()</code>
¿Queremos seleccionar elementos?	<code>filter()</code>
¿Queremos encontrar uno y detenernos?	<code>find()</code> o <code>for...of</code>
¿Queremos saber si alguno cumple?	<code>some()</code>
¿Queremos acumular un resultado?	<code>reduce()</code> o un bucle con acumulador

Los métodos de arrays también controlan la iteración, pero no reemplazan todos los bucles.

`for...of` permite `break`, `continue`, `return` desde la función contenedora y `await` de manera secuencial cuando corresponda. `forEach()` no ofrece esas mismas salidas.

17. Excepciones: una salida abrupta

Una excepción representa que la operación no puede continuar normalmente. Puede ser generada por el lenguaje:

```
const alumno = null;
console.log(alumno.nombre);
// TypeError
```

O lanzada por nuestro programa:

```
function dividir(dividendo, divisor) {
  if (divisor === 0) {
    throw new RangeError("El divisor no puede ser cero");
  }

  return dividendo / divisor;
}
```

`throw` termina inmediatamente la ejecución actual. Las instrucciones siguientes no se ejecutan:

```
throw new Error("No se pudo continuar");
console.log("Esta línea es inalcanzable");
```

JavaScript permite lanzar cualquier valor:

```
throw "falló";
throw 404;
```

Pero debe preferirse un objeto `Error` o una subclase. Así se conservan propiedades como `name`, `message`, `stack` y `cause`.

Tipos frecuentes de error

Tipo	Uso habitual
<code>Error</code>	error general de la aplicación
<code>TypeError</code>	valor de un tipo o forma incompatible
<code>RangeError</code>	valor fuera del rango admitido
<code>ReferenceError</code>	referencia a un nombre inexistente o inaccesible
<code>SyntaxError</code>	código o texto analizado con sintaxis inválida
<code>AggregateError</code>	agrupación de varios errores

El lenguaje genera algunos de estos errores automáticamente. Nuestro código también puede elegir una clase que comunique el contrato incumplido.

18. Propagación de excepciones

Cuando una función lanza y no captura una excepción, su ejecución termina y la excepción se propaga a quien la llamó:

```
function convertirCantidad(texto) {
  const cantidad = Number(texto);

  if (!Number.isInteger(cantidad) || cantidad <= 0) {
    throw new RangeError("Cantidad inválida");
  }

  return cantidad;
}

function crearLinea(datos) {
  const cantidad = convertirCantidad(datos.cantidad);
  return { productoId: datos.productoId, cantidad };
}

function procesarSolicitud(datos) {
  return crearLinea(datos);
}
```

Si `convertirCantidad()` lanza:

1. abandona `convertirCantidad()`;
2. como `crearLinea()` no captura, también la abandona;
3. como `procesarSolicitud()` no captura, continúa propagándose;
4. termina en el primer `catch` apropiado o en el manejador del entorno.

Este proceso suele llamarse **desenrollado de la pila**.

Las excepciones permiten separar el lugar donde se detecta un problema del nivel que sabe cómo responder. Una función profunda puede detectar una cantidad inválida; una ruta HTTP puede decidir devolver una respuesta `400`.

19. `try...catch`: capturar y manejar

```
try {
  // operación que puede lanzar
} catch (error) {
  // respuesta a la excepción
}
```

Ejemplo con JSON:

```
function leerConfiguracion(texto) {
  try {
    return JSON.parse(texto);
  } catch (error) {
    console.error("La configuración no contiene JSON válido");
    return null;
  }
}
```

Si `JSON.parse()` termina normalmente, `catch` se omite. Si lanza, el resto del bloque `try` se abandona y el control pasa a `catch`.

Capturar solamente lo que podemos manejar

Un `catch` demasiado amplio puede ocultar un error de programación:

```
try {
  const configuracion = JSON.parse(texto);
  usarConfiguracion(configuracoin); // nombre mal escrito
} catch {
  return null; // también oculta el ReferenceError
}
```

Conviene limitar el `try` o distinguir el tipo:

```
function parsearConfiguracion(texto) {
  try {
    return JSON.parse(texto);
  } catch (error) {
    if (error instanceof SyntaxError) {
      throw new Error("La configuración tiene formato inválido", {
        cause: error,
      });
    }

    throw error;
  }
}
```

`throw error` vuelve a lanzar una excepción que este nivel no sabe resolver.

Preservar la causa

Cuando agregamos contexto, la propiedad `cause` conserva el error original:


```
try {
  conectarBaseDeDatos();
} catch (error) {
  throw new Error("No fue posible iniciar el repositorio de alumnos", {
    cause: error,
  });
}
```

El mensaje superior explica qué operación del dominio falló; `cause` conserva el motivo técnico.

20. `finally` : ejecutar la limpieza

`finally` se ejecuta antes de salir de la construcción, tanto si hubo una excepción como si no:

```
const conexion = abrirConexion();

try {
  guardarDatos(conexion);
} catch (error) {
  console.error("No se pudieron guardar los datos", error);
} finally {
  cerrarConexion(conexion);
}
```

Su propósito habitual es liberar recursos o restaurar estado:

- cerrar archivos o conexiones;
- liberar bloqueos;
- detener indicadores de carga;
- restaurar una configuración temporal.

`finally` también se ejecuta antes de completar un `return` o propagar un `throw` :

```
function ejemplo() {
  try {
    return "resultado";
  } finally {
    console.log("limpieza");
  }
}
```

Debe evitarse `return`, `throw`, `break` o `continue` dentro de `finally`, porque pueden reemplazar una salida o excepción anterior:

```
function peligroso() {
  try {
    throw new Error("error original");
  } finally {
    return "oculta el error";
  }
}
```

`finally` debería limpiar, no decidir el resultado principal.

21. Errores personalizados

Una clase de error propia permite distinguir situaciones del dominio sin analizar mensajes:

```
class StockInsuficienteError extends Error {
  constructor(productoId, solicitado, disponible) {
    super(`Stock insuficiente para el producto ${productoId}`);
    this.name = "StockInsuficienteError";
    this.productoId = productoId;
    this.solicitado = solicitado;
    this.disponible = disponible;
  }
}
```

Puede lanzarse donde se detecta el problema:

```
function descontarStock(producto, cantidad) {
  if (cantidad > producto.stock) {
    throw new StockInsuficienteError(
      producto.id,
      cantidad,
      producto.stock
    );
  }

  return {
    ...producto,
    stock: producto.stock - cantidad,
  };
}
```

Y manejarse según su tipo:

```
try {
  const actualizado = descontarStock(producto, cantidad);
  guardar(actualizado);
} catch (error) {
  if (error instanceof StockInsuficienteError) {
    mostrarMensaje(`Solo quedan ${error.disponible} unidades`);
  } else {
    throw error;
  }
}
```

La excepción esperada se traduce a una respuesta del sistema. Cualquier error desconocido vuelve a propagarse.

22. Excepciones como control de flujo: alcance y límites

Una excepción es una forma de control de flujo porque interrumpe la secuencia, abandona funciones y busca un `catch`. Pero eso no significa que deba utilizarse para cualquier alternativa.

Decisión esperada

```
function buscarAlumno(legajo) {
  const alumno = alumnos.find((item) => item.legajo === legajo);
  return alumno ?? null;
}
```

Que una búsqueda no encuentre resultados puede ser una situación normal y representarse con `null`.

Operación que no puede cumplir su contrato

```
function obtenerAlumnoObligatorio(legajo) {
  const alumno = alumnos.find((item) => item.legajo === legajo);

  if (!alumno) {
    throw new Error(`No existe el alumno ${legajo}`);
  }

  return alumno;
}
```

Aquí la función promete entregar un alumno. Si no existe, no puede producir un resultado válido.

Por qué no reemplazar `if` con excepciones

Esta versión utiliza una excepción para una decisión completamente esperada:

```
try {
  if (!usuario.esAdministrador) {
    throw new Error("no es administrador");
  }

  mostrarPanel();
} catch {
  mostrarInicio();
}
```

Un `if...else` comunica mejor la regla:

```
if (usuario.esAdministrador) {
  mostrarPanel();
} else {
  mostrarInicio();
}
```

Regla orientativa:

- utilizar condiciones y resultados para alternativas normales del negocio;
 - utilizar excepciones cuando una operación no puede cumplir su contrato o falla una dependencia;
 - capturar en el nivel que pueda recuperar, traducir o agregar contexto;
 - no capturar solamente para ignorar el problema.
-

23. Caso integrador: procesar una solicitud de compra

```
class ValidacionError extends Error {
  constructor(mensaje, campo) {
    super(mensaje);
    this.name = "ValidacionError";
    this.campo = campo;
  }
}

function convertirCantidad(valor) {
  const cantidad = Number(valor);

  if (!Number.isInteger(cantidad) || cantidad <= 0) {
    throw new ValidacionError(
      "La cantidad debe ser un entero positivo",
      "cantidad"
    );
  }

  return cantidad;
}

function calcularCompra(productos, solicitud) {
  if (!solicitud) {
    throw new ValidacionError("Falta la solicitud", "solicitud");
  }

  const cantidad = convertirCantidad(solicitud.cantidad);
  let productoEncontrado = null;

  for (const producto of productos) {
    if (producto.id !== solicitud.productoId) {
      continue;
    }

    productoEncontrado = producto;
    break;
  }

  if (!productoEncontrado) {
    return { ok: false, motivo: "producto inexistente" };
  }

  if (productoEncontrado.stock < cantidad) {
    return { ok: false, motivo: "stock insuficiente" };
  }

  let descuento;

  switch (solicitud.tipoCliente) {
    case "premium":
      descuento = 0.15;
      break;
  }
}
```

```

    case "frecuente":
        descuento = 0.05;
        break;

    default:
        descuento = 0;
}

const subtotal = productoEncontrado.precio * cantidad;
const total = subtotal * (1 - descuento);

return {
    ok: true,
    producto: productoEncontrado.nombre,
    cantidad,
    total,
};
}

```

Un nivel exterior puede traducir las excepciones de validación:

```

function atenderCompra(productos, solicitud) {
    try {
        return calcularCompra(productos, solicitud);
    } catch (error) {
        if (error instanceof ValidacionError) {
            return {
                ok: false,
                motivo: error.message,
                campo: error.campo,
            };
        }

        throw error;
    }
}

```

El ejemplo combina:

- guardas para validar precondiciones;
- excepción para una entrada que viola el contrato;
- `for...of` para recorrer productos;
- `continue` para descartar los que no coinciden;
- `break` después de encontrar el producto;
- `if` para resultados normales del negocio;
- `switch` para valores discretos;
- `try...catch` en el nivel que sabe traducir el error.

24. Errores frecuentes y su explicación

Confundir asignación con comparación

```
if (estado = "activo") {  
    // asigna "activo" y la condición resulta truthy  
}
```

La comparación estricta es:

```
if (estado === "activo") {  
    // ...  
}
```

Ordenar mal un encadenamiento

Las condiciones más específicas suelen aparecer antes que las generales.

Anidar todas las condiciones

Las guardas permiten terminar los casos inválidos y mantener visible el camino principal.

Crear un bucle sin progreso

La condición nunca cambia y el bucle no termina. Debe identificarse qué estado lo acerca a la salida.

Actualizar después de un `continue` inalcanzable

En un `while`, conviene actualizar antes de una posible salida con `continue` o diseñar la cabecera para que el progreso sea inevitable.

Olvidar `break` en un `switch`

La ejecución continúa mediante *fall-through*. Si es intencional, debe quedar agrupado o comentado.

Utilizar `for...in` para valores de un array

`for...in` recorre claves como cadenas y puede incluir propiedades adicionales. `for...of` recorre valores.

Capturar y silenciar todos los errores

```
try {
  ejecutar();
} catch {
  // no hace nada
}
```

El sistema pierde la causa y puede continuar en un estado inválido. Debe manejarse, traducirse, registrarse o propagarse.

Lanzar cadenas

```
throw "falló";
```

Es legal, pero un objeto `Error` conserva tipo, mensaje, causa y traza.

Retornar desde `finally`

Puede ocultar un `return` o `throw` anterior. `finally` debe reservarse para limpieza.

25. Ideas que conviene conservar

1. El flujo de control determina cuál es la próxima instrucción.
2. `if` ejecuta un bloque cuando su condición es `truthy`; `else` cubre el camino contrario.
3. Una cadena `else if` ejecuta solamente la primera rama verdadera y su orden importa.
4. El anidamiento es válido, pero las guardas suelen hacer más visible el camino principal.
5. El ternario es una expresión apropiada para elegir entre dos valores breves.
6. `while` comprueba antes y puede ejecutar cero veces; `do...while` comprueba después y ejecuta al menos una.
7. Todo bucle necesita una condición de salida y un estado que progrese hacia ella.
8. `break` termina el bucle o `switch` más cercano; `continue` comienza la siguiente iteración.
9. `switch` compara una expresión con casos mediante igualdad estricta.
10. Sin `break`, `return` o `throw`, un caso continúa mediante *fall-through*.
11. El `for` reúne inicialización, condición y actualización.
12. `for...of` recorre valores de iterables; `for...in` recorre nombres de propiedades enumerables.
13. Un objeto común se recorre normalmente con `Object.entries()` y `for...of`.
14. `throw` abandona el flujo normal y propaga una excepción por la pila de llamadas.
15. `catch` debe capturar solamente aquello que puede manejar o traducir.
16. Al volver a lanzar con más contexto, `cause` conserva el error original.

17. `finally` se ejecuta para limpiar tanto en éxito como en error.
 18. Las excepciones son control de flujo abrupto, pero no sustituyen condiciones normales.
-

26. Preguntas de repaso

1. ¿Qué significa flujo de control?
2. ¿Qué valores puede utilizar una condición de `if`?
3. ¿Cuándo se ejecuta un bloque `else`?
4. ¿Por qué importa el orden de una cadena `else if`?
5. ¿Cuándo conviene un ternario y cuándo un `if...else`?
6. ¿Qué problema de legibilidad resuelven las guardas?
7. ¿Qué diferencia existe entre retornar un resultado inválido y lanzar una excepción?
8. ¿Cuántas veces como mínimo se ejecutan `while` y `do...while`?
9. ¿Qué debe cambiar para que un bucle termine?
10. ¿Qué diferencia existe entre `break` y `continue`?
11. ¿Qué peligro presenta `continue` dentro de un `while`?
12. ¿Cómo afecta un `break` a bucles anidados?
13. ¿Cómo compara `switch` el valor con cada `case`?
14. ¿Qué es el *fall-through* y cuándo puede ser útil?
15. ¿En qué orden se ejecutan las partes de un `for`?
16. ¿Qué diferencia existe entre `for...of` y `for...in`?
17. ¿Por qué un objeto común no puede recorrerse directamente con `for...of`?
18. ¿Qué ocurre con las funciones intermedias cuando una excepción se propaga?
19. ¿Cuándo se ejecuta `finally`?
20. ¿Por qué no debe retornarse desde `finally`?
21. ¿Por qué conviene lanzar objetos `Error` en lugar de cadenas?
22. ¿Cuándo una excepción es preferible a un resultado normal?

Ejercicio 1: simplificar con guardas

Reescribir con retornos tempranos:

```
function publicar(articulo, usuario) {
  if (usuario) {
    if (usuario.estaActivo) {
      if (articulo.titulo) {
        if (articulo.contenido) {
          return { ok: true };
        } else {
          return { ok: false, motivo: "falta contenido" };
        }
      } else {
        return { ok: false, motivo: "falta título" };
      }
    } else {
      return { ok: false, motivo: "usuario inactivo" };
    }
  } else {
    return { ok: false, motivo: "falta usuario" };
  }
}
```

Ejercicio 2: convertir bucles

Convertir este `while` en un `for` equivalente y explicar el orden de ejecución:

```
let numero = 2;

while (numero <= 20) {
  console.log(numero);
  numero += 2;
}
```

Luego escribir una versión con `for...of` a partir de un array de números.

Ejercicio 3: `switch` y *fall-through*

Construir una función que reciba un mes y devuelva la cantidad de días. Agrupar mediante *fall-through* los meses con 30 días y contemplar febrero por separado. Rechazar valores desconocidos.

Ejercicio 4: excepciones

Implementar `convertirNota(valor)` para que:

1. convierta la entrada a número;
2. lance `TypeError` si no produce un número válido;
3. lance `RangeError` si queda fuera de 0 a 10;
4. devuelva la nota en caso correcto;

5. sea utilizada desde otra función que capture únicamente esos errores esperados y vuelva a lanzar cualquier error desconocido.
-

Fuentes y lecturas recomendadas

- Flujo de control y manejo de errores — MDN
- `if...else` — MDN
- Operador condicional ternario — MDN
- Bucles e iteración — MDN
- `switch` y *fall-through* — MDN
- `for...of` — MDN
- `for...in` — MDN
- `try...catch...finally` — MDN
- Objeto `Error` — MDN
- Causa de un error — MDN
- ECMAScript Language Specification — TC39

try...catch en JavaScript, desde primeros principios

try...catch existe para resolver un problema muy concreto: hay operaciones que pueden fallar mientras el programa se está ejecutando y queremos decidir qué hacer cuando eso ocurre.

La idea fundamental es:

```
~~~js try { // Intento hacer algo. } catch (error) { // Si falla, reacciono al problema. } ~~~
```

Pero para entenderlo bien conviene empezar antes: por qué existe un error, qué significa lanzar una excepción y cómo cambia el flujo del programa.

1. Un programa normalmente avanza línea por línea

Supongamos:

```
~~~js console.log("A"); console.log("B"); console.log("C"); ~~~
```

La ejecución es:

```
~~~text A B C ~~~
```

Conceptualmente:

```
~~~text línea 1 ↓ línea 2 ↓ línea 3 ~~~
```

Pero ahora:

```
~~~js console.log("A");
```

```
algoQueNoExiste();
```

```
console.log("C"); ~~~
```

JavaScript encuentra la llamada a una función que no está definida y produce un error parecido a:

```
~~~text ReferenceError: algoQueNoExiste is not defined ~~~
```

En ese momento ocurre algo fundamental: **JavaScript deja de ejecutar normalmente ese camino del programa.**

Por lo tanto, la última línea:

```
~~~js console.log("C"); ~~~
```

no se ejecuta.

2. Un error altera el flujo normal

Podemos pensar que existe un camino normal:

~~~text A ↓ B ↓ C ↓ D ~~~

Pero si C produce un error:

~~~text A ↓ B ↓ C ✨ ~~~

JavaScript ya no continúa hacia D. El error empieza a propagarse buscando algún código que pueda manejarlo.

Aquí aparece **try...catch**.

Una excepción no es simplemente un valor que informa que algo salió mal. También es un mecanismo de control: interrumpe el recorrido normal y transfiere la ejecución a otro lugar.

3. try significa literalmente «intentá ejecutar esto»

~~~js try { console.log("A");

algoQueNoExiste();

console.log("B"); } ~~~

El bloque **try** no evita que ocurra el error.

No significa:

Ejecutá esto sin errores.

Significa:

Ejecutá esto normalmente, pero si durante esta ejecución aparece una excepción, dame la oportunidad de manejarla.

El bloque **try** delimita la región de código cuyas excepciones queremos poder capturar.

---

## 4. catch captura la excepción

---

```
~~~js try { console.log("A");  
algoQueNoExiste();
console.log("B"); } catch (error) { console.log("Ocurrió un problema"); }
console.log("C"); ~~~
```

Resultado:

```
~~~text A Ocurrió un problema C ~~~
```

Observá algo fundamental: B **no se ejecutó**.

Cuando ocurrió el error, JavaScript abandonó inmediatamente el bloque **try**:

```
~~~text try | └── A | └── error ✨ | └── B ← nunca llega | ↓ catch | └── "Ocurrió un  
problema" | ↓ C ~~~
```

Después de ejecutar el **catch**, el programa puede continuar con la instrucción posterior a toda la estructura **try...catch**.

---

## 5. El objeto Error

---

Normalmente escribimos:

```
~~~js catch (error) { console.log(error); } ~~~
```

La variable **error** recibe aquello que fue lanzado. Cuando JavaScript produce el fallo —o cuando nuestro código lanza un objeto Error— contiene información sobre lo ocurrido.

Por ejemplo:

```
~~~js try { x(); } catch (error) { console.log(error.name); console.log(error.message); } ~~~
```

Podría imprimir:

```
~~~text ReferenceError x is not defined ~~~
```

Las propiedades más utilizadas son:

- **name**: nombre o categoría del error.
- **message**: explicación legible del problema.
- **stack**: rastro de las llamadas activas cuando ocurrió; es muy útil para depurar.
- **cause**: causa original opcional, cuando el error fue creado con esa información.

Ejemplo:

```
~~~js try { x(); } catch (error) { console.log(error.name); console.log(error.message);  
console.log(error.stack); } ~~~
```

El rastro puede verse así:

```
~~~text ReferenceError: x is not defined at programa.js:4 at ... ~~~
```

## No siempre se lanza un objeto Error

JavaScript permite técnicamente lanzar cualquier valor:

```
~~~js throw "falló"; throw 404; throw { motivo: "sin permiso" }; ~~~
```

Por eso un **catch** recibe exactamente lo que se lanzó. Sin embargo, es una buena práctica lanzar instancias de Error o de sus subclases:

```
~~~js throw new Error("Falló la operación"); ~~~
```

Así se conservan de forma coherente el mensaje, el tipo y el rastro de la pila.

---

## 6. throw: lanzar una excepción

---

El concepto central del mecanismo es **throw**:

```
~~~js throw new Error("Algo salió mal"); ~~~
```

En cuanto se ejecuta **throw**, se interrumpe el flujo normal del bloque actual.

```
~~~js console.log("A");
```

```
throw new Error("Problema");
```

```
console.log("B"); ~~~
```

B nunca se ejecuta.

**throw** es una instrucción; el valor que aparece después es la información que viaja con la excepción.

También podemos detectar una regla inválida de nuestro propio dominio y lanzar un error:

```
~~~js function retirar(saldo, monto) { if (monto > saldo) { throw new Error("Saldo insuficiente"); }  
return saldo - monto; } ~~~
```

---

## 7. throw y catch son dos caras del mismo mecanismo

---

```
~~~js try { throw new Error("No hay conexión"); } catch (error) { console.log(error.message); } ~~~
```

Resultado:

```
~~~text No hay conexión ~~~
```

Modelo básico:

```
~~~text throw | | lanza una excepción ↓ busca un catch | ↓ catch ~~~
```

Por eso suele decirse que una excepción **sube por la pila de llamadas** hasta encontrar un **catch**.

---

## 8. Un ejemplo más real: dividir

---

Supongamos esta función:

```
~~~js function dividir(a, b) { return a / b; } ~~~
```

En JavaScript:

```
~~~js dividir(10, 0); ~~~
```

no produce una excepción; devuelve:

```
~~~text Infinity ~~~
```

Pero tal vez, para nuestro programa, dividir por cero sea inválido. Podemos imponer esa regla:

```
~~~js function dividir(a, b) { if (b === 0) { throw new Error("No se puede dividir por cero"); }  
return a / b; } ~~~
```

Quien llama a la función puede decidir cómo responder:

```
~~~js try { const resultado = dividir(10, 0); console.log(resultado); } catch (error) {  
console.log(error.message); } ~~~
```

Resultado:

```
~~~text No se puede dividir por cero ~~~
```

Las excepciones no sirven únicamente para errores internos de JavaScript. También podemos usarlas para señalar situaciones que vuelven imposible completar una operación de nuestra aplicación.

---



## 9. ¿Por qué no simplemente devolver false?

---

Podríamos escribir:

```
~~~js function dividir(a, b) { if (b === 0) { return false; }  
return a / b; } ~~~
```

Pero aparece un problema: ¿qué significa **false**?

Podría significar:

- error;
- resultado válido;
- dato faltante;
- permiso denegado;
- cualquier otra cosa.

Una excepción separa claramente:

```
~~~text resultado normal ~~~
```

de:

```
~~~text la operación no pudo completarse ~~~
```

Por ejemplo:

```
~~~js function buscarUsuario(id) { if (id < 0) { throw new Error("ID inválido"); }  
return { id, nombre: "Ana" }; } ~~~
```

Entonces quien llama decide:

```
~~~js try { const usuario = buscarUsuario(-5); console.log(usuario); } catch (error) {  
console.log("No pude obtener el usuario"); } ~~~
```

Esto no significa que siempre haya que usar excepciones. Para un resultado esperable como «no se encontró ningún usuario», una función también puede devolver **null**, **undefined** o un objeto de resultado explícito. La decisión depende del contrato de la función. Las excepciones son más apropiadas cuando la operación no puede cumplir ese contrato.

---

## 10. La pila de llamadas

---

Cuando una función llama a otra, JavaScript necesita recordar dónde debe regresar. Esas llamadas pendientes forman la **pila de llamadas** o *call stack*.

```
~~~js function c() { throw new Error("Problema"); }
```

```
function b() { c(); }
```

```
function a() { b(); }
```

```
a(); ~~~
```

La pila queda conceptualmente así:

```
~~~text a() └── b() └── c() ~~~
```

Cuando c lanza el error:

```
~~~text a() └── b() └── c() ✨ ~~~
```

JavaScript busca un **catch**:

1. Primero en c. No hay.
2. Sale de c y sube a b. No hay.
3. Sale de b y sube a a. No hay.
4. Sale de a.
5. Al no encontrar ningún manejador, queda una excepción no capturada.

En un navegador, normalmente se informa en la consola. En Node.js, una excepción síncrona no capturada normalmente hace terminar el proceso.

---

## 11. Podemos atraparla en una capa superior

---

```
~~~js function c() { throw new Error("Problema"); }
```

```
function b() { c(); }
```

```
function a() { b(); }
```

```
try { a(); } catch (error) { console.log("Capturado:", error.message); } ~~~
```

Resultado:

```
~~~text Capturado: Problema ~~~
```

Visualmente:

```
~~~text try └── a() └── b() └── c() | throw ↑ | no catch ↑ no catch ↑ no catch ↑ try | ↓  
catch ~~~
```

Este mecanismo permite que una función profunda diga «no puedo continuar» sin necesitar saber quién finalmente va a resolver, registrar o presentar el problema.

---

## 12. Propagación de excepciones

---

Considera:

```
~~~js function leerConfiguracion() { return convertirJSON(); }  
  
function convertirJSON() { throw new Error("JSON inválido"); } ~~~
```

La función **convertirJSON** no tiene que saber si el programa quiere:

- mostrar un mensaje;
- volver a intentar;
- escribir un registro;
- terminar;
- utilizar valores por defecto.

Simplemente informa que no pudo completar su trabajo:

```
~~~js throw new Error("JSON inválido"); ~~~
```

Una capa superior decide:

```
~~~js try { const config = leerConfiguracion(); console.log(config); } catch (error) {  
console.log("No se pudo cargar la configuración"); } ~~~
```

Esta separación de responsabilidades es una de las grandes ventajas de las excepciones:

```
~~~text capa profunda → detecta e informa capa superior → decide cómo responder ~~~
```

---

## 13. Error es una clase

---

Cuando escribimos:

```
~~~js new Error("Algo salió mal"); ~~~
```

estamos creando un objeto:

```
~~~js const error = new Error("Algo salió mal");  

console.log(typeof error); console.log(error instanceof Error); console.log(error.message); ~~~
```

Resultado:

```
~~~text object true Algo salió mal ~~~
```

Crear el objeto no lo lanza:

```
~~~js const error = new Error("Algo salió mal");
```

```
console.log("El programa continúa"); ~~~
```

Para interrumpir el flujo hay que usar **throw**:

```
~~~js throw error; ~~~
```

La diferencia es:

```
~~~text new Error(...) → construye información sobre un error throw new Error(...) → construye esa información y lanza la excepción ~~~
```

---

## 14. Tipos de Error

---

JavaScript posee varias clases incorporadas:

- **Error**: error genérico.
- **TypeError**: se usó un valor de una forma incompatible con su tipo.
- **RangeError**: un valor numérico está fuera de un rango permitido.
- **ReferenceError**: se intentó usar una referencia que no existe.
- **SyntaxError**: el texto analizado no respeta la sintaxis esperada.
- **URIError**: uso inválido de ciertas funciones relacionadas con URI.
- **AggregateError**: agrupa varios errores, por ejemplo en algunas operaciones con Promises.

Ejemplos:

```
~~~js const nombre = null; nombre.toUpperCase(); // TypeError ~~~
```

```
~~~js const arreglo = new Array(-1); // RangeError ~~~
```

```
~~~js console.log(variableInexistente); // ReferenceError ~~~
```

```
~~~js JSON.parse("{}"); // SyntaxError ~~~
```

Podemos comprobar el tipo con **instanceof**:

```
~~~js try { const nombre = null; nombre.toUpperCase(); } catch (error) { console.log(error instanceof TypeError); } ~~~
```

Resultado:

```
~~~text true ~~~
```

## 15. Podemos actuar de forma diferente según el error

---

```
~~~js try { hacerAlgo(); } catch (error) { if (error instanceof TypeError) { console.log("Error de tipo"); } else if (error instanceof RangeError) { console.log("Valor fuera de rango"); } else { console.log("Otro error"); } } ~~~
```

Pero hay una regla importante: si el **catch** no sabe manejar un tipo de error, normalmente debe volver a lanzarlo en vez de ocultarlo:

```
~~~js try { hacerAlgo(); } catch (error) { if (error instanceof TypeError) { corregirDatos(); } else { throw error; } } ~~~
```

---

## 16. Errores personalizados

Podemos crear clases que representen problemas específicos de nuestra aplicación:

```
~~~js class UsuarioNoEncontradoError extends Error { constructor(id) { super("No existe el usuario con ID " + id); this.name = "UsuarioNoEncontradoError"; this.id = id; } } ~~~
```

Luego:

```
~~~js function buscarUsuario(id) { if (id !== 123) { throw new UsuarioNoEncontradoError(id); } return { id: 123, nombre: "Ana" }; } ~~~
```

Y al usarla:

```
~~~js try { buscarUsuario(999); } catch (error) { if (error instanceof UsuarioNoEncontradoError) { console.log("Usuario inexistente:", error.id); } else { throw error; } } ~~~
```

Un error personalizado permite:

- reconocer el problema con **instanceof**;
- adjuntar datos estructurados, como el ID;
- separar el comportamiento del texto del mensaje;
- evitar tomar decisiones comparando cadenas frágiles.

No conviene escribir:

```
~~~js if (error.message === "El usuario no existe") { // ... } ~~~
```

El mensaje puede cambiar, traducirse o contener detalles. El tipo de error expresa el significado de manera más estable.

### Conservar la causa original

Si una capa agrega contexto, puede conservar la causa:

```
~~~js function cargarConfiguracion(texto) { try { return JSON.parse(texto); } catch (error) { throw new Error("No se pudo cargar la configuración", { cause: error }); } } ~~~
```

Así, el nuevo error explica la operación de alto nivel y **error.cause** conserva el fallo original.

---

## 17. finally

---

Existe un tercer bloque:

```
~~~js try { // Operación. } catch (error) { // Manejo del error. } finally { // Se ejecuta siempre. } ~~~
```

**finally** significa:

Ejecutá este bloque haya ocurrido un error o no.

Sin error:

```
~~~js try { console.log("Intentando"); } catch (error) { console.log("Error"); } finally { console.log("Terminando"); } ~~~
```

Resultado:

```
~~~text Intentando Terminando ~~~
```

Con error:

```
~~~js try { throw new Error("Problema"); } catch (error) { console.log("Error"); } finally { console.log("Terminando"); } ~~~
```

Resultado:

```
~~~text Error Terminando ~~~
```

---

## 18. ¿Para qué sirve finally?

---

Principalmente, para garantizar una limpieza o restauración:

```
~~~js abrirArchivo();
```

```
try { procesarArchivo(); } finally { cerrarArchivo(); } ~~~
```

Conceptualmente:

```
~~~text abrir recurso ↓ try | trabajar ↓ finally | cerrar recurso ~~~
```

En JavaScript también puede utilizarse para:

- ocultar un indicador de carga;
- liberar un bloqueo;
- cerrar una conexión;
- restaurar un estado temporal;
- eliminar un manejador que ya no se necesita.

Ejemplo asíncrono:

```
~~~js async function guardar() { mostrarIndicador();  
  
try { await enviarDatos(); } finally { ocultarIndicador(); } } ~~~
```

Aunque **enviarDatos** falle, el indicador se oculta.

---

## 19. finally se ejecuta incluso con return

---

```
~~~js function ejemplo() { try { return 10; } finally { console.log("finally"); } }  

console.log(ejemplo()); ~~~
```

Resultado:

```
~~~text finally 10 ~~~
```

Antes de abandonar la función, JavaScript ejecuta **finally**.

Conviene evitar un **return** dentro de **finally**:

```
~~~js function ejemploPeligroso() { try { return 10; } finally { return 20; } } ~~~
```

El resultado será 20: el **return** de **finally** reemplaza al anterior. También puede ocultar una excepción. Por eso **finally** debería concentrarse en la limpieza y no cambiar el resultado de la operación.

---

## 20. finally se ejecuta después de throw

---

```
~~~js function ejemplo() { try { throw new Error("Problema"); } finally { console.log("Limpiando"); }  
}
```

```
ejemplo(); ~~~
```

Primero se imprime:

```
~~~text Limpiando ~~~
```

y después el error continúa propagándose:

~~~text throw ↓ finally ↓ el error sigue subiendo ~~~

---

## 21. try...finally sin catch

---

Esto es válido:

```
~~~js try { hacerAlgo(); } finally { limpiar(); } ~~~
```

Aquí no queremos resolver el error. Solo queremos garantizar la limpieza.

Si **hacerAlgo** falla:

```
~~~text hacerAlgo() ✨ ↓ limpiar() ↓ el error continúa propagándose ~~~
```

La sintaxis admite estas combinaciones:

```
~~~text try + catch try + finally try + catch + finally ~~~
```

Un bloque **try** no puede aparecer solo.

---

## 22. catch sin parámetro

---

Si no necesitamos conocer el error:

```
~~~js try { hacerAlgo(); } catch { console.log("Falló"); } ~~~
```

En lugar de:

```
~~~js catch (error) { // ... } ~~~
```

Esto es válido. Sin embargo, si estamos depurando, registrando o decidiendo según el tipo, necesitaremos el parámetro.

---

## 23. Un error común: atrapar demasiado

---

Esto es tentador:

```
~~~js try { // 200 líneas de código. } catch { console.log("Algo salió mal"); } ~~~
```

Pero un bloque tan grande dificulta saber qué operación falló y puede mezclar errores que requieren respuestas diferentes.

Peor todavía:

```
~~~js try { hacerAlgo(); } catch { } ~~~
```



Esto se conoce como «tragarse» la excepción. El programa falló y el código fingió que no pasó nada. Luego puede continuar con un estado inválido y producir un problema mucho más difícil de localizar.

Es preferible:

- mantener el **try** tan pequeño como resulte razonable;
- capturar en una capa que pueda tomar una decisión útil;
- registrar contexto suficiente;
- dejar propagarse lo que no sabemos resolver.

---

## 24. Volver a lanzar el error: rethrow

---

Supongamos:

```
~~~js try { procesar(); } catch (error) { console.log("Falló el procesamiento");  
throw error; } ~~~
```

Primero hacemos algo local —por ejemplo, registrar, agregar contexto o limpiar— y después:

```
~~~js throw error; ~~~
```

permite que otra capa también maneje el problema. Esto se denomina **rethrow** o relanzamiento.

Relanzar el mismo objeto conserva su identidad y su rastro original:

```
~~~js throw error; ~~~
```

Crear otro error puede ser útil si queremos traducir el problema a una abstracción superior, pero conviene conservar la causa:

```
~~~js throw new Error("No se pudo preparar el informe", { cause: error }); ~~~
```

---

## 25. Un patrón útil: capturar solo lo que podemos manejar

---

```
~~~js try { operacion(); } catch (error) { if (puedoResolverlo(error)) { resolver(error); } else { throw  
error; } } ~~~
```

La idea es:

Capturá únicamente aquello que realmente sabés manejar.

«Manejar» puede significar:

- corregir los datos;
- reintentar de forma segura;
- usar un valor alternativo válido;
- transformar el error en una respuesta adecuada;
- informar al usuario y dejar la aplicación en un estado coherente.

Imprimir un mensaje y continuar no siempre equivale a haber manejado el fallo.

---

## 26. try...catch y JSON.parse

---

Un uso natural en JavaScript:

```
~~~js const texto = '{"nombre":"Ana"}';

const persona = JSON.parse(texto);

console.log(persona.nombre); ~~~
```

Pero si recibimos:

```
~~~js const texto = '{"nombre":'; ~~~
```

**JSON.parse** lanza un `SyntaxError` porque no puede producir un valor válido.

```
~~~js function convertirPersona(texto) { try { return JSON.parse(texto); } catch (error) { if (error
instanceof SyntaxError) { console.log("El JSON es inválido"); return null; }

throw error; } } ~~~
```

Aquí el **try...catch** es razonable porque estamos llamando a una operación cuyo contrato especifica que un texto inválido genera una excepción.

También podemos agregar contexto y propagar:

```
~~~js function convertirPersona(texto) { try { return JSON.parse(texto); } catch (error) { throw new
Error("No se pudo interpretar la persona", { cause: error }); } } ~~~
```

---

## 27. La asincronía cambia dónde puede capturarse un error

---

Consideremos:

```
~~~js try { setTimeout(() => { throw new Error("Boom"); }, 1000); } catch (error) {
console.log("Capturado"); } ~~~
```

Uno podría esperar que aparezca «Capturado», pero no ocurre.

Cuando se ejecuta el callback de **setTimeout**, el bloque **try** exterior ya terminó:

~~~text try |—— programa el callback |—— termina

...un segundo después...

callback |—— throw ✨ ~~~

Son ejecuciones separadas. El **catch** solo puede capturar excepciones lanzadas mientras se está ejecutando dinámicamente el código protegido por su **try**.

Para capturar esa excepción síncrona dentro del callback, el **try** debe estar dentro:

```
~~~js setTimeout(() => { try { throw new Error("Boom"); } catch (error) { console.log("Capturado:", error.message); } }, 1000); ~~~
```

En código moderno, lo habitual es representar el resultado asíncronico con una Promise y manejar su rechazo.

28. async/await y try...catch

Una función **async** siempre devuelve una Promise. Si dentro devuelve un valor, la Promise se cumple con ese valor. Si lanza una excepción, la Promise se rechaza con ese error.

```
~~~js async function obtenerDatos() { throw new Error("No se pudieron obtener datos"); } ~~~
```

Podemos manejarla con **await**:

```
~~~js async function main() { try { await obtenerDatos(); } catch (error) { console.log(error.message); } }

main(); ~~~
```

Resultado:

~~~text No se pudieron obtener datos ~~~

**await** suspende esa función async hasta que la Promise se resuelva. Si se rechaza, **await** se comporta como si lanzara el motivo del rechazo en ese punto. Por eso el **catch** que rodea al **await** puede capturarlo.

El **await** debe estar dentro del **try**:

```
~~~js async function cargar() { try { const datos = await obtenerDatos(); console.log(datos); } catch (error) { console.error("Falló la carga:", error); } } ~~~
```

### Un detalle importante

Crear una Promise dentro de un **try**, pero no esperarla, no permite capturar su rechazo:

```
~~~js async function ejemplo() { try { obtenerDatos(); // Falta await. } catch (error) {  
console.log("Este catch no captura el rechazo"); } } ~~~
```

Debe utilizarse:

```
~~~js await obtenerDatos(); ~~~
```

o devolverse la Promise a quien vaya a manejarla:

```
~~~js return obtenerDatos(); ~~~
```

---

## 29. El equivalente con Promises

---

Con **async/await**:

```
~~~js async function main() { try { const datos = await obtenerDatos(); console.log(datos); } catch  
(error) { console.error(error); } } ~~~
```

Con métodos de Promise:

```
~~~js obtenerDatos() .then((datos) => { console.log(datos); }) .catch((error) => {  
console.error(error); }); ~~~
```

Ambos modelan:

~~~text Promise cumplida → valor Promise rechazada → motivo del rechazo ~~~

Además, si una función de **then** lanza un error, la Promise resultante se rechaza y un **catch** posterior puede manejarlo:

```
~~~js obtenerDatos() .then((datos) => { return JSON.parse(datos); }) .catch((error) => {  
console.error("La descarga o la conversión falló:", error); }); ~~~
```

El método **finally** de Promise sirve para una limpieza que no depende del resultado:

```
~~~js obtenerDatos() .then(procesar) .catch(mostrarError) .finally(ocultarIndicador); ~~~
```

No hay que mezclar los dos estilos sin necesidad. Una cadena coherente suele ser más fácil de leer y razonar.

30. fetch tiene una sutileza importante

Esto captura, por ejemplo, un rechazo causado por un problema de red o por una cancelación:

```
~~~js try { const respuesta = await fetch(url); } catch (error) { console.log("No se pudo realizar la  
solicitud"); } ~~~
```

Pero una respuesta HTTP con estado 404, 403 o 500 normalmente **no hace que fetch rechace la Promise**. El servidor respondió correctamente a nivel del protocolo, aunque su respuesta represente un resultado fallido para nuestra aplicación.

Hay que verificar:

```
~~~js const respuesta = await fetch(url);

if (!respuesta.ok) { throw new Error("HTTP " + respuesta.status); } ~~~
```

Ejemplo completo:

```
~~~js async function cargarDatos(url) { try { const respuesta = await fetch(url);

if (!respuesta.ok) { throw new Error("HTTP " + respuesta.status); }

const datos = await respuesta.json(); return datos; } catch (error) { console.error("No se pudo
obtener la información:", error); throw error; } } ~~~
```

Este único **catch** puede recibir errores diferentes:

- un rechazo de **fetch** por un fallo de red;
- el error HTTP que nuestro código lanzó;
- un `SyntaxError` de **respuesta.json** si el cuerpo no es JSON válido;
- cualquier bug ocurrido dentro del bloque.

Por eso, en una aplicación real, puede convenir distinguirlos con tipos personalizados o separar las etapas.

31. Un ejemplo completo con fetch y un error de dominio

```
~~~js class ErrorHttp extends Error { constructor(status, url) { super("La solicitud respondió HTTP
" + status); this.name = "ErrorHttp"; this.status = status; this.url = url; } }

async function obtenerUsuario(id) { const url = "/api/usuarios/" + id; const respuesta = await
fetch(url);

if (!respuesta.ok) { throw new ErrorHttp(respuesta.status, url); }

return respuesta.json(); }

async function mostrarUsuario(id) { try { const usuario = await obtenerUsuario(id);
console.log(usuario.nombre); } catch (error) { if (error instanceof ErrorHttp && error.status ===
404) { console.log("El usuario no existe"); return; }

console.error("No se pudo mostrar el usuario:", error); throw error; } } ~~~
```

Observá la separación:

~~~text obtenerUsuario() | |—— sabe detectar que la operación falló | |—— throw | ↓  
mostrarUsuario() | |—— resuelve el 404 que sí conoce | |—— deja propagarse lo demás ~~~

Esta arquitectura es más importante que memorizar la sintaxis.

---

## 32. Condiciones esperables frente a excepciones

---

No todo resultado negativo es excepcional.

Por ejemplo:

```
~~~js function edadValida(edad) { return edad >= 0 && edad <= 130; } ~~~
```

Una edad incorrecta en un formulario puede ser una situación perfectamente esperable:

```
~~~js if (!edadValida(edad)) { console.log("Edad inválida"); } ~~~
```

No necesitamos lanzar y capturar un error para cada dato que el usuario pueda corregir.

Una primera regla mental útil:

~~~text condición esperable ↓ if

fallo excepcional de una operación ↓ throw / try / catch ~~~

Ejemplos de condiciones normalmente esperables:

- un campo vacío antes de enviar un formulario;
- una búsqueda sin resultados;
- una opción que el usuario no seleccionó;
- una contraseña que no cumple todavía las reglas mientras se escribe.

Ejemplos que pueden justificar una excepción:

- no se pudo analizar una configuración que debía ser válida;
- se perdió la conexión durante una operación;
- una dependencia devolvió datos que violan su contrato;
- una función recibió un argumento imposible para su contrato interno;
- una operación no puede entregar el resultado prometido.

No es una ley absoluta. La API debe tener un contrato claro y consistente: quien la usa necesita saber qué se devuelve y qué puede lanzar o rechazar.

Excepción no es sinónimo de bug

Un error puede representar:

- un bug de programación;
- un fallo externo;
- datos corruptos;
- una regla de negocio que impide completar la operación.

Un **catch** no corrige automáticamente un bug. Si oculta un `TypeError` inesperado y deja continuar el programa, puede empeorar el problema.

33. No uses excepciones como un if sofisticado

Esto es posible:

```
~~~js try { if (edad < 18) { throw new Error(); }  
console.log("Mayor"); } catch { console.log("Menor"); } ~~~
```

Pero el flujo normal ya se expresa claramente con:

```
~~~js if (edad >= 18) { console.log("Mayor"); } else { console.log("Menor"); } ~~~
```

Las excepciones no deberían convertirse en el mecanismo habitual de decisión del programa. Al leer **throw**, debería entenderse que el camino normal de la operación ya no puede continuar.

34. Buenas prácticas

1. Lanzá objetos `Error`

```
~~~js throw new Error("Descripción concreta del problema"); ~~~
```

Esto conserva tipo, mensaje y pila.

2. Usá mensajes con contexto

```
~~~js throw new Error("No se pudo guardar el pedido " + pedido.id); ~~~
```

El mensaje debe ayudar a diagnosticar, sin incluir contraseñas, tokens ni otros datos sensibles.

3. Usá tipos personalizados para decisiones del programa

```
~~~js if (error instanceof UsuarioNoEncontradoError) { // Respuesta específica. } ~~~
```

No dependas del texto exacto de **message**.

4. Mantené pequeño el bloque try

Protegé la operación que puede fallar y evitá mezclar decenas de tareas independientes bajo un único **catch**.

5. No ocultes errores desconocidos

```
~~~js catch (error) { if (puedoManejar(error)) { manejar(error); } else { throw error; } } ~~~
```

6. Agregá contexto sin perder la causa

```
~~~js throw new Error("No se pudo importar el archivo", { cause: error }); ~~~
```

7. Usá finally para limpieza

No lo uses para cambiar el resultado con **return** o **throw** salvo que sea una decisión deliberada y muy justificada.

8. En asincronía, esperá o devolvé la Promise

```
~~~js await operacionAsincronica(); ~~~
```

o:

```
~~~js return operacionAsincronica(); ~~~
```

Una Promise ignorada también puede producir un rechazo ignorado.

9. Con fetch, verificá response.ok

Un estado HTTP fallido no se transforma solo en excepción.

10. Separá el mensaje para el usuario del diagnóstico técnico

El usuario quizá necesite:

```
~~~text No pudimos guardar los cambios. Intentá nuevamente. ~~~
```

El registro técnico puede conservar el error, la pila y el contexto necesarios para investigar.

11. Documentá el contrato

Si una función puede lanzar tipos concretos de error, quien la utiliza debería poder saberlo. Las excepciones no reemplazan un diseño claro de la API.

35. La sintaxis completa

La forma general es:

```
~~~js try { // Código que puede lanzar una excepción. } catch (error) { // Código para manejarla. }  
finally { // Código que debe ejecutarse siempre. } ~~~
```

Podemos lanzar nuestras propias excepciones:

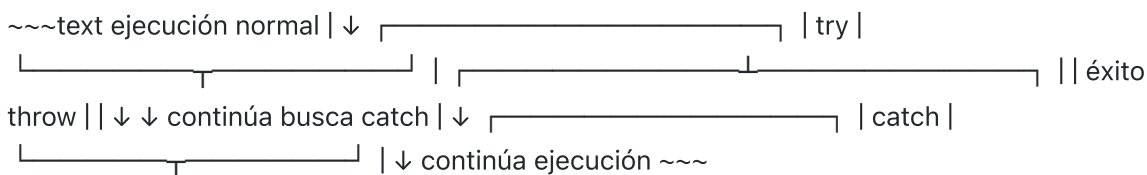
```
~~~js throw new Error("Descripción del problema"); ~~~
```

Las partes se relacionan así:

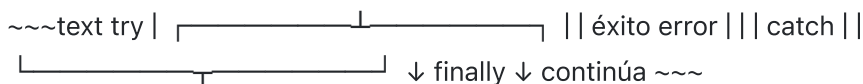
~~~text try delimita el código protegido throw interrumpe y transporta el fallo catch recibe y maneja la excepción finally garantiza una acción final ~~~

## 36. Modelo mental final

Podemos resumir **try...catch** en este esquema:



Si existe **finally**:



Si ninguna función de la pila captura la excepción:

~~~text función profunda | throw ↑ sale de cada llamada ↑ no encuentra catch ↑ excepción no capturada ~~~

Y, en asincronía:

~~~text callback posterior de setTimeout ≠ ejecución dinámica del try exterior

Promise rechazada | |—— await dentro de try → catch | |—— cadena de Promise → .catch() ~~~

Una forma precisa de explicarlo en una sola frase es:

**Ejecutá normalmente este código; si durante su ejecución se lanza una excepción, interrumpí ese camino, buscá el catch correspondiente y transferí allí el control junto con la información del fallo.**

Ese concepto de **transferencia del flujo de control** es el corazón de las excepciones.

**try...catch** no es realmente un mecanismo para «detectar errores». Es un mecanismo para cambiar abruptamente el flujo del programa y transportar información sobre un fallo desde el lugar donde se detecta hasta el lugar donde puede manejarse.

# 050-fundamentos-computacion-digital

---

Fundamentos de la computación digital

## 1. Una idea sorprendentemente simple

Una computadora puede realizar tareas extraordinariamente complejas:

- procesar textos e imágenes,
- ejecutar videojuegos,
- realizar cálculos científicos,
- reproducir música,
- comunicarse por Internet,
- ejecutar sistemas de inteligencia artificial.

Sin embargo, toda esa complejidad puede construirse a partir de elementos extremadamente simples.

La idea central que vamos a estudiar es:

Comportamientos complejos pueden construirse combinando una gran cantidad de operaciones extremadamente simples.

Para comprender cómo es posible, necesitamos tres ideas fundamentales:

1. representar información mediante valores discretos;
2. realizar operaciones lógicas sobre esos valores;
3. combinar operaciones simples para construir operaciones más complejas.

---

## 1. La representación binaria

La forma más sencilla de representar información consiste en distinguir solamente entre dos estados.

Podemos llamarlos:

0 1

Pero también podrían representar:

falso verdadero apagado encendido no sí abierto cerrado 0 V 5 V

Lo importante no es qué significado físico tengan esos estados.

Lo importante es que podamos distinguirlos claramente.

A cada unidad de información que puede tomar uno de dos valores la llamamos bit.

---

### 1. Codificar información

Los bits no tienen significado por sí mismos.

Nosotros decidimos qué representan.

Por ejemplo, imaginemos un dado.

Tiene seis resultados posibles:

1 2 3 4 5 6

Con un solo bit podemos representar solamente dos posibilidades.

Con dos bits:

00 01 10 11

podemos representar cuatro.

Con tres bits tenemos ocho combinaciones:

000 001 010 011 100 101 110 111

Por lo tanto, tres bits son suficientes para codificar las seis caras de un dado.

Por ejemplo:

Dado Binario 1 001 2 010 3 011 4 100 5 101 6 110

Las combinaciones 000 y 111 simplemente pueden quedar sin utilizar.

Esto muestra una idea importante:

Una secuencia de bits puede representar información que originalmente no tiene nada de "binaria".

Puede representar números, letras, colores, sonidos, posiciones de un juego o cualquier otra cosa que podamos codificar.

---

### 1. De representar información a procesarla

Supongamos ahora que tiramos dos dados.

Cada dado puede representarse mediante tres bits:

$A = a_2 a_1 a_0$   $B = b_2 b_1 b_0$

Queremos construir un sistema que responda una pregunta muy sencilla:

¿Los dos dados suman 7?

La salida también puede representarse mediante un bit:

0 → no gané 1 → gané

Tenemos entonces una función:

$f(A, B) \rightarrow \{0, 1\}$

Por ejemplo:

$1 + 6 \rightarrow 1$   $2 + 5 \rightarrow 1$   $3 + 4 \rightarrow 1$   $4 + 3 \rightarrow 1$   $5 + 2 \rightarrow 1$   $6 + 1 \rightarrow 1$

mientras que:

$1 + 1 \rightarrow 0$   $2 + 2 \rightarrow 0$   $3 + 2 \rightarrow 0 \dots$

Aquí aparece una idea fundamental de la computación digital:

Un cálculo puede entenderse como una función que transforma una combinación de bits de entrada en bits de salida.

---

## 1. Funciones booleanas

Cuando las variables solamente pueden tomar los valores 0 y 1, podemos trabajar con el álgebra de Boole.

Sus operaciones fundamentales pueden expresarse mediante tres operadores muy sencillos:

NOT AND OR

NOT

NOT invierte un valor.

A NOT A 0 1 1 0

Podemos escribir:

$\neg A$

---

AND

AND produce 1 solamente cuando ambas entradas son 1.

A B A AND B 0 0 0 0 1 0 1 0 0 1 1 1

Podemos escribir:

$A \wedge B$

---

OR

OR produce 1 cuando al menos una entrada es 1.

A B A OR B 0 0 0 0 1 1 1 0 1 1 1 1

Podemos escribir:

$A \vee B$

---

### 1. Construir comportamientos complejos

La potencia de estas operaciones aparece cuando comenzamos a combinarlas.

Por ejemplo:

$(A \text{ AND } B) \text{ OR } (\text{NOT } C)$

es decir:

$(A \wedge B) \vee \neg C$

Ya no estamos realizando una única operación elemental.

Estamos construyendo una función más compleja utilizando funciones más sencillas.

Este principio puede repetirse tantas veces como sea necesario:

operaciones simples ↓ combinaciones ↓ funciones más complejas ↓ combinaciones de funciones  
↓ sistemas todavía más complejos

Esta composición es una de las ideas esenciales de la computación.

---

### 1. La tabla de verdad

Una función booleana puede describirse completamente mediante una tabla de verdad.

Si tenemos dos entradas:

A B

existen solamente cuatro combinaciones posibles:

00 01 10 11

Por ejemplo, imaginemos una función cualquiera:

| A | B | F |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 0 |

La tabla no necesita explicar cómo calculamos F.

Simplemente enumera qué resultado debe producir para cada entrada posible.

Por eso podemos pensar una tabla de verdad como una definición por enumeración de una función.

---

#### 1. De una tabla de verdad a una expresión lógica

Aquí aparece un resultado particularmente importante.

Podemos convertir cualquier tabla de verdad en una expresión construida solamente con NOT, AND y OR.

Veamos el procedimiento.

Tomemos nuevamente:

| A | B | F |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 0 |

Nos interesan las filas donde:

$F = 1$

La primera es:

$A = 0 \ B = 1$

Queremos construir una expresión que sea verdadera exactamente para esa combinación.

Como A debe ser cero, lo negamos:

NOT A

Como B debe ser uno, lo dejamos como está.

Entonces:

$(\text{NOT } A) \text{ AND } B$

es decir:

$\neg A \wedge B$

---

La otra fila verdadera es:

$A = 1 \ B = 0$

Por lo tanto:

$A \text{ AND } (\text{NOT } B)$

o:

$A \wedge \neg B$

Finalmente queremos aceptar una combinación O la otra:

$((\text{NOT } A) \text{ AND } B) \text{ OR } (A \text{ AND } (\text{NOT } B))$

Matemáticamente:

$(\neg A \wedge B) \vee (A \wedge \neg B)$

Acabamos de obtener una expresión lógica directamente a partir de la tabla.

---

#### 1. El procedimiento general: forma normal disyuntiva

El procedimiento anterior se conoce como Forma Normal Disyuntiva (FND) o suma de productos.

El algoritmo conceptual es muy sencillo:

##### Paso 1

Buscar todas las filas donde:

salida = 1

##### Paso 2

Para cada una de esas filas, construir una expresión AND.

Para cada variable:

si aparece 1 → usar la variable si aparece 0 → usar NOT variable

Por ejemplo, la fila:

$A = 1 \ B = 0 \ C = 1$

produce:

$A \text{ AND } (\text{NOT } B) \text{ AND } C$

##### Paso 3

Unir todas esas expresiones mediante OR.



Es decir:

fila1 OR fila2 OR fila3 OR ...

El resultado es una expresión que reproduce exactamente la tabla de verdad original.

---

### 1. Una consecuencia fundamental

Este procedimiento demuestra algo muy poderoso.

Si podemos describir un comportamiento mediante una tabla de verdad, entonces podemos construirlo utilizando:

NOT AND OR

Por lo tanto:

NOT, AND y OR son suficientes para expresar cualquier función booleana.

No significa que la expresión obtenida sea siempre la más pequeña o eficiente.

Significa algo más fundamental:

siempre podemos construirla.

---

### 1. Álgebra de Boole

Las expresiones booleanas forman un álgebra.

Por lo tanto, podemos transformarlas utilizando reglas matemáticas.

Por ejemplo, existen propiedades semejantes a las del álgebra convencional.

Distributividad

$A \text{ AND } (B \text{ OR } C)$

puede distribuirse:

$(A \text{ AND } B) \text{ OR } (A \text{ AND } C)$

También podemos realizar el proceso inverso y factorizar:

$(A \text{ AND } B) \text{ OR } (A \text{ AND } C)$

como:

$A \text{ AND } (B \text{ OR } C)$

Esto permite simplificar expresiones y, eventualmente, construir circuitos más sencillos.

---

### 1. Leyes de De Morgan

Dos transformaciones particularmente importantes son las leyes de De Morgan:

$$\text{NOT } (A \text{ AND } B) = (\text{NOT } A) \text{ OR } (\text{NOT } B)$$

y:

$$\text{NOT } (A \text{ OR } B) = (\text{NOT } A) \text{ AND } (\text{NOT } B)$$

En símbolos:

$$\neg(A \wedge B) = \neg A \vee \neg B \quad \neg(A \vee B) = \neg A \wedge \neg B$$

Estas leyes permiten transformar unas operaciones lógicas en otras y serán especialmente importantes cuando estudiemos compuertas universales.

---

### 1. De las matemáticas a un mecanismo físico

Hasta aquí todo podría parecer simplemente matemática.

Pero podemos construir dispositivos físicos que implementen estas operaciones.

Necesitamos algún elemento que permita que una señal controle a otra.

Históricamente esto puede realizarse mediante relés.

Un relé puede pensarse, de manera simplificada, como:

un interruptor controlado eléctricamente.

Una señal eléctrica decide si otro circuito queda abierto o cerrado.

---

### 1. Construir AND mediante interruptores

Imaginemos dos interruptores conectados en serie:

+ ----[ A ]----[ B ]---- 💡 ---- -

La corriente solamente puede circular si:

A está cerrado Y B está cerrado

Por lo tanto:

serie → AND

Su comportamiento es:

A B Lámpara 0 0 0 0 1 0 1 0 0 1 1 1

Es exactamente la tabla de verdad de AND.

\_\_\_\_\_

#### 1. Construir OR mediante interruptores

Ahora conectemos los interruptores en paralelo:

+---[ A ]---+ + ---+ +---  --- - +---[ B ]---+

Existe corriente si está cerrado:

A O B

Por lo tanto:

paralelo → OR

Nuevamente obtenemos físicamente la misma función que habíamos definido matemáticamente.

\_\_\_\_\_

#### 1. De relés a computadoras

Una vez que podemos construir físicamente:

NOT AND OR

podemos construir combinaciones de ellas.

Y como vimos anteriormente:

NOT + AND + OR

permiten construir cualquier función booleana.

Por lo tanto podemos pasar de:

Álgebra de Boole ↓ expresiones lógicas ↓ compuertas ↓ circuitos ↓ sistemas digitales

Los primeros computadores electromecánicos utilizaron relés.

Posteriormente se utilizaron válvulas de vacío y finalmente transistores.

La tecnología física cambió radicalmente.

La lógica fundamental permaneció.

\_\_\_\_\_

#### 1. Representar números en binario

Hasta ahora utilizamos 0 y 1 principalmente como valores lógicos.

Pero también podemos utilizarlos para representar números.

Nuestro sistema decimal utiliza potencias de 10.

Por ejemplo:

583

significa:

$$5 \times 100 + 8 \times 10 + 3 \times 1$$

o:

$$5 \times 10^2 + 8 \times 10^1 + 3 \times 10^0$$

El sistema binario utiliza exactamente la misma idea, pero con potencias de 2.

Por ejemplo:

$1011_2$

significa:

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

es decir:

$$8 + 0 + 2 + 1 = 11$$

Por lo tanto:

$$1011_2 = 11_{10}$$

No estamos utilizando una matemática diferente.

Simplemente cambiamos la base de representación.

---

### 1. Sumar números binarios

La suma binaria funciona de manera completamente análoga a la suma decimal.

En decimal sabemos que:

$$9 + 1 = 10$$

Escribimos 0 y llevamos 1 a la siguiente columna.

En binario ocurre lo mismo cuando superamos el mayor dígito disponible:

$$0 + 0 = 0 \quad 0 + 1 = 1 \quad 1 + 0 = 1 \quad 1 + 1 = 10$$

Por ejemplo:

$$101 + 011 \text{ ----- } 1000$$

Es exactamente el mismo procedimiento que aprendimos para sumar números decimales.

---

### 1. El sumador como función lógica

Ahora podemos conectar dos partes de nuestra explicación.

La suma de dos bits puede describirse mediante una tabla de verdad.

| A | B | Suma | Acarreo |
|---|---|------|---------|
| 0 | 0 | 0    | 0       |
| 0 | 1 | 1    | 0       |
| 1 | 0 | 1    | 0       |
| 1 | 1 | 0    | 1       |

Tenemos dos entradas:

A B

y dos salidas:

S = suma C = acarreo

Cada salida es simplemente una función booleana.

Por lo tanto podemos obtener expresiones para ambas.

Para la suma:

$$S = (\text{NOT } A \text{ AND } B) \text{ OR } (A \text{ AND NOT } B)$$

Para el acarreo:

$$C = A \text{ AND } B$$

Este circuito se denomina medio sumador (half adder).

Aquí aparece una conexión crucial:

La aritmética puede construirse utilizando lógica booleana.

---

### 1. Multiplicación binaria

La multiplicación también funciona de manera análoga a la multiplicación decimal.

Pero resulta incluso más sencilla porque solamente existen dos dígitos.

Las reglas elementales son:

$$0 \times 0 = 0 \quad 0 \times 1 = 0 \quad 1 \times 0 = 0 \quad 1 \times 1 = 1$$

Por lo tanto, multiplicar un número por un bit significa esencialmente decidir entre:

0

o:

el mismo número

A partir de operaciones elementales de este tipo, desplazamientos y sumas podemos construir multiplicadores binarios.

Otra vez aparece el mismo principio:

operaciones elementales  $\downarrow$  combinación  $\downarrow$  operaciones aritméticas complejas

---

1. ¿Necesitamos realmente tres compuertas?

Hasta ahora dijimos que podemos construir cualquier función utilizando:

NOT AND OR

Pero podemos ir todavía más lejos.

Existe una operación llamada NAND:

NAND = NOT AND

Su tabla es:

| A | B | A NAND B |
|---|---|----------|
| 0 | 0 | 1        |
| 0 | 1 | 1        |
| 1 | 0 | 1        |
| 1 | 1 | 0        |

Sorprendentemente:

NAND por sí sola es suficiente para construir cualquier función booleana.

---

1. Construir NOT con NAND

Si conectamos las dos entradas:

$A \text{ NAND } A$

obtenemos:

$\text{NOT } A$

porque:

$$A \text{ NAND } A = \text{NOT } (A \text{ AND } A) = \text{NOT } A$$

# El tipo `Symbol` en JavaScript

`Symbol` es un tipo primitivo de JavaScript cuyo objetivo principal es crear **identificadores únicos**.

Aunque dos símbolos tengan la misma descripción, siguen siendo distintos:

```
const a = Symbol("id");
const b = Symbol("id");

console.log(a === b); // false
```

La cadena `"id"` es solamente una descripción útil para depuración. No define la identidad del símbolo.

## 1. Crear símbolos

Se crean con la función `Symbol()`:

```
const id = Symbol();
const nombre = Symbol("nombre");
```

Podemos inspeccionarlos:

```
console.log(typeof id);           // "symbol"
console.log(nombre);              // Symbol(nombre)
console.log(nombre.description);  // "nombre"
```

`Symbol` es uno de los tipos primitivos de JavaScript:

```
undefined
null
boolean
number
bigint
string
symbol
```

y, por supuesto, también existe el tipo `object`.

## 2. ¿Para qué sirve un símbolo?

Principalmente para crear **claves de propiedades que no colisionen con otras propiedades**.

```
const id = Symbol("id");

const persona = {
  nombre: "Ana",
  [id]: 1234
};

console.log(persona.nombre); // Ana
console.log(persona[id]);    // 1234
```

Los corchetes indican que usamos el valor almacenado en `id` como clave.

## 3. Una propiedad Symbol no es una propiedad String

```
const id = Symbol("id");

const persona = {
  id: 10,
  [id]: 20
};

console.log(persona.id); // 10
console.log(persona[id]); // 20
```

Conceptualmente:

```
persona
├── "id"      → 10
└── Symbol(id) → 20
```

Son claves completamente diferentes.

## 4. El problema que resuelve

Supongamos que una biblioteca recibe objetos creados por otra parte del programa y quiere agregar información interna:



```
function procesar(usuario) {
  usuario.estadoInterno = "procesado";
}
```

Podría haber una colisión:

```
const usuario = {
  nombre: "Ana",
  estadoInterno: "otra cosa"
};
```

Con un símbolo:

```
const estadoInterno = Symbol("estadoInterno");

function procesar(usuario) {
  usuario[estadoInterno] = "procesado";
}
```

Ahora ambas propiedades pueden coexistir.

## 5. Los símbolos son únicos

```
Symbol("x") === Symbol("x")
```

da:

```
false
```

Cada llamada crea un símbolo distinto, aunque la descripción sea la misma.

## 6. Registro global: `Symbol.for()`

JavaScript dispone de un registro global de símbolos:

```
const a = Symbol.for("usuario");
const b = Symbol.for("usuario");

console.log(a === b); // true
```

`Symbol.for()` busca primero si ya existe un símbolo con esa clave. Si existe, lo devuelve. Si no existe, lo crea.

## `Symbol()` versus `Symbol.for()`

```
const a = Symbol("x");
const b = Symbol("x");

console.log(a === b); // false
```

pero:

```
const a = Symbol.for("x");
const b = Symbol.for("x");

console.log(a === b); // true
```

## 7. `Symbol.keyFor()`

Permite recuperar la clave de un símbolo creado mediante `Symbol.for()`:

```
const s = Symbol.for("usuario");

console.log(Symbol.keyFor(s));
// "usuario"
```

Pero:

```
const s = Symbol("usuario");

console.log(Symbol.keyFor(s));
// undefined
```

porque ese símbolo no pertenece al registro global.

## 8. Las propiedades Symbol no aparecen normalmente al enumerar

```
const secreto = Symbol("secreto");

const usuario = {
  nombre: "Ana",
  edad: 25,
  [secreto]: 123
};
```

```
console.log(Object.keys(usuario));
// ["nombre", "edad"]
```

Tampoco aparecen en un `for...in`.

Eso no significa que sean privadas. Podemos obtenerlas con:

```
Object.getOwnPropertySymbols(usuario);
```

## 9. Symbol no es un mecanismo de seguridad

Una propiedad Symbol puede descubrirse:

```
const symbols = Object.getOwnPropertySymbols(usuario);
```

Por eso `Symbol` sirve para:

- evitar colisiones;
- agregar metadatos;
- implementar protocolos internos;
- personalizar comportamiento de objetos.

No sirve para proteger información secreta.

JavaScript define una serie de símbolos especiales denominados **Well-known Symbols**.

Entre los principales:

```
Symbol.iterator  
Symbol.asyncIterator  
Symbol.toPrimitive  
Symbol.toStringTag  
Symbol.hasInstance  
Symbol.match  
Symbol.replace  
Symbol.search  
Symbol.split  
Symbol.species  
Symbol.isConcatSpreadable  
Symbol.unscopables
```

Estos símbolos permiten que los objetos participen de protocolos internos del lenguaje.

## 11. Symbol.iterator

Determina cómo se recorre un objeto.

Cuando escribimos:

```
for (const x of objeto) {  
  // ...  
}
```

JavaScript busca:

```
objeto[Symbol.iterator]
```

Ejemplo:

```
const numeros = {
  desde: 1,
  hasta: 5,

  [Symbol.iterator]() {
    let actual = this.desde;
    const hasta = this.hasta;

    return {
      next() {
        if (actual <= hasta) {
          return {
            value: actual++,
            done: false
          };
        }

        return {
          done: true
        };
      }
    };
  }
};
```

Ahora:

```
for (const n of numeros) {
  console.log(n);
}
```

imprime:

```
1
2
3
4
5
```

Conceptualmente:

```
objeto
  |
  ▼
objeto[Symbol.iterator]()
  |
  ▼
iterator
  |
  ▼
next()
  |
  ├── { value: ..., done: false }
  ├── { value: ..., done: false }
  └── { done: true }
```

## 12. Por qué los arrays funcionan con `for...of`

Los arrays implementan:

```
Array.prototype[Symbol.iterator]
```

Por eso pueden recorrerse con `for...of`, spread y destructuring.

```
const numeros = [10, 20, 30];

const iterador = numeros[Symbol.iterator]();

console.log(iterador.next());
console.log(iterador.next());
console.log(iterador.next());
console.log(iterador.next());
```

## 13. `Symbol.asyncIterator`

Es el equivalente asincrónico de `Symbol.iterator`.

Se usa con:

```
for await (const elemento of fuente) {
  // ...
}
```

Ejemplo:

```
const fuente = {
  async *[Symbol.asyncIterator]() {
    yield 10;
    yield 20;
    yield 30;
  }
};

for await (const x of fuente) {
  console.log(x);
}
```

## 14. Symbol.toPrimitive

Permite controlar cómo un objeto se convierte en un valor primitivo.

```
const producto = {
  nombre: "Monitor",
  precio: 500,

  [Symbol.toPrimitive](hint) {
    if (hint === "number") {
      return this.precio;
    }

    return this.nombre;
  }
};
```

```
console.log(+producto); // 500
console.log(`${producto}`); // Monitor
```

El parámetro `hint` puede ser, según el contexto:

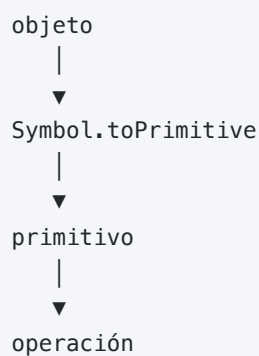
```
"number"
"string"
"default"
```

## 15. Relación con la coerción de tipos

Cuando JavaScript necesita convertir un objeto a un valor primitivo puede consultar:

```
objeto[Symbol.toPrimitive]
```

Conceptualmente:



Por eso este símbolo permite intervenir en la coerción implícita.

## 16. `Symbol.toStringTag`

Permite personalizar la identificación de un objeto:

```
const usuario = {
  [Symbol.toStringTag]: "Usuario"
};

console.log(
  Object.prototype.toString.call(usuario)
);
```

Resultado:

```
[object Usuario]
```

## 17. `Symbol.hasInstance`

Permite personalizar el comportamiento del operador `instanceof`.



```
class Adulto {
  static [Symbol.hasInstance](objeto) {
    return objeto.edad >= 18;
  }
}

const ana = {
  edad: 25
};

console.log(ana instanceof Adulto);
// true
```

Conceptualmente:

```
ana instanceof Adulto
  |
  ▼
Adulto[Symbol.hasInstance](ana)
```

## 18. Símbolos vinculados con expresiones regulares

Los principales son:

```
Symbol.match
Symbol.replace
Symbol.search
Symbol.split
```

Permiten personalizar cómo un objeto participa en operaciones como:

```
texto.match(objeto)
texto.replace(objeto, ...)
texto.search(objeto)
texto.split(objeto)
```

Ejemplo:

```
const buscador = {
  [Symbol.match](texto) {
    return texto.includes("JavaScript");
  }
};

console.log(
  "Me gusta JavaScript".match(buscador)
);
```

## 19. `Symbol.isConcatSpreadable`

Controla si un objeto se expande al utilizar `Array.prototype.concat()`.

```
const objeto = {
  0: "A",
  1: "B",
  length: 2,

  [Symbol.isConcatSpreadable]: true
};

const resultado = [1, 2].concat(objeto);

console.log(resultado);
// [1, 2, "A", "B"]
```

## 20. `Symbol.species`

Permite controlar qué constructor se utiliza cuando una operación crea objetos derivados.

```
class MiArray extends Array {
  static get [Symbol.species]() {
    return Array;
  }
}
```

Es un mecanismo avanzado y poco frecuente en código cotidiano.

## 21. `Symbol.unscopables`

Está relacionado con `with` y permite indicar qué propiedades no deben quedar disponibles dentro de ese bloque.

Como `with` está desaconsejado y no puede utilizarse en modo estricto, este símbolo tiene principalmente interés histórico.

## 22. Los símbolos más importantes para aprender

---

### Fundamentales

```
Symbol.iterator  
Symbol.asyncIterator  
Symbol.toPrimitive  
Symbol.toStringTag
```

### Para entender mecanismos internos

```
Symbol.hasInstance  
Symbol.match  
Symbol.replace  
Symbol.search  
Symbol.split
```

### Más especializados

```
Symbol.species  
Symbol.isConcatSpreadable  
Symbol.unscopables
```

## 23. Una forma poderosa de entender `Symbol`

---

La idea profunda no es solamente que `Symbol` genera claves únicas.

Los símbolos permiten que JavaScript tenga **protocolos sin depender de nombres de propiedades comunes**.

Si el lenguaje hubiera usado:

```
objeto.iterator()
```

podría entrar en conflicto con un método del usuario llamado `iterator`.

En cambio usa:

```
objeto[Symbol.iterator]()
```

Ese identificador pertenece al protocolo del lenguaje y no colisiona con:

```
objeto.iterator
```

## 24. **Symbol** como mecanismo de protocolo

Podemos pensar un objeto así:

|                       |
|-----------------------|
| objeto                |
| propiedades normales  |
| nombre                |
| edad                  |
| imprimir              |
| protocolos especiales |
| [Symbol.iterator]     |
| [Symbol.toPrimitive]  |
| [Symbol.toStringTag]  |
| ...                   |

Las propiedades normales describen los datos y comportamientos de nuestra aplicación.

Los Well-known Symbols permiten decirle al lenguaje:

Cuando quieras hacer determinada operación conmigo, hacela de esta manera.

## 25. Ejemplo integrador

```
class Rango {
  constructor(desde, hasta) {
    this.desde = desde;
    this.hasta = hasta;
  }

  *[Symbol.iterator]() {
    for (
      let n = this.desde;
      n <= this.hasta;
      n++
    ) {
      yield n;
    }
  }

  [Symbol.toPrimitive](hint) {
    if (hint === "number") {
      return this.hasta - this.desde + 1;
    }

    return `${this.desde}..${this.hasta}`;
  }

  get [Symbol.toStringTag]() {
    return "Rango";
  }
}
```

```
const rango = new Rango(3, 6);
```

Podemos recorrerlo:

```
for (const n of rango) {
  console.log(n);
}
```

Convertirlo a número:

```
console.log(+rango);
// 4
```

Convertirlo a texto:

```
console.log(`${rango}`);  
// 3..6
```

E inspeccionarlo:

```
console.log(  
  Object.prototype.toString.call(rango)  
);  
// [object Range]
```

## Idea para recordar

`Symbol` tiene dos usos relacionados pero diferentes.

Primero:

```
const id = Symbol("id");
```

nos da una **clave única**, útil para evitar colisiones.

Segundo, los símbolos especiales como:

```
Symbol.iterator  
Symbol.toPrimitive  
Symbol.hasInstance
```

actúan como **ganchos del lenguaje**.

Los Well-known Symbols constituyen un mecanismo mediante el cual los objetos pueden participar y personalizar protocolos internos de JavaScript.

# Expresiones regulares en JavaScript

Una **expresión regular** es una forma compacta de describir **patrones de texto**.

No sirve solamente para preguntar:

"¿Este texto es exactamente igual a este otro?"

También permite preguntar cosas como:

- ¿contiene un número?
- ¿empieza con una letra?
- ¿tiene formato de código postal?
- ¿hay una palabra repetida?
- ¿quiero extraer todos los números?
- ¿quiero reemplazar ciertos fragmentos?
- ¿quiero validar una estructura simple?

La idea fundamental es:

Una expresión regular describe un **conjunto de cadenas posibles**.

Por ejemplo, el patrón:

`\d`

no describe un texto concreto. Describe cualquiera de estos:

0  
1  
2  
3  
4  
5  
6  
7  
8  
9

# 1. El problema que intentan resolver

---

Supongamos que tenemos:

```
const texto = "El alumno tiene 27 años";
```

Queremos saber si contiene un número.

Sin expresiones regulares podríamos recorrer carácter por carácter:

```
function contieneNumero(texto) {  
  for (const caracter of texto) {  
    if (  
      caracter >= "0" &&  
      caracter <= "9"  
    ) {  
      return true;  
    }  
  }  
  
  return false;  
}
```

Funciona, pero estamos expresando un concepto relativamente simple de una manera bastante mecánica.

Con una expresión regular:

```
\d
```

podemos decir:

```
buscá un dígito.
```

Entonces:

```
const texto = "El alumno tiene 27 años";  
  
console.log(/\d/.test(texto));  
// true
```

La expresión regular describe **qué buscamos** y no tanto **cómo recorrer el texto para encontrarlo**.



## 2. Cómo se escribe una expresión regular en JavaScript

---

La forma más común es:

```
/patron/
```

Por ejemplo:

```
/ana/
```

Esto busca exactamente la secuencia:

```
a n a
```

Ejemplo:

```
const texto = "Me llamo Ana";  
  
console.log(/Ana/.test(texto));  
// true
```

Pero:

```
console.log(/ana/.test(texto));  
// false
```

porque por defecto las expresiones regulares distinguen mayúsculas de minúsculas.

## 3. El método `.test()`

---

Es la operación más sencilla para comenzar.

```
regex.test(texto)
```

devuelve un booleano:

```
const patron = /JavaScript/;

console.log(patron.test("Curso de JavaScript"));
// true

console.log(patron.test("Curso de Python"));
// false
```

Conceptualmente:

```
texto
  |
  ▼
¿contiene algo que coincida con el patrón?
  |
  ├── sí → true
  └── no → false
```

## 4. Caracteres especiales

La barra invertida introduce construcciones especiales.

```
\d  dígito
\D  no dígito
\w  carácter de palabra
\W  no carácter de palabra
\s  espacio en blanco
\S  no espacio en blanco
```

Ejemplo:

```
console.log(/\d/.test("abc"));
// false

console.log(/\d/.test("abc7"));
// true
```

## 5. Repeticiones

Los cuantificadores indican cuántas veces puede aparecer algo.

\*      cero o más  
+      una o más  
?      cero o una  
{n}    exactamente n  
{n,m} entre n y m  
{n,}   al menos n

## **+ : una o más veces**

`\d+`

significa:

uno o más dígitos consecutivos.

```
const texto = "Tengo 27 años";  
  
console.log(texto.match(/\d+/) [0]);  
// 27
```

## **\* : cero o más veces**

`a*`

puede coincidir con:

```
""  
"a"  
"aa"  
"aaa"
```

Por eso `*` puede coincidir incluso con una cadena vacía.

## **? : opcional**

`https?`

coincide con:

```
http  
https
```

porque la `s` es opcional.

## Cantidades exactas

```
\d{4}
```

significa exactamente cuatro dígitos.

```
console.log(/\d{4}/.test("Año 2026"));  
// true
```

También:

```
\d{2,4}
```

significa entre dos y cuatro dígitos.

## 6. Clases de caracteres

Los corchetes representan:

```
uno de estos caracteres.
```

```
[abc]
```

significa:

```
a o b o c
```

Ejemplo:

```
console.log(/[abc]/.test("mesa"));  
// true
```

porque aparece una `a`.

## Rangos

[a-z]  
[A-Z]  
[0-9]

También pueden combinarse:

[a-zA-Z0-9]

## Negar una clase

Dentro de `[]`, `^` al comienzo significa:

cualquier carácter excepto estos.

[^0-9]

Ejemplo:

```
console.log(/[^0-9]/.test("12345"));  
// false  
  
console.log(/[^0-9]/.test("123A5"));  
// true
```

## 7. El punto `.`

El punto significa:

casi cualquier carácter.

c.sa

puede coincidir con:

casa  
cosa  
c9sa  
c-sa

Si queremos buscar un punto literal:

```
\.
```

Ejemplo:

```
console.log(/\./test("ejemplo.com"));  
// true
```

## 8. Metacaracteres

Algunos caracteres participan de la sintaxis de las expresiones regulares:

```
. * + ? ^ $ ( ) [ ] { } | \
```

Para buscarlos literalmente suele ser necesario escaparlos:

```
\.  
\+  
\?  
\(  
\)
```

## 9. Inicio y final de cadena

```
^ inicio  
$ final
```

Por ejemplo:

```
^\d+$
```

significa que toda la cadena debe estar formada exclusivamente por números.

```
console.log(/^\d+$/test("123"));  
// true  
  
console.log(/^\d+$/test("123abc"));  
// false
```

## Buscar vs validar

```
/\d+/.test("abc123")  
// true
```

pregunta si existe un número en algún lugar.

En cambio:

```
/^\d+$/.test("abc123")  
// false
```

pregunta si toda la cadena es numérica.

## 10. Ejemplo: validar un legajo

Supongamos que un legajo tiene exactamente seis dígitos:

```
^\d{6}$
```

```
function legajoValido(legajo) {  
  return /\d{6}$/.test(legajo);  
}
```

```
console.log(legajoValido("123456"));  
// true  
  
console.log(legajoValido("12345"));  
// false  
  
console.log(legajoValido("A12345"));  
// false
```

## 11. Alternativas con |

La barra vertical significa:

una cosa o la otra.

gato|perro

Ejemplo:

```
const patron = /rojo|verde|azul/;  
  
console.log(patron.test("verde"));  
// true
```

## 12. Agrupación con paréntesis

(ab)+

significa repetir **ab** una o más veces.

Coincide con:

```
ab  
abab  
ababab
```

En cambio:

ab+

significa una **a** seguida de una o más **b**.

## 13. Flags

Después del cierre **/.../** pueden colocarse opciones:

/ana/i

La **i** ignora mayúsculas y minúsculas.

```
console.log(/ana/i.test("ANA"));  
// true
```

Flags importantes:



```
i  ignore case
g  global
m  multiline
s  dotAll
u  Unicode
```

## El flag **g**

```
const texto = "10 manzanas, 20 peras y 30 bananas";

console.log(texto.match(/\d+/g));
// ["10", "20", "30"]
```

## 14. Extraer información con **.match()**

```
const texto = "Juan tiene 25 años";

const resultado = texto.match(/\d+/);

console.log(resultado[0]);
// 25
```

## 15. **.matchAll()**

```
const texto = "Ana: 20, Juan: 30, Pedro: 40";

const resultados = texto.matchAll(/\w+:\s*\d+/g);

for (const resultado of resultados) {
  console.log(resultado[0]);
}
```

Resultado:

```
Ana: 20
Juan: 30
Pedro: 40
```

## 16. Grupos de captura

```
(\d{2})/(\d{2})/(\d{4})
```

Para:

```
15/08/2026
```

los grupos son:

```
grupo 1 → día  
grupo 2 → mes  
grupo 3 → año
```

```
const fecha = "15/08/2026";  
  
const resultado =  
  fecha.match(/(\d{2})/(\d{2})/(\d{4})/);  
  
console.log(resultado[1]); // 15  
console.log(resultado[2]); // 08  
console.log(resultado[3]); // 2026
```

### Grupos con nombre

```
(?<dia>\d{2})/(?<mes>\d{2})/(?<anio>\d{4})
```

```
const resultado = fecha.match(  
  /(?<dia>\d{2})/(?<mes>\d{2})/(?<anio>\d{4})/  
);  
  
console.log(resultado.groups.dia);  
console.log(resultado.groups.mes);  
console.log(resultado.groups.anio);
```

## 17. Reemplazar texto con `.replace()`

```
const telefono = "381-555-1234";

const limpio =
  telefono.replace(/-/g, "");

console.log(limpio);
// 3815551234
```

### Normalizar espacios

```
const texto = "Ana    Pérez";

const limpio =
  texto.replace(/\s+/g, " ");

console.log(limpio);
// Ana Pérez
```

### Limpiar un teléfono

```
const telefono = "+54 9 (381) 555-1234";

const limpio =
  telefono.replace(/\D/g, "");

console.log(limpio);
// 5493815551234
```

`\D` significa cualquier carácter que no sea un dígito.

## 18. Dividir texto con `.split()`

```
const texto = "Ana,Juan;Pedro María";

const nombres =
  texto.split(/[,\s]+/);

console.log(nombres);
```

Resultado:

```
["Ana", "Juan", "Pedro", "María"]
```

## 19. Unicode

Una regex como:

```
^[A-Za-z]+$
```

no contempla adecuadamente nombres como:

```
José  
María  
Ángel  
Ñúñez
```

Podemos utilizar propiedades Unicode:

```
const patron = /^p{L}+$/u;
```

`\p{L}` significa cualquier carácter Unicode clasificado como letra.

```
console.log(patron.test("José"));  
// true  
  
console.log(patron.test("Ñandú"));  
// true
```

## 20. Grupos que no capturan

Si queremos agrupar pero no guardar el resultado:

```
(?:...)
```

Ejemplo:

```
https?:/(?:www\.)?
```

`www.` queda agrupado para hacerlo opcional, pero no se captura.

## 21. Fronteras de palabra

`\b` representa una frontera de palabra.

```
\bpan\b
```

Ejemplo:

```
console.log(/\bpan\b/.test("compro pan"));  
// true  
  
console.log(/\bpan\b/.test("pantalla"));  
// false
```

## 22. Cuantificadores codiciosos

Los cuantificadores suelen ser **greedy**: consumen todo lo posible.

```
const html = "<b>uno</b><b>dos</b>";  
  
console.log(  
  html.match(/<b>.*</b>/) [0]  
);
```

Resultado:

```
<b>uno</b><b>dos</b>
```

Porque `.*` consume todo lo posible.

### Cuantificadores no codiciosos

```
.*?
```

consume lo mínimo posible.

```
console.log(  
  html.match(/<b>.*?</b>/) [0]  
);
```

Resultado:

`<b>uno</b>`

## 23. Lookahead

`\d+(?=.*€)`

significa:

uno o más dígitos seguidos de €, pero sin incluir el €.

En:

Precio: 100€

encuentra:

100

## 24. Lookbehind

`(?<= \$)\d+`

significa:

uno o más dígitos precedidos por \$.

En:

Precio: \$150

encuentra:

150

## 25. Crear expresiones regulares dinámicamente

---

Además del literal:

```
/ana/i
```

existe:

```
new RegExp(...)
```

Ejemplo:

```
const palabra = "JavaScript";

const patron =
  new RegExp(palabra, "i");

console.log(
  patron.test("javascript")
);
// true
```

### Cuidado con las barras invertidas

En un literal:

```
/\d+/
```

En una cadena:

```
new RegExp("\\d+")
```

porque la propia cadena de JavaScript también interpreta `\`.

## 26. Ejemplo: encontrar todos los números

---

```
const texto =  
  "Los valores son 12, 45 y 103";  
  
const numeros =  
  texto.match(/\d+/g);  
  
console.log(numeros);  
// ["12", "45", "103"]
```

Si queremos números reales:

```
const numeros =  
  texto.match(/\d+/g)  
    .map(Number);
```

Resultado:

```
[12, 45, 103]
```

## 27. Ejemplo: encontrar palabras

---

Para texto Unicode:

```
const texto =  
  "JavaScript es muy flexible";  
  
const palabras =  
  texto.match(/\p{L}+/gu);  
  
console.log(palabras);
```

## 28. Hashtags y menciones

---

Hashtags:

```
#\w+
```



```
const texto =  
  "Curso de #JavaScript y #Web";  
  
console.log(  
  texto.match(/#\w+/g)  
);
```

Menciones:

```
@\w+
```

```
const texto =  
  "Hola @ana, habla con @juan";  
  
console.log(  
  texto.match(/@\w+/g)  
);
```

## 29. Detectar palabras repetidas

```
const texto =  
  "Esto es muy muy importante";  
  
const patron =  
  /\b(\w+)\s+\1\b/i;  
  
console.log(patron.test(texto));  
// true
```

`\1` significa:

lo mismo que capturó el grupo 1.

## 30. Reorganizar una fecha

De:

15/08/2026

a:

2026-08-15

```
const fecha = "15/08/2026";

const resultado = fecha.replace(
  /(\d{2})\/(\d{2})\/(\d{4})/,
  "$3-$2-$1"
);

console.log(resultado);
// 2026-08-15
```

\$1 , \$2 y \$3 hacen referencia a los grupos capturados.

## 31. Validar una hora

Una regex básica:

```
^\d{2}:\d{2}$
```

aceptaría incluso:

```
99:99
```

Una mejor versión:

```
^(?:[01]\d|2[0-3]):[0-5]\d$
```

Descomposición:

```
^
inicio

(?:[01]\d|2[0-3])
hora entre 00 y 23

:
dos puntos

[0-5]\d
minutos entre 00 y 59

$
final
```

## 32. Ejemplo: validar un código

---

Formato:

```
ABC-1234
```

Regex:

```
^[A-Z]{3}-\d{4}$
```

Si queremos ignorar mayúsculas y minúsculas:

```
/^[A-Z]{3}-\d{4}$/i
```

## 33. Email: no exagerar

---

Una validación sencilla:

```
^[^\s@]+@[^\s@]+\.[^\s@]+$
```

Sirve para detectar errores evidentes, pero no conviene intentar reproducir toda la especificación formal del correo electrónico mediante una regex gigantesca.

La validación definitiva suele consistir en enviar un correo de confirmación.

## 34. No usar regex para todo

---

Las expresiones regulares son excelentes para:

- patrones lexicales;
- extraer fragmentos;
- validar estructuras simples;
- reemplazar texto;
- tokenizar formatos sencillos.

No son ideales para estructuras complejas y anidadas, como HTML completo.

Si el problema consiste principalmente en reconocer secuencias de caracteres, regex puede ser excelente.

Si hay una estructura compleja y jerárquica, probablemente necesitemos un parser.

## 35. Pensar una regex como un pequeño lenguaje

---

Una expresión regular tiene:

### Átomos

```
a  
\d  
[abc]  
.
```

### Repetición

```
*  
+  
?  
{n}
```

### Composición

```
abc
```

significa:

```
a seguido de b seguido de c
```

### Alternativa

```
a|b
```

### Agrupación

```
(...)
```

### Posición

```
^  
$  
\b
```

Con unas pocas piezas podemos construir patrones muy ricos.

## 36. De primeros principios: una regex describe posibilidades

---

Consideremos:

```
[AB]\d{2}
```

Podemos interpretarla como:

```
primer carácter:  
  A o B
```

```
segundo:  
  dígito
```

```
tercero:  
  dígito
```

Describe:

```
A00  
A01  
...  
A99  
  
B00  
B01  
...  
B99
```

Una regex es una descripción compacta de un conjunto potencialmente enorme de cadenas.

## 37. Relación con autómatas

---

Existe una conexión profunda con la teoría de computación.

Una expresión regular representa un **lenguaje regular**.

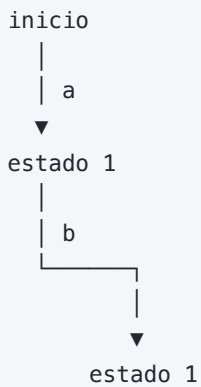
Por ejemplo:

```
ab*
```

describe:

a  
ab  
abb  
abbb  
abbbb  
...

Podemos imaginar una máquina que lee caracteres uno por uno:



El nombre “expresión regular” proviene justamente de la teoría de lenguajes regulares.

## 38. Ejemplo integrador

Supongamos líneas como:

Ana Perez;123456;3815551234  
Juan Gomez;654321;3814449876

Queremos reconocer:

nombre ; legajo ; teléfono

Podemos usar:

`^(.+);(\d{6});(\d+)$`

```
const linea =
  "Ana Perez;123456;3815551234";

const patron =
  /^(.+);(\d{6});(\d+)$/;

const resultado =
  linea.match(patron);

console.log(resultado[1]);
// Ana Perez

console.log(resultado[2]);
// 123456

console.log(resultado[3]);
// 3815551234
```

Una versión más clara usa grupos con nombre:

```
const patron =
  /^(?<nombre>.+);(?<legajo>\d{6});(?<telefono>\d+)$/;

const resultado =
  linea.match(patron);

console.log(resultado.groups);
```

Resultado aproximado:

```
{
  nombre: "Ana Perez",
  legajo: "123456",
  telefono: "3815551234"
}
```

La regex no solamente valida: también **extrae estructura de un texto plano**.

| Expresión                | Significado                                                               |
|--------------------------|---------------------------------------------------------------------------|
| <code>abc</code>         | secuencia literal <code>abc</code>                                        |
| <code>.</code>           | cualquier carácter                                                        |
| <code>d</code>           | dígito                                                                    |
| <code>D</code>           | no dígito                                                                 |
| <code>w</code>           | carácter de palabra                                                       |
| <code>W</code>           | no carácter de palabra                                                    |
| <code>s</code>           | espacio                                                                   |
| <code>S</code>           | no espacio                                                                |
| <code>[abc]</code>       | <code>a</code> , <code>b</code> o <code>c</code>                          |
| <code>[a-z]</code>       | letra de <code>a</code> a <code>z</code>                                  |
| <code>[^abc]</code>      | cualquier carácter salvo <code>a</code> , <code>b</code> , <code>c</code> |
| <code>*</code>           | cero o más                                                                |
| <code>+</code>           | uno o más                                                                 |
| <code>?</code>           | cero o uno                                                                |
| <code>{3}</code>         | exactamente tres                                                          |
| <code>{2,5}</code>       | entre dos y cinco                                                         |
| <code>^</code>           | comienzo                                                                  |
| <code>\$</code>          | final                                                                     |
| <code>b</code>           | frontera de palabra                                                       |
| <code>a b</code>         | <code>a</code> o <code>b</code>                                           |
| <code>(abc)</code>       | grupo y captura                                                           |
| <code>(?:abc)</code>     | grupo sin captura                                                         |
| <code>1</code>           | contenido del primer grupo                                                |
| <code>(?=...)</code>     | lookahead                                                                 |
| <code>(?&lt;=...)</code> | lookbehind                                                                |



# Operaciones principales en JavaScript

## Comprobar

---

```
/patron/.test(texto)
```

## Encontrar

---

```
texto.match(/patron/)
```

## Encontrar todos

---

```
texto.match(/patron/g)
```

o:

```
texto.matchAll(/patron/g)
```

## Reemplazar

---

```
texto.replace(/patron/g, reemplazo)
```

## Separar

---

```
texto.split(/patron/)
```

## Idea final

Las expresiones regulares parecen crípticas solamente cuando se intenta leerlas como una cadena de símbolos.

La forma correcta de pensarlas es:

una regex es un pequeño lenguaje para describir secuencias de caracteres.

Con unas pocas ideas:

qué carácter quiero  
+  
cuántas veces  
+  
en qué orden  
+  
entre qué alternativas  
+  
en qué posición

podemos construir la mayoría de las expresiones regulares útiles.

Por ejemplo:

```
^[A-Z]{3}-\d{4}$
```

se puede leer como:

inicio  
+  
3 letras  
+  
guion  
+  
4 números  
+  
final

La mejor manera de aprenderlas es **construirlas de izquierda a derecha y traducir cada fragmento a una regla sencilla.**

# Funciones en JavaScript

---

Una función es una pieza de código que podemos **definir una vez y ejecutar todas las veces que necesitamos**.

La idea más elemental es:

```
datos de entrada
  ↓
función
  ↓
resultado de salida
```

Por ejemplo, una función que recibe dos números y devuelve su suma:

```
function sumar(a, b) {
  return a + b;
}
```

Para utilizarla, la invocamos:

```
const resultado = sumar(4, 7);

console.log(resultado); // 11
```

En este ejemplo:

- `sumar` es el nombre de la función;
- `a` y `b` son sus parámetros;
- `4` y `7` son los argumentos enviados;
- `return a + b` produce el resultado;
- `resultado` recibe el valor retornado.

## 1. ¿Por qué necesitamos funciones?

---

Sin funciones podríamos escribir:

```
const resultado1 = 4 + 7;
const resultado2 = 10 + 20;
const resultado3 = 100 + 50;
```

Pero si la operación fuera más compleja, terminaríamos repitiendo mucho código.

Una función permite darle nombre a una operación:

```
function calcularPrecioFinal(precio, descuento) {  
  const rebaja = precio * descuento / 100;  
  
  return precio - rebaja;  
}
```

Entonces podemos reutilizarla:

```
console.log(calcularPrecioFinal(1000, 10)); // 900  
console.log(calcularPrecioFinal(500, 20)); // 400  
console.log(calcularPrecioFinal(800, 5)); // 760
```

La función separa dos cosas:

```
qué operación queremos realizar  
      ↓  
calcularPrecioFinal  
  
con qué datos queremos realizarla  
      ↓  
1000 y 10
```

## 2. Definir una función no es ejecutarla

Cuando escribimos:

```
function saludar() {  
  console.log("Hola");  
}
```

estamos **definiendo** la función.

El código interno todavía no se ejecutó.

Para ejecutarla debemos usar paréntesis:

```
saludar();
```

La diferencia es fundamental:

```
saludar
```

representa la función como valor.

En cambio:

```
saludar()
```

ejecuta la función.

Ejemplo:

```
console.log(saludar);  
// [Function: saludar]  
  
console.log(saludar());  
// ejecuta la función
```

Los paréntesis significan:

```
Invocá esta función ahora.
```

### 3. Función clásica declarada

---

La forma tradicional de declarar una función es:

```
function nombre(parametros) {  
    instrucciones;  
  
    return resultado;  
}
```

Ejemplo:

```
function multiplicar(a, b) {  
    return a * b;  
}
```

Uso:

```
const resultado = multiplicar(6, 7);

console.log(resultado); // 42
```

Esta forma se denomina habitualmente **function declaration** o declaración de función.

## Las declaraciones de función son elevadas

JavaScript permite invocar una función declarada antes de su aparición textual:

```
console.log(sumar(3, 4)); // 7

function sumar(a, b) {
  return a + b;
}
```

Esto ocurre porque las declaraciones de función participan del mecanismo llamado **hoisting**.

Conceptualmente, JavaScript conoce la declaración antes de comenzar a ejecutar el bloque:

1. Registrar la función sumar
2. Ejecutar el programa

Esto no significa que el código se mueva literalmente, pero es un buen primer modelo mental.

## 4. Expresiones de función

Una función también puede crearse como un valor y almacenarse en una variable:

```
const restar = function(a, b) {
  return a - b;
};
```

Uso:

```
console.log(restar(10, 3)); // 7
```

Aquí tenemos una función anónima:

```
function(a, b) {
  return a - b;
}
```

que se almacena en la constante:

```
restar
```

Podemos pensarlo así:

```
restar
|
▼
función
```

La función es un valor, igual que un número, una cadena o un objeto.

## Diferencia con la declaración clásica

Esto funciona:

```
console.log(sumar(2, 3));

function sumar(a, b) {
  return a + b;
}
```

Pero esto no funciona:

```
console.log(restar(5, 2));

const restar = function(a, b) {
  return a - b;
};
```

La variable `restar` no puede usarse antes de su inicialización.

## Expresión de función con nombre

Una expresión también puede tener un nombre interno:

```
const factorial = function calcularFactorial(n) {
  if (n <= 1) {
    return 1;
  }

  return n * calcularFactorial(n - 1);
};
```

Uso:

```
console.log(factorial(5)); // 120
```

El nombre interno puede ser útil para:

- recursividad;
- mensajes de error;
- depuración.

## 5. Funciones flecha

---

Las funciones flecha, o **arrow functions**, son una forma más compacta de escribir funciones.

La versión completa sería:

```
const sumar = (a, b) => {  
  return a + b;  
};
```

Equivale, para este caso sencillo, a:

```
const sumar = function(a, b) {  
  return a + b;  
};
```

Uso:

```
console.log(sumar(4, 5)); // 9
```

La flecha:

```
=>
```

separa los parámetros del cuerpo de la función.

```
parámetros => resultado
```

## 6. Abreviaturas de las funciones flecha

---

Las funciones flecha poseen varias abreviaturas. Conviene aprenderlas gradualmente.



## Forma completa

```
const doble = (numero) => {  
  return numero * 2;  
};
```

## Un solo parámetro: pueden omitirse los paréntesis

```
const doble = numero => {  
  return numero * 2;  
};
```

Las siguientes dos funciones son equivalentes:

```
const doble1 = (numero) => {  
  return numero * 2;  
};  
  
const doble2 = numero => {  
  return numero * 2;  
};
```

Los paréntesis solamente pueden omitirse cuando existe **exactamente un parámetro simple**.

## Una sola expresión: puede omitirse `return`

```
const doble = numero => numero * 2;
```

Esto significa:

```
const doble = numero => {  
  return numero * 2;  
};
```

La forma:

```
numero => numero * 2
```

se denomina retorno implícito.

## Sin parámetros

Cuando no hay parámetros, los paréntesis son obligatorios:

```
const obtenerSaludo = () => "Hola";
```

Uso:

```
console.log(obtenerSaludo()); // Hola
```

## Varios parámetros

Con varios parámetros, los paréntesis también son obligatorios:

```
const sumar = (a, b) => a + b;
```

No podemos escribir:

```
const sumar = a, b => a + b;
```

Eso es sintácticamente incorrecto.

## Retornar un objeto

Queremos hacer esto:

```
const crearPersona = nombre => {  
  return {  
    nombre: nombre,  
    activo: true  
  };  
};
```

Podríamos intentar abreviarlo así:

```
const crearPersona = nombre => {  
  nombre: nombre,  
  activo: true  
};
```

Pero no funciona como esperamos, porque JavaScript interpreta las llaves como el cuerpo de la función.

Para retornar un objeto de manera implícita debemos envolverlo entre paréntesis:

```
const crearPersona = nombre => ({
  nombre: nombre,
  activo: true
});
```

Con abreviatura de propiedad:

```
const crearPersona = nombre => ({
  nombre,
  activo: true
});
```

Uso:

```
console.log(crearPersona("Ana"));
```

Resultado:

```
{
  nombre: "Ana",
  activo: true
}
```

La regla para recordar es:

```
x => ({ valor: x })
```

Los paréntesis indican que las llaves pertenecen a un objeto y no al cuerpo de la función.

## 7. Resumen de abreviaturas

Estas funciones son equivalentes:

```
const cuadrado = function(numero) {
  return numero * numero;
};
```

```
const cuadrado = (numero) => {
  return numero * numero;
};
```

```
const cuadrado = numero => {  
  return numero * numero;  
};
```

```
const cuadrado = numero => numero * numero;
```

La última es la más compacta, pero no siempre es la más clara.

La brevedad es útil cuando la operación es sencilla:

```
const esPar = numero => numero % 2 === 0;
```

Para operaciones más largas conviene conservar el bloque:

```
const calcularPrecio = (precio, descuento, impuesto) => {  
  const rebaja = precio * descuento / 100;  
  const precioRebajado = precio - rebaja;  
  const recargo = precioRebajado * impuesto / 100;  
  
  return precioRebajado + recargo;  
};
```

El objetivo no es ganar un campeonato mundial de eliminación de caracteres. El objetivo es que el código se entienda.

## 8. Diferencias entre una función clásica y una función flecha

Las funciones flecha no son solamente una abreviatura. Tienen diferencias de comportamiento.

### Las funciones flecha no tienen su propio `this`

En una función clásica, el valor de `this` depende de cómo se invoque la función.

```
const persona = {  
  nombre: "Ana",  
  
  saludar: function() {  
    console.log(`Hola, soy ${this.nombre}`);  
  }  
};  
  
persona.saludar();  
// Hola, soy Ana
```

También podemos usar la sintaxis abreviada de método:

```
const persona = {
  nombre: "Ana",

  saludar() {
    console.log(`Hola, soy ${this.nombre}`);
  }
};
```

En cambio, una función flecha captura el `this` del entorno exterior.

Por eso no suele convenir usar una flecha como método principal de un objeto:

```
const persona = {
  nombre: "Ana",

  saludar: () => {
    console.log(this.nombre);
  }
};
```

Ese `this` no representa necesariamente a `persona`.

## Las flechas son útiles dentro de métodos

```
const persona = {
  nombre: "Ana",

  saludarMasTarde() {
    setTimeout(() => {
      console.log(`Hola, soy ${this.nombre}`);
    }, 1000);
  }
};
```

La flecha captura el `this` del método `saludarMasTarde`.

Conceptualmente:

```
saludarMasTarde()
├── this representa persona
│   └── función flecha
│       └── conserva ese this
```

## Las funciones flecha no tienen `arguments`

Una función clásica posee un objeto especial llamado `arguments`:

```
function mostrarArgumentos() {  
    console.log(arguments);  
}  
  
mostrarArgumentos(10, 20, 30);
```

Una función flecha no tiene su propio `arguments`.

En código moderno se prefieren los parámetros rest:

```
const mostrarArgumentos = (...valores) => {  
    console.log(valores);  
};
```

## Las funciones flecha no pueden utilizarse como constructor

Una función clásica puede utilizarse con `new`:

```
function Persona(nombre) {  
    this.nombre = nombre;  
}  
  
const ana = new Persona("Ana");
```

Una función flecha no puede:

```
const Persona = nombre => {  
    this.nombre = nombre;  
};  
  
const ana = new Persona("Ana");  
// TypeError
```

Para constructores modernos normalmente usamos clases:

```
class Persona {  
    constructor(nombre) {  
        this.nombre = nombre;  
    }  
}
```

## 9. Parámetros y argumentos

---

Es importante distinguir estos conceptos.

En la definición:

```
function sumar(a, b) {  
    return a + b;  
}
```

`a` y `b` son **parámetros**.

En la llamada:

```
sumar(10, 20);
```

`10` y `20` son **argumentos**.

```
definición:  
sumar(a, b)  
    ↑  ↑  
  parámetros  
  
invocación:  
sumar(10, 20)  
    ↑  ↑  
  argumentos
```

## 10. JavaScript no exige la cantidad exacta de argumentos

---

Podemos declarar:

```
function mostrar(a, b) {  
    console.log(a);  
    console.log(b);  
}
```

Y llamarla con un solo argumento:

```
mostrar(10);
```

Resultado:

```
10
undefined
```

Los parámetros faltantes reciben `undefined`.

También podemos enviar argumentos adicionales:

```
mostrar(10, 20, 30, 40);
```

La función utiliza `10` y `20`. Los demás argumentos no se asignan a parámetros declarados.

JavaScript no controla automáticamente la cantidad de argumentos como lo haría un lenguaje con una firma estricta.

## 11. Parámetros con valores predeterminados

Podemos asignar un valor predeterminado:

```
function saludar(nombre = "invitado") {
  return `Hola, ${nombre}`;
}
```

```
console.log(saludar("Ana"));
// Hola, Ana

console.log(saludar());
// Hola, invitado
```

El valor predeterminado se utiliza cuando el argumento es `undefined`:

```
console.log(saludar(undefined));
// Hola, invitado
```

Pero no cuando es `null`:

```
console.log(saludar(null));
// Hola, null
```

Esto sucede porque `null` es un valor enviado explícitamente.



## Los valores predeterminados pueden depender de parámetros anteriores

```
function crearRango(desde, hasta = desde + 10) {  
  return {  
    desde,  
    hasta  
  };  
}
```

```
console.log(crearRango(5));  
// { desde: 5, hasta: 15 }  
  
console.log(crearRango(5, 20));  
// { desde: 5, hasta: 20 }
```

## 12. Parámetros rest

Un parámetro rest permite recibir una cantidad variable de argumentos:

```
function sumarTodos(...numeros) {  
  let total = 0;  
  
  for (const numero of numeros) {  
    total += numero;  
  }  
  
  return total;  
}
```

Uso:

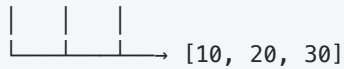
```
console.log(sumarTodos(1, 2));  
// 3  
  
console.log(sumarTodos(1, 2, 3, 4, 5));  
// 15
```

El parámetro:

```
...numeros
```

reúne los argumentos restantes en un array.

```
sumarTodos(10, 20, 30)
```



El parámetro rest debe aparecer al final:

```
function registrar(nombre, ...telefonos) {  
  console.log(nombre);  
  console.log(telefonos);  
}
```

```
registrar(  
  "Ana",  
  "3815551111",  
  "3815552222"  
);
```

Resultado:

```
"Ana"  
["3815551111", "3815552222"]
```

## 13. Parámetros mediante desestructuración

Si una función necesita muchos datos relacionados, puede recibir un objeto:

```
function registrarAlumno(alumno) {  
  console.log(alumno.nombre);  
  console.log(alumno.legajo);  
  console.log(alumno.telefono);  
}
```

Podemos desestructurarlo directamente en los parámetros:

```
function registrarAlumno({  
  nombre,  
  legajo,  
  telefono  
) {  
  console.log(nombre);  
  console.log(legajo);  
  console.log(telefono);  
}
```

Uso:

```
registrarAlumno({
  nombre: "Ana Pérez",
  legajo: "123456",
  telefono: "3815551234"
});
```

Esto tiene una ventaja importante: los argumentos tienen nombre.

Compárese:

```
registrarAlumno(
  "Ana Pérez",
  "123456",
  "3815551234",
  true,
  8
);
```

con:

```
registrarAlumno({
  nombre: "Ana Pérez",
  legajo: "123456",
  telefono: "3815551234",
  activo: true,
  nota: 8
});
```

La segunda forma es mucho más fácil de interpretar.

## Desestructuración con valores predeterminados

```
function crearUsuario({
  nombre,
  activo = true,
  rol = "usuario"
}) {
  return {
    nombre,
    activo,
    rol
  };
}
```

```
console.log(
  crearUsuario({
    nombre: "Ana"
  })
);
```

Resultado:

```
{
  nombre: "Ana",
  activo: true,
  rol: "usuario"
}
```

## Desestructuración de arrays

También puede desestructurarse un array:

```
function calcularDistancia([x1, y1], [x2, y2]) {
  const diferenciaX = x2 - x1;
  const diferenciaY = y2 - y1;

  return Math.sqrt(
    diferenciaX ** 2 +
    diferenciaY ** 2
  );
}
```

Uso:

```
console.log(
  calcularDistancia(
    [0, 0],
    [3, 4]
  )
);
// 5
```

## 14. ¿Los parámetros se pasan por valor o por referencia?

JavaScript pasa todos los argumentos **por valor**.

Pero cuando enviamos un objeto, el valor copiado es una referencia al objeto.

## Con valores primitivos

```
function cambiar(numero) {  
  numero = 100;  
}  
  
let valor = 10;  
  
cambiar(valor);  
  
console.log(valor); // 10
```

La función recibió una copia del número.

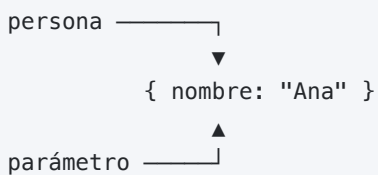
```
valor → 10  
  
cambiar recibe otra variable:  
numero → 10
```

Modificar `numero` no modifica `valor`.

## Con objetos

```
function cambiarNombre(persona) {  
  persona.nombre = "María";  
}  
  
const persona = {  
  nombre: "Ana"  
};  
  
cambiarNombre(persona);  
  
console.log(persona.nombre);  
// María
```

La función recibió una copia de la referencia, pero ambas referencias apuntan al mismo objeto:



Por eso una mutación realizada sobre el objeto es visible fuera de la función.

## Reemplazar la referencia no reemplaza la variable exterior

```
function reemplazarPersona(persona) {  
  persona = {  
    nombre: "María"  
  };  
}  
  
const persona = {  
  nombre: "Ana"  
};  
  
reemplazarPersona(persona);  
  
console.log(persona.nombre);  
// Ana
```

La función solamente reemplazó su copia local de la referencia.

La descripción precisa es:

JavaScript pasa todo por valor. En el caso de los objetos, el valor pasado es una referencia al objeto.

## 15. Retorno de valores

La instrucción `return` cumple dos funciones:

1. termina inmediatamente la ejecución de la función;
2. entrega un valor al código que la invocó.

```
function cuadrado(numero) {  
  return numero * numero;  
}
```

```
const resultado = cuadrado(5);  
  
console.log(resultado); // 25
```

## **return** termina la función

```
function clasificarEdad(edad) {  
  if (edad < 0) {  
    return "Edad inválida";  
  }  
  
  if (edad < 18) {  
    return "Menor";  
  }  
  
  return "Adulto";  
}
```

Cuando JavaScript encuentra un **return**, no continúa ejecutando las instrucciones siguientes de esa función.

## Una función sin **return** devuelve **undefined**

```
function saludar(nombre) {  
  console.log(`Hola, ${nombre}`);  
}
```

```
const resultado = saludar("Ana");  
  
console.log(resultado);  
// undefined
```

La función produce un efecto visible, pero no retorna explícitamente ningún valor.

Conceptualmente, JavaScript actúa como si existiera:

```
function saludar(nombre) {  
  console.log(`Hola, ${nombre}`);  
  
  return undefined;  
}
```

## 16. Retornar varios valores

Una función solamente puede ejecutar un **return**, pero ese valor puede ser un objeto o un array que contenga varios datos.

## Mediante un objeto

```
function dividir(dividendo, divisor) {  
  return {  
    cociente: Math.trunc(dividendo / divisor),  
    resto: dividendo % divisor  
  };  
}
```

```
const resultado = dividir(17, 5);  
  
console.log(resultado.cociente); // 3  
console.log(resultado.resto);    // 2
```

También podemos desestructurar:

```
const { cociente, resto } = dividir(17, 5);  
  
console.log(cociente);  
console.log(resto);
```

## Mediante un array

```
function minimoYMaximo(numeros) {  
  return [  
    Math.min(...numeros),  
    Math.max(...numeros)  
  ];  
}
```

```
const [minimo, maximo] =  
  minimoYMaximo([5, 2, 9, 3]);  
  
console.log(minimo); // 2  
console.log(maximo); // 9
```

Los objetos son preferibles cuando cada resultado tiene un significado claro. Los arrays son apropiados cuando importa principalmente la posición.

## 17. Cuidado con el salto de línea después de `return`

Esto es incorrecto:



```
function crearResultado() {  
    return  
    {  
        correcto: true  
    };  
}
```

JavaScript puede interpretarlo como:

```
function crearResultado() {  
    return;  
  
    {  
        correcto: true  
    }  
}
```

La función devuelve `undefined`.

Debe escribirse:

```
function crearResultado() {  
    return {  
        correcto: true  
    };  
}
```

La regla práctica es sencilla:

No coloques un salto de línea inmediatamente después de `return`.

## 18. Alcance de las variables

El **alcance**, o *scope*, indica en qué parte del programa puede utilizarse una variable.

Una variable no existe necesariamente en todo el programa.

Existen principalmente:

- alcance global;
- alcance de función;
- alcance de bloque;
- alcance léxico.

## 19. Alcance global

Una variable declarada fuera de todas las funciones y bloques pertenece al ámbito exterior:

```
const nombreAplicacion = "Gestor de alumnos";

function mostrarNombre() {
  console.log(nombreAplicacion);
}

mostrarNombre();
```

La función puede leer una variable de su entorno exterior.

Pero conviene evitar llenar el ámbito global de variables, porque distintas partes del programa pueden interferir entre sí.

## 20. Alcance de función

Los parámetros y las variables declaradas dentro de una función solamente existen dentro de ella:

```
function calcular() {
  const subtotal = 100;
  const impuesto = 21;

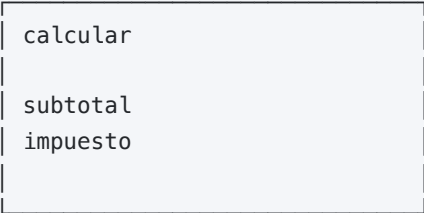
  return subtotal + subtotal * impuesto / 100;
}
```

Esto no funciona:

```
console.log(subtotal);
// ReferenceError
```

`subtotal` pertenece al ámbito de `calcular`.

Podemos imaginar que la función crea una pequeña región privada:



```
calcular
subtotal
impuesto
```

Al terminar la llamada, esas variables locales dejan de ser accesibles, salvo que alguna clausura las conserve.

## 21. Alcance de bloque

Las variables declaradas con `let` y `const` respetan los bloques:

```
if (true) {  
  const mensaje = "Hola";  
  let contador = 0;  
  
  console.log(mensaje);  
}  
  
console.log(mensaje);  
// ReferenceError
```

Un bloque está delimitado por llaves:

```
{  
  // bloque  
}
```

Esto se aplica a:

- `if`;
- `for`;
- `while`;
- bloques independientes;
- funciones.

## 22. `var` tiene alcance de función

`var` no respeta el alcance de bloque de la misma manera que `let` y `const`:

```
function ejemplo() {  
  if (true) {  
    var numero = 10;  
  }  
  
  console.log(numero); // 10  
}
```

En cambio:

```
function ejemplo() {  
  if (true) {  
    let numero = 10;  
  }  
  
  console.log(numero);  
  // ReferenceError  
}
```

Por eso, en JavaScript moderno, normalmente se utilizan:

```
const  
let
```

y se evita `var`, salvo que exista una razón específica.

## 23. Alcance léxico

JavaScript utiliza **alcance léxico**.

Esto significa que el ámbito de una función se determina por el lugar donde fue escrita, no por el lugar desde el cual fue invocada.

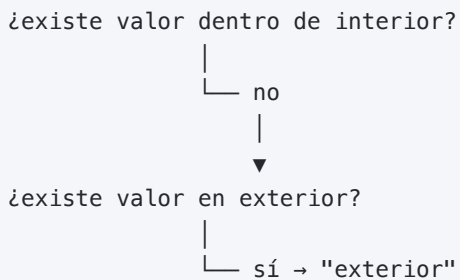
```
const valor = "global";  
  
function exterior() {  
  const valor = "exterior";  
  
  function interior() {  
    console.log(valor);  
  }  
  
  interior();  
}  
  
exterior();
```

Resultado:

```
exterior
```

La función `interior` encuentra la variable `valor` en el entorno donde fue definida.

La búsqueda conceptual es:



Si tampoco existiera allí, continuaría buscando hacia afuera.

## 24. Ocultamiento de variables

Una variable interior puede ocultar temporalmente una variable exterior con el mismo nombre:

```
const mensaje = "mensaje global";  
  
function mostrar() {  
  const mensaje = "mensaje local";  
  
  console.log(mensaje);  
}  
  
mostrar();  
// mensaje local  
  
console.log(mensaje);  
// mensaje global
```

Las dos variables son distintas:

```
ámbito global:  
mensaje → "mensaje global"  
  
ámbito de mostrar:  
mensaje → "mensaje local"
```

Este fenómeno se conoce como **shadowing** u ocultamiento.

## 25. Clausuras

Una **clausura**, o *closure*, aparece cuando una función conserva acceso a las variables del entorno donde fue creada, incluso después de que la función exterior haya terminado.

Comencemos con un ejemplo sencillo:

```
function crearSumador(cantidad) {  
  return numero => numero + cantidad;  
}
```

Uso:

```
const sumar10 = crearSumador(10);  
const sumar100 = crearSumador(100);  
  
console.log(sumar10(5)); // 15  
console.log(sumar100(5)); // 105
```

¿Qué ocurrió?

Al ejecutar:

```
const sumar10 = crearSumador(10);
```

la función exterior recibe:

```
cantidad = 10
```

y retorna:

```
numero => numero + cantidad
```

Esa función interior conserva acceso a `cantidad`.

```
sumar10  
  |  
  ▼  
numero => numero + cantidad  
                  |  
                  └─ cantidad = 10
```

La llamada a `crearSumador` ya terminó, pero la variable `cantidad` continúa disponible para la función retornada.

Eso es una clausura.

## 26. Las clausuras conservan estado

Podemos construir un contador privado:

```
function crearContador(inicial = 0) {  
  let valor = inicial;  
  
  return function() {  
    valor++;  
  
    return valor;  
  };  
}
```

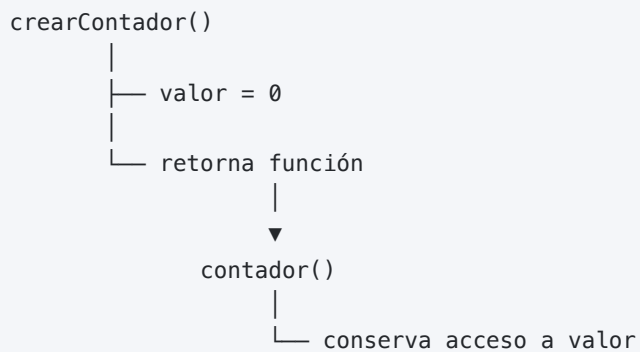
Uso:

```
const contador = crearContador();  
  
console.log(contador()); // 1  
console.log(contador()); // 2  
console.log(contador()); // 3
```

La variable `valor` no es global.

Tampoco puede modificarse directamente desde afuera.

Pero la función retornada conserva acceso a ella:



## Un objeto con operaciones privadas

```
function crearContador(inicial = 0) {  
  let valor = inicial;  
  
  return {  
    incrementar() {  
      valor++;  
    },  
  
    decrementar() {  
      valor--;  
    },  
  
    obtenerValor() {  
      return valor;  
    }  
  };  
}
```

Uso:

```
const contador = crearContador(10);  
  
contador.incrementar();  
contador.incrementar();  
contador.decrementar();  
  
console.log(contador.obtenerValor());  
// 11
```

No podemos hacer:

```
contador.valor = 100;
```

porque `valor` no es una propiedad del objeto. Es una variable local conservada por las funciones.

La clausura permite obtener una forma de encapsulamiento:

```
valor privado  
|  
├─ incrementar  
├─ decrementar  
└─ obtenerValor
```



## 27. La clausura conserva la variable, no una fotografía

```
function crearFunciones() {
  let valor = 10;

  const leer = () => valor;
  const cambiar = nuevoValor => {
    valor = nuevoValor;
  };

  return {
    leer,
    cambiar
  };
}
```

```
const estado = crearFunciones();

console.log(estado.leer()); // 10

estado.cambiar(50);

console.log(estado.leer()); // 50
```

Las funciones comparten la misma variable `valor`.

No conservaron una copia fija de su valor inicial.

## 28. Clausuras y bucles

Este ejemplo clásico utiliza `var`:

```
const funciones = [];

for (var i = 0; i < 3; i++) {
  funciones.push(() => i);
}

console.log(funciones[0]()); // 3
console.log(funciones[1]()); // 3
console.log(funciones[2]()); // 3
```

Las tres funciones conservan acceso a la misma variable `i`.

Cuando se ejecutan, el bucle ya terminó y `i` vale `3`.

Con `let`:

```
const funciones = [];  
  
for (let i = 0; i < 3; i++) {  
    funciones.push(() => i);  
}  
  
console.log(funciones[0]()); // 0  
console.log(funciones[1]()); // 1  
console.log(funciones[2]()); // 2
```

`let` crea una vinculación diferente para cada iteración.

## 29. Las funciones son valores

En JavaScript, las funciones son **ciudadanos de primera clase**.

Eso significa que pueden:

- almacenarse en variables;
- guardarse en arrays;
- colocarse en objetos;
- enviarse como argumento;
- retornarse desde otra función.

### Almacenar funciones

```
const sumar = (a, b) => a + b;
```

### Guardar funciones en un array

```
const operaciones = [  
    (a, b) => a + b,  
    (a, b) => a - b,  
    (a, b) => a * b  
];
```

```
console.log(operaciones[0](10, 5)); // 15  
console.log(operaciones[1](10, 5)); // 5  
console.log(operaciones[2](10, 5)); // 50
```

## Guardar funciones en un objeto

```
const operaciones = {  
  sumar: (a, b) => a + b,  
  restar: (a, b) => a - b,  
  multiplicar: (a, b) => a * b  
};
```

```
console.log(operaciones.sumar(4, 7));  
// 11
```

## 30. Funciones de orden superior

Una función de orden superior es una función que cumple al menos una de estas condiciones:

1. recibe una función como argumento;
2. retorna una función.

Por ejemplo:

```
function aplicarOperacion(numero, operacion) {  
  return operacion(numero);  
}
```

La función `aplicarOperacion` recibe otra función:

```
const doble = numero => numero * 2;  
const cuadrado = numero => numero ** 2;  
  
console.log(aplicarOperacion(5, doble));  
// 10  
  
console.log(aplicarOperacion(5, cuadrado));  
// 25
```

También podemos enviar la función directamente:

```
console.log(  
  aplicarOperacion(  
    5,  
    numero => numero + 100  
  )  
);  
// 105
```

## 31. Callback

Una función enviada a otra función suele denominarse **callback**.

```
function procesar(valor, callback) {  
  return callback(valor);  
}
```

Aquí:

```
valor => valor * 2
```

es el callback:

```
procesar(  
  10,  
  valor => valor * 2  
);
```

El nombre no implica necesariamente asincronía. Un callback es simplemente una función que entregamos para que otra función decida cuándo y cómo ejecutarla.

## 32. Funciones que retornan funciones

```
function multiplicarPor(factor) {  
  return numero => numero * factor;  
}
```

```
const duplicar = multiplicarPor(2);  
const triplicar = multiplicarPor(3);  
  
console.log(duplicar(10)); // 20  
console.log(triplicar(10)); // 30
```

`multiplicarPor` es de orden superior porque retorna una función.

También utiliza una clausura, porque la función retornada conserva el parámetro `factor`.

```
multiplicarPor(3)  
  |  
  └─ retorna numero => numero * factor  
        |  
        └─ factor = 3
```

Las funciones de orden superior y las clausuras suelen trabajar juntas.

## 33. Composición de funciones

---

Podemos crear una función que combine otras funciones:

```
function componer(f, g) {  
  return valor => f(g(valor));  
}
```

Ejemplo:

```
const duplicar = numero => numero * 2;  
const sumarUno = numero => numero + 1;  
  
const duplicarYSumarUno =  
  componer(sumarUno, duplicar);  
  
console.log(duplicarYSumarUno(5));  
// 11
```

El orden es:

```
5  
↓  
duplicar  
↓  
10  
↓  
sumarUno  
↓  
11
```

Esto permite construir operaciones complejas combinando transformaciones pequeñas.

## 34. Funciones de orden superior sobre arrays

---

Los arrays de JavaScript incluyen varias funciones de orden superior.

Las más importantes son:

```
map
filter
reduce
find
some
every
sort
flatMap
forEach
```

Estas funciones reciben callbacks y permiten expresar operaciones sobre colecciones.

## 35. `map`: transformar cada elemento

Supongamos:

```
const numeros = [1, 2, 3, 4];
```

Queremos obtener el doble de cada número.

Con un bucle:

```
const dobles = [];

for (const numero of numeros) {
  dobles.push(numero * 2);
}
```

Con `map`:

```
const dobles =
  numeros.map(numero => numero * 2);
```

Resultado:

```
[2, 4, 6, 8]
```

`map` significa:

Por cada elemento de entrada, producir un elemento de salida.

```
[1, 2, 3, 4]
|  |  |  |
▼  ▼  ▼  ▼
x2 x2 x2 x2
|  |  |  |
▼  ▼  ▼  ▼
[2, 4, 6, 8]
```

El array resultante tiene la misma cantidad de elementos.

## Transformar objetos

```
const alumnos = [
  {
    nombre: "Ana",
    nota: 8
  },
  {
    nombre: "Juan",
    nota: 5
  },
  {
    nombre: "María",
    nota: 9
  }
];
```

Podemos obtener solamente los nombres:

```
const nombres =
  alumnos.map(alumno => alumno.nombre);
```

Resultado:

```
["Ana", "Juan", "María"]
```

También podemos crear objetos nuevos:

```
const resumen = alumnos.map(alumno => ({
  nombre: alumno.nombre,
  aprobado: alumno.nota >= 6
}));
```

Resultado:

```
[
  {
    nombre: "Ana",
    aprobado: true
  },
  {
    nombre: "Juan",
    aprobado: false
  },
  {
    nombre: "María",
    aprobado: true
  }
]
```

## 36. Cómo podríamos construir nuestro propio `map`

```
function mapear(array, transformar) {
  const resultado = [];

  for (let indice = 0; indice < array.length; indice++) {
    const elementoTransformado = transformar(
      array[indice],
      indice,
      array
    );

    resultado.push(elementoTransformado);
  }

  return resultado;
}
```

Uso:

```
const numeros = [1, 2, 3];

const dobles = mapear(
  numeros,
  numero => numero * 2
);

console.log(dobles);
// [2, 4, 6]
```

Esto muestra qué hace `map` conceptualmente:



```
para cada elemento:  
  llamar a transformar(elemento)  
  guardar el resultado
```

## 37. **filter**: seleccionar elementos

`filter` conserva solamente los elementos que cumplen una condición.

```
const numeros = [1, 2, 3, 4, 5, 6];  
  
const pares =  
  numeros.filter(numero => numero % 2 === 0);
```

Resultado:

```
[2, 4, 6]
```

El callback debe producir un valor booleano:

```
1 → false → descartar  
2 → true  → conservar  
3 → false → descartar  
4 → true  → conservar
```

## Filtrar objetos

```
const aprobados =  
  alumnos.filter(alumno => alumno.nota >= 6);
```

Resultado:

```
[  
  {  
    nombre: "Ana",  
    nota: 8  
  },  
  {  
    nombre: "María",  
    nota: 9  
  }  
]
```

## Construir nuestro propio **filter**

```
function filtrar(array, condicion) {
  const resultado = [];

  for (let indice = 0; indice < array.length; indice++) {
    const elemento = array[indice];

    if (condicion(elemento, indice, array)) {
      resultado.push(elemento);
    }
  }

  return resultado;
}
```

Uso:

```
const pares = filtrar(
  [1, 2, 3, 4, 5, 6],
  numero => numero % 2 === 0
);
```

## 38. **reduce** : acumular un resultado

**reduce** recorre una colección y mantiene un acumulador.

Ejemplo: sumar números.

```
const numeros = [10, 20, 30];

const total = numeros.reduce(
  (acumulador, numero) => acumulador + numero,
  0
);
```

Resultado:

60

Paso por paso:

```
acumulador inicial = 0
```

```
0 + 10 = 10
```

```
10 + 20 = 30
```

```
30 + 30 = 60
```

La estructura es:

```
array.reduce(  
  (acumulador, elemento) => nuevoAcumulador,  
  valorInicial  
);
```

## Construir nuestro propio `reduce`

```
function reducir(array, combinar, valorInicial) {  
  let acumulador = valorInicial;  
  
  for (let indice = 0; indice < array.length; indice++) {  
    acumulador = combinar(  
      acumulador,  
      array[indice],  
      indice,  
      array  
    );  
  }  
  
  return acumulador;  
}
```

Uso:

```
const total = reducir(  
  [10, 20, 30],  
  (acumulador, numero) =>  
    acumulador + numero,  
  0  
);
```

## `reduce` puede producir cualquier tipo de resultado

No tiene que devolver necesariamente un número.

```
const personas = [  
  {  
    nombre: "Ana",  
    curso: "C1"  
  },  
  {  
    nombre: "Juan",  
    curso: "C3"  
  },  
  {  
    nombre: "María",  
    curso: "C1"  
  }  
];
```

```
const porCurso = personas.reduce(  
  (grupos, persona) => {  
    const curso = persona.curso;  
  
    if (!grupos[curso]) {  
      grupos[curso] = [];  
    }  
  
    grupos[curso].push(persona);  
  
    return grupos;  
  },  
  {}  
);
```

Resultado:

```
{
  C1: [
    {
      nombre: "Ana",
      curso: "C1"
    },
    {
      nombre: "María",
      curso: "C1"
    }
  ],
  C3: [
    {
      nombre: "Juan",
      curso: "C3"
    }
  ]
}
```

## 39. **find** : encontrar el primer elemento

```
const alumno = alumnos.find(
  alumno => alumno.nombre === "Juan"
);
```

Devuelve el primer elemento que cumple la condición.

Si no encuentra ninguno, retorna:

undefined

## 40. **some** : comprobar si alguno cumple

```
const hayDesaprobados =
  alumnos.some(alumno => alumno.nota < 6);
```

Resultado:

true

**some** termina apenas encuentra un elemento que cumple la condición.

Conceptualmente:

¿existe al menos uno?

## 41. `every` : comprobar si todos cumplen

```
const todosAprobaron =  
  alumnos.every(alumno => alumno.nota >= 6);
```

Conceptualmente:

¿todos cumplen?

`every` termina apenas encuentra un elemento que no cumple la condición.

## 42. `forEach` : ejecutar una acción

```
alumnos.forEach(alumno => {  
  console.log(alumno.nombre);  
});
```

`forEach` se utiliza principalmente para producir efectos:

- imprimir;
- modificar una interfaz;
- registrar información;
- enviar datos.

No crea un nuevo array útil.

```
const resultado = alumnos.forEach(  
  alumno => alumno.nombre  
);  
  
console.log(resultado);  
// undefined
```

Cuando buscamos transformar una colección, normalmente corresponde utilizar `map`, no `forEach`.

## 43. flatMap : transformar y aplanar

```
const alumnos = [
  {
    nombre: "Ana",
    temas: ["JavaScript", "HTML"]
  },
  {
    nombre: "Juan",
    temas: ["CSS", "JavaScript"]
  }
];
```

Con `map`:

```
const temas =
  alumnos.map(alumno => alumno.temas);
```

Resultado:

```
[
  ["JavaScript", "HTML"],
  ["CSS", "JavaScript"]
]
```

Con `flatMap`:

```
const temas =
  alumnos.flatMap(alumno => alumno.temas);
```

Resultado:

```
[
  "JavaScript",
  "HTML",
  "CSS",
  "JavaScript"
]
```

`flatMap` equivale aproximadamente a:

```
alumnos
  .map(alumno => alumno.temas)
  .flat();
```

## 44. Ordenar elementos

---

`sort` recibe una función de comparación:

```
const numeros = [20, 3, 100, 8];

numeros.sort((a, b) => a - b);

console.log(numeros);
// [3, 8, 20, 100]
```

La función compara dos elementos:

```
resultado negativo → a antes que b
resultado positivo → b antes que a
resultado cero     → equivalentes para el orden
```

Para ordenar en forma descendente:

```
numeros.sort((a, b) => b - a);
```

### `sort` modifica el array original

```
const originales = [3, 1, 2];

const ordenados = originales.sort(
  (a, b) => a - b
);

console.log(originales);
// [1, 2, 3]
```

Para evitarlo:

```
const ordenados = [...originales].sort(
  (a, b) => a - b
);
```

La copia se ordena y el array original se conserva.



## Ordenar objetos

```
const ordenadosPorNota = [...alumnos].sort(  
  (a, b) => b.nota - a.nota  
);
```

Por nombre:

```
const ordenadosPorNombre = [...alumnos].sort(  
  (a, b) =>  
    a.nombre.localeCompare(b.nombre)  
);
```

## 45. Encadenamiento: comportamiento similar a LINQ

Como muchos métodos retornan arrays, podemos encadenarlos:

```
const resultado = alumnos  
  .filter(alumno => alumno.nota >= 6)  
  .map(alumno => ({  
    nombre: alumno.nombre,  
    nota: alumno.nota  
  })))  
  .sort((a, b) => b.nota - a.nota);
```

La lectura es:

```
alumnos  
  ↓  
conservar los aprobados  
  ↓  
proyectar nombre y nota  
  ↓  
ordenar por nota descendente
```

Esto es muy parecido a una consulta LINQ:

```
var resultado = alumnos
    .Where(alumno => alumno.Nota >= 6)
    .Select(alumno => new
    {
        alumno.Nombre,
        alumno.Nota
    })
    .OrderByDescending(alumno => alumno.Nota)
    .ToList();
```

En JavaScript:

```
const resultado = alumnos
    .filter(alumno => alumno.nota >= 6)
    .map(alumno => ({
        nombre: alumno.nombre,
        nota: alumno.nota
    })))
    .sort((a, b) => b.nota - a.nota);
```

## 46. Ejemplo práctico completo

```
const alumnos = [  
  {  
    nombre: "Ana",  
    curso: "C1",  
    nota: 9,  
    asistencia: 0.95,  
    temas: ["JavaScript", "HTML"]  
  },  
  {  
    nombre: "Juan",  
    curso: "C1",  
    nota: 5,  
    asistencia: 0.80,  
    temas: ["JavaScript", "CSS"]  
  },  
  {  
    nombre: "María",  
    curso: "C3",  
    nota: 8,  
    asistencia: 0.70,  
    temas: ["React", "JavaScript"]  
  },  
  {  
    nombre: "Pedro",  
    curso: "C3",  
    nota: 7,  
    asistencia: 0.90,  
    temas: ["React", "Next.js"]  
  }  
];
```

Queremos:

1. conservar solamente alumnos aprobados;
2. exigir al menos 75 % de asistencia;
3. quedarnos con nombre, curso y nota;
4. ordenar por nota descendente.

```
const regularesAprobados = alumnos
  .filter(alumno =>
    alumno.nota >= 6 &&
    alumno.asistencia >= 0.75
  )
  .map(alumno => ({
    nombre: alumno.nombre,
    curso: alumno.curso,
    nota: alumno.nota
  })))
  .sort((a, b) => b.nota - a.nota);
```

Resultado:

```
[
  {
    nombre: "Ana",
    curso: "C1",
    nota: 9
  },
  {
    nombre: "Pedro",
    curso: "C3",
    nota: 7
  }
]
```

Cada paso recibe el resultado del anterior:

```
alumnos originales
  |
  ▼
filter
  |
  ▼
alumnos aprobados y regulares
  |
  ▼
map
  |
  ▼
objetos con nombre, curso y nota
  |
  ▼
sort
  |
  ▼
resultado ordenado
```

## 47. Equivalencias aproximadas entre LINQ y JavaScript

| LINQ              | JavaScript               | Propósito                  |
|-------------------|--------------------------|----------------------------|
| Where             | filter                   | conservar elementos        |
| Select            | map                      | transformar elementos      |
| SelectMany        | flatMap                  | transformar y aplanar      |
| Aggregate         | reduce                   | acumular                   |
| Any               | some                     | comprobar si alguno cumple |
| All               | every                    | comprobar si todos cumplen |
| FirstOrDefault    | find                     | encontrar el primero       |
| OrderBy           | sort con comparador      | ordenar                    |
| OrderByDescending | sort inverso             | ordenar descendente        |
| Count             | length o filter().length | contar                     |
| Contains          | includes                 | comprobar existencia       |
| Skip              | slice                    | omitir elementos           |
| Take              | slice                    | tomar elementos            |
| Distinct          | Set                      | eliminar repetidos         |
| Sum               | reduce                   | sumar                      |
| GroupBy           | reduce                   | agrupar                    |

## 48. Ejemplos de equivalencias

### Where

LINQ:

```
alumnos.Where(a => a.Nota >= 6);
```

JavaScript:

```
alumnos.filter(a => a.nota >= 6);
```

## Select

LINQ:

```
alumnos.Select(a => a.Nombre);
```

JavaScript:

```
alumnos.map(a => a.nombre);
```

## Any

LINQ:

```
alumnos.Any(a => a.Nota == 10);
```

JavaScript:

```
alumnos.some(a => a.nota === 10);
```

## All

LINQ:

```
alumnos.All(a => a.Asistencia >= 0.75);
```

JavaScript:

```
alumnos.every(  
  a => a.asistencia >= 0.75  
);
```

## FirstOrDefault

LINQ:

```
alumnos.FirstOrDefault(  
  a => a.Nombre == "Ana"  
);
```

JavaScript:

```
alumnos.find(  
  a => a.nombre === "Ana"  
);
```

JavaScript devuelve `undefined` cuando no encuentra el elemento.

## Sum

LINQ:

```
alumnos.Sum(a => a.Nota);
```

JavaScript:

```
const suma = alumnos.reduce(  
  (total, alumno) =>  
    total + alumno.nota,  
  0  
);
```

## Promedio

```
const promedio =  
  alumnos.reduce(  
    (total, alumno) =>  
      total + alumno.nota,  
    0  
  ) / alumnos.length;
```

Para un array vacío conviene controlar el caso:

```
const promedio = alumnos.length === 0  
  ? 0  
  : alumnos.reduce(  
    (total, alumno) =>  
      total + alumno.nota,  
    0  
  ) / alumnos.length;
```

## Skip y Take

LINQ:

```
elementos
  .Skip(10)
  .Take(5);
```

JavaScript:

```
elementos.slice(10, 15);
```

## Distinct

```
const valores = [
  "JS",
  "HTML",
  "JS",
  "CSS",
  "HTML"
];

const distintos = [...new Set(valores)];
```

Resultado:

```
["JS", "HTML", "CSS"]
```

## SelectMany y Distinct

Queremos todos los temas cursados, sin repetición:

```
const temas = [
  ...new Set(
    alumnos.flatMap(
      alumno => alumno.temas
    )
  )
];
```

Resultado:



```
[
  "JavaScript",
  "HTML",
  "CSS",
  "React",
  "Next.js"
]
```

## 49. Los callbacks reciben más de un argumento

Métodos como `map`, `filter` y `forEach` llaman al callback con:

```
elemento
índice
array completo
```

Ejemplo:

```
const letras = ["A", "B", "C"];

letras.forEach(
  (letra, indice, array) => {
    console.log(
      letra,
      indice,
      array
    );
  }
);
```

Por eso podemos usar el índice:

```
const nombres = ["Ana", "Juan", "María"];

const numerados = nombres.map(
  (nombre, indice) =>
    `${indice + 1}. ${nombre}`
);
```

Resultado:

```
[  
  "1. Ana",  
  "2. Juan",  
  "3. María"  
]
```

## 50. Error frecuente: olvidar `return` en una flecha con llaves

Esto no funciona como se espera:

```
const numeros = [1, 2, 3];  
  
const dobles = numeros.map(numero => {  
  numero * 2;  
});  
  
console.log(dobles);  
// [undefined, undefined, undefined]
```

Al usar llaves desaparece el retorno implícito.

Debe escribirse:

```
const dobles = numeros.map(numero => {  
  return numero * 2;  
});
```

O:

```
const dobles =  
  numeros.map(numero => numero * 2);
```

## 51. Las funciones usadas en pipelines deberían ser claras

Podemos escribir:

```
const resultado = alumnos  
  .filter(a => a.nota >= 6)  
  .map(a => a.nombre);
```

También podemos nombrar las operaciones:

```
const estaAprobado =  
  alumno => alumno.nota >= 6;  
  
const obtenerNombre =  
  alumno => alumno.nombre;  
  
const resultado = alumnos  
  .filter(estaAprobado)  
  .map(obtenerNombre);
```

Esto permite leer:

```
filtrar usando estaAprobado  
transformar usando obtenerNombre
```

Nombrar callbacks es especialmente útil cuando:

- la condición es compleja;
- se utiliza en varios lugares;
- queremos probarla por separado;
- su nombre explica mejor la intención.

## 52. Funciones puras y efectos secundarios

Una función pura:

1. para los mismos argumentos produce siempre el mismo resultado;
2. no modifica datos externos.

Ejemplo puro:

```
function sumar(a, b) {  
  return a + b;  
}
```

Otro ejemplo puro:

```
function aplicarDescuento(  
  precio,  
  porcentaje  
) {  
  return precio -  
    precio * porcentaje / 100;  
}
```

Una función con efecto secundario:

```
let total = 0;

function agregar(valor) {
  total += valor;
}
```

`agregar` modifica una variable exterior.

Los callbacks de `map`, `filter` y `reduce` suelen ser más fáciles de razonar cuando son puros:

```
const dobles =
  numeros.map(numero => numero * 2);
```

## 53. Diferencia importante con LINQ: ejecución inmediata

Los métodos de arrays como:

```
filter
map
reduce
```

se ejecutan inmediatamente.

```
const resultado = alumnos
  .filter(alumno => {
    console.log("filtrando", alumno.nombre);

    return alumno.nota >= 6;
  })
  .map(alumno => {
    console.log("transformando", alumno.nombre);

    return alumno.nombre;
  });
```

Primero se ejecuta completamente `filter` y genera un array.

Luego `map` recorre ese nuevo array.

```
array original
  ↓ filter completo
array intermedio
  ↓ map completo
array final
```

En cambio, LINQ sobre `IEnumerable<T>` suele utilizar ejecución diferida: la consulta se ejecuta al enumerarla.

Por eso el comportamiento es parecido, pero no idéntico.

## 54. Intermedios y costo de las cadenas

Esta expresión:

```
const resultado = numeros
  .filter(numero => numero % 2 === 0)
  .map(numero => numero ** 2);
```

crea normalmente:

1. un array intermedio con los pares;
2. un array final con sus cuadrados.

Para colecciones normales esto suele ser perfectamente razonable y muy claro.

Si la cantidad de datos fuera enorme, podríamos evitar el array intermedio con un único bucle:

```
const resultado = [];

for (const numero of numeros) {
  if (numero % 2 === 0) {
    resultado.push(numero ** 2);
  }
}
```

La cadena es más declarativa:

```
filtrar pares
↓
elevar al cuadrado
```

El bucle puede ser más eficiente en casos extremos, pero mezcla ambas operaciones.

Primero conviene elegir claridad. Después, si existe un problema medido, se optimiza.

## 55. Una implementación más parecida a LINQ diferido

JavaScript posee iteradores y generadores, con los que podemos construir operaciones perezosas.

### where

```
function* where(iterable, condicion) {
  for (const elemento of iterable) {
    if (condicion(elemento)) {
      yield elemento;
    }
  }
}
```

### select

```
function* select(iterable, transformar) {
  for (const elemento of iterable) {
    yield transformar(elemento);
  }
}
```

### take

```
function* take(iterable, cantidad) {
  let tomados = 0;

  for (const elemento of iterable) {
    if (tomados >= cantidad) {
      return;
    }

    yield elemento;
    tomados++;
  }
}
```

Ahora podemos crear una fuente infinita:

```
function* naturales() {
  let numero = 1;

  while (true) {
    yield numero;
    numero++;
  }
}
```

Y construir una consulta:

```
const pares = where(
  naturales(),
  numero => numero % 2 === 0
);

const cuadrados = select(
  pares,
  numero => numero ** 2
);

const primerosCinco = take(
  cuadrados,
  5
);

console.log([...primerosCinco]);
```

Resultado:

```
[4, 16, 36, 64, 100]
```

La fuente es infinita, pero solamente se generan los valores necesarios.

```
naturales
  ↓
where: solamente pares
  ↓
select: elevar al cuadrado
  ↓
take: detenerse después de 5
```

Esto se aproxima más al funcionamiento diferido de `IEnumerable<T>`.

También muestra cómo se relacionan varios conceptos:

- funciones de orden superior;
- callbacks;

- funciones generadoras;
- iteradores;
- clausuras;
- composición de operaciones.

## 56. Modelo mental de **map**, **filter** y **reduce**

Estas tres operaciones cubren una enorme cantidad de casos.

### **map**

muchos elementos  
↓  
misma cantidad de elementos transformados

```
[1, 2, 3]
  .map(x => x * 2)

// [2, 4, 6]
```

### **filter**

muchos elementos  
↓  
algunos de esos mismos elementos

```
[1, 2, 3, 4]
  .filter(x => x % 2 === 0)

// [2, 4]
```

### **reduce**

muchos elementos  
↓  
un resultado acumulado



```
[1, 2, 3, 4]
  .reduce(
    (total, x) => total + x,
    0
  )

// 10
```

En forma resumida:

```
map    → transformar
filter → seleccionar
reduce → acumular
```

## 57. Ejemplo integrador final

---

```
const ventas = [
  {
    producto: "Teclado",
    categoria: "Periféricos",
    precio: 50,
    cantidad: 2
  },
  {
    producto: "Mouse",
    categoria: "Periféricos",
    precio: 20,
    cantidad: 5
  },
  {
    producto: "Monitor",
    categoria: "Pantallas",
    precio: 300,
    cantidad: 1
  },
  {
    producto: "Webcam",
    categoria: "Periféricos",
    precio: 80,
    cantidad: 0
  }
];
```

Queremos:

1. conservar productos vendidos;
2. calcular el importe de cada producto;

3. ordenar por importe descendente;
4. calcular el total general.

## Detalle de ventas

```
const detalle = ventas
  .filter(venta => venta.cantidad > 0)
  .map(venta => ({
    producto: venta.producto,
    categoria: venta.categoria,
    importe:
      venta.precio * venta.cantidad
  }))
  .sort(
    (a, b) => b.importe - a.importe
  );
```

Resultado:

```
[
  {
    producto: "Monitor",
    categoria: "Pantallas",
    importe: 300
  },
  {
    producto: "Teclado",
    categoria: "Periféricos",
    importe: 100
  },
  {
    producto: "Mouse",
    categoria: "Periféricos",
    importe: 100
  }
]
```

## Total general

```
const total = detalle.reduce(
  (acumulador, venta) =>
    acumulador + venta.importe,
  0
);

console.log(total);
// 500
```

La consulta completa expresa claramente la intención:

```
ventas
↓
quedarse con las que tienen cantidad
↓
convertir cada venta en un importe
↓
ordenar por importe
↓
sumar los importes
```

Una función relaciona entradas con una salida:

```
argumentos
↓
función
↓
valor retornado
```

Las funciones clásicas se escriben con:

```
function sumar(a, b) {
  return a + b;
}
```

Las funciones flecha permiten formas compactas:

```
const sumar = (a, b) => a + b;
```

Los parámetros pueden incluir:

```
function ejemplo(
  obligatorio,
  predeterminado = 10,
  ...restantes
) {
  // ...
}
```

El alcance indica dónde existe una variable:

```
global
función
bloque
entorno léxico
```

Una clausura permite que una función conserve acceso a su entorno:

```
function multiplicarPor(factor) {
  return numero => numero * factor;
}
```

Las funciones de orden superior reciben o retornan funciones:

```
function aplicar(valor, operacion) {
  return operacion(valor);
}
```

Finalmente, los métodos de arrays permiten trabajar de una forma parecida a LINQ:

```
const resultado = elementos
  .filter(condicion)
  .map(transformacion)
  .sort(comparacion)
  .reduce(acumulacion, inicial);
```

La idea más importante es que una función no es solamente un bloque reutilizable. En JavaScript, una función también es un **valor que puede almacenarse, combinarse, enviarse y retornarse**. Esa característica es la base de los callbacks, las clausuras, las funciones de orden superior y las consultas encadenadas sobre colecciones.

# Gestión de archivos con JavaScript y Node.js

Un archivo parece algo sencillo: tiene un nombre, lo abrimos, guardamos información y lo cerramos. Sin embargo, para programar correctamente con archivos conviene construir primero un modelo mental preciso.

En este capítulo vamos a aprender, desde los primeros principios, a:

- comprender qué es un archivo y qué es una ruta;
- crear directorios y archivos;
- leer, escribir y agregar contenido;
- entender el texto, Unicode y las codificaciones;
- listar, buscar, copiar, mover y renombrar archivos;
- consultar sus metadatos;
- extraer información de TXT, JSON, CSV y registros de eventos;
- procesar archivos grandes sin cargarlos completos en memoria;
- manejar errores y evitar pérdidas accidentales de información.

Los ejemplos usan JavaScript ejecutado con Node.js y, salvo que se indique lo contrario, la API asíncrona basada en promesas.

## 1. El modelo más elemental: un archivo contiene bytes

Una computadora almacena información como secuencias de bits:

```
0 1 1 0 0 0 01 ...
```

Normalmente agrupamos esos bits de ocho en ocho. Cada grupo forma un **byte**:

```
8 bits = 1 byte
```

Un archivo puede pensarse, en una primera aproximación, como una secuencia ordenada de bytes guardada en un dispositivo:

```
archivo
├─ byte 0
├─ byte 1
├─ byte 2
└─ ...
```

Los bytes, por sí solos, no dicen qué representan. La misma secuencia podría interpretarse como:

- texto;
- una imagen;
- audio;
- un documento PDF;
- un programa ejecutable;
- datos comprimidos.

El **formato** establece cómo interpretar esos bytes.

## La extensión no transforma el contenido

El nombre `foto.jpg` sugiere que el archivo contiene una imagen JPEG, pero la extensión es solamente parte del nombre.

Si renombramos:

```
foto.jpg → foto.txt
```

los bytes internos no cambian. El archivo continúa conteniendo una imagen, aunque ahora su nombre resulte engañoso.

Esta diferencia será importante:

```
nombre y extensión → orientan sobre el contenido  
bytes del archivo → son el contenido real
```

---

## 2. Archivo, directorio, ruta y metadatos

Un sistema de archivos organiza la información mediante varios conceptos relacionados, pero distintos.

### Archivo

Contiene una secuencia de bytes.

### Directorio

Contiene entradas que relacionan nombres con archivos u otros directorios. Coloquialmente también lo llamamos **carpeta**.

```
curso/  
├── alumnos.txt  
├── notas.json  
└── entregas/  
    └── trabajo-01.txt
```

## Ruta

Es una descripción de la ubicación de una entrada dentro del sistema de archivos:

```
curso/entregas/trabajo-01.txt
```

## Metadatos

Son datos acerca del archivo, por ejemplo:

- tamaño;
- fechas de modificación y acceso;
- permisos;
- tipo de entrada;
- propietario, según el sistema operativo.

El nombre no forma parte de los bytes almacenados dentro del archivo. El directorio relaciona ese nombre con la entrada correspondiente.

Por eso **renombrar** un archivo normalmente no implica reescribir todo su contenido.

---

## 3. Node.js como intermediario

JavaScript, por sí solo, no define una operación universal para abrir un archivo del disco. Node.js ofrece módulos que hablan con el sistema operativo.

Los principales son:

|                  |                                        |
|------------------|----------------------------------------|
| node:fs          | operaciones con archivos y directorios |
| node:fs/promises | las mismas ideas usando promesas       |
| node:path        | construcción y análisis de rutas       |
| node:readline    | lectura de texto línea por línea       |
| node:buffer      | manipulación explícita de bytes        |

Usaremos especialmente:

```
import * as fs from "node:fs/promises";
import path from "node:path";
```

El prefijo `node:` deja claro que estamos importando un módulo incorporado en Node.js y no un paquete descargado.

## Tres estilos de API

Muchas operaciones del módulo `fs` tienen tres variantes.

### Asíncrona con promesas

```
import { readFile } from "node:fs/promises";

const texto = await readFile("mensaje.txt", "utf8");
```

Será nuestro estilo principal: no bloquea el hilo de JavaScript mientras Node.js espera la operación y se combina bien con `async` y `await`.

### Asíncrona con callback

```
import { readFile } from "node:fs";

readFile("mensaje.txt", "utf8", (error, texto) => {
  if (error) {
    console.error(error);
    return;
  }

  console.log(texto);
});
```

Es común en código antiguo y en algunas interfaces basadas en eventos.

### Síncrona

```
import { readFileSync } from "node:fs";

const texto = readFileSync("mensaje.txt", "utf8");
```

Detiene la ejecución hasta terminar. Puede ser razonable en un script muy pequeño, durante el inicio de un programa o en una herramienta que realiza una única tarea. En un servidor puede impedir que se atiendan otras solicitudes mientras espera al disco.



## 4. Preparar un proyecto mínimo

---

Creemos un directorio y un archivo `package.json` :

```
archivos-node/  
└─ package.json
```

Contenido de `package.json` :

```
{  
  "type": "module"  
}
```

La propiedad `"type": "module"` permite usar `import` y `export` en archivos con extensión `.js`.

Ahora podemos crear `inicio.js` :

```
import { writeFile } from "node:fs/promises";  
  
await writeFile("saludo.txt", "Hola desde Node.js\n", "utf8");  
  
console.log("Archivo creado");
```

Y ejecutarlo:

```
node inicio.js
```

También podríamos usar la extensión `.mjs` sin crear el `package.json`.

---

## 5. Rutas relativas y absolutas

---

Una operación necesita saber **dónde** está el archivo.

### Ruta relativa

```
datos/alumnos.txt
```

Se interpreta a partir del directorio de trabajo actual del proceso.

Podemos consultarlo con:

```
console.log(process.cwd());
```

`cwd` significa *current working directory*.

Si ejecutamos el mismo programa desde dos directorios diferentes, una misma ruta relativa puede señalar lugares diferentes.

## Ruta absoluta

Describe una ubicación desde la raíz del sistema:

```
/home/ana/proyecto/datos/alumnos.txt
```

En Windows podría verse así:

```
C:\Users\Ana\proyecto\datos\alumnos.txt
```

## Construir rutas con `node:path`

No conviene concatenar separadores manualmente:

```
// Frágil
const ruta = directorio + "/" + nombre;
```

Es preferible:

```
import path from "node:path";

const ruta = path.join("datos", "alumnos", "ana.txt");

console.log(ruta);
```

`path.join()` utiliza las convenciones del sistema operativo y normaliza separadores.

## Resolver una ruta absoluta

```
import path from "node:path";

const rutaAbsoluta = path.resolve("datos", "alumnos.txt");

console.log(rutaAbsoluta);
```

`path.resolve()` parte del directorio de trabajo, salvo que encuentre antes una ruta absoluta.

## Obtener partes de una ruta

```
import path from "node:path";

const ruta = "/curso/datos/alumnos.csv";

console.log(path.basename(ruta)); // alumnos.csv
console.log(path.extname(ruta)); // .csv
console.log(path.dirname(ruta)); // /curso/datos
console.log(path.parse(ruta));
```

`path.parse()` devuelve un objeto semejante a este:

```
const partes = {
  root: "/",
  dir: "/curso/datos",
  base: "alumnos.csv",
  ext: ".csv",
  name: "alumnos"
};
```

---

## 6. Crear directorios

Usamos `mkdir`:

```
import { mkdir } from "node:fs/promises";

await mkdir("datos");
```

Esto falla si `datos` ya existe o si falta algún directorio intermedio.

Para crear toda la cadena necesaria y no fallar si ya existe:

```
import { mkdir } from "node:fs/promises";

await mkdir("datos/2026/agosto", { recursive: true });
```

Modelo mental:

```
mkdir("a/b/c", { recursive: true })
|
├─ crea a si falta
├─ crea b si falta
└─ crea c si falta
```

---

## 7. Crear y escribir un archivo de texto

---

La operación más directa es `writeFile`:

```
import { writeFile } from "node:fs/promises";

await writeFile(
  "mensaje.txt",
  "Primera línea\nSegunda línea\n",
  "utf8"
);
```

Si `mensaje.txt` no existe, se crea. Si ya existe, su contenido se reemplaza.

La cadena contiene saltos de línea representados por `\n`:

```
Primera línea
Segunda línea
```

### Evitar sobrescribir un archivo existente

La opción `flag: "wx"` significa: abrir para escritura, pero fallar si el archivo ya existe.

```
import { writeFile } from "node:fs/promises";

await writeFile(
  "configuracion.txt",
  "modo=desarrollo\n",
  {
    encoding: "utf8",
    flag: "wx"
  }
);
```

Las banderas más habituales al escribir son:

| Bandera           | Significado práctico                   |
|-------------------|----------------------------------------|
| <code>"w"</code>  | Escribe desde cero; crea o reemplaza   |
| <code>"wx"</code> | Escribe desde cero; falla si ya existe |
| <code>"a"</code>  | Agrega al final; crea si no existe     |
| <code>"ax"</code> | Agrega; falla si ya existe             |

`writeFile` usa `"w"` por defecto.

---

## 8. Agregar contenido sin borrar lo anterior

---

`appendFile` agrega bytes al final:

```
import { appendFile } from "node:fs/promises";

await appendFile(
  "eventos.log",
  "El programa comenzó\n",
  "utf8"
);
```

Podemos llamarlo varias veces:

```
import { appendFile } from "node:fs/promises";

await appendFile("eventos.log", "Inicio\n", "utf8");
await appendFile("eventos.log", "Alumno creado\n", "utf8");
await appendFile("eventos.log", "Fin\n", "utf8");
```

Resultado:

```
Inicio
Alumno creado
Fin
```

La diferencia esencial es:

```
writeFile → reemplaza el contenido
appendFile → conserva lo anterior y agrega al final
```

---

## 9. Leer un archivo completo

---

Usamos `readFile`:

```
import { readFile } from "node:fs/promises";

const contenido = await readFile("mensaje.txt", "utf8");

console.log(contenido);
```

Al indicar `"utf8"`, pedimos que los bytes se **decodifiquen** y el resultado sea un `string` de JavaScript.

Si omitimos la codificación:

```
import { readFile } from "node:fs/promises";

const contenido = await readFile("mensaje.txt");

console.log(contenido);
```

obtenemos un `Buffer`, es decir, una representación de los bytes:

```
<Buffer 48 6f 6c 61 ...>
```

Podemos decodificarlo después:

```
const bytes = await readFile("mensaje.txt");
const texto = bytes.toString("utf8");

console.log(texto);
```

Esta distinción es fundamental:

```
readFile(ruta)           → Buffer, bytes
readFile(ruta, "utf8") → string, texto decodificado
```

---

## 10. Texto, Unicode y codificación

---

Un archivo no contiene “letras” directamente: contiene bytes. Para guardar texto necesitamos una regla que relacione caracteres y bytes. Esa regla es una **codificación de caracteres**.

### Unicode

Unicode asigna un número llamado **punto de código** a una enorme cantidad de caracteres.

Ejemplos conceptuales:

```
A    → U+0041
ñ    → U+00F1
€    → U+20AC
😊    → U+1F600
```

Unicode define qué número representa cada carácter. Una codificación como UTF-8 define cómo expresar esos números mediante bytes.

## UTF-8

UTF-8 usa una cantidad variable de bytes:

```
carácter ASCII común      → 1 byte
muchos caracteres latinos → 2 bytes
otros caracteres          → 3 bytes
muchos emojis             → 4 bytes
```

Podemos observarlo:

```
console.log(Buffer.from("A", "utf8"));
console.log(Buffer.from("ñ", "utf8"));
console.log(Buffer.from("😄", "utf8"));

console.log(Buffer.byteLength("A", "utf8")); // 1
console.log(Buffer.byteLength("ñ", "utf8")); // 2
console.log(Buffer.byteLength("😄", "utf8")); // 4
```

Por eso no debemos suponer que “un carácter ocupa un byte”.

Tampoco debemos confundir bytes, unidades internas de JavaScript y caracteres visibles:

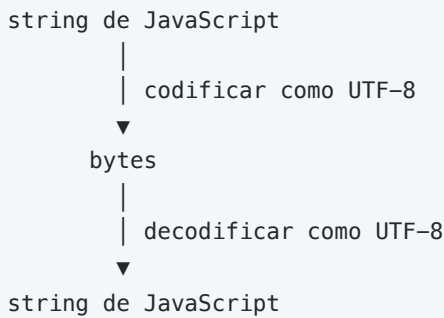
```
const ejemplo = "😄";

console.log(Buffer.byteLength(ejemplo, "utf8")); // 4 bytes UTF-8
console.log(ejemplo.length);                     // 2 unidades UTF-16
console.log(Array.from(ejemplo).length);         // 1 punto de código
```

JavaScript representa internamente sus strings mediante unidades de código UTF-16.

`Array.from()` recorre puntos de código completos, pero incluso un punto de código no siempre equivale a un símbolo percibido por una persona: ciertos acentos y emojis se construyen combinando varios. Por eso toda medición debe aclarar qué unidad está contando.

## Codificar y decodificar



En código:

```
const textoOriginal = "Tucumán 😊";

const bytes = Buffer.from(textoOriginal, "utf8");
const textoRecuperado = bytes.toString("utf8");

console.log(bytes);
console.log(textoRecuperado);
```

## ¿Qué ocurre si usamos la codificación equivocada?

Si los bytes fueron producidos con una codificación y los interpretamos con otra, aparecen caracteres incorrectos:

información correcta → bytes → interpretación incorrecta → texto dañado

Los textos como `JosÃ©` suelen ser síntoma de una codificación mal interpretada.

## Codificaciones disponibles en Node.js

Node.js reconoce, entre otras:

- `utf8` o `utf-8`;
- `utf16le`;
- `latin1`;
- `ascii` por compatibilidad;
- `base64`, `base64url` y `hex` para representar bytes mediante texto.

Para archivos de texto nuevos, UTF-8 es normalmente la elección apropiada.



## UTF-8 no guarda necesariamente una marca inicial

Algunos archivos comienzan con una marca llamada **BOM**. UTF-8 no la necesita, pero ciertas herramientas la agregan. Si aparece como primer carácter después de leer JSON o CSV, podemos retirarla:

```
const sinBOM = texto.replace(/^\uFEFF/, "");
```

## Salto de línea

Los sistemas han usado diferentes secuencias para separar líneas:

```
Linux y macOS actuales → LF    → \n
Windows                → CRLF → \r\n
macOS clásico          → CR    → \r
```

Para dividir texto procedente de distintas plataformas:

```
const lineas = texto.split(/\r\n|\n|\r/);
```

Para escribir con el separador propio del sistema actual:

```
import { EOL } from "node:os";

const texto = ["uno", "dos", "tres"].join(EOL);
```

En formatos intercambiados entre sistemas suele ser más simple elegir `\n` de manera consistente.

---

## 11. Transformar texto y volver a guardarlo

La secuencia general es:

```
leer → decodificar → transformar → codificar → escribir
```

Ejemplo: convertir un archivo a mayúsculas.

```
import { readFile, writeFile } from "node:fs/promises";

const texto = await readFile("entrada.txt", "utf8");
const transformado = texto.toLocaleUpperCase("es");

await writeFile("salida.txt", transformado, "utf8");
```

Ejemplo: reemplazar una palabra:

```
const texto = await readFile("entrada.txt", "utf8");
const transformado = texto.replaceAll("alumno", "estudiante");

await writeFile("salida.txt", transformado, "utf8");
```

Leer y escribir el mismo archivo es válido:

```
const texto = await readFile("entrada.txt", "utf8");
const transformado = texto.trim() + "\n";

await writeFile("entrada.txt", transformado, "utf8");
```

Pero si el proceso falla durante la escritura podríamos dejar el archivo incompleto. Cuando el dato es importante, es más seguro escribir primero un archivo temporal y luego renombrarlo:

```
import { rename, writeFile } from "node:fs/promises";

await writeFile("entrada.txt.tmp", transformado, "utf8");
await rename("entrada.txt.tmp", "entrada.txt");
```

Conviene que el temporal esté en el mismo directorio que el destino. La política exacta para reemplazar un archivo existente puede variar entre sistemas, por lo que una aplicación crítica debe probar el comportamiento en las plataformas que admite.

---

## 12. Listar el contenido de un directorio

---

`readdir` devuelve los nombres de las entradas:

```
import { readdir } from "node:fs/promises";

const nombres = await readdir("datos");

for (const nombre of nombres) {
  console.log(nombre);
}
```

Para distinguir archivos de directorios usamos `withFileTypes`:

```
import { readdir } from "node:fs/promises";

const entradas = await readdir("datos", {
  withFileTypes: true
});

for (const entrada of entradas) {
  if (entrada.isDirectory()) {
    console.log(`[DIR] ${entrada.name}`);
  } else if (entrada.isFile()) {
    console.log(`[FILE] ${entrada.name}`);
  } else {
    console.log(`[OTRO] ${entrada.name}`);
  }
}
```

Cada elemento es un `Dirent`. Entre otros métodos, ofrece:

```
isFile()      ¿es un archivo común?
isDirectory() ¿es un directorio?
isSymbolicLink() ¿es un enlace simbólico?
```

## Listar solamente archivos `.txt`

```
import { readdir } from "node:fs/promises";
import path from "node:path";

const entradas = await readdir("datos", {
  withFileTypes: true
});

const archivosTxt = entradas
  .filter((entrada) => entrada.isFile())
  .filter((entrada) => path.extname(entrada.name).toLowerCase() === ".txt")
  .map((entrada) => entrada.name);

console.log(archivosTxt);
```

## 13. Recorrer subdirectorios

Para buscar en todo un árbol necesitamos recorrer cada subdirectorio.

```
import { readdir } from "node:fs/promises";
import path from "node:path";

async function listarArchivos(directorio) {
  const resultados = [];

  const entradas = await readdir(directorio, {
    withFileTypes: true
  });

  for (const entrada of entradas) {
    const rutaCompleta = path.join(directorio, entrada.name);

    if (entrada.isDirectory()) {
      const archivosInternos = await listarArchivos(rutaCompleta);
      resultados.push(...archivosInternos);
    } else if (entrada.isFile()) {
      resultados.push(rutaCompleta);
    }
  }

  return resultados;
}

const archivos = await listarArchivos("datos");

console.log(archivos);
```

La función es recursiva:

```
visitar directorio
├── si encuentra archivo → guardarlo
└── si encuentra directorio → visitar ese directorio
```

Este ejemplo ignora enlaces simbólicos. Es una decisión deliberada: seguirlos sin cuidado podría producir ciclos o salir del árbol esperado.

## 14. Consultar información de un archivo

`stat` obtiene metadatos:

```
import { stat } from "node:fs/promises";

const informacion = await stat("mensaje.txt");

console.log("Tamaño:", informacion.size, "bytes");
console.log("Modificado:", informacion.mtime);
console.log("Es archivo:", informacion.isFile());
console.log("Es directorio:", informacion.isDirectory());
```

Algunas propiedades útiles:

| Propiedad              | Significado                                                |
|------------------------|------------------------------------------------------------|
| <code>size</code>      | Tamaño en bytes                                            |
| <code>mtime</code>     | Última modificación del contenido o metadatos relacionados |
| <code>atime</code>     | Último acceso, si el sistema lo registra                   |
| <code>birthtime</code> | Fecha de creación cuando el sistema puede proporcionarla   |

No todos los sistemas de archivos mantienen todas las fechas con la misma precisión o semántica. No conviene tratarlas como verdades universales.

## `stat` y `lstat`

Si la ruta es un enlace simbólico:

- `stat()` informa sobre el destino del enlace;
- `lstat()` informa sobre el enlace mismo.

```
import { lstat } from "node:fs/promises";

const informacion = await lstat("acceso-directo");

console.log(informacion.isSymbolicLink());
```

## 15. Comprobar si una ruta existe

Podemos intentar acceder:

```
import { access } from "node:fs/promises";

async function existe(ruta) {
  try {
    await access(ruta);
    return true;
  } catch {
    return false;
  }
}
```

Uso:

```
if (await existe("mensaje.txt")) {
  console.log("Existe");
}
```

Sin embargo, no siempre conviene preguntar primero y operar después:

1. comprobar que existe
2. otro proceso lo elimina
3. intentar leerlo

El estado puede cambiar entre los pasos. Cuando lo que realmente queremos es leer, suele ser mejor intentar leer y manejar el posible error:

```
import { readFile } from "node:fs/promises";

try {
  const texto = await readFile("mensaje.txt", "utf8");
  console.log(texto);
} catch (error) {
  if (error.code === "ENOENT") {
    console.log("El archivo no existe");
  } else {
    throw error;
  }
}
```

---

## 16. Buscar archivos por nombre

---

Combinamos el recorrido recursivo con una condición:

```

import { readdir } from "node:fs/promises";
import path from "node:path";

async function buscarPorNombre(directorio, fragmento) {
  const coincidencias = [];
  const buscado = fragmento.toLocaleLowerCase("es");

  const entradas = await readdir(directorio, {
    withFileTypes: true
  });

  for (const entrada of entradas) {
    const rutaCompleta = path.join(directorio, entrada.name);

    if (entrada.isDirectory()) {
      const internas = await buscarPorNombre(rutaCompleta, fragmento);
      coincidencias.push(...internas);
    } else if (
      entrada.isFile() &&
      entrada.name.toLocaleLowerCase("es").includes(buscado)
    ) {
      coincidencias.push(rutaCompleta);
    }
  }

  return coincidencias;
}

const resultados = await buscarPorNombre("datos", "alumno");

console.log(resultados);

```

Esto busca una parte literal del nombre. Para patrones más sofisticados podríamos usar una expresión regular.

## 17. Buscar texto dentro de archivos

Buscar por contenido requiere:

encontrar archivos → leerlos como texto → comparar el contenido

```

import { readFile, readdir } from "node:fs/promises";
import path from "node:path";

async function buscarEnTextos(directorio, textoBuscado) {
  const coincidencias = [];
  const buscado = textoBuscado.toLocaleLowerCase("es");

  const entradas = await readdir(directorio, {
    withFileTypes: true
  });

  for (const entrada of entradas) {
    const rutaCompleta = path.join(directorio, entrada.name);

    if (entrada.isDirectory()) {
      const internas = await buscarEnTextos(rutaCompleta, textoBuscado);
      coincidencias.push(...internas);
      continue;
    }

    const esTexto = entrada.isFile() &&
      path.extname(entrada.name).toLowerCase() === ".txt";

    if (!esTexto) {
      continue;
    }

    const contenido = await readFile(rutaCompleta, "utf8");

    if (contenido.toLocaleLowerCase("es").includes(buscado)) {
      coincidencias.push(rutaCompleta);
    }
  }

  return coincidencias;
}

console.log(await buscarEnTextos("datos", "javascript"));

```

Filtramos por extensión porque intentar decodificar indiscriminadamente imágenes, ejecutables o archivos comprimidos no constituye una búsqueda de texto confiable.



## Informar número y contenido de cada línea coincidente

```
import { readFile } from "node:fs/promises";

async function buscarLineas(ruta, textoBuscado) {
  const contenido = await readFile(ruta, "utf8");
  const lineas = contenido.split(/\r\n|\n|\r/);
  const buscado = textoBuscado.toLocaleLowerCase("es");

  return lineas
    .map((texto, indice) => ({
      numero: indice + 1,
      texto
    }))
    .filter((linea) =>
      linea.texto.toLocaleLowerCase("es").includes(buscado)
    );
}

console.log(await buscarLineas("mensaje.txt", "node"));
```

`indice + 1` convierte el índice de un arreglo, que empieza en cero, en un número de línea para personas, que normalmente empieza en uno.

## 18. Copiar archivos y directorios

### Copiar un archivo

```
import { copyFile } from "node:fs/promises";

await copyFile("original.txt", "copia.txt");
```

Por defecto, si el destino existe puede ser reemplazado.

Para impedirlo:

```
import { constants } from "node:fs";
import { copyFile } from "node:fs/promises";

await copyFile(
  "original.txt",
  "copia.txt",
  constants.COPYFILE_EXCL
);
```

## Copiar un directorio completo

```
import { cp } from "node:fs/promises";

await cp("datos", "respaldo/datos", {
  recursive: true
});
```

Algunas opciones útiles de `cp` son:

```
await cp("datos", "respaldo/datos", {
  recursive: true,
  force: false,
  errorOnExist: true,
  preserveTimestamps: true
});
```

La combinación exacta depende de la política deseada: reemplazar, omitir o fallar cuando el destino ya existe.

---

## 19. Renombrar y mover

Usamos `rename`:

```
import { rename } from "node:fs/promises";

await rename("borrador.txt", "informe.txt");
```

Si cambiamos también el directorio, estamos moviendo la entrada:

```
await rename(
  "pendientes/informe.txt",
  "terminados/informe.txt"
);
```

Renombrar y mover dentro de un mismo sistema de archivos son, conceptualmente, variaciones de la misma operación: cambiar la ruta con la que se encuentra la entrada.

El directorio de destino debe existir:

```
import { mkdir, rename } from "node:fs/promises";

await mkdir("terminados", { recursive: true });
await rename(
  "pendientes/informe.txt",
  "terminados/informe.txt"
);
```

Mover entre dispositivos o sistemas de archivos diferentes puede fallar con `EXDEV`. En ese caso se necesita una estrategia de copiar, verificar y luego eliminar el origen.

---

## 20. Eliminar archivos y directorios

---

Eliminar es una operación destructiva. Debemos validar cuidadosamente la ruta.

### Eliminar un archivo

```
import { unlink } from "node:fs/promises";

await unlink("temporal.txt");
```

### Eliminar un directorio vacío

```
import { rmdir } from "node:fs/promises";

await rmdir("directorio-vacio");
```

### Eliminar un árbol completo

```
import { rm } from "node:fs/promises";

await rm("temporales", {
  recursive: true,
  force: true
});
```

`recursive: true` amplía mucho el alcance de la operación. Antes de usarlo, conviene resolver e inspeccionar la ruta y nunca aplicarlo a una ruta vacía o no confiable.

---

## 21. Extraer información de un texto

Una vez leído, el contenido es un `string`. Podemos usar las operaciones habituales de JavaScript.

### Contar líneas

```
import { readFile } from "node:fs/promises";

function contarLineas(texto) {
  if (texto.length === 0) {
    return 0;
  }

  const partes = texto.split(/\r\n|\n|\r/);
  const terminaConSalto = /(?:\r\n|\n|\r)$/.test(texto);

  return partes.length - (terminaConSalto ? 1 : 0);
}

const texto = await readFile("informe.txt", "utf8");

console.log("Cantidad:", contarLineas(texto));
```

Aquí adoptamos una convención explícita: el archivo vacío tiene cero líneas y el salto final termina la última línea, pero no crea una línea adicional. Otras herramientas pueden definir el conteo de otra manera; lo importante es conocer y documentar la regla elegida.

### Contar palabras Unicode

```
function extraerPalabras(texto) {
  return texto.match(/[\p{L}\p{N}]+/gu) ?? [];
}

const palabras = extraerPalabras(texto);

console.log("Palabras:", palabras.length);
```

`\p{L}` representa letras Unicode y `\p{N}` representa números Unicode. La bandera `u` activa el tratamiento Unicode.

La definición de “palabra” depende del problema. Este patrón es útil como aproximación, pero no reemplaza un segmentador lingüístico en todos los idiomas.

## Contar apariciones

```
function contarApariciones(texto, palabra) {
  const palabras = texto.match(/\p{L}\p{N}+/gu) ?? [];
  const buscada = palabra.toLocaleLowerCase("es");

  return palabras.filter(
    (actual) => actual.toLocaleLowerCase("es") === buscada
  ).length;
}

console.log(contarApariciones(texto, "JavaScript"));
```

## Obtener números

```
const texto = "Ana obtuvo 8, Juan 10 y Sol 7";
const coincidencias = texto.match(/\d+/g) ?? [];
const numeros = coincidencias.map(Number);

console.log(numeros); // [8, 10, 7]
```

---

## 22. Extraer datos de JSON

JSON es texto con una gramática específica.

Archivo `alumnos.json`:

```
[
  { "nombre": "Ana", "nota": 8 },
  { "nombre": "Luis", "nota": 6 },
  { "nombre": "Sol", "nota": 10 }
]
```

Lectura:

```
import { readFile } from "node:fs/promises";

const texto = await readFile("alumnos.json", "utf8");
const alumnos = JSON.parse(texto.replace(/^\uFEFF/, ""));

const aprobados = alumnos.filter((alumno) => alumno.nota >= 6);
const promedio = alumnos.reduce(
  (acumulado, alumno) => acumulado + alumno.nota,
  0
) / alumnos.length;

console.log(aprobados);
console.log(promedio);
```

`JSON.parse()` convierte texto JSON en valores de JavaScript.

Para guardar datos:

```
import { writeFile } from "node:fs/promises";

const alumnos = [
  { nombre: "Ana", nota: 8 },
  { nombre: "Luis", nota: 6 }
];

const texto = JSON.stringify(alumnos, null, 4) + "\n";

await writeFile("alumnos.json", texto, "utf8");
```

`JSON.stringify()` convierte valores compatibles de JavaScript en texto JSON. Los argumentos `null, 4` agregan sangría para facilitar la lectura humana.

## Validar la estructura además de la sintaxis

Que el JSON sea válido no significa que contenga los datos esperados:

```
if (!Array.isArray(alumnos)) {
  throw new TypeError("Se esperaba un arreglo de alumnos");
}

for (const alumno of alumnos) {
  if (
    typeof alumno.nombre !== "string" ||
    typeof alumno.nota !== "number"
  ) {
    throw new TypeError("Alumno inválido");
  }
}
```

Hay dos preguntas diferentes:

¿el texto respeta la gramática JSON? → JSON.parse  
¿los datos tienen la forma que necesito? → validación

## 23. Extraer datos de un CSV sencillo

Archivo `alumnos.csv`:

```
nombre,nota
Ana,8
Luis,6
Sol,10
```

Si controlamos el formato y sabemos que ningún campo contiene comas, comillas o saltos de línea, podemos hacer un análisis elemental:

```
import { readFile } from "node:fs/promises";

const texto = await readFile("alumnos.csv", "utf8");
const lineas = texto
  .replace(/^\uFEFF/, "")
  .trim()
  .split(/\r\n|\n|\r/);

const [encabezado, ...filas] = lineas;

if (encabezado !== "nombre,nota") {
  throw new Error("Encabezado CSV inesperado");
}

const alumnos = filas.map((fila) => {
  const [nombre, notaTexto] = fila.split(",");

  return {
    nombre,
    nota: Number(notaTexto)
  };
});

console.log(alumnos);
```

Pero un CSV real puede contener:

```
"Pérez, Ana",8
"Texto con ""comillas""",9
```

En esos casos, `split(",")` es incorrecto. Para archivos CSV generales conviene usar una biblioteca que implemente correctamente la variante del formato que recibimos.

## 24. Extraer información de un registro de eventos

Supongamos este archivo `servidor.log`:

```
2026-08-24 INFO servidor iniciado
2026-08-24 ERROR conexión rechazada
2026-08-24 INFO reintento
```

Podemos extraer fecha, nivel y mensaje con una expresión regular:

```
import { readFile } from "node:fs/promises";

const texto = await readFile("servidor.log", "utf8");
const lineas = texto.split(/\r\n|\n|\r/);

const patron = /^(\d{4}-\d{2}-\d{2}) (INFO|WARN|ERROR) (.+)$/;

const eventos = [];

for (const linea of lineas) {
  const coincidencia = linea.match(patron);

  if (!coincidencia) {
    continue;
  }

  const [, fecha, nivel, mensaje] = coincidencia;

  eventos.push({ fecha, nivel, mensaje });
}

const errores = eventos.filter(
  (evento) => evento.nivel === "ERROR"
);

console.log(errores);
```

La expresión regular no “comprende” el archivo: describe la estructura que esperamos encontrar.



## 25. Leer archivos grandes con streams

`readFile` carga el archivo completo en memoria:

disco → archivo completo en RAM → procesamiento

Es cómodo para archivos pequeños y medianos. Para un archivo de varios gigabytes puede ser imposible o innecesario.

Un **stream** entrega fragmentos a medida que están disponibles:

disco → fragmento → procesar  
→ fragmento → procesar  
→ fragmento → procesar

### Leer línea por línea

```
import { createReadStream } from "node:fs";
import { createInterface } from "node:readline";

const flujo = createReadStream("servidor.log", {
  encoding: "utf8"
});

const lineas = createInterface({
  input: flujo,
  crlfDelay: Infinity
});

let cantidadErrores = 0;

for await (const linea of lineas) {
  if (linea.includes(" ERROR ")) {
    cantidadErrores++;
  }
}

console.log({ cantidadErrores });
```

`crlfDelay: Infinity` hace que `\r\n` sea tratado como un solo salto de línea.

La memoria utilizada ya no crece proporcionalmente al tamaño total del archivo.

### Copiar con streams

Para una copia sencilla es preferible `copyFile`. Los streams son útiles cuando también queremos transformar o inspeccionar el contenido durante el recorrido.

```
import { createReadStream, createWriteStream } from "node:fs";
import { pipeline } from "node:stream/promises";

await pipeline(
  createReadStream("origen.bin"),
  createWriteStream("destino.bin")
);
```

`pipeline` conecta los flujos, propaga errores y espera que el proceso termine.

---

## 26. Abrir un archivo mediante un descriptor

---

Las funciones de alto nivel abren, operan y cierran el archivo por nosotros. También podemos abrirlo explícitamente:

```
import { open } from "node:fs/promises";

const archivo = await open("mensaje.txt", "r");

try {
  const contenido = await archivo.readFile("utf8");
  console.log(contenido);
} finally {
  await archivo.close();
}
```

El sistema operativo mantiene un recurso abierto y Node.js lo representa mediante un `FileHandle`. Debemos cerrarlo incluso si ocurre un error. Por eso se utiliza `finally`.

Abrir explícitamente resulta útil cuando necesitamos varias operaciones sobre el mismo archivo o control de bajo nivel sobre posiciones y fragmentos.

---

## 27. Errores habituales

---

Las operaciones pueden fallar por muchas razones:

- la ruta no existe;
- el proceso no tiene permiso;
- una parte de la ruta no es un directorio;
- el destino ya existe;
- el disco está lleno;
- el archivo está siendo utilizado o bloqueado;

- se intenta mover entre sistemas de archivos diferentes;
- la ruta es demasiado larga para alguna capa del sistema.

Node.js entrega objetos de error que suelen incluir una propiedad `code`.

| Código               | Significado habitual                           |
|----------------------|------------------------------------------------|
| <code>ENOENT</code>  | No existe la ruta                              |
| <code>EACCES</code>  | Acceso denegado                                |
| <code>EEXIST</code>  | El destino ya existe                           |
| <code>EISDIR</code>  | Se esperaba un archivo, pero era un directorio |
| <code>ENOTDIR</code> | Una parte que debía ser directorio no lo es    |
| <code>ENOSPC</code>  | No queda espacio disponible                    |
| <code>EXDEV</code>   | La operación cruza sistemas de archivos        |

Ejemplo:

```
import { readFile } from "node:fs/promises";

try {
  const contenido = await readFile("config.json", "utf8");
  console.log(contenido);
} catch (error) {
  switch (error.code) {
    case "ENOENT":
      console.error("No se encontró config.json");
      break;

    case "EACCES":
      console.error("No hay permiso para leer config.json");
      break;

    default:
      throw error;
  }
}
```

No conviene ocultar todos los errores con un `catch` vacío. Si no sabemos resolver uno, debemos volver a lanzarlo o finalizar informando el problema.

## 28. Operaciones secuenciales y concurrentes

Con `await`, estas escrituras ocurren una después de otra:

```
await writeFile("a.txt", "A", "utf8");
await writeFile("b.txt", "B", "utf8");
await writeFile("c.txt", "C", "utf8");
```

Si son independientes, podemos iniciarlas juntas:

```
await Promise.all([
  writeFile("a.txt", "A", "utf8"),
  writeFile("b.txt", "B", "utf8"),
  writeFile("c.txt", "C", "utf8")
]);
```

No debemos lanzar varias modificaciones concurrentes sobre el mismo archivo sin diseñar cómo se coordinarán:

```
// Resultado no confiable: compiten por el mismo destino.
await Promise.all([
  writeFile("resultado.txt", "uno", "utf8"),
  writeFile("resultado.txt", "dos", "utf8")
]);
```

El sistema de archivos no convierte automáticamente una secuencia lógica de varias operaciones en una transacción.

---

## 29. Seguridad de las rutas

---

Si un usuario controla una parte de la ruta, podría intentar salir del directorio permitido:

```
../../archivo-secreto.txt
```

Esto se conoce como **path traversal** o recorrido de rutas.

Una estrategia básica es resolver la ruta y verificar que permanezca dentro del directorio autorizado:

```
import path from "node:path";

function resolverDentroDe(base, entrada) {
  const baseAbsoluta = path.resolve(base);
  const candidata = path.resolve(baseAbsoluta, entrada);
  const relativa = path.relative(baseAbsoluta, candidata);

  const saleDeLaBase =
    relativa === ".." ||
    relativa.startsWith(`..${path.sep}`) ||
    path.isAbsolute(relativa);

  if (saleDeLaBase) {
    throw new Error("La ruta sale del directorio permitido");
  }

  return candidata;
}

console.log(resolverDentroDe("datos", "alumnos/ana.txt"));
```

En aplicaciones expuestas a usuarios también hay que definir políticas sobre enlaces simbólicos, permisos, nombres permitidos, tipos de archivo y tamaños máximos.

## 30. Programa integrador: analizador de textos

El siguiente programa reúne varias ideas. Recorre un directorio, encuentra archivos `.txt`, obtiene sus metadatos y calcula líneas, palabras y caracteres.

Archivo `analizar-textos.js`:

```

import { readFile, readdir, stat } from "node:fs/promises";
import path from "node:path";

function contarLineas(texto) {
  if (texto.length === 0) {
    return 0;
  }

  const partes = texto.split(/\r\n|\n|\r/);
  const terminaConSalto = /(?:\r\n|\n|\r)$/.test(texto);

  return partes.length - (terminaConSalto ? 1 : 0);
}

function contarPalabras(texto) {
  return (texto.match(/\p{L}\p{N}+/gu) ?? []).length;
}

async function encontrarTextos(directorio) {
  const rutas = [];
  const entradas = await readdir(directorio, {
    withFileTypes: true
  });

  for (const entrada of entradas) {
    const ruta = path.join(directorio, entrada.name);

    if (entrada.isDirectory()) {
      rutas.push(...await encontrarTextos(ruta));
    } else if (
      entrada.isFile() &&
      path.extname(entrada.name).toLowerCase() === ".txt"
    ) {
      rutas.push(ruta);
    }
  }

  return rutas;
}

async function analizarArchivo(ruta) {
  const [texto, metadatos] = await Promise.all([
    readFile(ruta, "utf8"),
    stat(ruta)
  ]);

  return {
    ruta,
    bytes: metadatos.size,
    puntosDeCodigo: Array.from(texto).length,
    lineas: contarLineas(texto),
    palabras: contarPalabras(texto),
    modificado: metadatos.mtime.toISOString()
  };
}

```

```

}

const directorio = process.argv[2] ?? ".";

try {
  const rutas = await encontrarTextos(directorio);
  const informes = [];

  for (const ruta of rutas) {
    informes.push(await analizarArchivo(ruta));
  }

  console.table(informes);
} catch (error) {
  console.error(`No se pudo analizar: ${error.message}`);
  process.exitCode = 1;
}

```

Ejecución:

```
node analizar-textos.js datos
```

## Qué integra este programa

```

argumento de la terminal
  |
  ▼
recorrido recursivo del directorio
  |
  ▼
selección de archivos .txt
  |
  ▼
lectura de contenido + metadatos
  |
  ▼
extracción de información
  |
  ▼
tabla de resultados

```

`process.argv[2]` representa el primer argumento escrito después del nombre del programa. Si no se proporciona, se usa `"."`, que representa el directorio actual.

## 31. Criterios para elegir una técnica

| Necesidad                                      | Herramienta apropiada                                 |
|------------------------------------------------|-------------------------------------------------------|
| Leer un archivo pequeño como texto             | <code>readFile(ruta, "utf8")</code>                   |
| Obtener los bytes sin interpretarlos           | <code>readFile(ruta)</code>                           |
| Reemplazar o crear contenido                   | <code>writeFile</code>                                |
| Agregar al final                               | <code>appendFile</code>                               |
| Crear directorios intermedios                  | <code>mkdir(..., { recursive: true })</code>          |
| Listar una carpeta                             | <code>readdir</code>                                  |
| Obtener tamaño y fechas                        | <code>stat</code>                                     |
| Copiar un archivo                              | <code>copyFile</code>                                 |
| Copiar un árbol                                | <code>cp(..., { recursive: true })</code>             |
| Mover o cambiar el nombre                      | <code>rename</code>                                   |
| Procesar un archivo grande por líneas          | <code>createReadStream</code> + <code>readline</code> |
| Conectar lectura y escritura por fragmentos    | <code>pipeline</code>                                 |
| Realizar varias operaciones sobre una apertura | <code>open</code> y <code>FileHandle</code>           |

## 32. Errores conceptuales frecuentes

### Confundir extensión con formato

Cambiar `.jpg` por `.txt` no convierte una imagen en texto.

### Omitir la codificación al esperar texto

```
const datos = await readFile("archivo.txt");
```

devuelve bytes. Si queremos un string UTF-8:

```
const texto = await readFile("archivo.txt", "utf8");
```

### Suponer que un carácter ocupa un byte

En UTF-8, `A`, `ñ` y `🍌` ocupan cantidades diferentes de bytes.



## Suponer que una ruta relativa parte del archivo JavaScript

Normalmente parte de `process.cwd()`, el directorio de trabajo del proceso.

## Sobrescribir sin querer

`writeFile` reemplaza el contenido existente. Si eso no está permitido, debemos usar la bandera `wx` o verificar la política de destino.

## Cargar archivos enormes con `readFile`

Para procesamiento secuencial de archivos grandes debemos preferir streams.

## Dividir cualquier CSV con una coma

Los campos CSV pueden contener comas, comillas y saltos de línea. Un formato general necesita un analizador correcto.

## Ignorar errores

Un `catch` vacío transforma fallos reales en resultados silenciosamente incorrectos.

## Mezclar operaciones concurrentes sobre el mismo archivo

El orden final puede no ser el que imaginamos y pueden perderse datos.

---

## 33. Ejercicios propuestos

---

### Ejercicio 1: diario

Crear un programa que reciba una frase y la agregue a `diario.txt` con la fecha actual.

### Ejercicio 2: contador

Leer un archivo y mostrar:

- bytes;
- caracteres;
- palabras;
- líneas.

Comparar `Buffer.byteLength(texto, "utf8")` con `texto.length` y explicar por qué pueden diferir.

### Ejercicio 3: buscador

Recorrer un directorio y mostrar cada archivo `.md` que contenga una palabra dada, incluyendo los números de línea coincidentes.

### Ejercicio 4: respaldo

Copiar todos los `.txt` de un directorio a una carpeta `respaldo`, creándola si no existe. No reemplazar archivos existentes.

### Ejercicio 5: notas

Leer un JSON con alumnos y notas. Generar otro JSON que contenga:

- cantidad de alumnos;
- promedio;
- nota máxima;
- alumnos aprobados.

### Ejercicio 6: registros grandes

Procesar un archivo de eventos línea por línea y contar cuántos registros hay de cada nivel: `INFO`, `WARN` y `ERROR`.

### Ejercicio 7: organizador

Listar los archivos de un directorio y moverlos a subdirectorios según su extensión:

```
imagenes/  
textos/  
json/  
otros/
```

Antes de mover, resolver las rutas y evitar reemplazar destinos existentes.

---

## 34. Síntesis final

La gestión de archivos se vuelve más clara si mantenemos separadas estas capas:

1. El sistema de archivos organiza nombres, rutas y metadatos.
2. Un archivo contiene bytes.
3. Un formato indica cómo interpretar esos bytes.
4. Una codificación relaciona texto y bytes.
5. Node.js solicita las operaciones al sistema operativo.
6. Nuestro programa transforma los datos y decide cómo manejar errores.

El recorrido típico de un programa es:

```
construir ruta
    ↓
abrir o solicitar operación
    ↓
leer bytes
    ↓
decodificar si son texto
    ↓
buscar, validar o transformar
    ↓
codificar
    ↓
escribir, copiar o mover
    ↓
manejar el resultado o el error
```

La idea más importante es que un archivo no es, en esencia, “un documento de texto” ni “una imagen”. Es una secuencia de bytes a la que damos significado mediante convenciones. A partir de ese modelo, operaciones como leer, escribir, buscar o extraer información dejan de ser acciones misteriosas y se convierten en transformaciones bien definidas.

## Referencias oficiales

---

- Sistema de archivos ( `node:fs` )
- Rutas ( `node:path` )
- Buffers y codificaciones
- Lectura línea por línea ( `node:readline` )
- Streams

# Gestión de archivos con JavaScript y Node.js

Este tutorial parte de una idea central:

**Un archivo no contiene texto, imágenes, objetos JSON ni registros. Un archivo contiene bytes.**

"Texto", "imagen", "PDF" o "JSON" son interpretaciones de esos bytes.

A lo largo del tutorial usaremos exclusivamente módulos nativos de Node.js:

```
import * as path from 'node:path';
import {
  appendFile,
  copyFile,
  cp,
  mkdir,
  open,
  readFile,
  readdir,
  rename,
  rm,
  stat,
  writeFile,
} from 'node:fs/promises';

import {
  constants,
  createReadStream,
} from 'node:fs';

import { createInterface } from 'node:readline';
```

Los ejemplos utilizan módulos ES. Podés guardarlos en archivos con extensión `.mjs` y ejecutarlos así:

```
node programa.mjs
```

De esa forma no hace falta configurar `package.json`.

Conceptualmente, un archivo es:

una secuencia finita de bytes  
+  
metadatos

Los metadatos incluyen, según el sistema de archivos:

- nombre;
- tamaño;
- fechas de acceso y modificación;
- permisos;
- propietario;
- ubicación dentro de un directorio;
- indicador de archivo, directorio o enlace simbólico.

Un directorio, por su parte, organiza nombres y referencias a otros archivos o directorios.

Podemos imaginar esta estructura:

```
proyecto/  
├─ alumnos.json  
├─ notas.txt  
└─ imagenes/  
    └─ logo.png
```

Aquí:

- `proyecto` es un directorio;
- `notas.txt` es un archivo;
- `proyecto/notas.txt` es una ruta;
- `.txt` es una extensión.

La extensión es solamente una convención. Nada impide guardar una imagen con el nombre `foto.txt`. La extensión es una etiqueta, no un detector de mentiras.

## Las capas de interpretación

Tomemos este archivo:

```
datos/clientes.json
```

Podemos verlo en distintos niveles:

```
Ruta
↓
Archivo
↓
Bytes
  ↓ decodificación UTF-8
Texto
  ↓ análisis de JSON
Objetos y arrays
  ↓ interpretación del programa
Clientes, nombres y direcciones
```

Cada operación trabaja en una capa diferente:

| Operación                | Capa principal          |
|--------------------------|-------------------------|
| Renombrar                | Ruta                    |
| Copiar                   | Bytes                   |
| Leer como texto          | Codificación            |
| Buscar una palabra       | Texto                   |
| Interpretar JSON         | Formato                 |
| Extraer clientes morosos | Información del dominio |

Esto explica por qué copiar un archivo no necesita conocer su codificación, pero buscar una palabra sí.

## 2. Abrir, operar y cerrar

En un nivel bajo, para trabajar con un archivo el sistema operativo realiza aproximadamente este proceso:

1. Localizar la ruta.
2. Abrir el archivo.
3. Obtener un descriptor o manejador.
4. Leer o escribir bytes.
5. Cerrar el manejador.

Node.js ofrece funciones de alto nivel, como `readFile()` y `writeFile()`, que realizan automáticamente ese ciclo.

También permite trabajar explícitamente con un `FileHandle` mediante `open()`. En ese caso debemos cerrarlo nosotros.

Node proporciona tres familias de APIs:

| API        | Ejemplo                               | Comportamiento                |
|------------|---------------------------------------|-------------------------------|
| Sincrónica | <code>readFileSync()</code>           | Detiene el hilo de JavaScript |
| Callbacks  | <code>readFile(path, callback)</code> | Asincrónica                   |
| Promesas   | <code>await readFile(path)</code>     | Asincrónica y más cómoda      |

Usaremos `node:fs/promises`. Las operaciones sincrónicas bloquean el event loop; las APIs de promesas realizan el trabajo de archivos fuera del hilo principal, aunque las modificaciones concurrentes sobre un mismo archivo no se sincronizan automáticamente. Los `FileHandle` deben cerrarse explícitamente. (nodejs.org)

Ejemplo mínimo:

```
import { readFile } from 'node:fs/promises';

const contenido = await readFile('notas.txt', 'utf8');

console.log(contenido);
```

## 3. Las rutas

Una ruta es una dirección dentro del sistema de archivos.

Puede ser relativa:

```
datos/alumnos.json
```

O absoluta:

```
/Users/alejandro/proyecto/datos/alumnos.json
```

En Windows podría ser:

```
C:\Users\Alejandro\proyecto\datos\alumnos.json
```

## Nunca concatenar separadores manualmente

Esto es frágil:

```
const archivo = directorio + '/' + nombre;
```

En su lugar:

```
import * as path from 'node:path';  
  
const archivo = path.join(directorio, nombre);
```

Node elegirá el separador correspondiente al sistema operativo.

```
const archivo = path.join(  
  'datos',  
  'alumnos',  
  'comision-c1.json',  
);  
  
console.log(archivo);
```

En sistemas POSIX puede producir:

```
datos/alumnos/comision-c1.json
```

En Windows:

```
datos\alumnos\comision-c1.json
```

El módulo `node:path` proporciona implementaciones adaptadas al sistema actual y, cuando hace falta interpretar deliberadamente rutas de otro sistema, también ofrece `path.win32` y `path.posix`. (nodejs.org)



## Operaciones frecuentes con rutas

---

```
import * as path from 'node:path';

const archivo = path.join(
  'datos',
  'alumnos',
  'comision-c1.json',
);

console.log(path.basename(archivo));
// comision-c1.json

console.log(path.dirname(archivo));
// datos/alumnos

console.log(path.extname(archivo));
// .json

console.log(path.parse(archivo));
```

`path.parse()` devuelve algo semejante a:

```
{
  root: '',
  dir: 'datos/alumnos',
  base: 'comision-c1.json',
  ext: '.json',
  name: 'comision-c1'
}
```

### `path.resolve()`

`path.resolve()` construye una ruta absoluta:

```
const directorio = path.resolve('datos');

console.log(directorio);
```

Una ruta relativa se resuelve desde el directorio actual del proceso:

```
console.log(process.cwd());
```

`process.cwd()` es el directorio desde el cual se inició Node, que no necesariamente coincide con el directorio donde está el programa. (nodejs.org)

Por ejemplo:

```
cd /Users/alejandro  
node proyectos/archivos/programa.mjs
```

Dentro del programa:

```
process.cwd();
```

devolverá:

```
/Users/alejandro
```

No devolverá necesariamente:

```
/Users/alejandro/proyectos/archivos
```

## Ruta relativa al propio programa

---

En módulos ES podemos obtener la ubicación del archivo ejecutado así:

```
import * as path from 'node:path';  
import { fileURLToPath } from 'node:url';  
  
const nombreArchivoActual = fileURLToPath(import.meta.url);  
const directorioActual = path.dirname(nombreArchivoActual);  
  
const archivo = path.join(  
  directorioActual,  
  'datos',  
  'alumnos.json',  
);
```

Esto permite localizar recursos junto al programa independientemente del directorio desde el cual se lo ejecute.

---

## 4. Diferencias entre sistemas operativos

Para escribir código multiplataforma conviene no asumir detalles del sistema.

| Aspecto                        | Windows                                       | Linux/macOS y otros POSIX            |
|--------------------------------|-----------------------------------------------|--------------------------------------|
| Separador habitual             | <code>\</code>                                | <code>/</code>                       |
| Raíz                           | <code>C:</code> , <code>D:</code> , rutas UNC | <code>/</code>                       |
| Separador de <code>PATH</code> | <code>;</code>                                | <code>:</code>                       |
| Fin de línea tradicional       | <code>\r\n</code>                             | <code>\n</code>                      |
| Permisos POSIX                 | Limitados o diferentes                        | <code>rw</code> , propietario, grupo |
| Sensibilidad a mayúsculas      | Frecuentemente insensible                     | Depende del sistema de archivos      |

Reglas prácticas:

1. Usar siempre `path.join()` y `path.resolve()`.
2. No crear dos archivos que se distingan solamente por mayúsculas.
3. Usar nombres conservadores.
4. No depender de permisos POSIX para que el programa funcione en Windows.
5. No asumir que el orden devuelto por un directorio será siempre el mismo.
6. Ordenar explícitamente cuando el orden sea importante.

Para nombres realmente portables conviene evitar:

```
< > : " / \ | ? *
```

También conviene evitar nombres que terminen en punto o espacio.

## 5. Crear directorios

Antes de escribir:

```
datos/alumnos/c1.json
```

los directorios `datos` y `alumnos` deben existir.

```
import { mkdir } from 'node:fs/promises';

await mkdir('datos', {
  recursive: true,
});
```

Con `recursive: true`, Node crea también los directorios intermedios que falten y no falla simplemente porque el directorio ya exista.

```
import * as path from 'node:path';
import { mkdir } from 'node:fs/promises';

const archivo = path.join(
  'datos',
  'alumnos',
  'c1.json',
);

await mkdir(path.dirname(archivo), {
  recursive: true,
});
```

Una función reutilizable:

```
import * as path from 'node:path';
import { mkdir } from 'node:fs/promises';

async function crearDirectorioContenedor(archivo) {
  await mkdir(path.dirname(archivo), {
    recursive: true,
  });
}
```

La opción `mode` representa permisos de creación al estilo POSIX y no tiene el mismo soporte en Windows. (nodejs.org)

---

## 6. Bytes, texto y codificación

Supongamos que tenemos este texto:

Año 😊

Dentro de JavaScript es un `string`.

Para guardarlo en un archivo hay que convertirlo en bytes:

```
String de JavaScript
    ↓ codificación
Secuencia de bytes
    ↓ escritura
Archivo
```

Para leerlo sucede lo contrario:

```
Archivo
    ↓ lectura
Secuencia de bytes
    ↓ decodificación
String de JavaScript
```

## Unicode no es lo mismo que UTF-8

Unicode asigna números abstractos a caracteres:

```
A   → U+0041
ñ   → U+00F1
😊  → U+1F600
```

UTF-8 especifica cómo representar esos números mediante bytes.

Un mismo texto puede codificarse de diferentes maneras:

- UTF-8;
- UTF-16LE;
- ISO-8859-1;
- Windows-1252;
- otras codificaciones históricas.

Actualmente, para archivos de texto nuevos, UTF-8 suele ser la elección correcta.

## Los strings de JavaScript

Los strings de JavaScript están organizados como unidades de código UTF-16. Eso significa que `length` no siempre cuenta caracteres visibles. (tc39.es)

```
const texto = 'Año 😊';

console.log(texto.length);
// 6 unidades de código UTF-16

console.log([...texto].length);
// 5 puntos de código
```

El emoji ocupa dos unidades UTF-16.

Pero en UTF-8 el texto ocupa nueve bytes:

```
const texto = 'Año 😊';
const bytes = Buffer.from(texto, 'utf8');

console.log(bytes);
console.log(bytes.length);
// 9
```

La relación es:

```
texto.length      → unidades UTF-16
[...texto].length → puntos de código
Buffer.byteLength() → bytes codificados
```

```
console.log(Buffer.byteLength('Año 😊', 'utf8'));
// 9
```

Un punto de código tampoco equivale siempre a un símbolo visual: algunas letras y emojis pueden estar formados por varios puntos de código. Unicode tiene más capas que una cebolla, pero por lo menos no hace llorar siempre.

## Buffer : los bytes de Node.js

Un `Buffer` representa una secuencia de bytes:

```
const bytes = Buffer.from('Tucumán', 'utf8');

console.log(bytes);
```

Podemos volver a obtener el texto:

```
const texto = bytes.toString('utf8');  
  
console.log(texto);  
// Tucumán
```

Node admite varias codificaciones en `Buffer`, y utiliza UTF-8 como codificación predeterminada para numerosas operaciones de texto. Al decodificar bytes UTF-8 inválidos con `Buffer`, los errores se representan normalmente mediante el carácter de reemplazo `?`. (nodejs.org)

## El problema de usar una codificación incorrecta

```
const bytes = Buffer.from('canción', 'utf8');  
  
console.log(bytes.toString('latin1'));
```

Resultado:

```
canciÃ³n
```

Los bytes no cambiaron. Cambió la manera de interpretarlos.

```
bytes UTF-8  
  ↓ interpretados erróneamente como Latin-1  
texto deformado
```

Ese fenómeno suele denominarse *mojibake*.

## Decodificación estricta

Cuando necesitamos rechazar un archivo que no sea UTF-8 válido:

```
import { readFile } from 'node:fs/promises';

const bytes = await readFile('datos.txt');

const decoder = new TextDecoder('utf-8', {
  fatal: true,
});

try {
  const texto = decoder.decode(bytes);
  console.log(texto);
} catch {
  console.error('El archivo no contiene UTF-8 válido');
}
```

Con `fatal: true`, un error de decodificación produce una excepción en lugar de insertar silenciosamente ` `. `TextDecoder` tambi n permite controlar el tratamiento del marcador BOM. (nodejs.org)

## El BOM

Algunos archivos comienzan con una marca denominada *Byte Order Mark*. Una vez decodificado, puede aparecer como `U+FEFF`.

Podemos retirarlo expl citamente:

```
function quitarBom(texto) {
  return texto.charCodeAt(0) === 0xfeff
    ? texto.slice(1)
    : texto;
}
```

Esto resulta especialmente  til antes de interpretar JSON proveniente de herramientas externas:

```
const texto = quitarBom(
  await readFile('datos.json', 'utf8'),
);

const datos = JSON.parse(texto);
```

## No existe una detecci n universal infalible

Si un archivo no declara su codificaci n, no siempre puede deducirse de manera segura.



UTF-8 puede validarse, pero ciertos conjuntos de bytes son válidos bajo varias codificaciones. Por eso, un programa serio debe establecer un contrato:

Todos los archivos de entrada estarán codificados en UTF-8.

Mucho mejor que pedirle al programa que practique espiritismo con los bytes.

---

## 7. Fines de línea

Hay dos secuencias tradicionales:

Linux/macOS: `\n`  
Windows: `\r\n`

Node expone el fin de línea nativo mediante:

```
import { EOL } from 'node:os';  
  
console.log(JSON.stringify(EOL));
```

`os.EOL` vale `\n` en POSIX y `\r\n` en Windows. (nodejs.org)

Para archivos de proyecto, JSON, Markdown y datos internos suele ser más simple generar siempre:

`'\n'`

Los editores modernos lo entienden en todas las plataformas.

Cuando se lee texto proveniente de cualquier sistema, puede separarse así:

```
const lineas = texto.split(/\r\n|\n|\r/);
```

Esto acepta:

- `\r\n`;
  - `\n`;
  - el antiguo `\r`.
-

## 8. Crear y escribir archivos de texto

### Escritura básica

---

```
import * as path from 'node:path';
import {
  mkdir,
  writeFile,
} from 'node:fs/promises';

const archivo = path.join(
  'datos',
  'saludo.txt',
);

await mkdir(path.dirname(archivo), {
  recursive: true,
});

await writeFile(
  archivo,
  'Hola desde Node.js\n',
  {
    encoding: 'utf8',
  },
);
```

Si el archivo no existe, se crea.

Si ya existe, `writeFile()` lo reemplaza: su contenido anterior se trunca.

La función no crea automáticamente los directorios contenedores; por eso primero usamos `mkdir()`.

## Función reutilizable

---

```
import * as path from 'node:path';
import {
  mkdir,
  writeFile,
} from 'node:fs/promises';

async function escribirTexto(archivo, texto) {
  await mkdir(path.dirname(archivo), {
    recursive: true,
  });

  await writeFile(archivo, texto, {
    encoding: 'utf8',
  });
}
```

Uso:

```
await escribirTexto(
  path.join('datos', 'nota.txt'),
  'Primera línea\nSegunda línea\n',
);
```

## Crear sin sobrescribir

---

A veces queremos crear el archivo solamente si todavía no existe.

```
await writeFile(
  'configuracion.json',
  '{}\n',
  {
    encoding: 'utf8',
    flag: 'wx',
  },
);
```

La bandera se interpreta así:

```
w → escribir, creando o truncando
x → fallar si ya existe
```

Manejo del error:

```
try {
  await writeFile(
    'configuracion.json',
    '{}\n',
    {
      encoding: 'utf8',
      flag: 'wx',
    },
  );

  console.log('Archivo creado');
} catch (error) {
  if (error.code === 'EEXIST') {
    console.log('El archivo ya existía');
  } else {
    throw error;
  }
}
```

Esta técnica es mejor que:

```
// No recomendado
if (!(await existe(archivo))) {
  await writeFile(archivo, contenido);
}
```

Entre la comprobación y la escritura otro proceso podría crear el archivo. Es una condición de carrera.

## Crear un archivo vacío

```
await writeFile(
  'vacio.txt',
  '',
  {
    flag: 'wx',
  },
);
```

## 9. Agregar contenido

`appendFile()` agrega bytes al final.

```
import { appendFile } from 'node:fs/promises';

await appendFile(
  'registro.log',
  'Aplicación iniciada\n',
  {
    encoding: 'utf8',
  },
);
```

Si el archivo no existe, normalmente se crea.

```
async function registrar(mensaje) {
  const linea =
    `${new Date().toISOString()} ${mensaje}\n`;

  await appendFile(
    'aplicacion.log',
    linea,
    {
      encoding: 'utf8',
    },
  );
}
```

Uso:

```
await registrar('Servidor iniciado');
await registrar('Usuario autenticado');
```

Resultado:

```
2026-08-24T06:00:00.000Z Servidor iniciado
2026-08-24T06:00:01.000Z Usuario autenticado
```

## Cuidado con escrituras simultáneas

Esto es conceptualmente problemático:

```
await Promise.all([
  appendFile('datos.txt', 'A\n'),
  appendFile('datos.txt', 'B\n'),
  appendFile('datos.txt', 'C\n'),
]);
```

No debe asumirse un orden determinado.

Cuando el orden importa:

```
await appendFile('datos.txt', 'A\n');
await appendFile('datos.txt', 'B\n');
await appendFile('datos.txt', 'C\n');
```

Las APIs de archivos basadas en promesas no sincronizan por sí solas múltiples modificaciones concurrentes sobre el mismo archivo. (nodejs.org)

---

## 10. Leer archivos

### Leer todo el archivo como texto

---

```
import { readFile } from 'node:fs/promises';

const texto = await readFile(
  'datos.txt',
  {
    encoding: 'utf8',
  },
);

console.log(texto);
```

También puede escribirse:

```
const texto = await readFile(
  'datos.txt',
  'utf8',
);
```

Al indicar la codificación, Node:

1. lee los bytes;
2. los decodifica;
3. devuelve un `string`.

## Leer como bytes

---

```
const bytes = await readFile('imagen.png');

console.log(Buffer.isBuffer(bytes));
// true

console.log(bytes.length);
// Cantidad de bytes
```

Al no indicar una codificación, `readFile()` devuelve un `Buffer`.

Esto es correcto para:

- imágenes;
- PDFs;
- archivos comprimidos;
- audio;
- ejecutables;
- datos binarios.

Nunca debe hacerse esto con un archivo binario:

```
const imagen = await readFile(
  'imagen.png',
  'utf8',
);
```

Eso intenta interpretar los bytes de la imagen como texto UTF-8.

## Leer solamente si existe

---

En lugar de comprobar primero su existencia, intentamos leerlo:

```
async function leerTextoOrNull(archivo) {
  try {
    return await readFile(archivo, 'utf8');
  } catch (error) {
    if (error.code === 'ENOENT') {
      return null;
    }

    throw error;
  }
}
```

Uso:

```
const configuracion =
  await leerTextoONull('configuracion.json');

if (configuracion === null) {
  console.log('Todavía no existe');
}
```

Node desaconseja comprobar con `access()` antes de abrir, leer o escribir, porque el estado puede cambiar entre ambas operaciones. Es preferible ejecutar la operación y tratar `error.code`. (nodejs.org)

## 11. Trabajar con un manejador explícito

Las funciones anteriores son operaciones completas:

abrir → operar → cerrar

Cuando necesitamos realizar varias operaciones sobre el mismo archivo, podemos abrirlo explícitamente.

```
import { open } from 'node:fs/promises';

const manejador = await open(
  'datos.txt',
  'r',
);

try {
  const contenido = await manejador.readFile({
    encoding: 'utf8',
  });

  console.log(contenido);
} finally {
  await manejador.close();
}
```

El `finally` garantiza el cierre incluso si ocurre un error.

Las banderas habituales son:



| Bandera         | Significado                       |
|-----------------|-----------------------------------|
| <code>r</code>  | Leer                              |
| <code>r+</code> | Leer y escribir                   |
| <code>w</code>  | Escribir, creando o truncando     |
| <code>wx</code> | Crear sin reemplazar              |
| <code>a</code>  | Agregar al final                  |
| <code>ax</code> | Agregar, pero fallar si ya existe |

Para una lectura o escritura completa, `readFile()` y `writeFile()` son más sencillas. El `FileHandle` resulta útil cuando necesitamos:

- leer sectores concretos;
- mantener el archivo abierto;
- escribir varias veces;
- controlar posiciones;
- sincronizar explícitamente con el dispositivo.

## 12. Guardar y leer JSON

JSON es texto con una sintaxis determinada.

## Escribir

---

```
import { writeFile } from 'node:fs/promises';

const alumnos = [
  {
    legajo: 63355,
    nombre: 'Jazmín Vargas',
  },
  {
    legajo: 61124,
    nombre: 'José Medina',
  },
];

const texto = JSON.stringify(
  alumnos,
  null,
  2,
);

await writeFile(
  'alumnos.json',
  `${texto}\n`,
  {
    encoding: 'utf8',
  },
);
```

`JSON.stringify()` convierte estructuras de JavaScript en texto JSON.

El archivo resultante:

```
[
  {
    "legajo": 63355,
    "nombre": "Jazmín Vargas"
  },
  {
    "legajo": 61124,
    "nombre": "José Medina"
  }
]
```

## Leer

---

```
import { readFile } from 'node:fs/promises';

const texto = await readFile(
  'alumnos.json',
  'utf8',
);

const alumnos = JSON.parse(
  quitarBom(texto),
);

console.log(alumnos[0].nombre);
```

`JSON.parse()` no lee archivos. Recibe texto.

El proceso completo es:

```
readFile()
  ↓
string
  ↓ JSON.parse()
array u objeto
```

## Manejar JSON inválido

---

```
async function leerJson(archivo) {
  const texto = quitarBom(
    await readFile(archivo, 'utf8'),
  );

  try {
    return JSON.parse(texto);
  } catch (error) {
    throw new Error(
      `El archivo ${archivo} no contiene JSON válido`,
      {
        cause: error,
      },
    );
  }
}
```

## 13. Obtener información y metadatos

`stat()` obtiene información del sistema de archivos.

```
import { stat } from 'node:fs/promises';

const informacion = await stat('datos.txt');

console.log(informacion.size);
console.log(informacion.mtime);
console.log(informacion.isFile());
console.log(informacion.isDirectory());
```

Una función más clara:

```
async function describirEntrada(ruta) {
  const informacion = await stat(ruta);

  return {
    ruta,
    tipo: informacion.isFile()
      ? 'archivo'
      : informacion.isDirectory()
        ? 'directorio'
        : 'otro',
    tamañoEnBytes: informacion.size,
    modificacion: informacion.mtime,
    acceso: informacion.atime,
    creacion: informacion.birthtime,
  };
}
```

Uso:

```
console.log(
  await describirEntrada('datos.txt'),
);
```

`size` se expresa en bytes. `mtime` representa la última modificación del contenido. `birthtime` no es igual de fiable en todos los sistemas de archivos: cuando la fecha de creación no está disponible, Node puede devolver otro valor o la época Unix. (nodejs.org)

### `stat()` y enlaces simbólicos

`stat()` sigue un enlace simbólico y describe su destino.

Para describir el enlace mismo se utiliza:

```
import { lstat } from 'node:fs/promises';

const informacion = await lstat('enlace');
```

## 14. Listar un directorio

La forma más simple:

```
import { readdir } from 'node:fs/promises';

const nombres = await readdir('datos');

console.log(nombres);
```

Resultado posible:

```
[
  'alumnos.json',
  'notas.txt',
  'respaldos'
]
```

Esto devuelve solamente nombres.

## Obtener también el tipo

```
const entradas = await readdir(
  'datos',
  {
    withFileTypes: true,
  },
);

for (const entrada of entradas) {
  console.log({
    nombre: entrada.name,
    archivo: entrada.isFile(),
    directorio: entrada.isDirectory(),
    enlace: entrada.isSymbolicLink(),
  });
}
```

Con `withFileTypes: true`, cada elemento es un `Dirent`.

Una salida más legible:

```
function obtenerTipo(entrada) {
  if (entrada.isDirectory()) {
    return 'directorio';
  }

  if (entrada.isFile()) {
    return 'archivo';
  }

  if (entrada.isSymbolicLink()) {
    return 'enlace';
  }

  return 'otro';
}

const entradas = await readdir(
  'datos',
  {
    withFileTypes: true,
  },
);

entradas.sort((a, b) =>
  a.name.localeCompare(b.name, 'es'),
);

for (const entrada of entradas) {
  console.log(
    obtenerTipo(entrada),
    entrada.name,
  );
}
```

El orden debe establecerse explícitamente cuando sea relevante. `readdir()` también dispone de una opción recursiva en versiones modernas de Node, aunque construir primero nuestro propio recorrido ayuda a comprender el algoritmo. (nodejs.org)

## 15. Recorrer directorios recursivamente

Un sistema de directorios forma un árbol:

```
datos/
├── alumnos.json
├── cursos/
│   ├── c1.json
│   └── c3.json
└── textos/
    ├── introduccion.md
    └── unidad-1.md
```

Para recorrerlo:

1. listamos el directorio actual;
2. si encontramos un archivo, lo devolvemos;
3. si encontramos un directorio, repetimos el proceso dentro de él.

```
import * as path from 'node:path';
import { readdir } from 'node:fs/promises';

async function* recorrerArchivos(directorio) {
  const entradas = await readdir(
    directorio,
    {
      withFileTypes: true,
    },
  );

  entradas.sort((a, b) =>
    a.name.localeCompare(b.name, 'es'),
  );

  for (const entrada of entradas) {
    const rutaCompleta = path.join(
      directorio,
      entrada.name,
    );

    if (entrada.isDirectory()) {
      yield* recorrerArchivos(rutaCompleta);
    } else if (entrada.isFile()) {
      yield rutaCompleta;
    }
  }
}
```

Uso:

```
for await (  
  const archivo of recorrerArchivos('datos')  
) {  
  console.log(archivo);  
}
```

La función es un generador asíncrono:

```
async function* ...
```

No devuelve todos los archivos de golpe. Los va produciendo uno por uno.

Eso permite procesar árboles grandes sin construir primero un array gigantesco.

Esta implementación no sigue enlaces simbólicos, lo cual evita accidentalmente entrar en ciclos como:

```
A enlaza a B  
B enlaza a A
```

---



# 16. Buscar archivos por nombre o extensión

## Por extensión

```
import * as path from 'node:path';

async function buscarPorExtensiones(
  directorio,
  extensiones,
) {
  const aceptadas = new Set(
    extensiones.map(extension =>
      extension.startsWith('.')
        ? extension.toLowerCase()
        : `.${extension.toLowerCase()}`,
    ),
  );

  const encontrados = [];

  for await (
    const archivo of recorrerArchivos(directorio)
  ) {
    const extension =
      path.extname(archivo).toLowerCase();

    if (aceptadas.has(extension)) {
      encontrados.push(archivo);
    }
  }

  return encontrados;
}
```

Uso:

```
const archivos = await buscarPorExtensiones(
  'datos',
  ['txt', 'md', '.json'],
);

console.log(archivos);
```

## Por parte del nombre

```
async function buscarPorNombre(
  directorio,
  fragmento,
) {
  const buscado = fragmento.toLocaleLowerCase('es');
  const encontrados = [];

  for await (
    const archivo of recorrerArchivos(directorio)
  ) {
    const nombre = path
      .basename(archivo)
      .toLocaleLowerCase('es');

    if (nombre.includes(buscado)) {
      encontrados.push(archivo);
    }
  }

  return encontrados;
}
```

Uso:

```
const encontrados = await buscarPorNombre(
  'datos',
  'alumno',
);
```

## Búsqueda con patrones `glob`

En Node 22.17 o posterior, `fsPromises.glob()` es una API estable. Permite patrones como:

```
**/*.txt
**/*.json
src/**/*.js
```

La versión estable admite opciones como `cwd`, exclusiones y resultados como iterador asíncrono. (nodejs.org)

```
import { glob } from 'node:fs/promises';

for await (
  const archivo of glob(
    [
      '**/*.txt',
      '**/*.md',
    ],
    {
      cwd: 'datos',
      exclude: [
        'node_modules/**',
        '.git/**',
      ],
    },
  )
) {
  console.log(archivo);
}
```

Los símbolos principales son:

| Patrón   | Significado                            |
|----------|----------------------------------------|
| *        | Cualquier secuencia dentro de un nivel |
| ?        | Un carácter                            |
| **       | Cero o más niveles de directorios      |
| *.txt    | Archivos TXT del directorio actual     |
| **/*.txt | TXT en cualquier profundidad           |

El recorrido manual enseña el algoritmo. `glob()` resuelve el caso cotidiano con menos código.

## 17. Buscar texto dentro de un archivo pequeño

Para un archivo pequeño podemos cargarlo completamente.

```

function buscarEnTexto(
  texto,
  consulta,
  {
    distinguirMayusculas = false,
  } = {},
) {
  if (consulta.length === 0) {
    throw new Error(
      'La consulta no puede estar vacía',
    );
  }

  const transformar = valor =>
    distinguirMayusculas
      ? valor
      : valor.toLocaleLowerCase('es');

  const buscado = transformar(consulta);
  const lineas = texto.split(/\r\n|\n|\r/);
  const coincidencias = [];

  for (
    let indiceLinea = 0;
    indiceLinea < lineas.length;
    indiceLinea++
  ) {
    const lineaOriginal = lineas[indiceLinea];
    const lineaComparada =
      transformar(lineaOriginal);

    let desde = 0;

    while (desde < lineaComparada.length) {
      const indice = lineaComparada.indexOf(
        buscado,
        desde,
      );

      if (indice === -1) {
        break;
      }

      coincidencias.push({
        linea: indiceLinea + 1,
        columna: indice + 1,
        texto: lineaOriginal,
      });

      desde = indice + buscado.length;
    }
  }
}

```

```
    return coincidencias;
}
```

Uso:

```
const texto = await readFile(
  'notas.txt',
  'utf8',
);

const coincidencias = buscarEnTexto(
  texto,
  'node',
);

console.log(coincidencias);
```

Resultado posible:

```
[
  {
    linea: 3,
    columna: 12,
    texto: 'Trabajaremos con Node.js'
  }
]
```

La columna está medida en unidades de código de JavaScript. Para textos corrientes esto suele ser suficiente; una aplicación editorial que deba contar símbolos visuales necesitaría segmentación Unicode más precisa.

---

## 18. Buscar dentro de un archivo grande

`readFile()` carga el archivo completo en memoria.

Para un archivo de varios gigabytes eso sería una idea creativa, pero del tipo que termina con el sistema operativo golpeando la puerta.

En archivos grandes conviene leer por partes mediante un stream.

```

import { createReadStream } from 'node:fs';
import { createInterface } from 'node:readline';

async function* buscarPorLineas(
  archivo,
  consulta,
  {
    distinguirMayusculas = false,
  } = {},
) {
  if (consulta.length === 0) {
    throw new Error(
      'La consulta no puede estar vacía',
    );
  }

  const transformar = valor =>
    distinguirMayusculas
      ? valor
      : valor.toLocaleLowerCase('es');

  const buscado = transformar(consulta);

  const entrada = createReadStream(
    archivo,
    {
      encoding: 'utf8',
    },
  );

  const lector = createInterface({
    input: entrada,
    crlfDelay: Infinity,
  });

  let numeroLinea = 0;

  for await (const linea of lector) {
    numeroLinea++;

    const comparada = transformar(linea);
    const columna = comparada.indexOf(buscado);

    if (columna !== -1) {
      yield {
        archivo,
        linea: numeroLinea,
        columna: columna + 1,
        texto: linea,
      };
    }
  }
}

```

Uso:

```
for await (
  const coincidencia of buscarPorLineas(
    'servidor.log',
    'error',
  )
) {
  console.log(coincidencia);
}
```

`readline` permite consumir un stream línea por línea. Con `crlfDelay: Infinity`, las secuencias `\r\n` se interpretan correctamente como un único fin de línea. (nodejs.org)

## Buscar en todos los archivos de un árbol

---

Primero determinamos cuáles extensiones trataremos como texto:

```
import * as path from 'node:path';

const EXTENSIONES_DE_TEXTO = new Set([
  '.txt',
  '.md',
  '.json',
  '.csv',
  '.js',
  '.mjs',
  '.ts',
  '.html',
  '.css',
]);
```

Después recorreremos los archivos uno por uno:

```

async function buscarEnDirectorio(
  directorio,
  consulta,
) {
  const resultados = [];

  for await (
    const archivo of recorrerArchivos(directorio)
  ) {
    const extension =
      path.extname(archivo).toLowerCase();

    if (!EXTENSIONES_DE_TEXTO.has(extension)) {
      continue;
    }

    for await (
      const coincidencia of buscarPorLineas(
        archivo,
        consulta,
      )
    ) {
      resultados.push(coincidencia);
    }
  }

  return resultados;
}

```

Uso:

```

const resultados = await buscarEnDirectorio(
  'proyecto',
  'TODO',
);

console.table(resultados);

```

Esta versión abre solamente un archivo por vez. Es más prudente que lanzar miles de lecturas simultáneas con `Promise.all()`.



# 19. Copiar archivos

## Copiar un archivo

---

```
import { copyFile } from 'node:fs/promises';

await copyFile(
  'original.txt',
  'copia.txt',
);
```

Esto copia los bytes.

No importa si el archivo contiene:

- UTF-8;
- una imagen;
- un PDF;
- audio;
- datos comprimidos.

La copia no decodifica ni vuelve a codificar.

Por defecto, `copyFile()` puede reemplazar el archivo de destino.

## Evitar el reemplazo

---

```
import { constants } from 'node:fs';
import { copyFile } from 'node:fs/promises';

await copyFile(
  'original.txt',
  'copia.txt',
  constants.COPYFILE_EXCL,
);
```

Si `copia.txt` ya existe, la operación falla con `EEXIST`. La constante `COPYFILE_EXCL` establece precisamente esa política. (nodejs.org)

Una función completa:

```
import * as path from 'node:path';
import { constants } from 'node:fs';
import {
  copyFile,
  mkdir,
} from 'node:fs/promises';

async function copiarSinReemplazar(
  origen,
  destino,
) {
  await mkdir(path.dirname(destino), {
    recursive: true,
  });

  await copyFile(
    origen,
    destino,
    constants.COPYFILE_EXCL,
  );
}
```

## Copiar directorios

```
import { cp } from 'node:fs/promises';

await cp(
  'datos',
  'respaldo-datos',
  {
    recursive: true,
  },
);
```

Para evitar reemplazar entradas existentes:

```
await cp(
  'datos',
  'respaldo-datos',
  {
    recursive: true,
    force: false,
    errorOnExist: true,
  },
);
```

`cp()` copia árboles de directorios y dispone de opciones para recursividad, reemplazo, filtros y preservación de fechas. La API dejó de ser experimental en Node 22.3. ([nodejs.org](https://nodejs.org))

---

## 20. Renombrar y mover

Renombrar:

```
import { rename } from 'node:fs/promises';

await rename(
  'notas.txt',
  'notas-programacion.txt',
);
```

Mover es la misma operación, pero cambiando el directorio:

```
await rename(
  path.join('entrada', 'datos.txt'),
  path.join('procesados', 'datos.txt'),
);
```

Es recomendable crear antes el directorio de destino:

```
import * as path from 'node:path';
import {
  mkdir,
  rename,
} from 'node:fs/promises';

async function mover(origen, destino) {
  await mkdir(path.dirname(destino), {
    recursive: true,
  });

  await rename(origen, destino);
}
```

`rename()` cambia `oldPath` por `newPath`; no interpreta el contenido. (nodejs.org)

---

### Mover entre discos o volúmenes

Un renombrado suele funcionar dentro del mismo sistema de archivos. Si el origen y el destino están en volúmenes diferentes, puede producirse `EXDEV`.

En ese caso hay que:

1. copiar;

2. comprobar que la copia terminó;
3. eliminar el original.

```

import * as path from 'node:path';
import { constants } from 'node:fs';
import {
  copyFile,
  cp,
  mkdir,
  rename,
  rm,
  stat,
} from 'node:fs/promises';

async function moverMultiplataforma(
  origen,
  destino,
) {
  await mkdir(path.dirname(destino), {
    recursive: true,
  });

  try {
    await rename(origen, destino);
    return;
  } catch (error) {
    if (error.code !== 'EXDEV') {
      throw error;
    }
  }

  const informacion = await stat(origen);

  if (informacion.isDirectory()) {
    await cp(origen, destino, {
      recursive: true,
      force: false,
      errorOnExist: true,
    });

    await rm(origen, {
      recursive: true,
    });

    return;
  }

  if (informacion.isFile()) {
    await copyFile(
      origen,
      destino,
      constants.COPYFILE_EXCL,
    );

    await rm(origen);
    return;
  }
}

```

```
    throw new Error(
      'El origen no es un archivo ni un directorio ordinario',
    );
  }
```

El reemplazo por copia y eliminación no es atómico. Si la copia termina pero la eliminación falla, quedarán ambas versiones. Eso es preferible a perder las dos, pero debe contemplarse en el diseño.

---

## 21. Eliminar

### Archivo

---

```
import { rm } from 'node:fs/promises';

await rm('temporal.txt');
```

### Ignorar si no existe

---

```
await rm(
  'temporal.txt',
  {
    force: true,
  },
);
```

### Directorio vacío o con contenido

---

```
await rm(
  'directorio-temporal',
  {
    recursive: true,
    force: true,
  },
);
```

`recursive: true` elimina todo el árbol. No hay papelera de reciclaje intermedia. El método hace exactamente lo que se le pide, incluso cuando el programador se arrepiente tres milisegundos después.

Una práctica prudente es resolver y verificar la ruta antes de eliminar:

```
const destino = path.resolve(
  'temporales',
  identificador,
);

console.log('Se eliminará:', destino);

await rm(destino, {
  recursive: true,
  force: true,
});
```

---

## 22. Extraer información de un archivo

“Extraer información” puede significar varias cosas.

### Extraer metadatos

---

```
const informacion = await stat(archivo);

console.log({
  bytes: informacion.size,
  modificado: informacion.mtime,
});
```

## Extraer estadísticas de texto

```
function analizarTexto(texto) {
  const lineas = texto === ''
    ? 0
    : texto.split(/\r\n|\n|\r/).length;

  const palabras = texto.trim() === ''
    ? 0
    : texto.trim().split(/\s+/u).length;

  const segmentador = new Intl.Segmenter(
    'es',
    {
      granularity: 'grapheme',
    },
  );

  return {
    bytesUtf8: Buffer.byteLength(
      texto,
      'utf8',
    ),
    unidadesUtf16: texto.length,
    puntosDeCodigo: [...texto].length,
    simbolosVisuales: [
      ...segmentador.segment(texto),
    ].length,
    lineas,
    palabrasAproximadas: palabras,
  };
}
```

Uso:

```
const texto = await readFile(
  'articulo.txt',
  'utf8',
);

console.log(analizarTexto(texto));
```

El conteo de palabras mediante espacios es aproximado. Definir qué es una “palabra” es una decisión lingüística y de negocio, no una propiedad universal del archivo.



## Extraer estructuras JSON

---

```
const datos = JSON.parse(
  await readFile('alumnos.json', 'utf8'),
);

const nombres = datos.map(
  alumno => alumno.nombre,
);

console.log(nombres);
```

## Extraer datos de un log

---

Supongamos este formato:

```
2026-08-24T09:10:00Z INFO usuario=ana accion=login
2026-08-24T09:12:00Z ERROR usuario=juan accion=guardar
```

Podemos interpretarlo con una expresión regular:

```

const PATRON_LOG =
  /^(?<fecha>\S+)\s+(?<nivel>INFO|WARN|ERROR)\s+usuario=(?<usuario>\S+)\s+accion=(?
<accion>\S+)\$/u;

function interpretarLog(texto) {
  const resultados = [];

  const lineas = texto.split(
    /\r\n|\n|\r/,
  );

  for (
    let indice = 0;
    indice < lineas.length;
    indice++
  ) {
    const coincidencia =
      PATRON_LOG.exec(lineas[indice]);

    if (!coincidencia) {
      continue;
    }

    resultados.push({
      linea: indice + 1,
      fecha: coincidencia.groups.fecha,
      nivel: coincidencia.groups.nivel,
      usuario:
        coincidencia.groups.usuario,
      accion:
        coincidencia.groups.accion,
    });
  }

  return resultados;
}

```

Uso:

```

const texto = await readFile(
  'aplicacion.log',
  'utf8',
);

const registros = interpretarLog(texto);

const errores = registros.filter(
  registro => registro.nivel === 'ERROR',
);

console.table(errores);

```

El archivo no “contiene objetos”. Contiene texto que nuestro programa interpreta siguiendo un formato acordado.

## CSV

---

Esto:

```
linea.split(',');
```

no es un analizador CSV general.

Un campo CSV puede contener:

```
"Di Battista, Alejandro"
```

También puede contener:

- comillas escapadas;
- saltos de línea;
- separadores dentro del campo.

Solamente sería válido usar `split(',')` bajo un contrato muy limitado:

```
Ningún campo contiene comas, comillas ni saltos de línea.
```

Para CSV real conviene utilizar un parser especializado.

## PDF, DOCX e imágenes

---

Node puede leer sus bytes:

```
const bytes = await readFile('documento.pdf');
```

Pero no puede obtener automáticamente párrafos, tablas o imágenes simplemente porque haya leído el archivo.

Hace falta un parser que comprenda el formato:

```
bytes
  ↓ parser de PDF, DOCX, PNG...
estructura interna
  ↓
información
```

Leer el archivo y comprender el archivo son operaciones distintas.

---

## 23. Seguridad: impedir que una ruta escape de un directorio

Supongamos que una aplicación guarda archivos dentro de:

```
/uploads
```

Y recibe del usuario:

```
../../configuracion-secreta.json
```

Si se concatena sin validar, el usuario puede escapar del directorio autorizado.

Una comprobación básica:

```
import * as path from 'node:path';

function resolverDentroDe(
  directorioBase,
  rutaSolicitada,
) {
  const base = path.resolve(
    directorioBase,
  );

  const candidata = path.resolve(
    base,
    rutaSolicitada,
  );

  const relativa = path.relative(
    base,
    candidata,
  );

  const escapa =
    relativa === '..' ||
    relativa.startsWith(
      `..${path.sep}`,
    ) ||
    path.isAbsolute(relativa);

  if (escapa) {
    throw new Error(
      'La ruta está fuera del directorio permitido',
    );
  }

  return candidata;
}
```

Uso:

```
const archivoSeguro = resolverDentroDe(
  'uploads',
  'usuarios/ana/foto.png',
);
```

Esto rechaza:

```
resolverDentroDe(
  'uploads',
  '../../secreto.txt',
);
```

Para entornos hostiles también hay que definir una política sobre enlaces simbólicos, porque un enlace dentro del directorio puede apuntar fuera de él.

---

## 24. Manejo de errores

Las operaciones de archivos pueden fallar por razones normales:

| Código    | Significado habitual                            |
|-----------|-------------------------------------------------|
| ENOENT    | La ruta no existe                               |
| EEXIST    | El destino ya existe                            |
| EACCES    | Acceso denegado                                 |
| EPERM     | Operación no permitida                          |
| EISDIR    | Se esperaba un archivo y apareció un directorio |
| ENOTDIR   | Se esperaba un directorio                       |
| ENOTEMPTY | El directorio no está vacío                     |
| EXDEV     | Operación entre dispositivos o volúmenes        |

Ejemplo:

```

try {
  const texto = await readFile(
    'datos.txt',
    'utf8',
  );

  console.log(texto);
} catch (error) {
  switch (error.code) {
    case 'ENOENT':
      console.error(
        'El archivo no existe',
      );
      break;

    case 'EACCES':
    case 'EPERM':
      console.error(
        'No hay permisos suficientes',
      );
      break;

    case 'EISDIR':
      console.error(
        'La ruta corresponde a un directorio',
      );
      break;

    default:
      throw error;
  }
}

```

Para tomar decisiones conviene utilizar `error.code`, no analizar el texto de `error.message`. Los errores de sistema de Node incluyen propiedades como `code`, `path`, `dest` y `syscall`. (nodejs.org)

## No ocultar todos los errores

Esto es peligroso:

```

try {
  await realizarOperacion();
} catch {
  // No hacer nada
}

```

Podría ocultar:

- corrupción de datos;
- falta de permisos;
- disco lleno;
- errores de programación;
- rutas equivocadas.

Debe tratarse solamente aquello que realmente se sabe resolver:

```
try {
  await readFile(archivo, 'utf8');
} catch (error) {
  if (error.code === 'ENOENT') {
    return null;
  }

  throw error;
}
```

## 25. Archivos pequeños frente a archivos grandes

No existe un número universal que separe "pequeño" de "grande". Depende de:

- memoria disponible;
- cantidad de solicitudes concurrentes;
- tamaño de los archivos;
- frecuencia de acceso;
- latencia aceptable.

La elección general es:

| Situación                        | Herramienta                                             |
|----------------------------------|---------------------------------------------------------|
| Leer todo un archivo pequeño     | <code>readFile()</code>                                 |
| Escribir todo un archivo pequeño | <code>writeFile()</code>                                |
| Procesar muchas líneas           | <code>createReadStream()</code> + <code>readline</code> |
| Transferir datos grandes         | Streams                                                 |
| Leer posiciones concretas        | <code>FileHandle</code>                                 |
| Listar un directorio             | <code>readdir()</code>                                  |
| Recorrer un árbol                | Generador recursivo o <code>glob()</code>               |

`readFile()` tiene una ventaja enorme: simplicidad.



Los streams tienen otra: no necesitan almacenar simultáneamente todo el contenido.

## 26. Diferencias con otros lenguajes

Los conceptos no cambian:

ruta → abrir → bytes → decodificar → interpretar → cerrar

Lo que cambia es la forma de expresarlos.

| Concepto                       | Node.js                          | Python                                            | C#                                                                                     | Java                                           |
|--------------------------------|----------------------------------|---------------------------------------------------|----------------------------------------------------------------------------------------|------------------------------------------------|
| Rutas                          | Strings + <code>node:path</code> | Objetos <code>Path</code>                         | <code>Path</code> , <code>File</code> , <code>Directory</code> , <code>FileInfo</code> | Objetos <code>Path</code> + <code>Files</code> |
| Leer texto completo            | <code>await readFile()</code>    | <code>Path.read_text()</code>                     | <code>await File.ReadAllTextAsync()</code>                                             | <code>Files.readString()</code>                |
| Escribir texto                 | <code>await writeFile()</code>   | <code>Path.write_text()</code>                    | <code>await File.WriteAllTextAsync()</code>                                            | <code>Files.writeString()</code>               |
| Bytes                          | <code>Buffer</code>              | <code>bytes</code>                                | <code>byte[]</code>                                                                    | <code>byte[]</code>                            |
| Operación asincrónica estándar | Promesas                         | <code>pathlib</code> es principalmente sincrónico | APIs <code>Async</code>                                                                | <code>Files</code> básico es sincrónico        |
| Cierre explícito               | <code>FileHandle.close()</code>  | <code>with</code>                                 | <code>using</code> / <code>await using</code>                                          | <code>try-with-resources</code>                |

Python concentra muchas operaciones directamente en objetos `Path`. C# ofrece métodos asincrónicos como `ReadAllTextAsync()` y `WriteAllTextAsync()`. Java separa objetos `Path` de la clase utilitaria `Files`; `Files.readString()` utiliza UTF-8 cuando no se especifica otro charset. (docs.python.org)

### Una diferencia importante de Node.js

Node está diseñado alrededor de un event loop. Por eso es habitual escribir:

```
const texto = await readFile(
  archivo,
  'utf8',
);
```

En lugar de:

```
const texto = readFileSync(  
  archivo,  
  'utf8',  
);
```

La segunda forma detiene la ejecución de JavaScript mientras termina la operación.

En un pequeño script de línea de comandos eso puede ser aceptable. En un servidor que atiende múltiples solicitudes puede bloquear a todos los usuarios.

---

## 27. Programa integrador completo

Este programa:

1. crea un directorio temporal;
2. crea un archivo;
3. agrega una línea;
4. lo lee;
5. guarda JSON;
6. lista el directorio;
7. busca texto;
8. copia;
9. renombra;
10. obtiene metadatos.

Guardarlo como `demo-archivos.mjs`:

```

import * as path from 'node:path';

import {
  constants,
  createReadStream,
} from 'node:fs';

import {
  appendFile,
  copyFile,
  mkdir,
  mkdtemp,
  readFile,
  readdir,
  rename,
  stat,
  writeFile,
} from 'node:fs/promises';

import { tmpdir } from 'node:os';
import { createInterface } from 'node:readline';

async function* buscarPorLineas(
  archivo,
  consulta,
) {
  const buscado =
    consulta.toLocaleLowerCase('es');

  const entrada = createReadStream(
    archivo,
    {
      encoding: 'utf8',
    },
  );

  const lector = createInterface({
    input: entrada,
    crlfDelay: Infinity,
  });

  let numeroLinea = 0;

  for await (const linea of lector) {
    numeroLinea++;

    const columna = linea
      .toLocaleLowerCase('es')
      .indexOf(buscado);

    if (columna !== -1) {
      yield {
        linea: numeroLinea,
        columna: columna + 1,
      };
    }
  }
}

```

```

        texto: linea,
    };
}
}
}

async function main() {
    const raiz = await mkdtemp(
        path.join(
            tmpdir(),
            'gestion-archivos-',
        ),
    );

    const originales = path.join(
        raiz,
        'originales',
    );

    const copias = path.join(
        raiz,
        'copias',
    );

    await mkdir(originales, {
        recursive: true,
    });

    await mkdir(copias, {
        recursive: true,
    });

    const notas = path.join(
        originales,
        'notas.txt',
    );

    // Crear y escribir.
    await writeFile(
        notas,
        'Primera línea\n',
        {
            encoding: 'utf8',
        },
    );

    // Agregar.
    await appendFile(
        notas,
        'Segunda línea con Tucumán\n',
        {
            encoding: 'utf8',
        },
    );
}

```

```

// Leer.
const contenido = await readFile(
  notas,
  'utf8',
);

console.log('Contenido:');
console.log(contenido);

// Guardar JSON.
const alumnos = [
  {
    legajo: 1,
    nombre: 'Ana',
  },
  {
    legajo: 2,
    nombre: 'Juan',
  },
];

const archivoJson = path.join(
  originales,
  'alumnos.json',
);

await writeFile(
  archivoJson,
  `${JSON.stringify(
    alumnos,
    null,
    2,
  )}\n`,
  {
    encoding: 'utf8',
  },
);

// Listar.
const entradas = await readdir(
  originales,
  {
    withFileTypes: true,
  },
);

console.log('Entradas:');

for (const entrada of entradas) {
  console.log({
    nombre: entrada.name,
    esArchivo: entrada.isFile(),
    esDirectorio: entrada.isDirectory(),
  });
}

```

```

// Buscar.
console.log('Coincidencias:');

for await (
  const coincidencia of buscarPorLineas(
    notas,
    'tucumán',
  )
) {
  console.log(coincidencia);
}

// Copiar sin reemplazar.
const copia = path.join(
  copias,
  'notas.txt',
);

await copyFile(
  notas,
  copia,
  constants.COPYFILE_EXCL,
);

// Renombrar.
const copiaRenombrada = path.join(
  copias,
  'notas-respaldo.txt',
);

await rename(
  copia,
  copiaRenombrada,
);

// Metadatos.
const informacion = await stat(
  copiaRenombrada,
);

console.log('Metadatos:');

console.log({
  ruta: copiaRenombrada,
  bytes: informacion.size,
  modificado: informacion.mtime,
  esArchivo: informacion.isFile(),
});

console.log(
  '\nDirectorio generado:',
  raiz,
);
}

```

```
main().catch(error => {  
  console.error(error);  
  process.exitCode = 1;  
});
```

Ejecutar:

```
node demo-archivos.mjs
```

El programa utiliza el directorio temporal correspondiente a Windows, macOS o Linux, por lo que no contiene rutas específicas de ninguna plataforma.

---

## 28. Mapa mental final

La gestión de archivos puede resumirse así:

¿Dónde está?  
node:path

¿Existe el directorio contenedor?  
mkdir({ recursive: true })

¿Quiero escribir todo?  
writeFile()

¿Quiero agregar al final?  
appendFile()

¿Quiero leer todo como texto?  
readFile(ruta, 'utf8')

¿Quiero leer bytes?  
readFile(ruta)

¿Es demasiado grande?  
createReadStream()

¿Quiero listar?  
readdir({ withFileTypes: true })

¿Quiero recorrer subdirectorios?  
generador recursivo o glob()

¿Quiero copiar un archivo?  
copyFile()

¿Quiero copiar un directorio?  
cp({ recursive: true })

¿Quiero cambiar nombre o ubicación?  
rename()

¿Quiero conocer tamaño y fechas?  
stat()

¿Quiero eliminar?  
rm()

¿Quiero comprender el contenido?  
decodificar + interpretar el formato

La regla fundamental es mantener separadas estas capas:



Ruta

≠

Archivo

≠

Bytes

≠

Texto

≠

Formato

≠

Información

Cuando esas diferencias están claras, la API deja de parecer una colección arbitraria de funciones. Cada operación cae naturalmente en el nivel que le corresponde.

# Gestión de archivos con JavaScript y Node.js

Cuando un programa maneja archivos está haciendo algo conceptualmente muy simple:

**guardar bytes en un dispositivo y recuperarlos posteriormente.**

Todo lo demás —archivos de texto, JSON, imágenes, carpetas, copiar, buscar, renombrar— se construye sobre esa idea.

En Node.js, la API fundamental para hacerlo es `fs` (**file system**). Node ofrece versiones síncronas, basadas en callbacks y basadas en `Promise`. Para programas modernos usaremos normalmente:

```
import fs from "node:fs/promises";
```

Esto nos permite trabajar naturalmente con `async` / `await`.

Para el sistema operativo un archivo puede imaginarse como:

```
nombre
  ↓
datos almacenados
  ↓
01001000 01101111 01101100 01100001
```

Esos datos son **bytes**.

Un archivo no sabe si contiene:

- texto;
- una imagen;
- audio;
- JSON;
- un programa;
- un PDF.

Sólo contiene bytes.

Por ejemplo:

```
48 6F 6C 61
```

son cuatro bytes expresados en hexadecimal.

Si los interpretamos como UTF-8 obtenemos:

```
Hola
```

La distinción importante es:

```
archivo
  ↓
bytes
  ↓
interpretación
```

Por eso podemos leer un archivo de dos maneras fundamentales en Node:

```
const datos = await fs.readFile("archivo.txt");
```

o:

```
const texto = await fs.readFile("archivo.txt", "utf8");
```

En el primer caso obtenemos bytes.

En el segundo le estamos diciendo:

```
Leé los bytes e interpretalos como texto UTF-8.
```

---

## 2. Nuestro primer archivo

Creemos un programa:

```
proyecto/
└─ index.js
```

Y escribamos:

```
import fs from "node:fs/promises";

await fs.writeFile("saludo.txt", "Hola mundo");
```

Ejecutamos:

```
node index.js
```

Aparecerá:

```
proyecto/  
├─ index.js  
└─ saludo.txt
```

con:

```
Hola mundo
```

`writeFile()` crea el archivo si no existe.

Si ya existe, por defecto reemplaza su contenido.

---

## 3. Escribir texto

Podemos especificar explícitamente la codificación:

```
await fs.writeFile(  
  "saludo.txt",  
  "Hola mundo",  
  "utf8"  
);
```

Aunque para strings UTF-8 es el valor predeterminado de `writeFile()`.

Podemos escribir cualquier string JavaScript:

```
const contenido = `  
Nombre: Ana  
Edad: 25  
Ciudad: Tucumán  
`;  
  
await fs.writeFile("persona.txt", contenido, "utf8");
```

El archivo resultante será:

Nombre: Ana  
Edad: 25  
Ciudad: Tucumán

---

## 4. Leer un archivo de texto

Leerlo es exactamente la operación inversa:

```
const contenido = await fs.readFile(  
  "persona.txt",  
  "utf8"  
);  
  
console.log(contenido);
```

Obtenemos un `string`.

Sin `"utf8"`:

```
const contenido = await fs.readFile("persona.txt");  
  
console.log(contenido);
```

obtendríamos aproximadamente:

```
<Buffer 4e 6f 6d 62 72 65 ...>
```

Un `Buffer` representa bytes.

Podemos convertirlo después:

```
const bytes = await fs.readFile("persona.txt");  
  
const texto = bytes.toString("utf8");  
  
console.log(texto);
```

Esto evidencia algo importante:

```
archivo
  ↓
bytes
  ↓
Buffer
  ↓ UTF-8
string
```

---

## 5. ¿Qué es una codificación?

Supongamos que queremos guardar:

```
A
```

Unicode identifica `A` mediante el código:

```
U+0041
```

Pero un disco solamente guarda bytes.

UTF-8 establece cómo convertir ese código Unicode en bytes:

```
A
↓
U+0041
↓ UTF-8
41
```

Otro ejemplo:

```
ñ
↓
U+00F1
↓ UTF-8
C3 B1
```

Y un emoji puede necesitar todavía más bytes:



↓

U+1F600

↓ UTF-8

F0 9F 98 80

Por eso:

```
Buffer.from("A").length
```

produce:

1

pero:

```
Buffer.from("ñ").length
```

produce:

2

y:

```
Buffer.from("😄").length
```

produce:

4

UTF-8 es una codificación de longitud variable.

---

## 6. Texto JavaScript y texto almacenado

Aquí aparece una distinción fundamental.

Dentro de JavaScript tenemos:

```
const texto = "Hola 😊";
```

Eso es un `string`.

Cuando queremos guardarlo:

```
await fs.writeFile("texto.txt", texto, "utf8");
```

Node realiza aproximadamente:

```
JavaScript string
    ↓
codificación UTF-8
    ↓
bytes
    ↓
archivo
```

Cuando lo leemos:

```
const texto = await fs.readFile(
  "texto.txt",
  "utf8"
);
```

ocurre lo contrario:

```
archivo
    ↓
bytes
    ↓
decodificación UTF-8
    ↓
JavaScript string
```

La codificación es, por tanto, **el convenio que permite pasar entre caracteres y bytes**.

---

## 7. Agregar información sin borrar lo anterior

Esto:



```
await fs.writeFile("log.txt", "primero");
await fs.writeFile("log.txt", "segundo");
```

deja solamente:

segundo

porque la segunda escritura reemplaza el archivo.

Para agregar información usamos:

```
await fs.appendFile(
  "log.txt",
  "Nueva línea\n",
  "utf8"
);
```

Por ejemplo:

```
await fs.appendFile("log.txt", "Inicio\n");
await fs.appendFile("log.txt", "Usuario conectado\n");
await fs.appendFile("log.txt", "Fin\n");
```

produce:

```
Inicio
Usuario conectado
Fin
```

Esto es especialmente útil para:

- logs;
- historiales;
- registros;
- archivos CSV.

---

## 8. El problema de los saltos de línea

Históricamente los sistemas operativos no utilizaron exactamente la misma representación.

Unix, Linux y macOS modernos utilizan:

```
LF
```

representado como:

```
"\n"
```

Windows tradicionalmente utiliza:

```
CR + LF
```

es decir:

```
"\r\n"
```

Por eso un archivo puede contener:

```
uno\n
dos\n
tres\n
```

o:

```
uno\r\n
dos\r\n
tres\r\n
```

Una manera robusta de separar líneas es:

```
const lineas = texto.split(/\r?\n/);
```

Acepta ambas variantes.

Node también expone:

```
import os from "node:os";

console.log(os.EOL);
```

que devuelve el salto de línea convencional del sistema actual.

Sin embargo, para formatos interoperables suele ser perfectamente razonable utilizar simplemente:

```
"\n"
```

en todas las plataformas.

---

## 9. Rutas de archivos

Un archivo se identifica mediante una **ruta**.

En Unix, Linux y macOS podemos encontrar:

```
/home/ana/documentos/notas.txt
```

o:

```
/Users/ana/documentos/notas.txt
```

En Windows:

```
C:\Users\Ana\documentos\notas.txt
```

Aquí aparece uno de los principales problemas de portabilidad.

Windows suele utilizar:

```
\
```

mientras que los sistemas POSIX utilizan:

```
/
```

No deberíamos construir rutas así:

```
const archivo = carpeta + "/" + nombre;
```

Node proporciona el módulo:

```
import path from "node:path";
```

Entonces hacemos:

```
const archivo = path.join(  
  "datos",  
  "usuarios",  
  "ana.txt"  
);
```

Node producirá la ruta apropiada para el sistema operativo.

Conceptualmente:

```
path.join(...)  
  ↓  
ruta correcta para la plataforma
```

---

## 10. Rutas relativas y absolutas

Si escribimos:

```
await fs.readFile("datos.txt", "utf8");
```

`datos.txt` es una **ruta relativa**.

Se interpreta respecto del directorio actual del proceso:

```
console.log(process.cwd());
```

Por ejemplo:

```
/Users/alejandro/proyecto
```

Entonces:

```
datos.txt
```

significa:

```
/Users/alejandro/proyecto/datos.txt
```

Podemos construir una ruta absoluta:

```
const archivo = path.resolve("datos.txt");  
  
console.log(archivo);
```

## 11. Analizar una ruta

Supongamos:

```
const archivo = "/usuarios/ana/documentos/informe.pdf";
```

Podemos obtener distintas partes.

Nombre:

```
path.basename(archivo);
```

Resultado:

```
informe.pdf
```

Directorio:

```
path.dirname(archivo);
```

Resultado:

```
/usuarios/ana/documentos
```

Extensión:

```
path.extname(archivo);
```

Resultado:

```
.pdf
```

Esto es muy útil para clasificar archivos.

Por ejemplo:

```
const extension = path.extname("foto.jpg");

if (extension === ".jpg") {
  console.log("Es una imagen JPEG");
}
```

---

## 12. Crear directorios

Podemos crear una carpeta:

```
await fs.mkdir("datos");
```

Pero fallará si ya existe.

Normalmente hacemos:

```
await fs.mkdir("datos", {
  recursive: true
});
```

Además permite crear varios niveles:

```
await fs.mkdir(
  path.join("datos", "usuarios", "imagenes"),
  { recursive: true }
);
```

---

## 13. Listar archivos

Supongamos:

```
datos/  
├─ ana.txt  
├─ juan.txt  
├─ foto.jpg  
└─ notas.md
```

Podemos listar:

```
const archivos = await fs.readdir("datos");  
  
console.log(archivos);
```

Resultado:

```
[  
  "ana.txt",  
  "juan.txt",  
  "foto.jpg",  
  "notas.md"  
]
```

---

## 14. Saber si algo es archivo o directorio

Un directorio puede contener ambos:

```
datos/  
├─ usuarios/  
├─ imagenes/  
└─ config.json
```

Podemos pedir información adicional:

```
const elementos = await fs.readdir(  
  "datos",  
  { withFileTypes: true }  
);
```

Entonces:

```
for (const elemento of elementos) {  
  
  if (elemento.isFile()) {  
    console.log("Archivo:", elemento.name);  
  }  
  
  if (elemento.isDirectory()) {  
    console.log("Directorio:", elemento.name);  
  }  
}
```

---

## 15. Obtener información sobre un archivo

Usamos:

```
const info = await fs.stat("datos.txt");
```

Podemos consultar:

```
console.log(info.size);  
console.log(info.mtime);  
console.log(info.birthtime);
```

y:

```
console.log(info.isFile());  
console.log(info.isDirectory());
```

Por ejemplo:

```
const info = await fs.stat("datos.txt");  
  
console.log(`Tamaño: ${info.size} bytes`);  
console.log(`Modificado: ${info.mtime}`);
```

Hay que tener cuidado con algunos metadatos: su significado y disponibilidad exactos pueden variar entre sistemas de archivos y sistemas operativos.

---



## 16. Copiar un archivo

Node tiene una operación específica:

```
await fs.copyFile(
  "original.txt",
  "copia.txt"
);
```

Para copiar estructuras completas existe:

```
await fs.cp(
  "origen",
  "destino",
  { recursive: true }
);
```

---

## 17. Renombrar un archivo

```
await fs.rename(
  "viejo.txt",
  "nuevo.txt"
);
```

Pero `rename()` también permite moverlo:

```
await fs.rename(
  "datos.txt",
  path.join("backup", "datos.txt")
);
```

¿Por qué?

Porque para el sistema de archivos **renombrar y mover son esencialmente variaciones de modificar la ruta asociada a una entrada.**

---

## 18. Eliminar archivos

```
await fs.unlink("datos.txt");
```

Otra API más general es:

```
await fs.rm("datos.txt");
```

Para eliminar un directorio completo:

```
await fs.rm(
  "datos",
  {
    recursive: true,
    force: true
  }
);
```

Hay que tratar esta operación con respeto: Node no manda mágicamente el contenido a una papelera portátil. Puede eliminarlo directamente.

---

## 19. Comprobar si existe un archivo

Es tentador hacer:

```
if (archivoExiste) {
  // abrir
}
```

pero entre la comprobación y la apertura otro proceso podría eliminarlo.

En muchas situaciones es mejor **intentar directamente la operación**:

```
try {  
  
    const texto = await fs.readFile(  
        "config.txt",  
        "utf8"  
    );  
  
    console.log(texto);  
  
} catch (error) {  
  
    console.error("No pude leer el archivo");  
}
```

Los sistemas de archivos pueden cambiar mientras nuestro programa se ejecuta.

Esto introduce una idea general importante:

Las operaciones con archivos pueden fallar y ese fallo forma parte normal del problema.

---

## 20. Los códigos de error

Podemos inspeccionar:

```
try {  
  
    await fs.readFile("inexistente.txt");  
  
} catch (error) {  
  
    console.log(error.code);  
}
```

Podríamos obtener:

ENOENT

Entonces:

```
try {  
  
  const texto = await fs.readFile(  
    "config.txt",  
    "utf8"  
  );  
  
} catch (error) {  
  
  if (error.code === "ENOENT") {  
    console.log("El archivo no existe");  
  } else {  
    throw error;  
  }  
}
```

Es mejor discriminar el error que asumir que todo fallo significa "archivo inexistente".

También podríamos tener:

- falta de permisos;
- ruta inválida;
- disco lleno;
- dispositivo desconectado;
- archivo bloqueado;
- errores de entrada/salida.

---

## 21. Buscar archivos por nombre o extensión

Supongamos:

```
documentos/  
├─ informe.txt  
├─ notas.md  
├─ foto.png  
└─ alumnos.txt
```

Podemos buscar todos los `.txt`:

```
const archivos = await fs.readdir("documentos");

const textos = archivos.filter(
  nombre => path.extname(nombre) === ".txt"
);

console.log(textos);
```

Resultado:

```
[
  "informe.txt",
  "alumnos.txt"
]
```

También podemos buscar por nombre:

```
const encontrados = archivos.filter(
  nombre => nombre.includes("informe")
);
```

---

## 22. Búsqueda recursiva

Ahora tenemos:

```
documentos/
├─ informe.txt
├─ universidad/
│   ├─ alumnos.txt
│   └─ notas/
│       └─ parcial.txt
└─ personal/
    └─ compras.txt
```

Para recorrer todo el árbol podemos usar recursividad:

```

async function buscar(directorio) {

    const elementos = await fs.readdir(
        directorio,
        { withFileTypes: true }
    );

    const archivos = [];

    for (const elemento of elementos) {

        const ruta = path.join(
            directorio,
            elemento.name
        );

        if (elemento.isDirectory()) {

            const encontrados = await buscar(ruta);

            archivos.push(...encontrados);

        } else {

            archivos.push(ruta);

        }
    }

    return archivos;
}

```

Uso:

```

const archivos = await buscar("documentos");

console.log(archivos);

```

Conceptualmente estamos recorriendo un árbol:

```

documentos
├── informe.txt
├── universidad
│   ├── alumnos.txt
│   ├── notas
│   └── parcial.txt
└── personal
    └── compras.txt

```

Un directorio contiene otros nodos, algunos de los cuales contienen a su vez otros nodos.

Por eso la recursividad encaja naturalmente.

---

## 23. Buscar texto dentro de archivos

Ahora ya sabemos encontrar archivos.

Podemos además examinar su contenido.

```
async function contieneTexto(
  archivo,
  buscado
) {

  const texto = await fs.readFile(
    archivo,
    "utf8"
  );

  return texto.includes(buscado);
}
```

Uso:

```
if (await contieneTexto(
  "informe.txt",
  "JavaScript"
)) {
  console.log("Encontrado");
}
```

Podemos combinar ambos conceptos:

```
recorrer directorios
    ↓
seleccionar .txt
    ↓
leer cada archivo
    ↓
buscar una expresión
```

Eso es esencialmente la base de herramientas como buscadores de código, indexadores y utilidades parecidas a `grep`.

---

## 24. Buscar línea por línea

Supongamos:

```
Juan,25,Tucumán  
Ana,31,Salta  
Pedro,19,Córdoba
```

Podemos hacer:

```
const texto = await fs.readFile(  
  "personas.txt",  
  "utf8"  
);  
  
const lineas = texto.split(/\r?\n/);  
  
for (const linea of lineas) {  
  if (linea.includes("Ana")) {  
    console.log(linea);  
  }  
}
```

Resultado:

```
Ana,31,Salta
```

---

## 25. Extraer información

Un archivo de texto suele contener **estructura**.

Por ejemplo:

```
Ana,31,Tucumán  
Juan,25,Salta
```

Primero obtenemos líneas:

```
const lineas = texto.split(/\r?\n/);
```

Después campos:



```
for (const linea of lineas) {  
    const campos = linea.split(",");  
    console.log(campos);  
}
```

Resultado:

```
["Ana", "31", "Tucumán"]  
["Juan", "25", "Salta"]
```

Podemos convertirlos en objetos:

```
const personas = lineas.map(linea => {  
    const [nombre, edad, ciudad] =  
        linea.split(",");  
    return {  
        nombre,  
        edad: Number(edad),  
        ciudad  
    };  
});
```

Ahora tenemos:

```
[  
  {  
    nombre: "Ana",  
    edad: 31,  
    ciudad: "Tucumán"  
  },  
  {  
    nombre: "Juan",  
    edad: 25,  
    ciudad: "Salta"  
  }  
]
```

Aquí ocurrió una transformación fundamental:

```
archivo
  ↓
texto
  ↓
líneas
  ↓
campos
  ↓
objetos
```

Gran parte del procesamiento de información consiste exactamente en esto.

---

## 26. Un CSV real es más complicado

El ejemplo anterior:

```
linea.split(",");
```

sirve para aprender el principio, pero no constituye un parser CSV completo.

Porque esto es perfectamente válido en CSV:

```
Ana,"San Miguel de Tucumán, Tucumán",31
```

La coma dentro de las comillas **no separa columnas**.

Por eso, para archivos CSV reales conviene utilizar una biblioteca especializada.

La lección general es:

```
No confundamos el formato conceptual que estamos aprendiendo con toda la complejidad de una especificación real.
```

---

## 27. JSON: texto que representa datos

JSON también es simplemente texto.

Archivo:

```
{  
  "nombre": "Ana",  
  "edad": 31  
}
```

Podemos leerlo:

```
const texto = await fs.readFile(  
  "persona.json",  
  "utf8"  
);
```

y convertirlo:

```
const persona = JSON.parse(texto);  
  
console.log(persona.nombre);
```

El proceso es:

```
archivo JSON  
  ↓  
bytes  
  ↓ UTF-8  
string  
  ↓ JSON.parse  
objeto JavaScript
```

Para guardar:

```
const persona = {  
  nombre: "Ana",  
  edad: 31  
};  
  
const texto = JSON.stringify(  
  persona,  
  null,  
  2  
);  
  
await fs.writeFile(  
  "persona.json",  
  texto,  
  "utf8"  
);
```

Aquí hacemos exactamente el recorrido inverso:

```
objeto JavaScript
  ↓
JSON.stringify
  ↓
string
  ↓ UTF-8
bytes
  ↓
archivo
```

## 28. Archivos pequeños y archivos grandes

`readFile()` hace conceptualmente:

```
archivo completo
  ↓
memoria
```

Esto es fantástico para:

```
10 KB
1 MB
10 MB
```

Pero imaginemos:

```
clientes.csv
80 GB
```

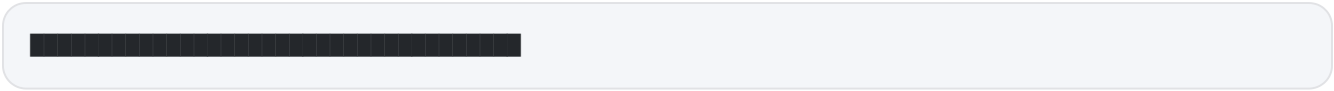
No queremos cargar 80 GB en RAM.

Necesitamos procesar el archivo **por partes**.

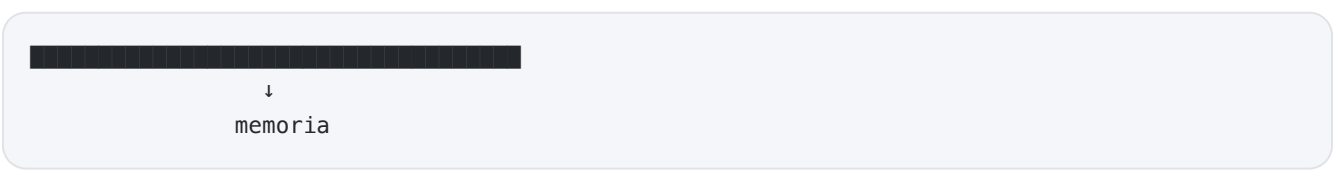
Aquí aparecen los streams.

## 29. Streams

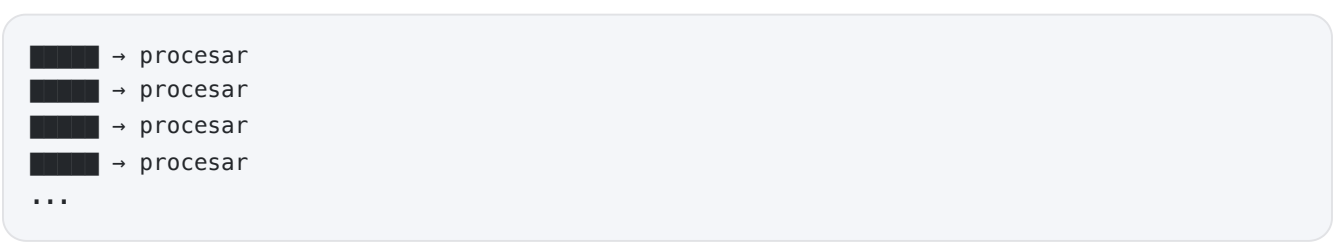
Podemos imaginar un archivo grande así:



`readFile()` intenta:



Un stream hace:



Importamos:

```
import fs from "node:fs";
```

y hacemos:

```
const stream = fs.createReadStream(
  "archivo-grande.txt",
  { encoding: "utf8" }
);
```

Podemos consumirlo:

```
for await (const fragmento of stream) {
  console.log(fragmento);
}
```

Cada `fragmento` contiene una parte del archivo.

Esto permite procesar datos cuyo tamaño puede ser mucho mayor que la memoria disponible.

# 30. Pero un fragmento no es una línea

Un error conceptual frecuente es pensar:

```
chunk = línea
```

No.

El sistema puede entregar:

```
Ana,31,Tucu
```

y después:

```
mán\nJuan,25,S
```

El corte depende del flujo de bytes, no de nuestra estructura lógica.

Para leer líneas existe:

```
import readline from "node:readline";
import fs from "node:fs";

const archivo = fs.createReadStream(
  "personas.txt"
);

const lineas = readline.createInterface({
  input: archivo,
  crlfDelay: Infinity
});

for await (const linea of lineas) {
  console.log(linea);
}
```

Esto resulta mucho más apropiado para archivos de texto grandes.

---

## 31. Síncrono contra asíncrono

Node ofrece, por ejemplo:

```
fs.readFile(...)
```

con callbacks,

```
await fs.promises.readFile(...)
```

con Promises,

y:

```
fs.readFileSync(...)
```

de forma síncrona.

Una operación síncrona:

```
import fs from "node:fs";

const texto = fs.readFileSync(
  "datos.txt",
  "utf8"
);
```

detiene la ejecución hasta terminar.

Conceptualmente:

```
programa
  ↓
leer archivo
  ↓
ESPERAR
  ↓
continuar
```

Mientras:

```
import fs from "node:fs/promises";

const texto = await fs.readFile(
  "datos.txt",
  "utf8"
);
```

utiliza la infraestructura asíncrona de Node.

Para scripts muy pequeños las funciones síncronas pueden ser cómodas.

Para servidores y aplicaciones concurrentes normalmente preferimos las APIs asíncronas.

## 32. File handles

Hasta ahora utilizamos operaciones de alto nivel:

```
readFile()  
writeFile()  
appendFile()
```

Pero conceptualmente un sistema operativo suele trabajar así:

```
abrir archivo  
  ↓  
obtener descriptor  
  ↓  
leer/escribir  
  ↓  
cerrar archivo
```

Node representa esta idea mediante `FileHandle`.

```
const archivo = await fs.open(  
  "datos.txt",  
  "r"  
);  
  
try {  
  
  const contenido =  
    await archivo.readFile("utf8");  
  
  console.log(contenido);  
  
} finally {  
  
  await archivo.close();  
  
}
```

El cierre es importante.

Aquí estamos un nivel más cerca del sistema operativo.

---

## 33. Modos de apertura

Cuando abrimos un archivo indicamos nuestra intención.



Por ejemplo:

```
await fs.open("datos.txt", "r");
```

r:

```
read
```

Lectura.

```
await fs.open("datos.txt", "w");
```

w:

```
write
```

Escritura.

Puede crear el archivo y truncar el existente.

```
await fs.open("datos.txt", "a");
```

a:

```
append
```

Agregar al final.

La idea es importante:

abrir un archivo no significa solamente encontrarlo; también establece cómo pretendemos utilizarlo.

---

## 34. Diferencias entre Windows, macOS y Linux

Node abstrae gran parte de las diferencias, pero no puede eliminarlas todas.

## Separador de rutas

---

Windows:

```
C:\datos\archivo.txt
```

POSIX:

```
/datos/archivo.txt
```

Solución:

```
path.join(...)
```

## Raíz

---

Windows puede tener varias unidades:

```
C:\  
D:\  
E:\
```

Unix/macOS tienen una raíz lógica:

```
/
```

## Mayúsculas y minúsculas

---

En muchos sistemas Linux:

```
Archivo.txt
```

y:

```
archivo.txt
```

son archivos diferentes.

En configuraciones habituales de Windows y macOS pueden referirse al mismo archivo.

Por eso conviene **no depender de diferencias solamente de mayúsculas y minúsculas**.

## Caracteres permitidos

---

Windows impone restricciones particulares sobre algunos caracteres y nombres.

Por ejemplo, nombres como:

```
CON  
NUL  
PRN
```

tienen significados históricos especiales.

Si nuestro programa crea nombres a partir de datos introducidos por usuarios debemos sanitizarlos.

## Permisos

---

Linux y macOS siguen fuertemente el modelo POSIX:

```
lectura  
escritura  
ejecución
```

para distintos propietarios/grupos.

Windows posee su propio modelo de permisos.

Node intenta ofrecer una API uniforme, pero las semánticas no siempre son idénticas entre plataformas.

---

## 35. No escribamos rutas específicas del sistema

Evitemos:

```
const archivo =  
  "C:\\Users\\Alejandro\\datos\\clientes.txt";
```

porque sólo tiene sentido en una determinada computadora Windows.

Y también:

```
const archivo =  
  "/Users/alejandro/datos/clientes.txt";
```

porque presupone una estructura específica.

Mejor:

```
const archivo = path.join(  
  "datos",  
  "clientes.txt"  
);
```

o construirla desde una ubicación conocida dinámicamente.

---

## 36. La carpeta del usuario

Node puede obtener ubicaciones del sistema:

```
import os from "node:os";  
  
console.log(os.homedir());
```

Podría producir:

macOS:

```
/Users/ana
```

Linux:

```
/home/ana
```

Windows:

```
C:\Users\Ana
```

Entonces:

```
const archivo = path.join(  
  os.homedir(),  
  "mi-programa",  
  "datos.json"  
);
```

Es muchísimo más portable que escribir rutas absolutas manualmente.

---

## 37. Construyamos una pequeña utilidad

Supongamos que queremos encontrar todos los `.txt` dentro de un directorio que contengan determinada palabra.

```

import fs from "node:fs/promises";
import path from "node:path";

async function buscar(directorio, palabra) {

    const encontrados = [];

    const elementos = await fs.readdir(
        directorio,
        { withFileTypes: true }
    );

    for (const elemento of elementos) {

        const ruta = path.join(
            directorio,
            elemento.name
        );

        if (elemento.isDirectory()) {

            const internos =
                await buscar(ruta, palabra);

            encontrados.push(...internos);

            continue;
        }

        if (path.extname(elemento.name) !== ".txt") {
            continue;
        }

        const texto = await fs.readFile(
            ruta,
            "utf8"
        );

        if (texto.includes(palabra)) {
            encontrados.push(ruta);
        }
    }

    return encontrados;
}

```

Uso:

```
const archivos = await buscar(
  "documentos",
  "JavaScript"
);

console.log(archivos);
```

Esta pequeña función reúne casi todos los conceptos fundamentales:

```
directorio
  ↓
readdir
  ↓
archivo o directorio
  ↓
recursión
  ↓
path.extname
  ↓
readFile
  ↓
decodificar UTF-8
  ↓
buscar información
```

---

## 38. Extraer estadísticas de muchos archivos

Ahora podemos hacer algo más interesante:

```
async function informacion(archivo) {

  const stat = await fs.stat(archivo);

  return {
    archivo,
    bytes: stat.size,
    modificado: stat.mtime
  };
}
```

Uso:

```
const info = await informacion("datos.txt");

console.log(info);
```

Podemos empezar a construir programas que:

- indexen documentos;
- calculen tamaños;
- encuentren duplicados;
- generen catálogos;
- extraigan estadísticas;
- realicen backups;
- transformen formatos.

Todas estas aplicaciones nacen de las mismas operaciones elementales.

---

## 39. El modelo mental completo

Podemos resumir la gestión de archivos en cuatro niveles.

### Nivel 1 — bytes

---

El disco guarda:

```
01001000
01101111
01101100
01100001
```

Node los puede representar mediante:

```
Buffer
```

### Nivel 2 — texto

---

Una codificación interpreta esos bytes:

```
bytes
↓ UTF-8
Hola
```

En JavaScript obtenemos:



```
string
```

## Nivel 3 — estructura

---

El texto puede representar estructuras:

```
Ana,31,Tucumán
```

que interpretamos como:

```
{  
  nombre: "Ana",  
  edad: 31,  
  ciudad: "Tucumán"  
}
```

## Nivel 4 — sistema de archivos

---

Los archivos están organizados mediante:

```
directorios  
  ↓  
subdirectorios  
  ↓  
archivos
```

y los manipulamos mediante rutas.

En Node:

|          |                    |
|----------|--------------------|
| Buffer   | → bytes            |
| string   | → texto            |
| fs       | → archivos         |
| path     | → rutas            |
| readline | → texto por líneas |
| streams  | → datos grandes    |

---

## 40. Las operaciones fundamentales

Prácticamente toda la gestión cotidiana de archivos se reduce a estas operaciones:

```
// crear / escribir
await fs.writeFile(archivo, datos);

// agregar
await fs.appendFile(archivo, datos);

// leer
await fs.readFile(archivo, "utf8");

// crear directorio
await fs.mkdir(directorio);

// listar
await fs.readdir(directorio);

// información
await fs.stat(archivo);

// copiar
await fs.copyFile(origen, destino);

// mover / renombrar
await fs.rename(origen, destino);

// eliminar
await fs.rm(archivo);

// construir rutas portables
path.join(...partes);
```

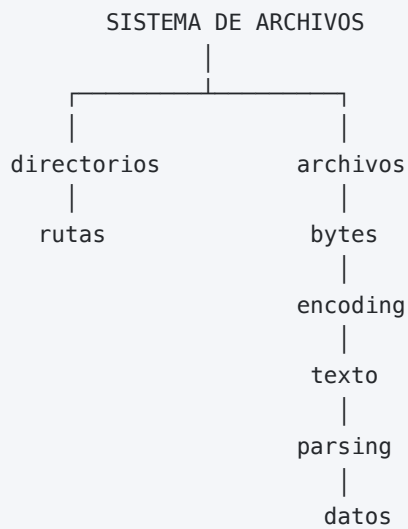
Node proporciona las tres variantes principales de acceso al sistema de archivos —síncrona, callback y `Promise`—; para código moderno general, `node:fs/promises` da una interfaz especialmente cómoda con `async/await`.

---

## 41. La idea más importante

No conviene aprender estas funciones como una colección de comandos independientes.

Hay un modelo mucho más simple:



De ahí salen casi todas las operaciones:

```
crear  
leer  
escribir  
agregar  
copiar  
mover  
renombrar  
eliminar  
listar  
buscar  
clasificar  
extraer  
transformar
```

Cuando entendemos esa jerarquía, `fs` deja de parecer una API llena de funciones arbitrarias.

En realidad, todas responden a una misma pregunta:

**¿Cómo transforma mi programa una estructura de bytes almacenada por el sistema operativo en información que JavaScript pueda manipular, y cómo realiza luego el camino inverso?**

Ese es el fundamento de la gestión de archivos.

# Byte Pair Encoding (BPE) en JavaScript con Node.js

---

## 1. El problema: ¿cómo convertir texto en números?

---

Una computadora no trabaja directamente con palabras como:

```
hola
```

Trabaja con números.

Por eso, antes de enviar texto a un modelo, necesitamos una función que transforme un texto en una secuencia de números:

```
"hola" -> [104, 111, 108, 97]
```

A esos números los llamaremos **tokens**.

Pero aparece inmediatamente una pregunta:

```
¿Qué debería representar cada token?
```

Podríamos usar:

- un token por carácter;
- un token por palabra;
- un token por byte;
- o tokens que representen fragmentos frecuentes de texto.

BPE, **Byte Pair Encoding**, parte de una idea muy simple:

```
empezar con unidades pequeñas y unir repetidamente los pares que aparecen con mayor frecuencia.
```

En nuestro caso las unidades iniciales serán **bytes**.

---

JavaScript maneja strings Unicode:

```
const texto = "hola";
```

Pero para trabajar con bytes necesitamos convertir ese string a una secuencia de bytes.

Node.js permite hacerlo fácilmente:

```
const bytes = Buffer.from("hola", "utf8");  
  
console.log([...bytes]);
```

Resultado:

```
[104, 111, 108, 97]
```

Cada número está entre 0 y 255, porque un byte tiene 8 bits:

```
00000000 -> 0  
11111111 -> 255
```

En ASCII muchos caracteres ocupan un solo byte.

Por ejemplo:

```
h -> 104  
o -> 111  
l -> 108  
a -> 97
```

Pero Unicode contiene muchos más caracteres.

Por ejemplo:

```
console.log([...Buffer.from("ñ", "utf8")]);
```

produce:

```
[195, 177]
```

La ñ necesita dos bytes en UTF-8.

Un emoji puede necesitar todavía más:

```
console.log([...Buffer.from("😄", "utf8")]);
```

produce cuatro bytes.

Esto es importante porque nuestro tokenizer no necesita conocer letras, palabras ni idiomas.

Solo necesita conocer bytes.

---

### 3. ¿Por qué empezar por bytes?

Supongamos que hacemos un tokenizer basado únicamente en palabras.

Durante el entrenamiento encontramos:

```
casa  
perro  
gato
```

Pero después aparece:

```
ornitorrinco
```

Si esa palabra nunca fue aprendida, podríamos no saber cómo representarla.

Con bytes esto nunca ocurre.

Todo texto UTF-8 puede convertirse en bytes.

Por lo tanto, comenzamos con un vocabulario universal de 256 tokens:

```
0  
1  
2  
...  
255
```

Cada token representa exactamente un byte.

Así nuestro tokenizer siempre puede representar cualquier texto.

---

## 4. El problema de usar solamente bytes

Funciona, pero produce demasiados tokens.

Por ejemplo:

banana

en UTF-8 es:

[98, 97, 110, 97, 110, 97]

Son seis tokens.

Pero observemos la estructura:

b a n a n a

El par:

a n

aparece dos veces.

Podríamos inventar un nuevo token que represente:

a n

Entonces:

b a n a n a

se convertiría aproximadamente en:

b [an] [an] a

Ahora tenemos cuatro tokens.

BPE hace exactamente eso.

---

## 5. La idea fundamental de BPE

El algoritmo de aprendizaje puede resumirse así:

1. Convertir el texto de entrenamiento en bytes.
2. Buscar todos los pares de tokens consecutivos.
3. Encontrar el par más frecuente.
4. Crear un nuevo token para ese par.
5. Reemplazar todas sus apariciones.
6. Repetir.

Supongamos:

banana

Inicialmente:

98 97 110 97 110 97

Los pares son:

98 97  
97 110  
110 97  
97 110  
110 97

Contamos:

98 97 → 1 vez  
97 110 → 2 veces  
110 97 → 2 veces

Hay un empate. Nuestro programa puede elegir simplemente el primero que encuentre.

Supongamos que selecciona:

97 110

Creamos el token:

256



con significado:

```
256 = [97, 110]
```

Entonces:

```
98 97 110 97 110 97
```

se transforma en:

```
98 256 256 97
```

Ahora volvemos a contar pares.

BPE continúa aprendiendo estructuras cada vez mayores.

---

## 6. Los tokens nuevos son composiciones

Nuestro vocabulario comienza así:

```
0..255 = bytes originales
```

Luego aprendemos tokens nuevos:

```
256 = [97, 110]  
257 = [98, 256]  
258 = [256, 97]  
...
```

Un token nuevo puede contener tokens que a su vez contienen otros tokens.

Por eso BPE construye una especie de vocabulario jerárquico.

Por ejemplo:

```
256 = "an"  
257 = "ban"  
258 = "ana"  
259 = "banana"
```

No necesariamente aprenderá exactamente esos tokens, pero conceptualmente puede hacerlo.

---

## 7. Qué necesitamos implementar

Construiremos una clase:

```
class BPE
```

con tres operaciones principales:

```
train(text, merges)
encode(text)
decode(tokens)
```

Su uso será:

```
const bpe = new BPE();

bpe.train(textoBase, 50);

const tokens = bpe.encode("banana");

console.log(tokens);

console.log(bpe.decode(tokens));
```

La propiedad fundamental debe cumplirse siempre:

```
bpe.decode(bpe.encode(texto)) === texto
```

---

## 8. Primera pieza: convertir un string en bytes

Creamos una función pequeña:

```
function toBytes(text) {
  return [...Buffer.from(text, "utf8")];
}
```

Ejemplo:

```
console.log(toBytes("hola"));
```

Resultado:

```
[104, 111, 108, 97]
```

Para reconstruir el texto:

```
function fromBytes(bytes) {  
  return Buffer.from(bytes).toString("utf8");  
}
```

Entonces:

```
const bytes = toBytes("hola");  
  
console.log(fromBytes(bytes));
```

produce:

```
hola
```

Esta ida y vuelta será la base de todo el tokenizer.

---

## 9. Contar pares

Supongamos que tenemos:

```
const tokens = [98, 97, 110, 97, 110, 97];
```

Queremos contar:

```
98,97  
97,110  
110,97  
97,110  
110,97
```

Podemos usar un `Map`.

```
function countPairs(tokens) {  
  const counts = new Map();  
  
  for (let i = 0; i < tokens.length - 1; i++) {  
    const key = `${tokens[i]},${tokens[i + 1]}`;  
  
    counts.set(key, (counts.get(key) ?? 0) + 1);  
  }  
  
  return counts;  
}
```

La clave:

```
"97,110"
```

representa el par:

```
[97, 110]
```

No es la representación más eficiente posible.

Es simplemente una representación muy fácil de entender.

---

## 10. Encontrar el par más frecuente

Ahora busquemos el máximo.

```
function mostFrequentPair(tokens) {
  const counts = countPairs(tokens);

  let bestPair = null;
  let bestCount = 0;

  for (const [pair, count] of counts) {
    if (count > bestCount) {
      bestPair = pair;
      bestCount = count;
    }
  }

  if (bestPair === null) {
    return null;
  }

  return bestPair.split(",").map(Number);
}
```

Si:

```
tokens = [98, 97, 110, 97, 110, 97];
```

podría devolver:

```
[97, 110]
```

## 11. Fusionar un par

Ahora necesitamos transformar:

```
98 97 110 97 110 97
```

usando:

```
97 110 -> 256
```

para obtener:

```
98 256 256 97
```

La función es:

```
function mergePair(tokens, pair, newToken) {  
  const result = [];  
  
  for (let i = 0; i < tokens.length; i++) {  
    if (  
      i < tokens.length - 1 &&  
      tokens[i] === pair[0] &&  
      tokens[i + 1] === pair[1]  
    ) {  
      result.push(newToken);  
      i++;  
    } else {  
      result.push(tokens[i]);  
    }  
  }  
  
  return result;  
}
```

El detalle importante es:

```
i++;
```

Cuando encontramos el par consumimos dos tokens.

Por eso debemos saltar el segundo.

---

## 12. El aprendizaje completo

Ya tenemos las piezas fundamentales.

El entrenamiento será:

```
texto  
↓  
bytes  
↓  
buscar par más frecuente  
↓  
crear token  
↓  
fusionar  
↓  
repetir
```

Nuestro primer token nuevo será:

256

porque los valores:

0..255

ya están reservados para bytes.

---

## 13. Nuestra clase BPE

Vamos ahora a reunir todo.

```

class BPE {
  constructor() {
    this.merges = [];
  }

  train(text, mergeCount) {
    let tokens = [...Buffer.from(text, "utf8")];

    this.merges = [];

    for (let n = 0; n < mergeCount; n++) {
      const pair = this.findBestPair(tokens);

      if (!pair) {
        break;
      }

      const newToken = 256 + this.merges.length;

      this.merges.push({
        pair,
        token: newToken
      });

      tokens = this.merge(tokens, pair, newToken);
    }
  }

  encode(text) {
    let tokens = [...Buffer.from(text, "utf8")];

    for (const merge of this.merges) {
      tokens = this.merge(
        tokens,
        merge.pair,
        merge.token
      );
    }

    return tokens;
  }

  decode(tokens) {
    const bytes = [];

    for (const token of tokens) {
      this.expand(token, bytes);
    }

    return Buffer.from(bytes).toString("utf8");
  }

  findBestPair(tokens) {
    const counts = new Map();
  }
}

```



```

for (let i = 0; i < tokens.length - 1; i++) {
  const key = `${tokens[i]},${tokens[i + 1]}`;

  counts.set(
    key,
    (counts.get(key) ?? 0) + 1
  );
}

let bestPair = null;
let bestCount = 0;

for (const [key, count] of counts) {
  if (count > bestCount) {
    bestPair = key;
    bestCount = count;
  }
}

if (bestPair === null) {
  return null;
}

return bestPair.split(",").map(Number);
}

merge(tokens, pair, newToken) {
  const result = [];

  for (let i = 0; i < tokens.length; i++) {
    if (
      i < tokens.length - 1 &&
      tokens[i] === pair[0] &&
      tokens[i + 1] === pair[1]
    ) {
      result.push(newToken);
      i++;
    } else {
      result.push(tokens[i]);
    }
  }

  return result;
}

expand(token, bytes) {
  if (token < 256) {
    bytes.push(token);
    return;
  }

  const merge = this.merges[token - 256];

  this.expand(merge.pair[0], bytes);
}

```

```
this.expand(merge.pair[1], bytes);  
}  
}
```

Esta es prácticamente toda la implementación.

El resto consiste en entender por qué funciona.

---

## 14. Cómo aprende `train`

Consideremos:

```
const bpe = new BPE();  
  
bpe.train("banana banana banana", 5);
```

Primero ejecutamos:

```
Buffer.from(text, "utf8")
```

y obtenemos bytes.

Luego repetimos:

```
const pair = this.findBestPair(tokens);
```

Por ejemplo podría aparecer:

```
[97, 110]
```

Creamos:

```
const newToken = 256;
```

y guardamos:

```
{  
  pair: [97, 110],  
  token: 256  
}
```

Conceptualmente:

```
256 = 97 + 110
```

o, interpretado como texto:

```
256 = "an"
```

Después reemplazamos las apariciones.

Luego buscamos nuevamente el par más frecuente.

El segundo token nuevo será:

```
257
```

el tercero:

```
258
```

y así sucesivamente.

---

## 15. La tabla de fusiones es el modelo aprendido

Después de entrenar:

```
console.log(bpe.merges);
```

podríamos obtener algo parecido a:

```
[
  { pair: [97, 110], token: 256 },
  { pair: [98, 256], token: 257 },
  { pair: [256, 97], token: 258 }
]
```

Esto significa:

```
256 = [97, 110]
257 = [98, 256]
258 = [256, 97]
```

Es importante entender esto:

```
BPE no aprende una red neuronal.
```

Aprende una **lista ordenada de fusiones**.

Ese es su modelo.

---

## 16. Cómo funciona `encode`

Supongamos que ya aprendimos:

```
97 110 -> 256
98 256 -> 257
```

Codificamos:

```
bpe.encode("banana");
```

Primero:

```
banana
```

se convierte en:

```
98 97 110 97 110 97
```

Aplicamos la primera regla:

```
97 110 -> 256
```

y obtenemos:

```
98 256 256 97
```

Después aplicamos:

```
98 256 -> 257
```

obteniendo:

```
257 256 97
```

Y esos son los tokens finales.

La parte crucial es esta:

```
for (const merge of this.merges) {  
  tokens = this.merge(tokens, merge.pair, merge.token);  
}
```

Las reglas se aplican **en el mismo orden en que fueron aprendidas**.

---

## 17. ¿Por qué importa el orden?

Supongamos:

```
256 = "an"  
257 = "ban"
```

El token `257` depende del `256`.

Es decir:

```
257 = [98, 256]
```

No podríamos crear `257` antes de haber creado `256`.

Por eso las reglas BPE tienen un orden.

---

## 18. Cómo funciona `decode`

Decodificar parece más complicado, pero en realidad es muy simple.

Recordemos:

```
257 = [98,256]
256 = [97,110]
```

Entonces:

```
257
```

se expande como:

```
98 256
```

y `256` se expande como:

```
97 110
```

Resultado:

```
98 97 110
```

que en UTF-8 representa:

```
ban
```

Nuestra función:

```
expand(token, bytes)
```

hace esto recursivamente.

---

## 19. El caso base de la recursión

La función empieza:

```
if (token < 256) {  
  bytes.push(token);  
  return;  
}
```

Si el token es menor que 256, ya es un byte original.

No hay nada más que expandir.

Este es el **caso base**.

---

## 20. El caso recursivo

Si:

```
token >= 256
```

entonces es un token aprendido.

Buscamos su regla:

```
const merge = this.merges[token - 256];
```

Por ejemplo:

```
token = 257
```

entonces:

```
257 - 256 = 1
```

y buscamos:

```
this.merges[1]
```

Luego expandimos las dos partes:

```
this.expand(merge.pair[0], bytes);  
this.expand(merge.pair[1], bytes);
```

Eventualmente todo termina convertido en bytes originales.

---

## **21. Ejemplo completo ejecutable**

Guardemos:



```
// bpe.js

class BPE {
  constructor() {
    this.merges = [];
  }

  train(text, mergeCount) {
    let tokens = [...Buffer.from(text, "utf8")];

    this.merges = [];

    for (let n = 0; n < mergeCount; n++) {
      const pair = this.findBestPair(tokens);

      if (!pair) break;

      const newToken = 256 + this.merges.length;

      this.merges.push({
        pair,
        token: newToken
      });

      tokens = this.merge(tokens, pair, newToken);
    }
  }

  encode(text) {
    let tokens = [...Buffer.from(text, "utf8")];

    for (const { pair, token } of this.merges) {
      tokens = this.merge(tokens, pair, token);
    }

    return tokens;
  }

  decode(tokens) {
    const bytes = [];

    for (const token of tokens) {
      this.expand(token, bytes);
    }

    return Buffer.from(bytes).toString("utf8");
  }

  findBestPair(tokens) {
    const counts = new Map();

    for (let i = 0; i < tokens.length - 1; i++) {
      const key = `${tokens[i]},${tokens[i + 1]}`;

```

```

        counts.set(
            key,
            (counts.get(key) ?? 0) + 1
        );
    }

    let bestPair = null;
    let bestCount = 0;

    for (const [key, count] of counts) {
        if (count > bestCount) {
            bestPair = key;
            bestCount = count;
        }
    }

    if (bestPair === null) {
        return null;
    }

    return bestPair.split(",").map(Number);
}

merge(tokens, pair, newToken) {
    const result = [];

    for (let i = 0; i < tokens.length; i++) {
        if (
            i < tokens.length - 1 &&
            tokens[i] === pair[0] &&
            tokens[i + 1] === pair[1]
        ) {
            result.push(newToken);
            i++;
        } else {
            result.push(tokens[i]);
        }
    }

    return result;
}

expand(token, bytes) {
    if (token < 256) {
        bytes.push(token);
        return;
    }

    const { pair } = this.merges[token - 256];

    this.expand(pair[0], bytes);
    this.expand(pair[1], bytes);
}
}

```

```
// -----  
// Ejemplo  
// -----  
  
const bpe = new BPE();  
  
const trainingText = `  
banana banana banana  
bandana banana  
`;  
  
bpe.train(trainingText, 10);  
  
console.log("Fusiones:");  
console.log(bpe.merges);  
  
const text = "banana";  
  
const tokens = bpe.encode(text);  
  
console.log("Texto:");  
console.log(text);  
  
console.log("Tokens:");  
console.log(tokens);  
  
console.log("Decodificado:");  
console.log(bpe.decode(tokens));
```

Ejecutamos:

```
node bpe.js
```

La propiedad que debemos comprobar es:

```
const text = "banana";  
  
const tokens = bpe.encode(text);  
const restored = bpe.decode(tokens);  
  
console.log(restored === text);
```

Debe producir:

```
true
```

## 22. Probemos Unicode

Nuestro algoritmo trabaja con bytes UTF-8.

Por eso también funciona con:

```
const text = "mañana 😊";
```

Podemos comprobar:

```
const tokens = bpe.encode(text);  
  
console.log(tokens);  
console.log(bpe.decode(tokens));
```

La salida reconstruida será:

```
mañana 😊
```

Aunque ñ y 😊 ocupen varios bytes.

Esta es precisamente una de las ventajas de construir el tokenizer sobre bytes.

---

## 23. Qué significa realmente un token

Al principio:

```
token 97 = byte 97
```

y ese byte representa:

```
"a"
```

Pero después:

```
token 256
```

podría representar:

"an"

y:

token 257

podría representar:

"ban"

y otro token podría representar:

"banana"

Por lo tanto un token no es necesariamente:

- una letra;
- una sílaba;
- una palabra.

Es simplemente:

una secuencia de bytes que el algoritmo decidió que era útil agrupar.

---

## 24. ¿Por qué esto reduce la cantidad de tokens?

Supongamos que usamos únicamente bytes:

banana

requiere:

6 tokens

Después de aprender ciertas fusiones podríamos obtener:

[257, 258]

es decir:

2 tokens

Cuanto más frecuente es una secuencia en el corpus de entrenamiento, mayor es la posibilidad de que BPE termine representándola con pocos tokens.

---

## 25. El compromiso fundamental

Si hacemos pocas fusiones:

vocabulario pequeño  
muchos tokens por texto

Si hacemos muchas fusiones:

vocabulario grande  
menos tokens por texto

Ese es uno de los compromisos fundamentales de un tokenizer.

En nuestro programa:

```
bpe.train(text, 100);
```

aprende cien fusiones.

El vocabulario total tendría como máximo:

$256 + 100 = 356$  tokens

---

## 26. Una pequeña mejora: no fusionar pares que aparecen una sola vez

Nuestra versión actual aprende el par más frecuente aunque aparezca una sola vez.

Para fines educativos esto simplifica el código.

Pero podemos detenernos cuando el mejor par tenga frecuencia menor que 2.

Podríamos modificar:

```
findBestPair()
```

para devolver también la frecuencia:

```
return {  
  pair: bestPair.split(",").map(Number),  
  count: bestCount  
};
```

y durante entrenamiento hacer:

```
if (!best || best.count < 2) {  
  break;  
}
```

Así solo aprendemos patrones que realmente se repiten.

No es necesario para comprender el algoritmo básico.

---

## 27. Qué hemos simplificado

Esta implementación está diseñada para mostrar el algoritmo, no para competir con tokenizadores de producción.

Deliberadamente omitimos:

- optimizaciones de rendimiento;
- estructuras especiales para contar pares;
- pretokenización;
- reglas especiales para espacios;
- regex complejas;
- tokens reservados;
- almacenamiento del vocabulario en archivos;
- entrenamiento con corpus enormes;
- paralelización;
- codificación incremental.

La esencia de BPE permanece intacta:

```
bytes
↓
contar pares
↓
fusionar el más frecuente
↓
guardar la regla
↓
repetir
```

---

## 28. La idea completa en pseudocódigo

El entrenamiento entero puede escribirse conceptualmente así:

```
tokens = UTF8(texto)

repetir N veces:

    contar pares consecutivos

    elegir el más frecuente

    crear nuevo token

    guardar:
        par -> nuevo token

    reemplazar el par
```

Codificar:

```
tokens = UTF8(texto)

para cada regla aprendida:

    aplicar la fusión

devolver tokens
```

Decodificar:



```
para cada token:
```

```
    si es byte:  
        devolver byte
```

```
    si es token aprendido:  
        expandir sus dos componentes
```

```
convertir bytes UTF-8 a string
```

Eso es BPE en su forma esencial.

---

## 29. La clase mínima final

Si eliminamos explicaciones y dejamos solamente el núcleo del algoritmo:

```

class BPE {
  constructor() {
    this.merges = [];
  }

  train(text, mergeCount) {
    let tokens = [...Buffer.from(text)];

    this.merges = [];

    for (let n = 0; n < mergeCount; n++) {
      const counts = new Map();

      for (let i = 0; i < tokens.length - 1; i++) {
        const key = `${tokens[i]},${tokens[i + 1]}`;
        counts.set(key, (counts.get(key) ?? 0) + 1);
      }

      let best = null;
      let max = 0;

      for (const [key, count] of counts) {
        if (count > max) {
          best = key;
          max = count;
        }
      }

      if (!best) break;

      const pair = best.split(",").map(Number);
      const token = 256 + this.merges.length;

      this.merges.push({ pair, token });

      tokens = this.merge(tokens, pair, token);
    }
  }

  encode(text) {
    let tokens = [...Buffer.from(text)];

    for (const { pair, token } of this.merges) {
      tokens = this.merge(tokens, pair, token);
    }

    return tokens;
  }

  decode(tokens) {
    const bytes = [];

    const expand = token => {
      if (token < 256) {

```

```

        bytes.push(token);
        return;
    }

    const { pair } = this.merges[token - 256];

    expand(pair[0]);
    expand(pair[1]);
};

for (const token of tokens) {
    expand(token);
}

return Buffer.from(bytes).toString("utf8");
}

merge(tokens, pair, newToken) {
    const result = [];

    for (let i = 0; i < tokens.length; i++) {
        if (
            tokens[i] === pair[0] &&
            tokens[i + 1] === pair[1]
        ) {
            result.push(newToken);
            i++;
        } else {
            result.push(tokens[i]);
        }
    }

    return result;
}
}

```

Esta versión contiene prácticamente todo lo conceptualmente importante de BPE.

---

## 30. Qué conviene recordar

No memorices el código.

Recordá el proceso:

```
texto
↓
UTF-8
↓
bytes
↓
pares frecuentes
↓
fusiones
↓
tokens cada vez mayores
```

El punto central es este:

BPE construye un vocabulario descubriendo secuencias frecuentes a partir de unidades elementales.

Al trabajar sobre bytes obtenemos además una propiedad extremadamente útil:

cualquier texto Unicode puede ser representado, incluso si contiene caracteres o palabras nunca vistos durante el entrenamiento.

Y finalmente, las tres operaciones de nuestro tokenizer son:

```
bpe.train(textoBase, cantidadDeFusiones);

const tokens = bpe.encode(texto);

const textoOriginal = bpe.decode(tokens);
```

Ese pequeño ciclo contiene la esencia de un tokenizer BPE.